

Angewandte Mathematik

Über schräge Typen, künstliche Beweise und krumme Zahlen

19. Oktober 2026

Einleitung

Worum es geht

Dieses Skript behandelt Grundlagen und einige Kapitel der angewandten Mathematik. Jeder Begriff kommt zweimal vor: einmal normal aufgeschrieben, einmal als Lean-Code, den der Compiler geprüft hat.

Das hat einen praktischen Grund. Wer etwas formalisiert, muss Fragen beantworten, die man sonst offenlassen kann:

- Worüber läuft der Quantor?
- Was ist der Definitionsbereich dieser Funktion?
- Welche Voraussetzung wird wo gebraucht?
- Was passiert dort, wo ein Ausdruck undefiniert ist?

Diese Fragen zu beantworten hilft beim Verstehen – auch ohne Computer.

Mathlib baut nicht auf ZFC auf

Im Studium lernt man meist: Die Mathematik steht auf der Zermelo–Fraenkel-Mengenlehre mit Auswahlaxiom (ZFC). Alles ist eine Menge, auch Zahlen, Paare und Funktionen.

Mathlib ist anders gebaut. Die Grundlage ist eine abhängige Typentheorie, der Calculus of Inductive Constructions. Dort ist nicht alles eine Menge. Stattdessen hat jeder Ausdruck einen *Typ*, und Typen sind der Grundbegriff. Das wirkt sich direkt darauf aus, wie man Begriffe bildet:

- Eine Menge ist immer Teilmenge eines Typs, den es schon gibt. Eine „Menge aller Mengen“ gibt es nicht. Die Russellsche Menge kann man nicht einmal hinschreiben (Kapitel 2).
- Ein Beweis ist ein Term, eine Aussage ist ein Typ (Kapitel 3).
- Funktionen sind *total*. Ausdrücke wie $1/0$ oder die Ableitung an einer nicht differenzierbaren Stelle bekommen in Mathlib irgendeinen Wert. Die eigentliche Voraussetzung steht dann im Satz.
- Grenzwerte laufen nicht über ε und N, sondern über *Filter*. Ein Begriff für alle Grenzwertarten (Kapitel 4).

Die klassischen Grundlagen kommen trotzdem vor. ZFC und NBG stehen in Kapitel 2 – aber als Theorien, die man *innerhalb* der Typentheorie definiert. Mathlib enthält beide; Abschnitt 3.6 zeigt, wie.

Wie man das Skript liest

Kapitel 1 steht vorweg und begründet das Ganze: warum Beweise überhaupt geprüft werden müssen, was ein Beweisassistent daran ändert, und warum seit 2024 auch Programme Beweise in Lean abliefern.

Lean erscheint erst ab Kapitel 3. Kapitel 2 bleibt bei der gewöhnlichen mathematischen Sprache – mit einer Ausnahme: Abschnitt 2.8 benutzt Haskell, weil die Kategorientheorie dort Sprachbestandteil ist. Abschnitt 3.6 nimmt die Mengenlehre später noch einmal auf und formalisiert sie in Lean.

Ab Kapitel 3 gehört zu jedem Kapitel eine Datei in `code/`:

Kapitel bzw. Abschnitt	**Datei**
2.8 Kategorientheorie	`code/Kategorien.hs`
3 Typentheorie, mit 3.6 Mengenlehre in Lean	`code/Kapitel3.lean`
4 Infinitesimalrechnung	`code/Kapitel4.lean`
5 Numerische Integration	`code/Kapitel5.lean`
6 Differentialgleichungen	`code/Kapitel6.lean`
7 Sprachmodelle	`code/Kapitel7.lean`

Alle Codeabschnitte sind unverändert aus diesen Dateien eingebunden. Was hier steht, hat also ein Compiler akzeptiert – Lean das eine, GHC das andere. Pseudo-Code gibt es nicht. Zu jeder Vorlesung gehört außerdem eine Datei `code/VorlesungNN.lean` mit dem Code in der Reihenfolge der Folien.

Zum Rechnen kommen fünf Jupyter-Notebooks in `notebooks/` dazu (Abschnitt 6.11). Sie lösen Differentialgleichungen mit `scipy` und erzeugen dabei die Abbildungen, die das Skript an dieser Stelle einbindet – dieselbe Regel wie beim Code: Gezeigt wird nur, was wirklich gelaufen ist. Dafür braucht man `numpy`, `scipy` und `matplotlib`; `make notebooks` führt sie aus.

Mathlib. Kästen wie dieser nennen den genauen Namen eines Begriffs in Mathlib und die Datei, in der er steht. Der erste davon erscheint in Kapitel 3.

Software

Man braucht Lean 4 mit Mathlib (getestet mit der Toolchain `leanprover/lean4:v4.26.0` und Mathlib `v4.26.0`) und eine LaTeX-Installation. Einzelne Lean-Dateien prüft man so:

```
lake env lean LeanMath/Angewandte_Mathematik/code/Kapitel4.lean
```

Alle auf einmal mit `make lean`. Das Skript baut `make`.

Inhaltsverzeichnis

Einleitung **1**

1 Warum Mathematik in Lean? **5**
1.1 Beweise, die niemand mehr prüft . . . 5
1.2 Was ein Beweisassistent ändert . . . 5
1.3 Mathlib: eine Bibliothek statt vieler Aufsätze . . . 6
1.4 Was bereits formalisiert ist . . . 6
1.5 Künstliche Beweise . . . 7
1.6 Drei Ergebnisse aus dem Jahr 2026 . . . 7
1.7 Warum Lean und KI zusammenpassen . . . 9
1.8 Wo es weiterhin klemmt . . . 10
1.9 Was das für dieses Skript heißt . . . 10

2 Mengenlehre **11**
2.1 Naive Mengenlehre . . . 11
2.2 Russells Paradoxon . . . 13
2.3 Nicht jede Formel ist eine Menge . . . 13
2.4 Formale Systeme . . . 14
2.5 ZFC . . . 25
2.6 Semantik und Modelle . . . 31
2.7 NBG: Mengen und Klassen . . . 41
2.8 Kategorientheorie . . . 46

3 Typentheorie **53**
3.1 Terme, Typen, Universen . . . 53
3.2 Induktive Typen, Strukturen, Typklassen . . . 56
3.3 Proof as Proposition: die Curry–Howard-Korrespondenz . . . 57
3.4 Gleichheit und die Axiome . . . 58
3.5 Inferenzregeln und Typprüfung . . . 60
3.6 Mengenlehre in Lean . . . 66
3.7 Lean: Term- und Taktikmodus . . . 74
3.8 Die Strukturen der Mathematik in Lean . . . 74

4 Infinitesimalrechnung **83**
4.1 Räume . . . 83
4.2 Konvergenz durch Filter . . . 91
4.3 Vollständigkeit . . . 96
4.4 Ableitung . . . 100
4.5 Gradientenabstieg . . . 103
4.6 Optima unter Nebenbedingungen . . . 107

5 Numerische Integration **110**
5.1 Das Integral in Mathlib . . . 110
5.2 Quadraturformeln . . . 111
5.3 Zusammengesetzte Formeln . . . 114
5.4 Mehrere Dimensionen . . . 115
5.5 Der Fluch der Dimension . . . 117
5.6 Numerisches Experiment . . . 118

6 Gewöhnliche Differentialgleichungen **121**
6.1 Wozu das gut ist . . . 121
6.2 Das Anfangswertproblem . . . 122
6.3 Stetigkeit reicht nicht . . . 123
6.4 Die Lipschitz-Bedingung . . . 124
6.5 Picard–Lindelöf . . . 125
6.6 Eindeutigkeit und Grönwall . . . 126
6.7 Lineare Systeme . . . 127
6.8 Das kleine Einmaleins . . . 132
6.9 Das Pendel, ausgeführt . . . 136
6.10 Das Euler-Verfahren . . . 141
6.11 Rechnen mit scipy . . . 142

7 Sprachmodelle **149**
7.1 Was ein Sprachmodell ist . . . 149
7.2 Softmax . . . 149
7.3 Die Verlustfunktion . . . 150
7.4 Der Transformer . . . 152
7.5 Training . . . 154
7.6 Was skaliert . . . 155
7.7 Was Sprachmodelle über Intelligenz verständlich machen . . . 155
7.8 Was sie nicht zeigen . . . 157

Kapitel 1

Warum Mathematik in Lean?

Mathematik gilt als das Fach, in dem man sich sicher sein kann. Ein Beweis ist richtig oder falsch, und wer ihn liest, kann das entscheiden. Das stimmt, solange Beweise kurz genug sind, dass jemand sie wirklich liest.

1.1 Beweise, die niemand mehr prüft

Drei Beispiele dafür, dass diese Voraussetzung nicht immer gilt.

Die endlichen einfachen Gruppen. Ihre Klassifikation steht in etwa 500 Arbeiten von mehr als hundert Autoren, zusammen rund 10 000 Seiten, entstanden zwischen 1955 und 1983. Niemand hat sie ganz gelesen. 1983 galt der Beweis als fertig. Später stellte sich heraus, dass ein Fall – die quasidünnen Gruppen – nicht vollständig behandelt war. Aschbacher und Smith schlossen die Lücke 2004, auf etwa 1200 weiteren Seiten.

Die Keplersche Vermutung. Thomas Hales bewies 1998, dass keine Kugelpackung dichter ist als die im Obstladen übliche. Der Beweis stützt sich auf umfangreiche Rechnungen. Die Gutachter der *Annals of Mathematics* gaben nach vier Jahren auf: Sie seien zu 99 Prozent überzeugt, mehr könnten sie nicht sagen. Hales begann daraufhin *Flyspeck* und formalisierte den ganzen Beweis. 2014 war das Projekt fertig, in HOL Light und Isabelle.

Fehler in eigenen Arbeiten. Vladimir Voevodsky, Fields-Medaille 2002, fand Jahre nach der Veröffentlichung Fehler in eigenen Sätzen, die niemandem aufgefallen waren. Er zog daraus den Schluss, dass die übliche Kontrolle durch Gutachter für sein Gebiet nicht ausreicht, und arbeitete seitdem an Grundlagen und Formalisierung. Was daraus wurde, steht in Bemerkung 3.1.

In allen drei Fällen ist nicht die Mathematik das Problem, sondern die Prüfung. Ein Beweis ist nur so viel wert, wie jemand nachrechnen kann.

1.2 Was ein Beweisassistent ändert

Ein Beweisassistent prüft keinen Text. Er prüft einen Term. Die Aussage ist ein Typ, der Beweis ein Ausdruck dieses Typs, und die Frage „ist der Beweis richtig?“ wird zur Frage „hat dieser Ausdruck diesen Typ?“. Das ist eine Rechnung, und die kann eine Maschine ausführen. Wie das genau geht, steht in Kapitel 3.

Entscheidend ist, wer diese Rechnung ausführt. In Lean ist das der *Kern*: ein Programm von einigen tausend Zeilen, das nichts weiter kann als Terme auf ihren Typ zu prüfen. Alles andere – Taktiken, Elaborator, Automatisierung, Bibliotheken – steht außerhalb. Diese Aufteilung heißt

de-Bruijn-Kriterium: Der bequeme Teil darf beliebig kompliziert sein, weil am Ende der kleine Teil nachrechnet.

Wer sich irren darf. Eine Taktik darf Fehler haben. Schlimmstenfalls erzeugt sie einen Term, den der Kern ablehnt – dann steht eine Fehlermeldung da, kein falscher Satz. Deshalb kann man Automatisierung bedenkenlos benutzen, auch schlecht getestete und auch solche, die eine KI geschrieben hat.

Umsonst ist das nicht. Drei Dinge muss man weiterhin glauben:

(1) **Der Kern rechnet richtig.** Dagegen hilft, dass er klein ist und dass es unabhängig geschriebene Nachprüfer gibt, die denselben Beweisterm noch einmal durchrechnen.

(2) **Die Axiome sind widerspruchsfrei.** Lean benutzt drei (Abschnitt 3.4). Welche in einem Beweis vorkommen, kann man sich anzeigen lassen.

(3) **Die Aussage ist die, die man meinte.** Dagegen hilft nichts. Darauf kommt Abschnitt 1.8 zurück.

1.3 Mathlib: eine Bibliothek statt vieler Aufsätze

Formalisieren lohnt sich erst, wenn man nicht bei null anfängt. Mathlib ist der Versuch, die Mathematik in *eine* Bibliothek zu schreiben: über eine Million Zeilen, mehrere hundert Mitwirkende, täglich erweitert.

Der Unterschied zu einer Sammlung einzelner Beiträge ist wichtig. In Mathlib gibt es den Begriff „Gruppe“ genau einmal. Jeder Satz über Gruppen gilt deshalb sofort für die ganzen Zahlen, für Permutationen, für invertierbare Matrizen und für alles, was später dazukommt. Dasselbe für Ringe, Körper, Vektorräume, Ordnungen.

Der Preis dafür ist, dass alles zusammenpassen muss. Wer die Definition von „Ring“ ändert, betrifft tausende Sätze. Daher die strenge Ordnung der Strukturen, die Kapitel 3 beschreibt, und daher auch Entscheidungen, die zunächst seltsam wirken – etwa dass jede Funktion überall definiert ist und $1/0$ einen Wert hat.

1.4 Was bereits formalisiert ist

Jahr	Ergebnis	System
2005	Vierfarbensatz	Coq
2012	Satz von Feit–Thompson (ungerade Ordnung)	Coq
2014	Keplersche Vermutung (Flyspeck)	HOL Light, Isabelle
2022	Liquid Tensor Experiment	Lean
2022	Sphärenumstülpung (Gromovs h-Prinzip)	Lean
2023	Polynomiale Freiman–Ruzsa-Vermutung	Lean
2025	Equational Theories Project	Lean
2026	Zetafunktion: 67,2 Prozent der Nullstellen	Lean
2026	Navier–Stokes mit Kraft: Blowup	Lean
laufend	Großer Fermatscher Satz	Lean

Die beiden Einträge von 2026 stammen nicht von Menschen; dazu Abschnitt 1.6. Zwei der älteren sind für dieses Skript aufschlussreich.

Das *Liquid Tensor Experiment* geht auf eine Frage von Peter Scholze zurück. Er stellte 2020 einen eigenen, schwierigen Satz zur Formalisierung und sagte dazu, er sei sich bei diesem Beweis selbst nicht sicher. Ein Jahr später war der Kern formalisiert, 2022 das Ganze. Scholzes Fazit

war nicht nur „es stimmt“, sondern dass die Formalisierung die Struktur des Beweises sichtbar gemacht hat.

Die *polynomiale Freiman–Ruzsa-Vermutung* wurde im November 2023 bewiesen, von Gowers, Green, Manners und Tao. Drei Wochen später lag der Beweis formalisiert vor – erarbeitet von rund zwanzig Leuten, die den Beweis in Teilstücke zerlegten und parallel bearbeiteten. Das ist der eigentliche Fortschritt: Formalisierung ist von einer Lebensaufgabe zu einem Projekt von Wochen geworden.

1.5 Künstliche Beweise

Seit 2024 kommt etwas dazu. Programme finden Beweise, die vorher niemand aufgeschrieben hat, und geben sie in Lean ab.

AlphaProof. Bei der Internationalen Mathematik-Olympiade 2024 löste ein System von Google DeepMind vier der sechs Aufgaben und erreichte 28 von 42 Punkten – einen Punkt unter der Goldgrenze. Drei Aufgaben übernahm AlphaProof, die Geometrieaufgabe ein zweites System. Die Lösungen von AlphaProof waren Lean-Beweise. Bewertet hat sie kein Mensch, sondern der Compiler.[1]

Das Training lief so: Ein Sprachmodell übersetzte etwa eine Million informell formulierter Aufgaben in Lean-Aussagen. Vieles davon war unbrauchbar, das macht nichts – unbrauchbare Aussagen lassen sich eben nicht beweisen. Auf diesem Vorrat lief ein Verfahren nach dem Muster von AlphaZero: einen Beweis vorschlagen, prüfen lassen, aus dem Ergebnis lernen, von vorn. Was dort der Gegner im Spiel ist, ist hier der Compiler.

Offene Modelle. DeepSeek-Prover, Kimina-Prover, Goedel-Prover, Seed-Prover und andere erzeugen ebenfalls Lean-Beweise; ihre Gewichte sind frei verfügbar. Gemessen werden sie an Sammlungen wie *miniF2F* (Aufgaben auf Wettbewerbsniveau) und *PutnamBench*. Auf miniF2F sind die Trefferquoten innerhalb weniger Jahre von etwa einem Drittel auf über achtzig Prozent gestiegen. Das gilt für Aufgaben mit bekanntem Zuschnitt. Wie es bei Forschungsfragen aussieht, steht im nächsten Abschnitt.

Zwei Arten von Goldmedaille. Bei der Olympiade 2025 meldeten mehrere Anbieter Ergebnisse auf Goldniveau. Ein Teil davon waren Beweise in natürlicher Sprache, die von Menschen bewertet wurden. Ein anderer Teil waren Lean-Beweise. Der Unterschied ist genau der Punkt dieses Kapitels: Im ersten Fall muss man den Gutachtern glauben, im zweiten dem Kern.

Werkzeuge für Menschen. Nebenbei ist das Formalisieren von Hand leichter geworden. Lean sucht passende Lemmas selbst, Taktiken schließen Routineschritte, und Ergänzungen schlagen ganze Beweisschritte vor. Was davon taugt, entscheidet sofort der Compiler – man muss den Vorschlag nicht lesen, um zu wissen, ob er hält.

1.6 Drei Ergebnisse aus dem Jahr 2026

Wettbewerbsaufgaben haben eine bekannte Antwort und passen auf eine Seite. 2026 kamen Ergebnisse dazu, bei denen beides nicht gilt. Drei davon, weil sie drei verschiedene Rollen der Maschine zeigen.

[1] Die Arbeit dazu erschien 2025 in *Nature*.

Ein Gegenbeispiel, das in einen Beitrag passt. Die *Jacobi-Vermutung* (Keller, 1939) besagt: Hat eine polynomiale Abbildung $F : \mathbb{C}^n \to \mathbb{C}^n$ überall dieselbe, von null verschiedene Jacobi-Determinante, dann hat sie eine polynomiale Umkehrung. Am 19. Juli 2026 veröffentlichte Levent Alpöge eine Abbildung $\mathbb{C}^3 \to \mathbb{C}^3$ mit Determinante -2, die drei verschiedene Punkte auf denselben Bildpunkt wirft:

$$F(0, 0, -\tfrac{1}{4}) \;=\; F(1, -\tfrac{3}{2}, \tfrac{13}{2}) \;=\; F(-1, \tfrac{3}{2}, \tfrac{13}{2}).$$

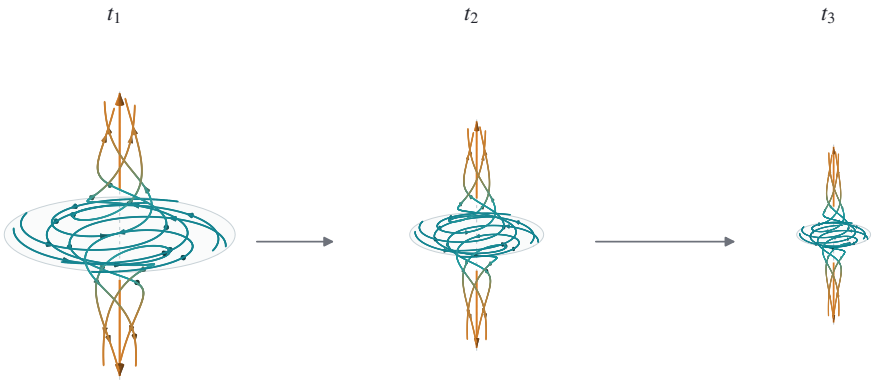
Damit ist die Vermutung für jedes $n \geq 3$ falsch; der Fall $n = 2$ bleibt offen. Gefunden hat das Beispiel ein Sprachmodell (Claude Fable 5); wie, ist nicht veröffentlicht.[2] Das spielt hier auch keine Rolle – die Rechnung macht jedes Computeralgebrasystem in Sekunden, und das ganze Gegenbeispiel passt in einen einzigen Kurznachrichtenbeitrag.

Eine Schranke, weiter getrieben. Ob die Riemannsche Vermutung stimmt, weiß niemand. Man weiß aber, dass ein Teil der nichttrivialen Nullstellen der Zetafunktion auf der kritischen Geraden liegt, und dieser Anteil wurde seit den 1970er Jahren in kleinen Schritten nach oben geschoben; zuletzt stand er bei 41,6 Prozent. Im August 2026 meldete Anthropic, eine Forschungsversion von Claude habe ihn auf etwa 67,2 Prozent gebracht.[3]

Der Ablauf ist aufschlussreich: 650 Ansätze, dann rund 60 Teilprozesse, 2400 Aufrufe der Kommandozeile, 31 Millionen ausgegebene Zeichen, anderthalb Tage Rechnen. Geprüft wurde danach dreifach – von Mathematikern im Haus, von zwei externen Fachleuten (Brian Conrey, Dan Goldston) und durch eine Formalisierung in Lean.

Was dabei *nicht* herauskam, ist ein neuer Gedanke. Das Ergebnis treibt ein bekanntes Verfahren weiter, als Menschen es getan hatten, und stützt sich auf Vorarbeiten der letzten Jahre. Anthropic schreibt selbst, dieser Weg werde die Riemannsche Vermutung nicht beweisen.

Navier–Stokes. Am 8. September 2026 veröffentlichte OpenAI einen Beweis, dass Lösungen der dreidimensionalen Navier–Stokes-Gleichungen mit einer glatten äußeren Kraft in endlicher Zeit unendlich werden können – die Geschwindigkeiten wachsen über alle Grenzen, die Energie bleibt endlich. Gefunden hat ihn ein nicht veröffentlichtes Modell; im Einsatz waren nach Angaben der Firma rund 10 000 Agenten und Rechenzeit im Millionenwert. Veröffentlicht wurden eine Ausarbeitung und eine Formalisierung in Lean.[4]



Der innere Kern der Strömung zu drei Zeitpunkten $t_1 < t_2 < t_3$ vor der Singularität: Das Fluid läuft spiralförmig nach innen (türkis) und strömt beiderseits einer Trennschicht entlang der Achse ab (orange). Der Kern wird immer schmaler, die Geschwindigkeit wächst über alle Grenzen. Mit $\tau = 1 - t$, der Zeit bis zur Singularität, schrumpft der Radius wie $\tau^{1/2}$, die Höhe wie $\tau^{1/2-h}$ für ein festes $0 < h < 1/100$, und die Geschwindigkeiten wachsen wie $\tau^{-1/2-h}$; dass der Kern schneller dünner als kürzer wird, ist im Bild übertrieben. Abbildung aus OpenAI, *Finite time blowup for Navier–Stokes* (2026), Abb. 1.

[2] Die Ausarbeitung dazu: arXiv:2608.00222.

[3] `https://www.anthropic.com/research/riemann-zeta`

[4] `https://openai.com/index/navier-stokes-solution/`

Drei Einschränkungen gehören dazu. Erstens ist das die Aussage (C) aus Feffermans offizieller Problembeschreibung, die Variante *mit* Kraft. Sie zählt nach dieser Beschreibung als Lösung des Millennium-Problems; OpenAI hat den Preis trotzdem nicht beansprucht. Die Frage, auf die es den meisten ankommt – ob glatte Lösungen ohne äußere Kraft immer existieren –, ist offen. Zweitens baut die Konstruktion auf einer Methode von Diego Córdoba und Luis Martínez-Zoroa von 2023 auf. Drittens war der Beweis Mitte September noch nicht unabhängig nachgeprüft.

Einen Tag zuvor hatten Tristan Buckmaster (NYU) und Levent Alpöge Ergebnisse zu den verwandten Euler-Gleichungen angekündigt, erarbeitet mit Codex und Claude. Darüber entstand ein Streit: Buckmaster warf OpenAI vor, von der eingeschlagenen Richtung erfahren zu haben, bevor sie öffentlich war; OpenAI bestreitet das.

Bemerkung 1.1 (Was daran neu ist – und was nicht). Die drei Fälle zeigen drei Rollen. Beim Gegenbeispiel sucht die Maschine ein Objekt, das ein Mensch danach in einer Zeile nachrechnet. Bei der Zetafunktion treibt sie ein bekanntes Verfahren weiter, als jemand es von Hand tun würde. Bei Navier–Stokes liefert sie eine Konstruktion, die so lang ist, dass ein Beweisassistent das einzige verlässliche Prüfmittel bleibt.

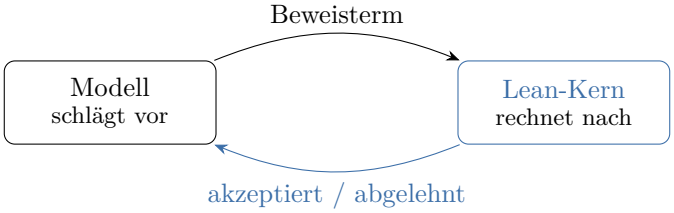
Nur im dritten Fall ist die Formalisierung wirklich nötig – und genau dort wird die Frage aus Abschnitt 1.8 scharf. Ob die formalisierte Aussage die ist, die Fefferman aufgeschrieben hat, prüft kein Compiler. Das muss ein Mensch lesen.

Und noch etwas: Im Streit um Navier–Stokes ging es nicht darum, ob der Beweis stimmt. Es ging darum, wer zuerst da war. Ein formaler Beweis entscheidet über Richtigkeit, nicht über Urheberschaft.

1.7 Warum Lean und KI zusammenpassen

Sudoku. Ein Sudoku zu lösen ist schwer. Nachzusehen, ob eine ausgefüllte Tabelle eine Lösung ist, ist leicht – man prüft Zeilen, Spalten, Kästchen, fertig. Beweise sind genauso: finden ist schwer, prüfen ist leicht.

Auf dieser Asymmetrie beruht alles Weitere. Ein Sprachmodell ist ein guter Vorschlaggeber und ein schlechter Prüfer: Es schreibt flüssigen Text, auch wenn nichts dahintersteht. Lean ist ein schlechter Vorschlaggeber und ein exakter Prüfer. Zusammen ergibt das eine Schleife:



Drei Eigenschaften dieser Schleife sind wichtig:

- **Das Signal ist exakt.** Es gibt kein „fast richtig“. Der Kern sagt ja oder nein, und er sagt es aus denselben Gründen wie bei jedem anderen Beweis.
- **Das Signal ist umsonst.** Kein Mensch muss etwas bewerten. Deshalb kann man Millionen Versuche machen. Beim Training an natürlichsprachlichen Beweisen geht das nicht.
- **Halluzinationen enden am Kern.** Ein erfundenes Lemma gibt es nicht, ein übersprungener Schritt typt nicht. Was durchkommt, folgt aus den Axiomen – unabhängig davon, wie es zustande kam.

Der letzte Punkt ist der Grund, warum Typentheorie plötzlich ein praktisches Thema ist und nicht nur ein grundlagentheoretisches. Man braucht dem Modell nicht zu vertrauen. Man muss nur den Kern prüfen lassen – was dabei im Einzelnen passiert, steht in Bemerkung 3.8.

1.8 Wo es weiterhin klemmt

Geprüft wird der Beweis, nicht die Aussage. Das ist die eigentliche Schwachstelle. Wer $\forall \varepsilon\, \exists N$ und $\exists N\, \forall \varepsilon$ vertauscht, bekommt einen einwandfreien Beweis einer anderen Aussage. Dazu kommen die Konventionen, mit denen Mathlib Funktionen überall definiert: $3 - 5 = 0$ auf $\mathbb{N}$, $x/0 = 0$, die Ableitung an einer Stelle, an der es keine gibt. Das sind keine Fehler, sondern Festlegungen – aber eine Aussage, die daraus ihre Wahrheit bezieht, sagt nichts.

Für Wettbewerbsaufgaben kann man die Formalisierung von Hand prüfen. Für Forschungsaussagen ist das Übersetzen selbst eine Arbeit, die Verständnis verlangt. Sie automatisch und zuverlässig zu erledigen, ist ungelöst.

Lücken und Axiome. `sorry` markiert in Lean eine offene Stelle. Der Compiler meldet das als Warnung, nicht als Fehler – die Datei übersetzt trotzdem. Wer einen fremden Beweis übernimmt, sieht also zuerst nach, ob `sorry` darin vorkommt und welche Axiome benutzt werden.

Richtig heißt nicht verständlich. Ein maschinell gefundener Beweis kann aus tausend Zeilen Fallunterscheidung bestehen. Er ist korrekt und erklärt nichts. Der Vierfarbensatz war 1976 das erste bekannte Beispiel, die Keplersche Vermutung das zweite; mit KI-Beweisen wird der Fall zum Normalfall. Formalisiert heißt nicht verstanden, und beides zu bekommen bleibt Handarbeit.

1.9 Was das für dieses Skript heißt

Jede Codezeile in diesem Skript stammt aus einer Datei, die der Compiler akzeptiert hat; herausgeschnitten wird sie maschinell. Was hier steht, ist also geprüft – dieselbe Mechanik wie oben, nur im Kleinen.

Der Weg dorthin geht über drei Stufen. Kapitel 2 klärt, was ein formales System überhaupt ist: Zeichen, Axiome, Regeln, Ableitungen – und warum die naive Mengenlehre dafür nicht taugt. Kapitel 3 zeigt Leans Grundlage und wie der Kern prüft. Ab Kapitel 4 wird damit gearbeitet.

Kapitel 2

Mengenlehre

Wer die Mathematik begründen will, fängt meist bei der Mengenlehre an. Dieses Kapitel geht den historischen Weg: naive Mengenlehre, Russellsches Paradoxon, axiomatische Reparatur. Am Ende steht ein Blick auf die Kategorientheorie, die Objekte ganz anders beschreibt.

Es geht dabei um eine Frage: Welche Zusammenfassungen sind erlaubt? Die naive Antwort „alle“ führt in den Widerspruch. ZFC und NBG geben darauf zwei verschiedene, tragfähige Antworten.

Bemerkung 2.1 (Noch kein Lean). Dieses Kapitel kommt ohne Lean-Code aus; nur an wenigen Stellen blickt es in Worten voraus, vor allem in den Abschnitten 2.5 und 2.7. Lean wird in Kapitel 3 eingeführt; Abschnitt 3.6 nimmt dann alles hier Besprochene noch einmal auf und formalisiert es.

Eine Ausnahme ist Abschnitt 2.8. Dort steht Haskell – nicht als Formalisierung, sondern weil die Kategorientheorie in der Informatik längst Alltag ist und sich die Axiome dort unmittelbar wiederfinden. Auch dieser Code ist vom Compiler geprüft.

2.1 Naive Mengenlehre

Cantors Definition von 1895 lautet:

> „Unter einer *Menge* verstehen wir jede Zusammenfassung M von bestimmten wohlunterschiedenen Objekten m unserer Anschauung oder unseres Denkens (welche die *Elemente* von M genannt werden) zu einem Ganzen.“

Objekte unseres Denkens

Drei Wörter in diesem Satz sind wichtiger, als sie aussehen.

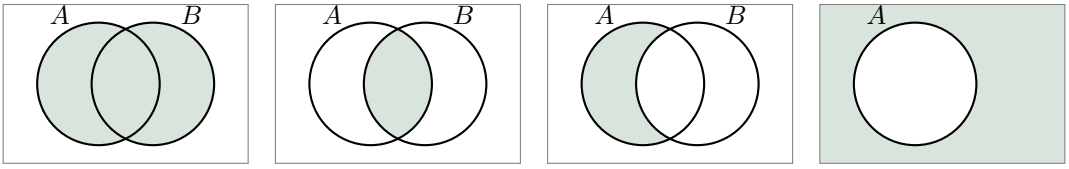
„Unserer Anschauung oder unseres Denkens.“ Cantor gründet die Menge auf *geistige* Objekte. Eine Menge ist danach nichts, was man vorfindet, sondern etwas, das im Denken zusammengefasst wird. Das war keine Nebenbemerkung, sondern Programm. Die unendlichen Mengen, um die es Cantor ging, kann niemand aufzählen und niemand anschauen. Entweder sie sind Gegenstände des Denkens, oder es gibt sie nicht.

„Bestimmten.“ Von jedem Objekt muss feststehen, ob es dazugehört. Ein „vielleicht“ ist nicht vorgesehen. Daraus wird später die Rolle der Formeln: Die Zugehörigkeit muss durch eine Bedingung festgelegt sein (Abschnitt 2.5).

„Wohlunterschiedenen.“ Von je zwei Objekten muss feststehen, ob sie dasselbe sind. Daraus wird das Extensionalitätsaxiom.

Die Stärke der Anschauung

Die naive Mengenlehre leuchtet vor allem deshalb ein, weil man sie *zeichnen* kann. Ein Kreis ist eine Menge, ein Punkt darin ein Element, und die Operationen liest man direkt am Bild ab.



$A \cup B$ $A \cap B$ $A \setminus B$ A^{c}

Solche Bilder gehen auf Euler zurück und heißen nach John Venn, der sie 1880 systematisch benutzte. Sie sind nützlich, weil aus Rechenregeln Bilder werden: Dass $(A \cup B)^{\mathrm{c}} = A^{\mathrm{c}} \cap B^{\mathrm{c}}$ gilt, muss man nicht nachrechnen – man sieht es. Kein anderes Gebiet der Mathematik ist so früh so zugänglich.

Wo das Bild aufhört. Und genau hier liegt das Problem. Das Diagramm zeigt Elemente als *Punkte* und Mengen als *Kreise*. Das sind zwei verschiedene Dinge: Ein Punkt ist kein Kreis, und ein Kreis liegt nicht in sich selbst.

Die naive Komprehension macht diese Unterscheidung nicht mit. Sie erlaubt, $x \in x$ hinzuschreiben – und dazu hat die Anschauung nichts zu sagen. Zeichnen lässt sich ein Kreis, der als Punkt in sich selbst liegt, nicht.

Die Bilder funktionieren also nur, solange man Mengen und Elemente sauber trennt. Wo die Theorie diese Trennung aufgibt, hört die Anschauung auf – und dort beginnt das Paradoxon des nächsten Abschnitts.

Warum der Rückgriff aufs Denken nicht trägt

Der erste Punkt hat sich nicht gehalten. Der Grund ist einfach: Er sagt nicht, welche Zusammenfassungen es gibt.

Denken lässt sich $\{\, x \mid x \notin x \,\}$ ohne jede Mühe. Der Ausdruck ist kurz, verständlich, und für jedes einzelne x ist entschieden, ob die Bedingung zutrifft. Trotzdem gibt es diese Menge nicht, wie der nächste Abschnitt zeigt. Dass wir etwas denken können, ist also kein Grund dafür, dass es existiert. Die Anschauung hat hier schlicht keine Meinung.

Die Auffassung, Logik und Mathematik handelten von geistigen Vorgängen, heißt *Psychologismus*. Frege hat sie scharf angegriffen: Ein Gedanke ist nicht deshalb wahr, weil jemand ihn denkt. Der Satz des Pythagoras galt, bevor ihn jemand fasste, und gälte weiter, wenn ihn niemand mehr dächte. Sonst hinge die Mathematik daran, wie viele Menschen es gibt und wie sie gebaut sind – und es gäbe nur endlich viele Mengen, weil nur endlich viele Gedanken gedacht werden.

Die Axiomatisierungen ziehen daraus die Konsequenz. Sie sagen nichts darüber, was man denken kann, sondern nur, welche Bildungen *erlaubt* sind. Statt Anschauung gibt es eine Liste von Regeln. Was eine Menge *ist*, wird gar nicht mehr erklärt. Festgelegt wird nur, was man mit Mengen tun darf.

Bemerkung 2.2. Von Cantors Definition bleibt damit ein Wort übrig: die „Zusammenfassung“. Nur ist das Zusammenfassen keine Tätigkeit mehr, sondern eine Erlaubnis.

Das Komprehensionsprinzip

Formal braucht man von Cantors Definition nur ein Prinzip.

Definition 2.3 (Uneingeschränkte Komprehension)**.** Zu jeder Eigenschaft φ gibt es die Menge

$$\{\, x \mid \varphi(x) \,\}$$

aller Objekte mit dieser Eigenschaft; es gilt $y \in \{x \mid \varphi(x)\} \iff \varphi(y)$.

Das Prinzip wirkt harmlos. Man darf zu jeder Formel die Menge ihrer Erfüller bilden, ohne jede Einschränkung. Genau daran scheitert die naive Theorie.

2.2 Russells Paradoxon

Satz 2.4 (Russell, 1901)**.** *Uneingeschränkte Komprehension (Definition 2.3) ist widersprüchlich.*

Beweis. Man bilde $R := \{\, x \mid x \notin x \,\}$. Nach Komprehension gilt für jedes y die Äquivalenz $y \in R \iff y \notin y$. Für $y := R$ folgt

$$R \in R \iff R \notin R,$$

ein Widerspruch: Keine Aussage ist zu ihrer eigenen Negation äquivalent. □

Der Barbier. In einem Dorf rasiert der Barbier genau diejenigen Männer, die sich *nicht* selbst rasieren.

Die Frage ist: Rasiert der Barbier sich selbst?

Rasiert er sich selbst, so gehört er zu denen, die sich selbst rasieren – dann darf er sich nach seiner eigenen Vorschrift gerade nicht rasieren. Rasiert er sich nicht, so gehört er zu denen, die sich nicht selbst rasieren – dann muss er sich nach derselben Vorschrift rasieren. Beide Antworten widerlegen sich selbst.

Einen solchen Barbier kann es also nicht geben. Nicht, weil das Dorf zu klein wäre, sondern weil die Beschreibung in sich widersprüchlich ist.

Ersetzt man „rasiert“ durch „enthält als Element“, so steht dort genau die Menge R aus dem Beweis.

Der Beweis besteht aus zwei Teilen, und man sollte sie trennen.

Der erste Teil ist reine Aussagenlogik: Aus $P \iff \neg P$ folgt ein Widerspruch, ganz gleich, was P bedeutet. Mengenlehre kommt darin nicht vor.

Der zweite Teil ist die Feststellung, dass uneingeschränkte Komprehension genau so eine Aussage liefert. Man braucht dafür nur zwei Dinge: eine Elementbeziehung und das Recht, zu jeder Formel die Menge ihrer Erfüller zu bilden. Wer beides annimmt, kann alles beweisen.

Bemerkung 2.5 (Wo eine Reparatur ansetzen kann). Aus der Trennung folgt, wo man eingreifen muss. Am logischen Teil lässt sich nichts ändern. Also muss die Komprehension eingeschränkt werden – entweder indem man vorschreibt, woraus ausgesondert werden darf (der Weg von ZFC), oder indem man jede Zusammenfassung zulässt und stattdessen einschränkt, was als Element auftreten darf (der Weg von NBG). Beide Wege kommen in diesem Kapitel vor.

2.3 Nicht jede Formel ist eine Menge

Russells Paradoxon ist kein Einzelfall. Der Satz von Cantor zeigt: Es gibt immer mehr „Eigenschaften“ als „Objekte“.

Satz 2.6 (Cantor)**.** *Für jede Menge A gibt es keine surjektive Abbildung $A \to \mathcal{P}(A)$ und keine injektive Abbildung $\mathcal{P}(A) \to A$.*

Beweis. Sei $f : A \to \mathcal{P}(A)$. Die Menge $D := \{\, a \in A \mid a \notin f(a) \,\}$ liegt nicht im Bild von f: Wäre $D = f(a_0)$, so folgte $a_0 \in D \iff a_0 \notin f(a_0) \iff a_0 \notin D$. Das ist das Argument aus Satz 2.4, diesmal diagonalisiert. □

Das Diagonalverfahren. Man stelle sich eine Tabelle vor. Die Zeilen sind die Mengen $f(a)$, die Spalten die Elemente a. In jedes Feld schreibt man, ob $a \in f(a)$ gilt:

	a_1	a_2	a_3	a_4
$f(a_1)$	$\in$	$\notin$	$\in$	$\notin$
$f(a_2)$	$\in$	$\in$	$\notin$	$\in$
$f(a_3)$	$\notin$	$\notin$	$\notin$	$\in$
$f(a_4)$	$\in$	$\notin$	$\in$	$\in$
D	$\notin$	$\notin$	$\in$	$\notin$

Die hervorgehobene *Diagonale* liest man ab und dreht jeden Eintrag um. Das ergibt die letzte Zeile: die Menge D.

D kann keine Zeile der Tabelle sein. Wäre $D = f(a_k)$, dann müssten sich Zeile k und die Zeile D überall gleichen – insbesondere in der Spalte a_k. Dort aber ist D gerade so gebaut, dass es *anders* ist. Die Tabelle hat also immer eine Zeile zu wenig, egal wie man sie anlegt.

Dasselbe Argument, mit Ziffern statt $\in$, zeigt, dass die reellen Zahlen nicht abzählbar sind.

Daraus folgt: Es gibt keine Menge, die alle ihre eigenen Teilmengen als Elemente enthält. Eine „Menge aller Mengen“ ist also nicht nur unpraktisch, sondern unmöglich.

Damit ist klar, dass die naive Komprehension nicht zu retten ist. Die beiden klassischen Antworten darauf sind ZFC und NBG. Sie greifen an verschiedenen Stellen an, und beide funktionieren.

2.4 Formale Systeme

Nach Russells Paradoxon fehlt etwas. „Man kann es denken“ taugt nicht als Kriterium dafür, dass es etwas gibt – das war das Ergebnis von Abschnitt 2.3. Was tritt an seine Stelle?

Im 20. Jahrhundert fand man eine Antwort: das *formale System*. Statt sich auf Einsicht zu verlassen, schreibt man auf, welche Zeichenreihen erlaubt sind und nach welchen Regeln man von einer zur nächsten kommt. Ob ein Beweis korrekt ist, wird damit zu einer Frage, die man *nachsehen* kann.

Woraus ein formales System besteht

Definition 2.7 (Formales System)**.** Ein *formales System* besteht aus

(a) einem *Zeichenvorrat*;

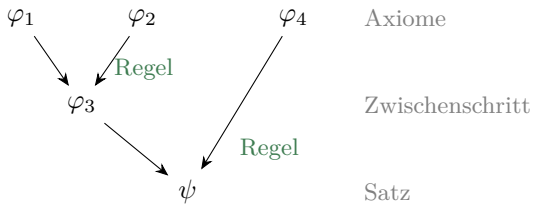
(b) Regeln, welche Zeichenreihen als *Formeln* gelten (der *Syntax*);

(c) einer Menge ausgezeichneter Formeln, den *Axiomen*;

(d) einer Menge von *Schlussregeln*, die aus Formeln neue Formeln erzeugen.

Eine *Schlussregel* hat dabei endlich viele *Prämissen* und eine *Konklusion*.

Was eine *Herleitung* ist, legt die Definition bewusst noch nicht fest. Die beiden Bauarten unten geben darauf verschiedene Antworten – einmal eine Folge, einmal ein Baum – und beide werden gebraucht.



Ein Spiel mit Zeichen. Ein formales System ist gebaut wie ein Brettspiel. Die Regeln sagen, welche Züge erlaubt sind – nicht, was die Figuren „bedeuten“. Ob eine Partie regelgerecht gespielt wurde, kann jemand nachprüfen, der von Strategie nichts versteht und nicht einmal weiß, wozu man das Spiel spielt.

Das ist der Vorteil. Ob eine Zeichenreihe eine Herleitung ist, hängt nicht mehr davon ab, ob jemand sie einleuchtend findet. Es ist eine Eigenschaft der Zeichenreihe selbst, die man Zeile für Zeile abhaken kann. Vorher brauchte man Anschauung, jetzt genügt Nachrechnen.

Die Sprache der Logik

Die Formeln der Logik werden aus wenigen Zeichen gebaut. Weil die folgenden Abschnitte ständig von ihnen reden, hier ihre Namen.

Zeichen	**gelesen**	**Name**
$\neg\varphi$	nicht φ	Negation
$\varphi \wedge \psi$	φ und ψ	Konjunktion
$\varphi \vee \psi$	φ oder ψ	Disjunktion
$\varphi \to \psi$	wenn φ, dann ψ	Implikation
$\varphi \leftrightarrow \psi$	φ genau dann, wenn ψ	Äquivalenz
$\bot$	falsch, der Widerspruch	Falsum
$\forall x\, \varphi$	für alle x gilt φ	Allquantor
$\exists x\, \varphi$	es gibt ein x mit φ	Existenzquantor

- Die *Junktoren* $\neg, \wedge, \vee, \to, \leftrightarrow$ verbinden Formeln zu neuen Formeln. Klassisch ist es eine Frage der Bequemlichkeit, wie viele davon man als Grundzeichen nimmt: Alle lassen sich durch $\neg$ und $\to$ ausdrücken, etwa $\varphi \vee \psi$ durch $\neg\varphi \to \psi$. Intuitionistisch geht das nicht; dort braucht man $\wedge$, $\vee$, $\to$ und $\bot$ und setzt nur $\neg\varphi := \varphi \to \bot$.

- Die *Quantoren* $\forall$ und $\exists$ *binden* eine Variable. In $\forall x\,(x = y)$ ist x *gebunden* – der Name spielt keine Rolle, solange man keinen schon vorkommenden nimmt: $\forall z\,(z = y)$ sagt dasselbe, $\forall y\,(y = y)$ nicht –, während y *frei* vorkommt: Wovon die Formel handelt, hängt davon ab, wofür y steht. Eine Formel ohne freie Variablen heißt *Aussage*.

- Die griechischen Buchstaben $\alpha, \beta, \gamma, \varphi, \psi, \chi$ stehen *in Formeln* für beliebige Formeln. Sie gehören nicht zur Sprache selbst, sondern zum Reden über sie. Ein *Schema* wie $\alpha \to (\beta \to \alpha)$ meint daher unendlich viele Formeln, seine *Einsetzungsinstanzen*: Für α und β darf man jede Formel einsetzen, an jeder Stelle dieselbe.

- Ist φ aus den Annahmen Σ herleitbar, schreibt man $\Sigma \vdash \varphi$, ohne Annahmen kurz $\vdash \varphi$. Eine ohne Annahmen herleitbare Formel heißt *Satz* des Systems. Bewiesene Behauptungen *über* ein System – etwa das Deduktionstheorem weiter unten – heißen dagegen *Metatheoreme*.

Viele Axiome oder viele Regeln?

Definition 2.7 lässt offen, wie man die Arbeit zwischen Axiomen und Regeln aufteilt. Man kann es an beiden Enden ansetzen, und beide Extreme werden benutzt.

Hilbert-Kalkül	**Natürliches Schließen**
viele Axiomenschemata, *eine* Regel (modus ponens)	(fast) kein Axiom, dafür zu jedem Junktor eine Einführungs- und eine Beseitigungsregel
Herleitung ist eine flache Liste	Herleitung ist ein Baum, Annahmen werden unterwegs entlassen
Beweise sind lang und unanschaulich	Beweise sehen aus wie gewöhnliches Schließen
Metatheorie ist leicht	Metatheorie ist aufwendig

Der Hilbert-Kalkül.

Definition 2.8 (Hilbert-Kalkül)**.** Die Formeln seien aus Aussagevariablen mit $\neg$ und $\to$ gebaut (die übrigen Junktoren gelten als Abkürzungen). *Axiome* sind alle Einsetzungsinstanzen der drei Schemata

$$\begin{aligned} &\text{(K)} && \alpha \to (\beta \to \alpha),\\ &\text{(S)} && \big(\alpha \to (\beta \to \gamma)\big) \to \big((\alpha \to \beta) \to (\alpha \to \gamma)\big),\\ &\text{(N)} && (\neg\beta \to \neg\alpha) \to (\alpha \to \beta). \end{aligned}$$

Einzige *Regel* ist der *modus ponens*:

$$\frac{\alpha \qquad \alpha \to \beta}{\beta}\ \text{(MP)}$$

Jede Einsetzungsinstanz eines der drei Schemata ist ein Axiom. Es sind also unendlich viele Axiome, aber man erkennt in endlicher Zeit, ob eine Formel eines ist.

Beide sagen etwas Harmloses. **(K)** besagt: Was gilt, gilt auch dann noch, wenn man eine zusätzliche Annahme β danebenlegt und sie nicht benutzt. **(S)** verteilt eine gemeinsame Annahme: Wer aus α die Implikation $\beta \to \gamma$ gewinnt und aus α auch β, der gewinnt aus α schon γ.

Definition 2.9 (Ableitung im Hilbert-Kalkül)**.** Sei Σ eine Menge von Formeln. Eine *Ableitung von φ aus Σ* ist eine endliche Folge $\varphi_1, \dots, \varphi_n$ von Formeln mit $\varphi_n = \varphi$, in der jedes φ_i

(a) ein Axiom ist, oder

(b) ein Element von Σ, oder

(c) durch modus ponens aus zwei früheren Gliedern entsteht: Es gibt $j, k < i$ mit $\varphi_k = (\varphi_j \to \varphi_i)$.

Gibt es eine solche Folge, so schreibt man $\Sigma \vdash \varphi$; für $\Sigma = \emptyset$ kurz $\vdash \varphi$.

Ableitbarkeit als Regelsystem. Definition 2.9 beschreibt, wie man eine Ableitung *aufschreibt*. Dieselbe Relation $\Sigma \vdash \varphi$ lässt sich auch induktiv durch Regeln festlegen – und in dieser Form wird sie mit dem natürlichen Schließen vergleichbar:

$$\frac{}{\Sigma \vdash \varphi}\ \text{(Ax)} \qquad \frac{}{\Sigma \vdash \varphi}\ \text{(Hyp)} \qquad \frac{\Sigma \vdash \varphi \to \psi \qquad \Sigma \vdash \varphi}{\Sigma \vdash \psi}\ \text{(MP)}$$

Nebenbedingungen: bei (Ax) ist φ eine Einsetzungsinstanz eines der Schemata (K), (S), (N); bei (Hyp) ist $\varphi \in \Sigma$. Beide Regeln haben keine Prämissen – sie sind die Blätter jeder Ableitung.

Bemerkung 2.10. Die beiden Fassungen definieren dieselbe Relation. Aus einer Folge $\varphi_1, \ldots, \varphi_n$ liest man die Regelanwendungen der Reihe nach ab; umgekehrt schreibt man einen Regelbaum in beliebiger Reihenfolge als Folge nieder.

Bemerkung 2.11 (Für die Prädikatenlogik). Für Quantoren kommen zwei Axiomenschemata hinzu – $\forall x\, \varphi \to \varphi[t/x]$ (falls t für x einsetzbar ist) und $\forall x(\varphi \to \psi) \to (\varphi \to \forall x\, \psi)$ (falls x in φ nicht frei vorkommt) – sowie die Regel der *Generalisierung*:

$$\frac{\varphi}{\forall x\, \varphi}\ (\textsc{Gen})$$

Für das Gleichheitszeichen kommen die *Gleichheitsaxiome* $x = x$ und $x = y \to (\varphi \to \varphi[y/x])$ hinzu (falls y für x einsetzbar ist); im natürlichen Schließen entsprechen ihnen eine Einführungs- und eine Beseitigungsregel für $=$. Auf die Struktur der Ableitung ändert das nichts. In der Regelfassung kommt entsprechend $\dfrac{\Sigma \vdash \varphi}{\Sigma \vdash \forall x\, \varphi}$ (Gen) hinzu, mit der Nebenbedingung, dass x in keiner Formel aus Σ frei vorkommt.

Wie man auf den Beweis kommt. Nun zu $\varphi \to \varphi$. Man arbeitet rückwärts.

Die einzige Regel ist der modus ponens. Der letzte Schritt muss also aus irgendeinem X und $X \to (\varphi \to \varphi)$ das Ziel erzeugen. Woher soll $X \to (\varphi \to \varphi)$ kommen? Die rechte Hälfte von (S) hat die Bauform $(\alpha \to \beta) \to (\alpha \to \gamma)$. Setzt man dort $\alpha := \varphi$ und $\gamma := \varphi$, so steht da

$$(\varphi \to \beta) \to (\varphi \to \varphi),$$

und das passt mit $X := \varphi \to \beta$. Was β ist, bleibt offen – und genau darauf kommt es an.

Um (S) benutzen zu dürfen, braucht man seine linke Hälfte, also $\varphi \to (\beta \to \varphi)$. Das ist genau (K), für *jedes* β. Und man braucht X selbst, also $\varphi \to \beta$. Wählt man nun

$$\beta := \varphi \to \varphi,$$

so wird auch $\varphi \to (\varphi \to \varphi)$ zu einer Instanz von (K). Beide Lücken sind mit demselben Axiom gefüllt, und der Beweis ist fertig.

Der Beweis, Zeile für Zeile. Mit $\alpha := \varphi$, $\beta := \varphi \to \varphi$, $\gamma := \varphi$:

(1) (S), eingesetzt:

$$(\varphi \to ((\varphi \to \varphi) \to \varphi)) \to ((\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi))$$

(2) (K) mit $\alpha := \varphi$, $\beta := \varphi \to \varphi$:

$$\varphi \to ((\varphi \to \varphi) \to \varphi)$$

Das ist genau die *linke* Hälfte von (1).

(3) Modus ponens aus (1) und (2) – die linke Hälfte fällt weg:

$$(\varphi \to (\varphi \to \varphi)) \to (\varphi \to \varphi)$$

(4) (K) mit $\alpha := \varphi$, $\beta := \varphi$:

$$\varphi \to (\varphi \to \varphi)$$

Das ist genau die linke Hälfte von (3).

(5) Modus ponens aus (3) und (4):

$$\varphi \to \varphi$$

Das Muster ist zweimal dasselbe: Eine Zeile hat die Form $A \to B$, die nächste liefert A, und der modus ponens streicht A weg. Beide Male ist das benötigte A eine Instanz von (K).

Dieselben zwei Bausteine kennt man aus der Programmierung. (K) und (S) sind keine willkürlichen Formeln. Liest man den Pfeil als Funktionstyp, so ist (K) die konstante Funktion `const :: a -> b -> a` und (S) die Anwendung unter gemeinsamem Argument, `(a -> b -> c) -> (a -> b) -> a -> c`. Der modus ponens ist das Anwenden einer Funktion auf ein Argument.

Und $\varphi \to \varphi$ ist dann `id`. Die fünf Zeilen von eben sind genau die Gleichung $\mathrm{id} = S\,K\,K$: Man rechne $S\,K\,K\,x = K\,x\,(K\,x) = x$. Dass das kein Zufall ist, steht in Abschnitt 3.3.

Bemerkung 2.12. Man sieht hier gut, warum Hilbert-Beweise so umständlich sind. Die Aussage „φ folgt aus φ“ ist trivial; die Herleitung ist es nicht. Der Grund: Man darf φ nirgends *annehmen*. Alles muss aus geschlossenen Axiomen zusammengesetzt werden, und β muss man raten.

Gentzens natürliches Schließen. Gentzen kehrte 1934 die Gewichtung um. Statt Axiome anzugeben, aus denen alles folgt, gibt er zu *jedem* Junktor zwei Regeln: eine, die ihn erzeugt (*Einführung*), und eine, die ihn wieder verbraucht (*Beseitigung*). Axiome braucht er dafür keine – jedenfalls nicht für die intuitionistische Logik. Für die klassische kommt ein einziges Axiomenschema hinzu, das tertium non datur $\varphi \vee \neg\varphi$ (Definition 2.28); Gentzen nennt seine Instanzen *Grundformeln*: Sie dürfen oben im Baum stehen, ohne als Annahme zu zählen.

$$\frac{\begin{array}{c}[\varphi]^n \\ \vdots \\ \psi\end{array}}{\varphi \to \psi}\ (\to\mathrm{I}) \qquad \frac{\varphi \to \psi \qquad \varphi}{\psi}\ (\to\mathrm{E})$$

$$\frac{\varphi \qquad \psi}{\varphi \wedge \psi}\ (\wedge\mathrm{I}) \qquad \frac{\varphi \wedge \psi}{\varphi}\ (\wedge\mathrm{E}) \qquad \frac{\varphi}{\varphi \vee \psi}\ (\vee\mathrm{I}) \qquad \frac{\bot}{\varphi}\ (\bot\mathrm{E})$$

$$\frac{\varphi \vee \psi \qquad \begin{array}{c}[\varphi]^n \\ \vdots \\ \chi\end{array} \qquad \begin{array}{c}[\psi]^n \\ \vdots \\ \chi\end{array}}{\chi}\ (\vee\mathrm{E}) \qquad \frac{\varphi(x)}{\forall x\,\varphi(x)}\ (\forall\mathrm{I}) \qquad \frac{\forall x\,\varphi(x)}{\varphi(t)}\ (\forall\mathrm{E})$$

Definition 2.13 (Ableitung im natürlichen Schließen)**.** Eine *Ableitung* ist ein endlicher Baum, dessen Knoten mit Formeln beschriftet sind:

(a) Jedes Blatt trägt eine Formel; sie heißt *Annahme* und kann mit einer Nummer markiert sein.

(b) Jeder innere Knoten entsteht aus den Formeln seiner Kinder durch eine der Regeln.

(c) Wendet man eine Regel mit *Entlassung* an ($\to$I, $\vee$E, $\exists$E), so darf man Blätter oberhalb dieser Stelle mit der zugehörigen Nummer markieren; sie heißen von da an *geschlossen*.

Die Wurzel heißt *Konklusion*, die nicht geschlossenen Blätter die *offenen Annahmen*. Man schreibt $\Sigma \vdash \varphi$, wenn es eine Ableitung mit Konklusion φ gibt, deren offene Annahmen sämtlich in Σ liegen.

Wie man einen solchen Beweis liest. Unten steht, was bewiesen wird; nach oben verzweigt der Baum sich zu dem, woraus es folgt. Ganz oben stehen die Annahmen.

Eine Annahme darf man jederzeit hinschreiben – sie kostet nichts, solange man sie am Ende wieder loswird. Genau das leistet →I: Hat man aus der Annahme φ irgendwann ψ erreicht, so darf man $\varphi \to \psi$ notieren und die Annahme *entlassen*. Sie ist dann verbraucht und taucht in der fertigen Aussage nicht mehr auf. In der Schreibweise heißt das:

- $[\varphi]^n$ – eine Annahme, die an der Stelle n entlassen wurde;
- die Ziffer an der Regel sagt, *welche* Annahme dort verschwindet.

Bei ∀I kommt eine Nebenbedingung hinzu: x darf in keiner noch offenen Annahme frei vorkommen – sonst könnte man aus einer Aussage über ein bestimmtes x eine über alle machen.

$\varphi \to \varphi$, Schritt für Schritt. Gesucht ist $\varphi \to \varphi$. Ein Pfeil entsteht nur durch →I, also muss der letzte Schritt diese Regel sein. Sie verlangt: aus der Annahme φ die Formel φ herleiten. Das ist schon erledigt, denn die Annahme *ist* φ. Also:

(1) Nimm φ an und nenne diese Annahme 1.

(2) Damit steht φ da – nichts zu tun.

(3) Wende →I an: notiere $\varphi \to \varphi$ und entlasse Annahme 1.

$$\frac{[\varphi]^1}{\varphi \to \varphi}\ (\to\text{I},\ 1)$$

Am Ende ist keine Annahme mehr offen. Die Formel gilt also ohne Voraussetzung.

Und dasselbe für (K). Interessanter ist das Axiom (K) von eben. Im Hilbert-Kalkül steht es am Anfang, hier muss man es herleiten:

(1) Nimm φ an (Annahme 1).

(2) Nimm ψ an (Annahme 2) – man darf, auch wenn man es gar nicht braucht.

(3) Es steht φ da, aus Schritt (1).

(4) →I entlässt Annahme 2 und liefert $\psi \to \varphi$. Dass ψ nie benutzt wurde, stört nicht; man nennt das eine *leere* Entlassung.

(5) →I entlässt Annahme 1 und liefert $\varphi \to (\psi \to \varphi)$.

$$\dfrac{\dfrac{[\varphi]^1}{\psi \to \varphi}\ (\to\text{I},\ 2)}{\varphi \to (\psi \to \varphi)}\ (\to\text{I},\ 1)$$

Im Baum sieht man Annahme 2 gar nicht, weil sie nie gebraucht wurde. Genau dieser Schritt – „nimm etwas an, benutze es nicht“ – ist im Hilbert-Kalkül das Axiom (K). Dort muss er als Formel dastehen, hier fällt er als Nebenwirkung einer Regel an.

Der Unterschied, in Regelform. Schreibt man auch das natürliche Schließen so auf, springt der Unterschied ins Auge. Dort lauten die beiden Regeln für die Implikation:

$$\frac{\Sigma, \varphi \vdash \psi}{\Sigma \vdash \varphi \to \psi}\ (\to\text{I}) \qquad\qquad \frac{\Sigma \vdash \varphi \to \psi \qquad \Sigma \vdash \varphi}{\Sigma \vdash \psi}\ (\to\text{E})$$

→E und (MP) sind dieselbe Regel. Der Unterschied steckt allein in →I: Diese Regel greift in die *Annahmenmenge* ein. Oben steht Σ, φ, unten nur noch Σ – eine Annahme ist verschwunden.

Im Hilbert-Kalkül gibt es keine solche Regel. Σ wird dort nirgends kleiner; die einzige Regel, die Σ überhaupt anfasst, ist (Hyp), und die liest nur daraus ab. Deshalb ist das Deduktionstheorem dort kein Handgriff, sondern ein Metatheorem, das man beweisen muss.

	Hilbert	**natürliches Schließen**
Annahme benutzen	(Hyp)	(Hyp)
Pfeil beseitigen	(MP)	→E
Pfeil erzeugen	Axiome (K), (S)	→I
Σ verkleinern	–	→I

Die beiden Kalküle verhalten sich also spiegelbildlich: Was im einen Grundannahme ist, ist im anderen ein Satz. (K) ist bei Hilbert Axiom und bei Gentzen herleitbar; die Einführungsregel →I ist bei Gentzen Regel und bei Hilbert das Deduktionstheorem.

Bemerkung 2.14 (Einführung und Beseitigung passen zusammen)*.* Die Regelpaare sind nicht beliebig gewählt. Die Beseitigungsregel nimmt genau das zurück, was die Einführungsregel hineingesteckt hat: Wer $\varphi \to \psi$ durch →I gewonnen hat, hatte einen Weg von φ nach ψ – und →E gibt ihn wieder heraus.

Man nennt das *Harmonie*. Sie erlaubt eine Umdeutung, die philosophisch einiges ausmacht: Die Bedeutung eines Junktors steckt nicht in einer Wahrheitstafel, sondern in seinen beiden Regeln. „Und“ heißt, was ∧I und ∧E mit ihm tun dürfen.

Gentzens Hauptsatz. Aus der Harmonie folgt der wichtigste Satz über das natürliche Schließen. Ein *Umweg* ist eine Stelle, an der ein Junktor eingeführt und sofort wieder beseitigt wird. Solche Umwege lassen sich herauskürzen, und eine Herleitung ohne Umwege heißt *normal*.

Satz 2.15 (Normalisierung; Prawitz 1965, im Sequenzenkalkül Gentzen 1935)**.** *Jede Herleitung lässt sich in eine normale überführen.*

Zur Geschichte: Gentzen bewies 1935 die entsprechende Aussage für seinen Sequenzenkalkül, die *Schnittelimination* – das ist sein „Hauptsatz“. Den ersten veröffentlichten Beweis für das natürliche Schließen selbst gab Dag Prawitz 1965. Dass Gentzen auch dafür einen Beweis hatte, für die intuitionistische Fassung, zeigte erst 2008 eine handschriftliche Fassung seiner Dissertation, die Jan von Plato fand.

Das hat eine Folge, die man *Teilformeleigenschaft* nennt: In einer normalen Herleitung kommen nur Teilformeln der Annahmen und der Behauptung vor. Nichts Fremdes muss erfunden werden.

Daraus folgt die Widerspruchsfreiheit fast von selbst. Wäre $\bot$ ohne Annahmen herleitbar, so gäbe es dafür eine normale Herleitung – und in der dürften nur Teilformeln von $\bot$ vorkommen, also gar nichts. Eine solche Herleitung gibt es nicht.

Bemerkung 2.16. Für die Arithmetik reicht dieses Argument nicht aus; dort brauchte Gentzen 1936 zusätzlich transfinite Induktion bis ε_0. Das steht nicht im Widerspruch zu Gödels Unvollständigkeitssatz (Satz 2.51 in Abschnitt 2.6), nach dem eine widerspruchsfreie Arithmetik ihre eigene Widerspruchsfreiheit nicht beweist, sondern bestätigt ihn: Das Beweismittel liegt außerhalb dessen, was die Arithmetik selbst hergibt.

Wo die Arbeit hinwandert. Der entscheidende Fall ist das Annehmen. In der Mathematik sagt man ständig „Sei φ angenommen … also gilt $\varphi \to \psi$“. Im natürlichen Schließen ist das eine *Regel* ($\to$I), man darf es einfach tun.

Im Hilbert-Kalkül gibt es diesen Schritt nicht. Dort ist er das *Deduktionstheorem* – ein Metatheorem, also ein Satz *über* den Kalkül, den man erst beweisen muss und der eine Herleitung in eine andere umrechnet. Was in dem einen System ein Handgriff ist, ist im anderen ein Theorem.

Die Arbeit fällt also so oder so an – entweder in den einzelnen Beweisen oder in der Theorie über den Kalkül.

Daraus ergibt sich, wofür man was nimmt.

Wenige Regeln sind günstig, wenn man über den Kalkül reden will. Jeder Beweis *über* ein formales System läuft per Induktion über den Aufbau von Herleitungen, und die hat so viele Fälle, wie es Regeln gibt. Bei einer einzigen Regel ist das eine Zeile. Gödel führte seine Sätze deshalb über einem Hilbert-System.

Viele Regeln sind günstig, wenn man in dem Kalkül rechnen will. Jede Regel entspricht einem Denkschritt, den man ohnehin macht. Beweise werden kurz, lesbar und lassen sich maschinell suchen – Letzteres vor allem im Sequenzenkalkül, wo Gentzens Schnittelimination garantiert, dass man bei der Suche nichts Fremdes erfinden muss.

Bemerkung 2.17. Die Wahl des Kalküls ist kein Streit über Wahrheit. Beide Bauarten gibt es klassisch und intuitionistisch: Gentzens Regeln mit $\bot$E ergeben die *intuitionistische* Logik, mit dem tertium non datur $\varphi \vee \neg\varphi$ als Axiomenschema die klassische (Abschnitt 2.5); Hilberts Kalkül wird intuitionistisch, wenn man (N) durch schwächere Schemata ersetzt. Bei gleicher Logik leiten beide Kalküle dieselben Formeln her, und man kann sie ineinander übersetzen. Die Wahl zwischen Hilbert und Gentzen ist also allein Arbeitsteilung. Die Regeln in Abschnitt 3.5 sind von der zweiten Art – Einführung und Beseitigung, mit Annahmen im Kontext.

Ein formales System ohne Logik: der λ-Kalkül

Beide bisherigen Kalküle leiten *Aussagen* her. Ein formales System muss das aber gar nicht tun. Der λ-Kalkül, 1936 von Church angegeben, hat keine Axiome, keine Aussagen und keinen Wahrheitsbegriff. Er hat nur Terme und eine Regel, wie man sie umformt.

Definition 2.18 (λ-Terme)**.** Die *Terme* sind durch drei Regeln gegeben:

(a) jede Variable x ist ein Term;

(b) ist M ein Term, so auch $\lambda x.\, M$ (*Abstraktion*);

(c) sind M, N Terme, so auch $M\,N$ (*Anwendung*).

Mehr gibt es nicht. Keine Zahlen, keine Wahrheitswerte, keine Operatoren – nur Variablen, Abstraktion und Anwendung. Dazu drei Umformungsregeln:

	Regel	**was sie tut**
α	$\lambda x.\, M \;\equiv\; \lambda y.\, M[y/x]$	gebundene Variablen umbenennen
β	$(\lambda x.\, M)\, N \;\rightsquigarrow\; M[N/x]$	anwenden: einsetzen
η	$\lambda x.\, (M\, x) \;\rightsquigarrow\; M$	überflüssige Abstraktion weglassen

Bei α und η gilt jeweils die Bedingung, dass y beziehungsweise x nicht frei vorkommt – sonst würde man Variablen einfangen, die nicht zusammengehören.

Die β-Regel ist die einzige, die wirklich rechnet. Und sie genügt: Mit ihr lässt sich jede berechenbare Funktion ausdrücken. Zahlen, Wahrheitswerte und Rekursion sind nichts als geschickt gebaute Terme. Church und Turing zeigten 1936/37, dass der λ-Kalkül und die Turingmaschine genau dieselben Funktionen berechnen.

Bemerkung 2.19 (Church–Rosser). Auch hier gilt Konfluenz: Führt man die β-Regel an verschiedenen Stellen aus, so lassen sich die Ergebnisse wieder zusammenführen. Ein Term hat damit höchstens eine Normalform. Anders als beim natürlichen Schließen muss die Reduktion aber *nicht* abbrechen – $(\lambda x.\, x\, x)\,(\lambda x.\, x\, x)$ reduziert zu sich selbst, für immer. Erst die Typentheorie in Kapitel 3 erzwingt das Abbrechen.

Logik und Zahlen als Terme

Der λ-Kalkül kennt weder Wahrheitswerte noch Zahlen. Trotzdem kann man beides in ihm ausdrücken – und zwar *ohne ein einziges neues Axiom*. Man schreibt einfach Terme hin, die sich so verhalten, wie sich Wahrheitswerte und Zahlen verhalten sollen.

Wahrheitswerte. Wozu braucht man sie? Um zwischen zwei Dingen zu wählen. Also definiert man sie als genau das:

$$\text{wahr} := \lambda x.\, \lambda y.\, x, \qquad \text{falsch} := \lambda x.\, \lambda y.\, y.$$

wahr nimmt das erste Argument, falsch das zweite. Die Fallunterscheidung ist damit keine eigene Konstruktion mehr, sondern bloße Anwendung: $b\, M\, N$ liefert M, wenn b wahr ist, und sonst N. Und die Junktoren fallen ab:

$$\text{nicht} := \lambda p.\, p\ \text{falsch wahr}, \qquad \text{und} := \lambda p.\, \lambda q.\, p\ q\ \text{falsch}, \qquad \text{oder} := \lambda p.\, \lambda q.\, p\ \text{wahr}\ q.$$

Bemerkung 2.20. wahr ist derselbe Term wie das Axiom (K) von Seite 16: $\alpha \to (\beta \to \alpha)$ als Funktion gelesen ist $\lambda x.\, \lambda y.\, x$. Was dort Grundannahme war, ist hier Wahrheitswert. Das ist kein Zufall, sondern ein erster Hinweis auf Kapitel 3.

Haskell code/Lambda.hs

```haskell
-- Ein Wahrheitswert ist eine Funktion, die zwischen zwei Dingen waehlt.
type Wahrheitswert = forall a. a -> a -> a

wahr, falsch :: Wahrheitswert
wahr   = \x _ -> x      -- nimm das erste
falsch = \_ y -> y      -- nimm das zweite

-- Die Fallunterscheidung ist dann gar keine eigene Konstruktion mehr:
wenn :: Wahrheitswert -> a -> a -> a
wenn b dann sonst = b dann sonst

nicht :: Wahrheitswert -> Wahrheitswert
nicht p = p falsch wahr

undD :: Wahrheitswert -> Wahrheitswert -> Wahrheitswert
undD p q = p q falsch

oderD :: Wahrheitswert -> Wahrheitswert -> Wahrheitswert
oderD p q = p wahr q
```

Natürliche Zahlen. Derselbe Trick. Wozu braucht man die Zahl n? Um etwas n-mal zu tun. Also *ist* n das n-malige Anwenden:

$$\bar{n} := \lambda f.\, \lambda x.\, \underbrace{f\,(f\,(\cdots(f}_{n\text{-mal}}\ x)\cdots))$$

also $\bar{0} = \lambda f.\lambda x.\, x$ (gar nicht anwenden), $\bar{1} = \lambda f.\lambda x.\, f\, x$, $\bar{2} = \lambda f.\lambda x.\, f\,(f\, x)$ und so fort. Diese Terme heißen nach Church *Church-Zahlen*.

Das Rechnen ergibt sich von selbst:

$$\begin{aligned} \text{nachfolger} &:= \lambda n.\, \lambda f.\, \lambda x.\, f\,(n\, f\, x), \\ \text{plus} &:= \lambda m.\, \lambda n.\, \lambda f.\, \lambda x.\, m\, f\,(n\, f\, x), \\ \text{mal} &:= \lambda m.\, \lambda n.\, \lambda f.\, m\,(n\, f). \end{aligned}$$

Warum das stimmt. plus liest sich so: Wende f erst n-mal auf x an, dann noch m-mal auf das Ergebnis. Zusammen $m + n$ mal.

mal ist noch knapper: $n\, f$ ist „f, aber n-mal“. Wendet man das seinerseits m-mal an, hat man f insgesamt $m \cdot n$ mal angewandt. Multiplikation ist hier Hintereinanderausführung – und deshalb kommt sie ohne jede Fallunterscheidung aus.

Auch die Abfrage auf Null braucht nichts Neues: $\text{istNull} := \lambda n.\, n\,(\lambda x.\,\text{falsch})$ wahr. Ist n gleich $\bar{0}$, so wird gar nicht angewandt und wahr bleibt stehen; sonst schlägt $\lambda x.\,\text{falsch}$ mindestens einmal zu.

Haskell code/Lambda.hs

```haskell
-- Eine Zahl ist das, was "n-mal anwenden" bedeutet.
type Zahl = forall a. (a -> a) -> a -> a

null', eins, zwei :: Zahl
null' = \_ x -> x            -- gar nicht anwenden
eins  = \f x -> f x          -- einmal
zwei  = \f x -> f (f x)      -- zweimal

nachfolger :: Zahl -> Zahl
nachfolger n = \f x -> f (n f x)

plus :: Zahl -> Zahl -> Zahl
plus m n = \f x -> m f (n f x)

mal :: Zahl -> Zahl -> Zahl
mal m n = \f -> m (n f)

istNull :: Zahl -> Wahrheitswert
istNull n = n (\_ -> falsch) wahr

-- Zum Nachsehen: eine Church-Zahl auf eine gewoehnliche abbilden.
zuInt :: Zahl -> Int
zuInt n = n (+ 1) 0

beispiel :: Int
beispiel = zuInt (mal zwei (nachfolger zwei))   -- 2 * 3 = 6
```

Bemerkung 2.21 (Kodierung, nicht Axiomatisierung). Der Wortgebrauch lohnt eine Präzisierung. Peano *axiomatisiert* die natürlichen Zahlen: Er setzt sie voraus und schreibt auf, welche Gesetze gelten sollen. Church *kodiert* sie: Er setzt nichts voraus, sondern baut Terme, die sich so verhalten.

Der Unterschied ist folgenreich. Die Kodierung liefert das *Rechnen* – mal $\bar{2}$ $\bar{3}$ reduziert wirklich zu $\bar{6}$. Sie liefert aber keinen *Beweis*, etwa dass $m + n = n + m$ gilt. Dazu müsste man über alle Church-Zahlen reden können, und dafür fehlt dem λ-Kalkül jedes Mittel: Er hat keine Aussagen. Erst wenn Typen dazukommen, wird daraus ein System, in dem man auch beweisen kann – Kapitel 3.

Alligator Eggs

Die β-Regel liest sich abstrakt, lässt sich aber spielen. Bret Victor hat den λ-Kalkül 2007 als Brettspiel aufgeschrieben, unter dem Namen *Alligator Eggs*.[1] Die Übersetzung ist eins zu eins:

λ-Kalkül	**Alligator Eggs**
Variable x	ein *Ei* in der Farbe x
Abstraktion $\lambda x.\, M$	ein *hungriger Alligator* der Farbe x, der M bewacht
Anwendung $M\, N$	M und N nebeneinander
Klammerung	ein *alter Alligator*, der nur bewacht
β-Reduktion	der Alligator *frisst* die Familie rechts und stirbt dabei
α-Umbenennung	die *Farbregel*: kollidierende Farben werden umgefärbt
Klammern weglassen	die *Altersregel*: ein alter Alligator, der nur *eines* bewacht, stirbt

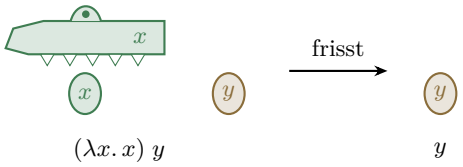
Das ganze Spielmaterial besteht aus drei Dingen:



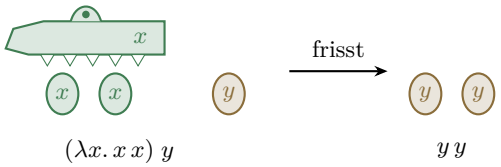
Der alte Alligator ist der unscheinbarste und doch nötig: Er hält zusammen, was zusammengehört. In der Schreibweise sind das die Klammern, die man im Bild sonst nicht unterbringen könnte.

Die Fressregel. Der oberste, linkeste hungrige Alligator frisst die ganze Familie rechts von ihm. Danach stirbt er – und jedes Ei, das er bewacht hat und das seine Farbe trägt, wird durch das Gefressene ersetzt. Eier anderer Farbe bleiben, wie sie sind.

Das ist genau $(\lambda x.\, M)\, N \rightsquigarrow M[N/x]$, nur mit Zähnen. Unten die einfachste Partie: Ein grüner Alligator bewacht ein grünes Ei – das ist die Identität. Er frisst das braune Ei, stirbt, und sein grünes Ei wird durch das braune ersetzt. Übrig bleibt das braune Ei.



Interessanter wird es, wenn der Alligator *mehrere* Eier seiner Farbe bewacht. Dann wird das Gefressene vervielfacht – und genau das macht die Substitution $M[N/x]$ auch:



[1] Bret Victor, *Alligator Eggs!*, 2007. Spielregeln, Spielmaterial und die Originalzeichnungen unter `https://worrydream.com/AlligatorEggs/`. Die Zeichnungen in diesem Abschnitt sind eigene und geben nur die Regeln wieder.

Die Farbregel. Ein Alligator darf nur seine *eigene* Farbe ersetzen. Trotzdem kann es schiefgehen: Frisst ein Alligator etwas, in dem eine Farbe vorkommt, die weiter innen schon vergeben ist, so gerät das Gefressene unter die falsche Bewachung.

Deshalb färbt man vorher um. Ein Alligator darf jederzeit die Farbe wechseln, solange er alle seine Eier mitfärbt und keine Farbe doppelt vorkommt. Das ist die α-Umbenennung – im Spiel sieht man sofort, wozu sie da ist, in der Formelschreibweise übersieht man sie leicht.

Der Witz daran ist nicht die Verpackung, sondern dass nichts weggelassen wurde. Wer die drei Spielregeln beherrscht, beherrscht den λ-Kalkül – und damit den Kern jeder funktionalen Programmiersprache. Die Regeln λ und App in Abschnitt 3.5 sind dieselbe Sache, nur mit Typen versehen.

Damit ist die *Syntax* beisammen: welche Zeichenreihen Formeln sind, welche Schritte erlaubt sind und was $\Sigma \vdash \varphi$ heißt. Nachprüfen lässt sich das alles, ohne zu wissen, was die Zeichen bedeuten. Der nächste Abschnitt schreibt auf diese Weise die Axiome der Mengenlehre auf: ZFC. Offen bleibt dabei, was es heißt, dass eine Formel *gilt*, und ob – und wie – man zeigen kann, dass sich aus den Axiomen kein Widerspruch herleiten lässt. Beides führt auf die Bedeutung der Zeichen, die *Semantik*. Sie folgt, wenn die Axiome dastehen, in Abschnitt 2.6.

2.5 ZFC

Die Sprache der Mengenlehre

ZFC ist ein formales System im Sinne von Definition 2.7, und zwar eines in der Logik *erster Stufe*: Die Quantoren laufen nur über die Objekte der Theorie, hier über Mengen – nicht über Eigenschaften oder Formeln. (Genau gefasst wird das in Definition 2.41.) Bleibt zu sagen, welchen Zeichenvorrat es benutzt – und der ist sehr klein:

- Variablen $x, y, z, \ldots$ – sie bezeichnen Mengen, sonst nichts;
- einem zweistelligen Relationssymbol $\in$;
- dem Gleichheitszeichen $=$;
- den Junktoren $\neg, \wedge, \vee, \to, \leftrightarrow$;
- den Quantoren $\forall$ und $\exists$.

Formeln entstehen daraus nach zwei Regeln. *Atomare* Formeln sind $x \in y$ und $x = y$. Und sind φ, ψ Formeln, so auch $\neg\varphi$, $\varphi \wedge \psi$, $\varphi \vee \psi$, $\varphi \to \psi$, $\varphi \leftrightarrow \psi$ sowie $\forall x\, \varphi$ und $\exists x\, \varphi$.

Mehr gibt es nicht. Zahlen, geordnete Paare, Funktionen, Relationen – nichts davon steht in der Sprache. Alles muss erst als Menge *definiert* werden, und jede solche Definition ist nur eine Abkürzung für eine Formel in $\in$ und $=$.

Dass die Quantoren nur über Mengen laufen dürfen, hat eine Folge. Die Aussonderung soll besagen: „Zu *jeder* Formel φ und jeder Menge x gibt es die Menge $\{y \in x \mid \varphi(y)\}$." Das „zu jeder Formel" lässt sich erststufig nicht hinschreiben. Also formuliert man statt eines Axioms ein *Axiomenschema*: für jede einzelne Formel φ ein eigenes Axiom. Das sind unendlich viele Axiome, aber jedes für sich ist erststufig. Bei der Ersetzung ist es genauso.

Bemerkung 2.22. Dass φ dabei zusätzliche freie Variablen enthalten darf, ist wichtig und wird leicht übersehen. Sie dienen als *Parameter*: In $\{y \in x \mid y \in z\}$ ist z so ein Parameter, und das Schema liefert die Aussonderung für jede Wahl von z.

Die Axiome

Die Zermelo–Fraenkel-Mengenlehre ersetzt Definition 2.3 durch ein schwächeres Schema.

Definition 2.23 (Aussonderung, Separation)**.** Zu jeder Menge x und jeder Formel φ gibt es die Menge $\{\, y \in x \mid \varphi(y) \,\}$.

Der Unterschied ist klein, aber er entscheidet alles: Ausgesondert wird immer aus einem x, das es schon gibt. Russells Konstruktion liefert dann keinen Widerspruch mehr, sondern eine harmlose Teilmenge.

Die übrigen Axiome sagen, welche Mengen es überhaupt gibt und wann zwei gleich sind:

Axiom	**Inhalt**
Extensionalität	gleiche Elemente $\Rightarrow$ gleiche Menge
Leermenge	$\emptyset$ existiert
Paarmenge	$\{a, b\}$ existiert
Vereinigung	$\bigcup x$ existiert
Potenzmenge	$\mathcal{P}(x)$ existiert
Aussonderung	$\{y \in x \mid \varphi(y)\}$ existiert
Ersetzung	Bilder von Mengen unter Funktionen sind Mengen
Unendlichkeit	es gibt eine induktive Menge
Fundierung	keine unendlich absteigenden $\in$-Ketten
Auswahl	jede Familie nichtleerer Mengen hat eine Auswahlfunktion

Aus der Fundierung folgt insbesondere $x \notin x$ für jede Menge x. Damit lässt sich beides nachrechnen: dass es keine Allmenge gibt, und dass die Russellsche Aussonderung harmlos ist.

Satz 2.24. *In ZFC gibt es keine Menge u mit $x \in u$ für alle Mengen x. Außerdem gilt für jede Menge x die Gleichheit $\{\, y \in x \mid y \notin y \,\} = x$.*

Beweis. Gäbe es so eine Allmenge u, dann wäre insbesondere $u \in u$. Das widerspricht der Fundierung. Aus demselben Grund ist die Bedingung $y \notin y$ für jedes y erfüllt; die Aussonderung ändert also nichts. □

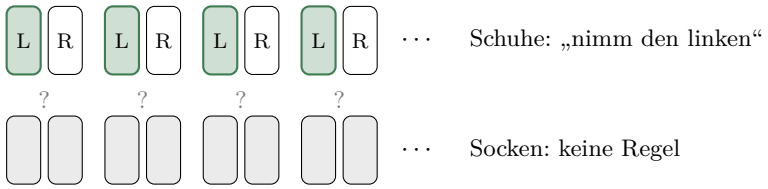
Das Auswahlaxiom

Ein Axiom der Liste fällt aus dem Rahmen. Die Axiome, die Mengen *bilden* – Paarmenge, Vereinigung, Potenzmenge, Aussonderung, Ersetzung –, legen die neue Menge eindeutig fest: Ihre Elemente stehen fest. Das Auswahlaxiom sagt nur, dass *irgendeine* Auswahlfunktion existiert; meist gibt es viele, und keine wird beschrieben.

Definition 2.25 (Auswahlaxiom, AC)**.** Ist $(X_i)_{i \in I}$ eine *Familie* nichtleerer Mengen – zu jedem i aus einer Indexmenge I eine Menge $X_i \neq \emptyset$ –, so gibt es eine Funktion f auf I mit $f(i) \in X_i$ für alle $i \in I$, eine *Auswahlfunktion*.

Für endlich viele Mengen braucht man das Axiom nicht: Man wählt aus X_1 ein Element, dann aus X_2 und so fort, und eine Induktion über die Anzahl macht daraus einen Beweis. Erst wenn unendlich viele Wahlen *auf einmal* getroffen werden sollen, muss ein Axiom die Funktion liefern. ZFC ohne das Auswahlaxiom heißt ZF.

Schuhe und Socken. Russells Bild: Aus unendlich vielen Paaren Schuhe kann man ohne das Auswahlaxiom auswählen – man nimmt jeweils den linken. Das ist eine *Regel*, und die Regel ist die Auswahlfunktion. Bei unendlich vielen Paaren Socken gibt es keine solche Regel; die beiden Socken eines Paares sind nicht zu unterscheiden. Dass es trotzdem eine Auswahl gibt, behauptet das Auswahlaxiom; ZF allein beweist es nicht.



Was das Axiom leistet, sieht man am besten an einer Fassung, die ohne Familien auskommt.

Satz 2.26 (Auswahl und Rechtsinverse)**.** *Über ZF ist das Auswahlaxiom gleichwertig zu: Jede surjektive Abbildung $g\colon A \to B$ hat ein* Rechtsinverses*, also eine Abbildung $s\colon B \to A$ mit $g(s(b)) = b$ für alle $b \in B$.*

Beweis. Aus dem Auswahlaxiom folgt die Aussage: Weil g surjektiv ist, sind die Urbilder $g^{-1}(b)$ für $b \in B$ nichtleer. Eine Auswahlfunktion s für die Familie $(g^{-1}(b))_{b\in B}$ wählt aus jedem Urbild ein Element, also gilt $g(s(b)) = b$.

Umgekehrt sei $(X_i)_{i\in I}$ eine Familie nichtleerer Mengen. Setze $A := \{\,(i,x) \mid i \in I,\ x \in X_i\,\}$ und $g(i,x) := i$. Weil jedes X_i nichtleer ist, ist $g\colon A \to I$ surjektiv. Ein Rechtsinverses s liefert zu jedem i ein Paar $s(i) = (i, x_i)$ mit $x_i \in X_i$, und $f(i) := x_i$ ist eine Auswahlfunktion. □

In dieser Form begegnet einem das Axiom ständig, meist unbemerkt: Wer zu jedem b „ein a mit $g(a) = b$“ wählt und damit weiterrechnet, benutzt es.

Bemerkung 2.27 (Was aus dem Auswahlaxiom folgt)*.* Einige Folgerungen, manche sogar gleichwertig zu ihm:

- *Wohlordnungssatz* (Zermelo 1904): Jede Menge lässt sich so ordnen, dass jede nichtleere Teilmenge ein kleinstes Element hat. Gleichwertig zum Auswahlaxiom.
- *Zornsches Lemma*: Hat in einer halbgeordneten Menge jede Kette – jede total geordnete Teilmenge – eine obere Schranke, so gibt es ein maximales Element. Ebenfalls gleichwertig; in dieser Form benutzt man das Axiom in der Algebra.
- Jeder Vektorraum hat eine Basis, auch $\mathbb{R}$ als Vektorraum über $\mathbb{Q}$. Der Beweis über das Zornsche Lemma nennt aber keine, und ohne Auswahlaxiom muss es keine geben: Die Aussage ist sogar gleichwertig zu ihm (Blass 1984).
- Es gibt Mengen reeller Zahlen, denen sich keine Länge zuordnen lässt (Vitali 1905). Und eine Kugel lässt sich in endlich viele Stücke zerlegen, aus denen man durch Drehen und Verschieben zwei Kugeln zusammensetzt, jede so groß wie die erste (Banach–Tarski 1924).

Aus den übrigen Axiomen folgt das Auswahlaxiom nicht, und widerlegen lässt es sich mit ihnen auch nicht – vorausgesetzt, ZF ist *widerspruchsfrei*, lässt also den Widerspruch $\bot$ nicht herleiten (Gödel 1938, Cohen 1963). Was das in der Sprache der Modelle heißt, steht in Abschnitt 2.6, Bemerkung 2.46. Ob man das Axiom annimmt, ist also eine Entscheidung, keine Entdeckung. Die übliche Mathematik nimmt es an; ZFC ist der Standard.

Der Satz vom ausgeschlossenen Dritten

ZFC ist eine Theorie in der *klassischen* Logik erster Stufe. Zu dieser Logik gehört ein Prinzip, das so selbstverständlich wirkt, dass man es meist nicht bemerkt.

Definition 2.28 (Tertium non datur)**.** Für jede Aussage P gilt $P \vee \neg P$. Als Regel der Logik meint man das Schema $\varphi \vee \neg\varphi$ für jede Formel φ, auch mit freien Variablen.

Es gilt auch für Aussagen, von denen niemand weiß, welche Seite zutrifft – etwa für die Goldbach-Vermutung, nach der jede gerade Zahl größer als 2 die Summe zweier Primzahlen ist. Als Prinzip für alle P gleichwertig dazu ist die *Beseitigung der doppelten Negation* $\neg\neg P \to P$ und damit der *Widerspruchsbeweis*: Um P zu zeigen, genügt es, aus $\neg P$ einen Widerspruch herzuleiten. Den umgekehrten Schluss – führt P zum Widerspruch, so gilt $\neg P$ – braucht man dafür nicht; er ist nur $\to$I und gilt auch intuitionistisch. Für Existenzaussagen heißt das: Führt $\forall x\, \neg\varphi(x)$ zum Widerspruch, so gilt $\exists x\, \varphi(x)$ – auch wenn man kein einziges solches x kennt.

Intuitionistische Logik. Brouwer bestritt 1908, dass das tertium non datur ein logisches Gesetz ist. In der *intuitionistischen* Logik, die daraus hervorging, heißt eine Aussage beweisen, eine *Konstruktion* angeben (Deutung nach Brouwer, Heyting und Kolmogorow):

- Ein Beweis von $A \wedge B$ ist ein Beweis von A zusammen mit einem Beweis von B.
- Ein Beweis von $A \vee B$ ist ein Beweis von A oder ein Beweis von B – und man weiß, welcher.
- Ein Beweis von $A \to B$ ist ein Verfahren, das jeden Beweis von A in einen Beweis von B verwandelt. $\bot$ hat keinen Beweis; $\neg A$ steht für $A \to \bot$.
- Ein Beweis von $\forall x\, \varphi(x)$ ist ein Verfahren, das zu jedem Objekt a einen Beweis von $\varphi(a)$ liefert.
- Ein Beweis von $\exists x\, \varphi(x)$ ist ein Objekt a zusammen mit einem Beweis von $\varphi(a)$.

Danach ist $P \vee \neg P$ kein Gesetz: Ein Beweis müsste für jedes P entscheiden, welche Seite gilt. Die Liste kehrt in Abschnitt 3.3 wieder, als Curry–Howard-Korrespondenz: Beweise sind dort Programme.

Satz 2.29 (Das tertium non datur ist nicht widerlegbar)**.** *Auch intuitionistisch gilt* $\neg\neg(P \vee \neg P)$.

Beweis. Angenommen $\neg(P \vee \neg P)$. Aus P folgte $P \vee \neg P$, also ein Widerspruch; das zeigt $\neg P$. Aus $\neg P$ folgt aber wieder $P \vee \neg P$ – ein Widerspruch. Also gilt $\neg\neg(P \vee \neg P)$. Der Beweis benutzt nur Einführung und Beseitigung von $\to$ und $\vee$. □

Intuitionistisch kann man das tertium non datur also nicht widerlegen. Man kann es nur nicht allgemein beweisen.

Bemerkung 2.30 (An den Kalkülen abgelesen). Gentzens Regeln aus Abschnitt 2.4 – Einführung und Beseitigung, mit $\bot$E als Beseitigung des Falsums – ergeben genau die intuitionistische Logik. Klassisch wird das natürliche Schließen erst, wenn man $\varphi \vee \neg\varphi$ als Axiomenschema zulässt. Im Hilbert-Kalkül steckt das Klassische im Schema (N): Mit $\alpha := \neg\neg\beta$ wird es zu $(\neg\beta \to \neg\neg\neg\beta) \to (\neg\neg\beta \to \beta)$. Die Prämisse ist schon intuitionistisch herleitbar (ein Fall von $\gamma \to \neg\neg\gamma$); (N) liefert also $\neg\neg\beta \to \beta$ und ist intuitionistisch nicht herleitbar – anders als (K) und (S).

Konstruktive und nicht konstruktive Beweise

Der Unterschied wird greifbar, sobald etwas *existieren* soll. Ein *konstruktiver* Beweis einer Existenzaussage liefert das Objekt mit – oder ein Verfahren, es zu berechnen. Ein *nicht konstruktiver* zeigt nur, dass es existieren muss. Ein bekanntes Beispiel lässt sich auf beide Arten beweisen.

Satz 2.31. *Es gibt irrationale Zahlen a und b, für die a^b rational ist.*

Erster Beweis, nicht konstruktiv. $\sqrt{2}$ ist irrational. Ist $\sqrt{2}^{\sqrt{2}}$ rational, so leisten $a = b = \sqrt{2}$ das Verlangte. Andernfalls ist $a := \sqrt{2}^{\sqrt{2}}$ irrational, und mit $b := \sqrt{2}$ wird

$$a^b = \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^{\,2} = 2. \qquad \square$$

Der Beweis ist vollständig – und verrät nicht, welches Paar es ist. Er benutzt das tertium non datur für die Aussage „$\sqrt{2}^{\sqrt{2}}$ ist rational“, von der er nicht weiß, welche Seite gilt.

Zweiter Beweis, konstruktiv. Setze $a := \sqrt{2}$ und $b := \log_2 9$. Dann ist

$$a^b = 2^{\frac{1}{2}\log_2 9} = 2^{\log_2 3} = 3.$$

Und b ist irrational: Wäre $\log_2 9 = p/q$ mit natürlichen Zahlen $p, q \geq 1$, so wäre $2^p = 9^q$ – links eine gerade, rechts eine ungerade Zahl. $\square$

Hier steht das Paar da. Welcher Fall im ersten Beweis vorliegt, weiß man übrigens: $\sqrt{2}^{\sqrt{2}}$ ist sogar transzendent, nach dem Satz von Gelfond und Schneider (1934). Das ist aber ein tiefer Satz, kein Nachrechnen.

Zwei Kästen. Ein nicht konstruktiver Beweis gleicht dem Nachweis, dass in einem von zwei Kästen ein Schatz liegt – ohne Auskunft, in welchem. Ein konstruktiver Beweis zeigt auf den Kasten. Für viele Zwecke reicht das Erste; wer den Schatz braucht, braucht das Zweite. Das Auswahlaxiom ist das bekannteste nicht konstruktive Prinzip: Liegt in *jedem* von unendlich vielen Kästen etwas, so versichert es, dass man aus allen zugleich etwas herausnehmen kann – ohne zu sagen, was.



nicht konstruktiv: in einem von beiden — konstruktiv: in diesem

Aus der Auswahl folgt das tertium non datur

Auswahl und tertium non datur hängen zusammen. In ZFC fällt das nicht auf, weil die Logik ohnehin klassisch ist. In einer *konstruktiven* Mengenlehre, die das tertium non datur nicht annimmt, zeigt es sich: Nimmt man das Auswahlaxiom hinzu, bekommt man das tertium non datur zurück.

Satz 2.32 (Diaconescu, 1975)**.** *Gelten Extensionalität, Leermenge, Paarmenge, Aussonderung und das Auswahlaxiom, so gilt für jede Aussage P das tertium non datur $P \vee \neg P$ – auch dann, wenn die Logik selbst nur intuitionistisch ist. „Nichtleer“ heißt dabei: Es gibt ein Element.*[2]

[2] Diaconescu bewies die Aussage für Topoi; die mengentheoretische Fassung mit diesem Beweis stammt von Goodman und Myhill (1978).

Beweis. Sei P eine Aussage. Bei jedem Schritt steht dabei, welches Axiom er benutzt.

(1) *Leermenge und Paarmenge* liefern $0 := \emptyset$, dann $1 := \{0\}$ – die Paarmenge von 0 mit sich selbst – und $\{0, 1\}$. Es ist $0 \neq 1$, denn $0 \in 1$, aber $0 \notin 0$.

(2) *Aussonderung* aus $\{0, 1\}$ mit den Formeln $x = 0 \vee P$ und $x = 1 \vee P$ liefert

$$U := \{\, x \in \{0, 1\} \mid x = 0 \vee P \,\}, \qquad V := \{\, x \in \{0, 1\} \mid x = 1 \vee P \,\}.$$

Beide sind nichtleer, denn $0 \in U$ und $1 \in V$.

(3) *Paarmenge* liefert die Menge $\{U, V\}$.

(4) Das *Auswahlaxiom*, angewandt auf die Familie mit Indexmenge $\{U, V\}$, die jedem Index sich selbst zuordnet, liefert eine Auswahlfunktion f auf der Menge $\{U, V\}$ mit $f(U) \in U$ und $f(V) \in V$. Nach (2) heißt das: $f(U) = 0 \vee P$ und $f(V) = 1 \vee P$. Für zwei Mengen bräuchte man kein Axiom – aber man weiß nicht, ob es zwei sind: Gilt P, so ist $U = V$, und $\{U, V\}$ hat nur ein Element. Dann muss f für U und V dasselbe wählen – darauf beruhen die Schritte (5) und (6).

(5) Die beiden Disjunktionen aus (4) sind bewiesen; man darf sie auch intuitionistisch nach Fällen zerlegen ($\vee$E). Steht in einer von ihnen rechts P, so gilt P. Bleibt der Fall $f(U) = 0$ und $f(V) = 1$. Wegen $0 \neq 1$ ist dann $f(U) \neq f(V)$, also $U \neq V$.

(6) In diesem Fall gilt $\neg P$, mit der *Extensionalität*: Gälte P, so hätten U und V dieselben Elemente, nämlich 0 und 1; also wäre $U = V$ – ein Widerspruch. Das ist $\to$I, kein Widerspruchsbeweis.

In jedem Fall gilt $P \vee \neg P$. □

Bemerkung 2.33 (Nicht nur die Auswahl). Auch die Fundierung in der Form „jede nichtleere Menge hat ein $\in$-minimales Element“ liefert intuitionistisch das tertium non datur: Für $X := \{\, x \in \{0, 1\} \mid x = 1 \vee P \,\}$ ist ein $\in$-minimales Element entweder 0 – dann gilt P – oder 1; dann liegt $0 \in 1$ nicht in X, also gilt $\neg P$. Konstruktive Mengenlehren nehmen deshalb statt der Fundierung die $\in$-Induktion.

Bemerkung 2.34 (Was das für Lean heißt). Leans Typentheorie ist ohne Zusatzaxiome konstruktiv; das tertium non datur ist dort nicht beweisbar. Leans Standardbibliothek stellt aber ein Auswahlaxiom bereit (`Classical.choice`) – sogar ein globales, das aus jedem nichtleeren Typ ein Element wählt –, und Mathlib benutzt es durchgehend. Daneben beruht Mathlib auf genau zwei weiteren Axiomen:

- *Aussagenextensionalität* (`propext`): Sind zwei Aussagen P und Q äquivalent, so sind sie gleich, $P = Q$.
- das *Quotientenaxiom* (`Quot.sound`): Ist $a \sim b$, so haben a und b im Quotienten nach $\sim$ dieselbe Klasse. Aus ihm beweist Lean die *Funktionsextensionalität* (`funext`): Stimmen zwei Funktionen f und g an jeder Stelle überein, so ist $f = g$.

Aussagen- und Funktionsextensionalität ersetzen zusammen genau Schritt (6), für den die Mengenlehre die Extensionalität braucht. In Lean sind U und V Prädikate, also Funktionen, die jedem x eine Aussage zuordnen. Gilt P, so sind $U(x)$ und $V(x)$ für jedes x beide wahr, also äquivalent; `propext` macht daraus $U(x) = V(x)$, und `funext` macht daraus $U = V$. Die Auswahl in Schritt (4) liefert `Classical.choice`. Die übrigen Schritte kosten in der Typentheorie nichts: Die Aussonderung ist umsonst, denn ein Prädikat ist einfach eine Funktion in die Aussagen, und statt der Menge $\{0, 1\}$ aus Leermenge und Paarmenge nimmt Lean den Typ aller Aussagen;

statt $0 \neq 1$ braucht der Beweis dort nur, dass `True` und `False` verschieden sind. So beweist Lean das tertium non datur, mit genau Diaconescus Argument (Abschnitt 3.4). Für jeden Satz lässt sich in Lean anzeigen, welche dieser Axiome sein Beweis benutzt. Definitionen, deren *Wert* vom Auswahlaxiom abhängt, kann Lean nicht ausrechnen.

Klassisch oder konstruktiv: ZFC und Lean

Der Vergleich in Kurzform:

ZFC	**Lean**
Klassische Logik ist eingebaut: $P \vee \neg P$ gilt immer.	Die Typentheorie ohne Zusatzaxiome ist konstruktiv: $P \vee \neg P$ ist dort nicht beweisbar.
Das Auswahlaxiom ist eines der Axiome – das „C“ in ZFC.	Das Auswahlaxiom ist ein eigenes Axiom der Standardbibliothek (`Classical.choice`). Zusammen mit Aussagen- und Funktionsextensionalität (`propext`; `funext`, bewiesen aus `Quot.sound`) folgt aus ihm das tertium non datur (Satz 2.32).
Ob ein Beweis Auswahl benutzt, sieht man im Text, wenn der Autor es sagt.	Welche Axiome ein Beweis benutzt, zeigt Lean auf Anfrage für jeden Satz.

Für maschinell gefundene Beweise heißt das: Leans Prüfprogramm, der Kern, akzeptiert klassische wie konstruktive Beweise, und man kann bei jedem nachfragen, worauf er beruht.

2.6 Semantik und Modelle

Abschnitt 2.4 hat geklärt, was $\Sigma \vdash \varphi$ heißt: Es gibt eine Herleitung, die nur erlaubte Schritte benutzt. Abschnitt 2.5 hat auf diese Weise die Axiome von ZFC aufgeschrieben. Zwei Fragen sind dabei offen geblieben. Was heißt es, dass eine Formel *gilt* – dass etwa die Axiome von ZFC wahr sind, und worin? Und ist ZFC widerspruchsfrei? Die erste Frage beantwortet die Semantik. Mit ihr lässt sich die zweite umformulieren: Eine Theorie ist genau dann widerspruchsfrei, wenn sie ein Modell hat (Satz 2.43). Für ZFC selbst gibt es darauf allerdings nur eine relative Antwort (Satz 2.51 am Ende des Abschnitts).

Syntax und Semantik

Es gibt also zwei ganz verschiedene Weisen, wie eine Formel „gelten“ kann.

$\Sigma \vdash \varphi$ – **ableitbar, *syntaktisch***
Es gibt eine Herleitung von φ aus den Annahmen Σ. Das ist eine Aussage über Zeichenreihen.

$\Sigma \models \varphi$ – **folgt, *semantisch***
In jeder Struktur, in der alle Aussagen aus Σ wahr sind, ist auch φ wahr. Das ist eine Aussage über Bedeutungen.

Die erste Zeile ist durch die Kalküle aus Abschnitt 2.4 genau erklärt. In der zweiten fehlt noch, was eine Struktur ist und was „wahr“ heißt. Die Antwort steckt in einem Satz:

Semantik ist eine Abbildung $[\![\cdot]\!]$ von der Syntax in die Mengen.

Eine Struktur legt diese Abbildung auf den Zeichen fest. Tarskis Rekursion über den Aufbau setzt sie dann eindeutig auf alle Terme und Formeln fort. Die Tabelle zeigt das Ergebnis vorab, für eine Struktur $\mathfrak{M}$ mit Träger M; die Definitionen danach füllen sie aus.

Syntax	**Bedeutung in $\mathfrak{M}$**
k-stelliges Relationszeichen R	Teilmenge $[\![R]\!] \subseteq M^k$
k-stelliges Funktionszeichen f	Abbildung $[\![f]\!] : M^k \to M$
Konstante c	Element $[\![c]\!] \in M$
Term t, Variablen aus $\vec{x} = (x_1, \ldots, x_n)$	Abbildung $[\![t]\!]_{\vec{x}} : M^n \to M$
Formel φ, freie Variablen aus $\vec{x}$	Teilmenge $[\![\varphi]\!]_{\vec{x}} \subseteq M^n$, ihre „Lösungsmenge“
Aussage φ, also $n = 0$	$[\![\varphi]\!] \subseteq M^0 = \{()\}$: $\emptyset$ (falsch) oder $\{()\}$ (wahr)

Über dem Strich steht, was die Struktur festlegt; darunter, was daraus folgt. Ein Beispiel vorab: In $\mathbb{R}$ bedeutet die Formel $x \cdot x = 1 + 1$ ihre Lösungsmenge $\{-\sqrt{2}, \sqrt{2}\}$, in $\mathbb{Q}$ die leere Menge – dieselbe Zeichenreihe, zwei Mengen.

Signatur und Struktur. Welche Zeichen außer den logischen vorkommen, hängt vom Gegenstand ab.

Definition 2.35 (Signatur)**.** Eine *Signatur* L ist eine Menge von Relationszeichen, Funktionszeichen und Konstanten; jedes Relations- und Funktionszeichen hat eine *Stelligkeit* $k \geq 1$. Variablen, Junktoren, Quantoren und das Gleichheitszeichen gehören nicht zur Signatur; sie stehen in jeder Sprache zur Verfügung. Das Gleichheitszeichen ist ein logisches Zeichen: Es bedeutet immer die Gleichheit.

Die Sprache der Mengenlehre aus Abschnitt 2.5 hat die Signatur $\{\in\}$: ein einziges zweistelliges Relationszeichen, denn $=$ gehört zur Logik. Die Gruppentheorie hat $\{\cdot, e, {}^{-1}\}$, ein zweistelliges und ein einstelliges Funktionszeichen und eine Konstante. Die Körper haben $\{0, 1, +, \cdot\}$, die Ordnungen $\{<\}$.

Aus den Zeichen entstehen *Terme*: die Variablen, die Konstanten und alles, was man durch Einsetzen in Funktionszeichen erhält, etwa $x \cdot x$ oder $1 + 1$. *Atomare Formeln* sind $t_1 = t_2$ und $R(t_1, \ldots, t_k)$ für Terme t_i. Aus ihnen bauen Junktoren und Quantoren alle weiteren Formeln.

Definition 2.36 (Struktur)**.** Eine *Struktur* $\mathfrak{M}$ für eine Signatur L besteht aus einer nichtleeren Menge M, dem *Träger*, und einer Zuordnung $[\![\cdot]\!]$ auf den Zeichen von L:

- jedem k-stelligen Relationszeichen R eine Teilmenge $[\![R]\!] \subseteq M^k$,
- jedem k-stelligen Funktionszeichen f eine Abbildung $[\![f]\!] : M^k \to M$,
- jeder Konstanten c ein Element $[\![c]\!] \in M$.

Sind mehrere Strukturen im Spiel, schreibt man $[\![\cdot]\!]^{\mathfrak{M}}$. Für die Sprache der Mengenlehre ist eine Struktur also ein Paar (M, E) mit $E = [\![\in]\!] \subseteq M \times M$.

Mehr ist es nicht – und das ist der erste überraschende Punkt. Die Relation E muss nicht das „wirkliche“ Enthaltensein sein. Jede Relation $E \subseteq M \times M$ ergibt eine Struktur; ein *Modell* der ZFC-Axiome ist (M, E), wenn sie darin gelten (Definition 2.38). Ob sie es tun, ist eine Frage an M und E, nicht an unsere Vorstellung von Mengen.

Wann eine Formel wahr ist. Tarski hat 1935 gezeigt, wie man Wahrheit sauber definiert, ohne in einen Zirkel zu geraten: durch Rekursion über den *Aufbau* der Formel. Diese Rekursion setzt die Zuordnung $[\![\cdot]\!]$ von den Zeichen auf alle Terme und Formeln fort.

Dazu braucht man einen *Kontext* $\vec{x} = (x_1, \ldots, x_n)$: paarweise verschiedene Variablen, unter denen alle freien Variablen des Terms oder der Formel vorkommen. Ein Tupel $\vec{a} = (a_1, \ldots, a_n) \in M^n$ gibt dann jeder Variablen x_i den Wert a_i.

Definition 2.37 (Semantik, nach Tarski)**.** Sei $\mathfrak{M}$ eine Struktur mit Träger M und $\vec{x}$ ein Kontext der Länge n. Den Index $\vec{x}$ lässt man weg, wo er sich nicht ändert.

(a) Ein Term t bedeutet die Abbildung $[\![t]\!]_{\vec{x}} : M^n \to M$ mit

$$[\![x_i]\!](\vec{a}) = a_i, \qquad [\![c]\!](\vec{a}) = [\![c]\!],$$
$$[\![f(t_1, \ldots, t_k)]\!](\vec{a}) = [\![f]\!]([\![t_1]\!](\vec{a}), \ldots, [\![t_k]\!](\vec{a})).$$

(b) Eine Formel φ bedeutet die Teilmenge $[\![\varphi]\!]_{\vec{x}} \subseteq M^n$ mit

$$\begin{aligned}
[\![t_1 = t_2]\!] &= \{\, \vec{a} \in M^n \mid [\![t_1]\!](\vec{a}) = [\![t_2]\!](\vec{a}) \,\}, \\
[\![R(t_1, \ldots, t_k)]\!] &= \{\, \vec{a} \in M^n \mid ([\![t_1]\!](\vec{a}), \ldots, [\![t_k]\!](\vec{a})) \in [\![R]\!] \,\}, \\
[\![\neg\varphi]\!] &= M^n \setminus [\![\varphi]\!], \qquad [\![\bot]\!] = \emptyset, \\
[\![\varphi \wedge \psi]\!] &= [\![\varphi]\!] \cap [\![\psi]\!], \qquad [\![\varphi \vee \psi]\!] = [\![\varphi]\!] \cup [\![\psi]\!], \\
[\![\varphi \to \psi]\!] &= (M^n \setminus [\![\varphi]\!]) \cup [\![\psi]\!], \\
[\![\exists y\, \varphi]\!]_{\vec{x}} &= \{\, \vec{a} \in M^n \mid \text{es gibt } b \in M \text{ mit } (\vec{a}, b) \in [\![\varphi]\!]_{\vec{x},y} \,\}, \\
[\![\forall y\, \varphi]\!]_{\vec{x}} &= \{\, \vec{a} \in M^n \mid \text{für alle } b \in M \text{ ist } (\vec{a}, b) \in [\![\varphi]\!]_{\vec{x},y} \,\}.
\end{aligned}$$

Dabei kommt y in $\vec{x}$ nicht vor; sonst benennt man die gebundene Variable vorher um – die α-Umbenennung, die die Alligatoren aus Abschnitt 2.4 schon kennen. $\varphi \leftrightarrow \psi$ kürzt $(\varphi \to \psi) \wedge (\psi \to \varphi)$ ab.

(c) Eine *Aussage* ist eine Formel ohne freie Variablen; ihr Kontext ist leer, $n = 0$. M^0 hat genau ein Element, das leere Tupel $()$. Also ist $[\![\varphi]\!]$ entweder $\emptyset$ – dann heißt φ *falsch* – oder $\{()\}$ – dann heißt φ *wahr*.

So liest man die Klauseln. Jede Klausel übersetzt einen Baustein der Formel in eine Operation auf Mengen. Bei den Junktoren sieht man das sofort: Die *Junktoren werden Mengenoperationen*, $\neg$ das Komplement in M^n, $\wedge$ der Schnitt, $\vee$ die Vereinigung. Mehr Mühe macht die Klausel für den Existenzquantor,

$$[\![\exists y\, \varphi]\!]_{\vec{x}} = \{\, \vec{a} \in M^n \mid \text{es gibt } b \in M \text{ mit } (\vec{a}, b) \in [\![\varphi]\!]_{\vec{x},y} \,\}.$$

Man liest sie in drei Schritten, hier in der Struktur $\mathbb{R}$ mit $0, 1, +, \cdot$.

(1) *Der Index bestimmt die Produktmenge.* Der Index ist der Kontext, und jede seiner Variablen bekommt eine Koordinate. Zum Index $\vec{x} = (x_1, \ldots, x_n)$ gehört M^n, zum Index $\vec{x}, y$ gehört $M^n \times M = M^{n+1}$ – eine Koordinate mehr, die letzte, für y. Zum Index x, y gehört in $\mathbb{R}$ also die Ebene $\mathbb{R}^2$: erste Koordinate x, zweite y.

(2) *Eine Formel bedeutet eine Teilmenge davon*, eine *Relation*. Das ist meist nicht die ganze Produktmenge und oft selbst kein Produkt $A \times B$. Drei Formeln im Index x, y:

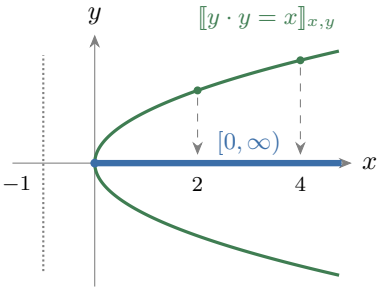
- $[\![y \cdot y = x]\!]_{x,y} = \{\, (a, b) \mid b \cdot b = a \,\}$ ist eine liegende Parabel (Bild unten), kein Produkt.
- $[\![x = x]\!]_{x,y} = \mathbb{R}^2$ ist die ganze Ebene.

- $[\![x \cdot x = 1 + 1]\!]_{x,y} = \{-\sqrt{2}, \sqrt{2}\} \times \mathbb{R}$ sind zwei senkrechte Geraden: y steht im Index, kommt in der Formel aber nicht vor und darf deshalb alles sein. Im Index x allein war es $\{-\sqrt{2}, \sqrt{2}\}$, wie im Beispiel vorab.

Im Index x, y bedeutet so jede Formel eine zweistellige Relation, auch wenn eine der beiden Variablen gar nicht vorkommt. Kommen beide vor, sieht man es am deutlichsten: In der Signatur $\{<\}$ ist $[\![x < y]\!]_{x,y}$ die Relation $[\![<]\!]$ selbst, in der Mengenlehre ist $[\![x \in y]\!]_{x,y} = E$.

(3) $\exists y$ *wirft den Schatten auf* M^n. Die y-Koordinate wird gestrichen: $\vec{a}$ liegt in $[\![\exists y\, \varphi]\!]_{\vec{x}}$, wenn über $\vec{a}$ mindestens ein Punkt $(\vec{a}, b)$ von $[\![\varphi]\!]_{\vec{x},y}$ liegt. Ein solches b heißt *Zeuge*. Das ist das Bild unter der *Projektion* $M^{n+1} \to M^n$, $(\vec{a}, b) \mapsto \vec{a}$: Jeder Punkt fällt senkrecht auf M^n, und was dabei getroffen wird, ist der Schatten. Für $\exists y\, (y \cdot y = x)$ prüft man das Punkt für Punkt:

		a im Schatten?	
a	Zeuge b mit $b \cdot b = a$	in $\mathbb{R}$	in $\mathbb{Q}$
4	$b = 2$	ja	ja
0	$b = 0$	ja	ja
2	$b = \sqrt{2}$	ja	nein, $\sqrt{2} \notin \mathbb{Q}$
-1	keiner, denn $b \cdot b \geq 0$	nein	nein



In $\mathbb{R}$ ist der Schatten der Parabel $[\![\exists y\, (y \cdot y = x)]\!]_x = [0, \infty)$. Die Senkrechte über $a = -1$, im Bild gepunktet, trifft die Parabel nicht. In $\mathbb{Q}$ fehlt $a = 2$, denn die Klausel sucht den Zeugen nur im Träger – „es gibt $b \in M$“ –, und $\sqrt{2} \notin \mathbb{Q}$. Dort besteht der Schatten nur aus den Quadraten rationaler Zahlen, etwa 0, 1, 4 und $\frac{1}{4}$.

Zum Vergleich $\forall$*:* Hier genügt nicht ein Punkt über $\vec{a}$; die ganze Senkrechte $\{\vec{a}\} \times M$, die *Faser* über $\vec{a}$, muss in $[\![\varphi]\!]$ liegen. Die zwei Geraden aus (2) bestehen aus ganzen Fasern, also ist $[\![\forall y\, (x \cdot x = 1 + 1)]\!]_x = \{-\sqrt{2}, \sqrt{2}\}$, dasselbe wie mit $\exists y$. Die Parabel enthält keine ganze Faser, also ist $[\![\forall y\, (y \cdot y = x)]\!]_x = \emptyset$, während $\exists y$ den Schatten $[0, \infty)$ lieferte.

Ohne freie Variablen, also für $n = 0$, fällt der Schatten auf $M^0 = \{()\}$, einen einzigen Punkt. Er ist $\{()\}$, also *wahr*, wenn es überhaupt etwas zu projizieren gibt, und sonst $\emptyset$, also *falsch*. So wird aus einer Lösungsmenge ein Wahrheitswert. Für die Aussage $\exists x\, (x \cdot x = 1 + 1)$ – die gebundene Variable heißt hier x statt y – projiziert man

$$[\![x \cdot x = 1 + 1]\!]_x = \{\, a \mid a \cdot a = 2 \,\} = \begin{cases} \{-\sqrt{2}, \sqrt{2}\} & \text{in } \mathbb{R}, \\ \emptyset & \text{in } \mathbb{Q}. \end{cases}$$

Die Aussage ist also in $\mathbb{R}$ wahr und in $\mathbb{Q}$ falsch.

Auch die Wahrheitswerte sind dabei Mengen. Fasst man das leere Tupel als leere Menge auf, so ist *falsch* $= \emptyset$ und *wahr* $= \{\emptyset\}$ – genau die Zahlen 0 und 1, wie sie im Beweis von Satz 2.32 aus Leermenge und Paarmenge gebaut wurden. Im λ-Kalkül waren *wahr* und *falsch* Terme, eine Kodierung im Kalkül; hier sind sie Bedeutungen.

Jede Klausel benutzt nur die Bedeutungen der unmittelbaren Teile. Die Bedeutung des Ganzen hängt also nur von den Bedeutungen der Teile ab; man nennt das *Kompositionalität*. Weil die Teile kürzer sind, bricht die Rekursion ab. Deshalb ist $[\![\cdot]\!]$ durch die Werte auf den Zeichen – also durch die Struktur – eindeutig bestimmt.

Punktweise gelesen. Tarski selbst formuliert die Klauseln für einen einzelnen Punkt: $\mathfrak{M} \models \varphi\,[\vec{a}]$ heißt $\vec{a} \in [\![\varphi]\!]_{\vec{x}}$, gelesen „$\varphi$ ist in $\mathfrak{M}$ wahr, wenn jedes x_i für a_i steht“. Die Klausel für $\wedge$ lautet dann: $\mathfrak{M} \models (\varphi \wedge \psi)[\vec{a}]$ genau dann, wenn $\mathfrak{M} \models \varphi\,[\vec{a}]$ und $\mathfrak{M} \models \psi\,[\vec{a}]$. Die für $\forall$ lautet: $\mathfrak{M} \models (\forall y\, \varphi)[\vec{a}]$ genau dann, wenn $\mathfrak{M} \models \varphi\,[\vec{a}, b]$ für alle $b \in M$ gilt. Mengen oder Punkte – es ist dieselbe Definition.

Modelle. Jetzt lässt sich die zweite Zeile vom Anfang genau sagen.

Definition 2.38 (Modell, Folgerung)**.** Sei $\mathfrak{M}$ eine Struktur und Σ eine Menge von Aussagen.

(a) $\mathfrak{M} \models \varphi$, gelesen „$\varphi$ gilt in $\mathfrak{M}$“, heißt $[\![\varphi]\!]^{\mathfrak{M}}_{\vec{x}} = M^n$: φ ist wahr, welche Werte die Variablen auch haben. Für eine Aussage heißt das $[\![\varphi]\!]^{\mathfrak{M}} = \{()\}$, also wahr.

(b) $\mathfrak{M}$ ist ein *Modell* von Σ, wenn $\mathfrak{M} \models \sigma$ für alle $\sigma \in \Sigma$ gilt.

(c) $\Sigma \models \varphi$, gelesen „φ *folgt* aus Σ“, heißt: Jedes Modell von Σ erfüllt φ. Gilt φ in jeder Struktur, so heißt φ *allgemeingültig*, kurz $\models \varphi$.

$\Sigma \models \varphi$ heißt also: Jede Struktur, deren $[\![\cdot]\!]$ alle Aussagen aus Σ auf *wahr* abbildet, bildet auch φ auf *wahr* ab.

Beispiel 2.39 (Dieselbe Formel, zwei Strukturen)**.** $\mathbb{R}$ und $\mathbb{Q}$, jeweils mit $0, 1, +, \cdot$, sind Körper, also Modelle der Körperaxiome. Die Aussage $\exists x\,(x \cdot x = 1 + 1)$ ist in $\mathbb{R}$ wahr und in $\mathbb{Q}$ falsch, wie oben nachgerechnet: dieselbe Formel, zwei Abbildungen $[\![\cdot]\!]$, zwei Wahrheitswerte. Darum folgt die Aussage nicht aus den Körperaxiomen – sonst wäre sie in $\mathbb{Q}$ wahr – und ihre Negation auch nicht – sonst wäre die Aussage in $\mathbb{R}$ falsch.

Beispiel 2.40 (Eine Struktur für die Mengenlehre)**.** $(\mathbb{N}, <)$ ist eine Struktur für die Signatur $\{\in\}$: Der Träger ist $\mathbb{N}$, und $[\![\in]\!] = \{\,(a, b) \mid a < b\,\}$. Dann ist $[\![x \in y]\!]_{x,y}$ die Relation $<$ selbst, und

$$[\![\forall z\, \neg(z \in x)]\!]_x = \{\,a \in \mathbb{N} \mid \text{für kein } c \in \mathbb{N} \text{ ist } c < a\,\} = \{0\}.$$

Die Zahl 0 spielt also die leere Menge. Die „Elemente“ von b sind die Zahlen $a < b$, also $0, 1, \ldots, b - 1$. Damit lassen sich die Axiome aus Abschnitt 2.5 einzeln nachprüfen, hier die ersten drei der Tabelle:

- Die *Extensionalität* gilt: Ist $\{\,c \mid c < a\,\} = \{\,c \mid c < b\,\}$, so ist $a = b$.
- Das Axiom der *Leermenge*, „es gibt eine leere Menge“, also $\exists x\, \forall z\, \neg(z \in x)$, ist wahr; der Zeuge ist 0.
- Das *Paarmengenaxiom* gilt nicht. Zu $a = b = 1$ müsste es ein c geben mit $\{\,z \mid z < c\,\} = \{1\}$. Jede Menge $\{\,z \mid z < c\,\}$, die 1 enthält, enthält aber auch 0.

$(\mathbb{N}, <)$ ist also ein Modell eines Teils der ZFC-Axiome. Seine Elemente sind Zahlen, und $\in$ wird durch die Ordnung gedeutet – von Enthaltensein ist nirgends die Rede. Für die Semantik spielt das keine Rolle. (Baut man die Zahlen, wie in ZFC üblich, als Mengen, $0 = \emptyset$, $1 = \{0\}$, $2 = \{0, 1\}$ und so fort, so ist $a < b$ sogar dasselbe wie $a \in b$. Die Semantik sieht davon nichts: Sie kennt nur den Träger und $[\![\in]\!]$.)

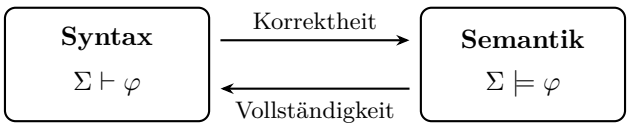
In beiden Beispielen laufen die Quantoren über die Elemente des Trägers – über Zahlen, nicht über Mengen von Zahlen. Das liegt nicht an den Beispielen, sondern an den Klauseln für $\exists$ und $\forall$: Das b dort ist ein Element von M.

Definition 2.41 (Logik erster Stufe)**.** Eine Logik heißt *erster Stufe*, wenn ihre Quantoren nur über *Objekte* laufen, in einer Struktur also über die Elemente des Trägers. Quantoren über Eigenschaften, Teilmengen von M oder Formeln („für alle Formeln φ gilt ...“) sind nicht erlaubt; das wäre zweite Stufe.

In diesem Sinn ist ZFC eine Theorie erster Stufe (Abschnitt 2.5): In einem Modell (M, E) laufen ihre Quantoren über die Elemente von M, und „zu jeder Formel“ lässt sich nur als Schema sagen.

Bemerkung 2.42 (Wahrheit lässt sich nicht von innen definieren). Diese Definitionen finden in einer *Metasprache* statt, und die muss schon eine Mengenlehre enthalten. Denn die Semantik ist eine Abbildung *in die Mengen*: $[\![\varphi]\!]$ ist eine Teilmenge von M^n. Und $\Sigma \models \varphi$ quantifiziert über *alle* Strukturen, also über alle Mengen M. Um von Modellen der Mengenlehre zu reden, braucht man daher bereits eine Mengenlehre – M und E sind ja Mengen. Das ist kein Versehen, sondern unvermeidlich: Tarski zeigte in derselben Arbeit, dass sich der Wahrheitsbegriff einer hinreichend starken Sprache in dieser Sprache selbst nicht definieren lässt. Man kann über die Wahrheit eines Systems nur von außen reden.

Ableitbarkeit und Folgerung haben zunächst nichts miteinander zu tun. Dass sie doch zusammenfallen, ist ein Satz:



Korrektheit ist die leichtere Hälfte: Die Schlussregeln sind so gewählt, dass sie nichts Falsches erzeugen. Mit $[\![\cdot]\!]$ prüft man das Regel für Regel nach. Der modus ponens etwa erhält die Eigenschaft „$[\![\cdot]\!]$ ist ganz M^n“: Aus $[\![\alpha]\!] = M^n$ und

$$[\![\alpha \to \beta]\!] = (M^n \setminus [\![\alpha]\!]) \cup [\![\beta]\!] = M^n$$

folgt $[\![\beta]\!] = M^n$, denn $M^n \setminus [\![\alpha]\!]$ ist leer. So prüft man jede Regel und jedes Axiom – das ist der Korrektheitssatz: Aus $\Sigma \vdash \varphi$ folgt $\Sigma \models \varphi$. Mit ihm ist auch klar, dass sich $\exists x\,(x \cdot x = 1 + 1)$ aus den Körperaxiomen nicht herleiten lässt; sonst gälte die Aussage in $\mathbb{Q}$.

Vollständigkeit ist der Satz von Gödel aus dem Jahr 1930 und gilt für die klassische Logik erster Stufe, etwa für den Hilbert-Kalkül mit den Quantoren- und Gleichheitsaxiomen: Was semantisch folgt, ist auch herleitbar; aus $\Sigma \models \varphi$ folgt $\Sigma \vdash \varphi$. Erst das rechtfertigt, das Beweisen durch Zeichenmanipulation zu ersetzen. *Klassisch* muss der Kalkül sein, weil es die Semantik ist: $[\![\varphi \vee \neg\varphi]\!] = [\![\varphi]\!] \cup (M^n \setminus [\![\varphi]\!]) = M^n$. Gentzens Regeln allein, ohne das tertium non datur, leiten das nicht immer her (Abschnitt 2.5).

Modell und Widerspruchsfreiheit. Eine *Theorie* Σ ist eine Menge von Aussagen, etwa die Körperaxiome oder die Axiome von ZFC. Sie heißt *widerspruchsfrei*, wenn sich aus ihr der Widerspruch $\bot$ nicht herleiten lässt, kurz $\Sigma \nvdash \bot$. Gleichwertig: Für keine Formel φ sind φ und $\neg\varphi$ beide herleitbar. Denn mit $\neg\varphi = (\varphi \to \bot)$ folgt aus φ und $\neg\varphi$ durch $\to$E der Widerspruch $\bot$, und aus $\bot$ folgt durch $\bot$E jede Formel. Eine widersprüchliche Theorie beweist also alles.

Satz 2.43 (Modell und Widerspruchsfreiheit)**.** *Für die klassische Logik erster Stufe mit Gleichheit gilt: Eine Theorie Σ ist genau dann widerspruchsfrei, wenn sie ein Modell hat.*

Beweis. ($\Leftarrow$) Das ist die Korrektheit. Sei $\mathfrak{M}$ ein Modell von Σ. Wäre $\Sigma \vdash \bot$, so wäre $\bot$ in $\mathfrak{M}$ wahr. Aber $[\![\bot]\!] = \emptyset$, also falsch.

($\Rightarrow$) Das ist die Vollständigkeit. Man zeigt die Kontraposition. Hat Σ kein Modell, so gilt $\Sigma \models \bot$ – jedes Modell von Σ erfüllt $\bot$, denn es gibt keines. Nach der Vollständigkeit ist dann $\Sigma \vdash \bot$, und Σ ist nicht widerspruchsfrei. □

Der Satz übersetzt eine Aussage über *alle* Herleitungen – keine endet bei $\bot$ – in eine über *ein* Objekt: Es gibt ein Modell. Um Widerspruchsfreiheit zu zeigen, genügt deshalb ein einziges Beispiel; die Körperaxiome etwa sind widerspruchsfrei, weil $\mathbb{Q}$ ein Körper ist. Für ZFC heißt der Satz:

„ZFC ist widerspruchsfrei“ und „ZFC hat ein Modell“ sind gleichwertig.

Das ist dieselbe Sache, einmal über Herleitungen und einmal über Mengen gesagt. Ein Modell ist dabei, wie in Definition 2.36, eine *Menge* M mit einer Relation $E \subseteq M \times M$, in der alle Axiome von ZFC wahr sind. Dass es eines gibt, kann ZFC selbst nicht beweisen, sofern ZFC widerspruchsfrei ist; das folgt aus dem zweiten Teil von Satz 2.51 (Gödel 1931) weiter unten.

Bemerkung 2.44 (Und die Mengen selbst?). Bilden nicht alle Mengen zusammen, mit dem gewöhnlichen $\in$, ein solches Modell? Nein. Die Gesamtheit aller Mengen ist keine Menge – es gibt keine Allmenge, nach Cantor (Abschnitt 2.3) und in ZFC auch nach Satz 2.24 – und taugt deshalb nicht als Träger im Sinn von Definition 2.36. Und „alle Axiome gelten in ihr“ lässt sich nicht einmal als eine einzige Formel hinschreiben: Dazu bräuchte man einen Wahrheitsbegriff für die ganze Mengenwelt, und den gibt es nach Tarski nicht von innen (Bemerkung 2.42).

Woher kommt in der Richtung ($\Rightarrow$) das Modell? Henkin hat 1949 genau diese Richtung direkt bewiesen und so die Vollständigkeit neu begründet: Er baut aus einer widerspruchsfreien Theorie ein Modell aus der Syntax selbst, dessen Elemente geschlossene Terme sind. (Dazu kommt für jede $\exists$-Aussage eine neue Konstante als Zeuge hinzu; dann erweitert man Σ widerspruchsfrei, bis jede Aussage entschieden ist, und zwei Terme gelten als gleich, wenn diese Erweiterung ihre Gleichheit beweist.) Dieselbe Idee, ein Modell aus Termen – ein *Termmodell* –, taucht in Kapitel 3 als syntaktische Kategorie wieder auf (Abschnitt 3.6).

Eine Aussage φ heißt aus Σ *widerlegbar*, wenn $\neg\varphi$ herleitbar ist, und *unabhängig* von Σ, wenn sie weder herleitbar noch widerlegbar ist. Auch das lässt sich mit Modellen sagen.

Korollar 2.45 (Unabhängigkeit heißt: zwei Modelle)**.** *Eine Aussage φ ist genau dann unabhängig von Σ, wenn $\Sigma \cup \{\varphi\}$ und $\Sigma \cup \{\neg\varphi\}$ beide ein Modell haben – nach Satz 2.43: wenn beide widerspruchsfrei sind.*

Beweis. Nach Korrektheit und Vollständigkeit ist φ genau dann nicht aus Σ herleitbar, wenn φ nicht aus Σ folgt, also genau dann, wenn es ein Modell von Σ gibt, in dem φ falsch und damit $\neg\varphi$ wahr ist – ein Modell von $\Sigma \cup \{\neg\varphi\}$. Ebenso ist $\neg\varphi$ genau dann nicht herleitbar, wenn $\Sigma \cup \{\varphi\}$ ein Modell hat. □

Ein Beispiel steht schon da: $\exists x\,(x \cdot x = 1 + 1)$ ist von den Körperaxiomen unabhängig, und $\mathbb{R}$ und $\mathbb{Q}$ sind die beiden Modelle (Beispiel 2.39). Dasselbe Muster, mit Modellen der Mengenlehre statt Körpern, steckt hinter der Unabhängigkeit des Auswahlaxioms (Bemerkung 2.46) und der Kontinuumshypothese (weiter unten) – dort allerdings nur relativ, unter der Voraussetzung, dass die Mengenlehre selbst widerspruchsfrei ist.

Bemerkung 2.46 (Das Auswahlaxiom: zwei Modelle von ZF). Gödel zeigte 1938: Ist ZF widerspruchsfrei, so auch ZF mit dem Auswahlaxiom. Cohen zeigte 1963: dann auch ZF mit seiner Negation. Nach Satz 2.43 heißt das: Hat ZF ein Modell, so auch eines, in dem das Auswahlaxiom gilt, und eines, in dem es falsch ist – zwei Modelle wie in Korollar 2.45, mit ZF an der Stelle der Körperaxiome und dem Auswahlaxiom an der Stelle von $\exists x\,(x \cdot x = 1 + 1)$.

Zwei weitere Sätze über Modelle. Aus Satz 2.43 folgt sofort ein Satz, in dem Herleitungen gar nicht mehr vorkommen.

Satz 2.47 (Kompaktheit)**.** *Hat jede endliche Teilmenge von Σ ein Modell, so hat Σ ein Modell.*

Mit Satz 2.43 liest man den Beweis in einer Zeile: Eine Herleitung von $\bot$ ist *endlich*, benutzt also nur endlich viele Aussagen aus Σ. Deshalb ist Σ widerspruchsfrei, sobald jede endliche Teilmenge es ist – und „widerspruchsfrei“ heißt nach dem Satz „hat ein Modell“.

Der zweite Satz handelt davon, wie *groß* ein Modell sein muss. Er braucht die Vollständigkeit nicht, nur die Semantik. Dafür ein paar Begriffe.

- Die *Mächtigkeit* einer Struktur ist die ihres Trägers. Ein *abzählbares* Modell ist also eines mit abzählbarem Träger M.

- Eine Signatur heißt *abzählbar*, wenn sie höchstens abzählbar viele Zeichen hat. Dann gibt es auch nur abzählbar viele Formeln: Jede ist eine endliche Zeichenreihe über einem abzählbaren Vorrat – den Zeichen der Signatur, den Variablen $x_1, x_2, \ldots$, den Junktoren, Quantoren und Klammern.
- ZFC ist eine Theorie in der Signatur $\{\in\}$ mit abzählbar vielen Axiomen; jedes Schema hat eine Instanz pro Formel.
- Eine *Teilstruktur* $\mathfrak{N}$ von $\mathfrak{M}$ hat einen Träger $N \subseteq M$, der die Konstanten enthält und unter den Funktionen abgeschlossen ist, und deutet die Zeichen wie $\mathfrak{M}$, auf N eingeschränkt.

Satz 2.48 (Löwenheim–Skolem, absteigend)**.** *Sei L eine abzählbare Signatur und Σ eine Theorie in L. Hat Σ ein Modell mit unendlichem Träger, so hat Σ auch eines mit abzählbar unendlichem Träger.*

Genauer: Ist $\mathfrak{M}$ eine Struktur für L und $A \subseteq M$ abzählbar, so gibt es eine abzählbare Teilstruktur $\mathfrak{N}$ mit $A \subseteq N$, in der jede Formel dasselbe bedeutet wie in $\mathfrak{M}$, geschnitten mit N^n:

$$[\![\varphi]\!]^{\mathfrak{N}}_{\vec{x}} = [\![\varphi]\!]^{\mathfrak{M}}_{\vec{x}} \cap N^n.$$

Der erste Teil folgt aus dem zweiten. Ist M unendlich, so wählt man A abzählbar unendlich; dann ist auch N abzählbar unendlich. Für Aussagen ist $n = 0$ und $N^0 = M^0 = \{()\}$. Die Gleichung sagt dann: Jede Aussage hat in $\mathfrak{N}$ denselben Wahrheitswert wie in $\mathfrak{M}$. Insbesondere ist $\mathfrak{N}$ wieder ein Modell von Σ.

Beweisskizze für den zweiten Teil. Man behält von M nur, was man als *Zeugen* braucht. Man beginnt mit $N_0 := A$. Zu jeder Formel $\exists y\, \varphi$ und jedem Tupel $\vec{a}$ aus N_0 mit $\vec{a} \in [\![\exists y\, \varphi]\!]^{\mathfrak{M}}$ wählt man *einen* Zeugen, also ein $b \in M$ mit $(\vec{a}, b) \in [\![\varphi]\!]^{\mathfrak{M}}$, und nimmt ihn dazu. Das ergibt N_1. Es gibt abzählbar viele Formeln und abzählbar viele Tupel, also kommen nur abzählbar viele Zeugen dazu. Dasselbe wiederholt man mit N_1, dann mit N_2 und so fort, und setzt $N := N_0 \cup N_1 \cup N_2 \cup \cdots$. Als abzählbare Vereinigung abzählbarer Mengen ist N abzählbar.

Zu jeder $\exists$-Formel, die in $\mathfrak{M}$ mit Parametern aus N gilt, liegt nun schon in N ein Zeuge: Die Parameter liegen alle in einem N_k, der Zeuge also in N_{k+1}. Funktionszeichen machen keine Extraarbeit: Der einzige Zeuge für $\exists y\, (y = f(\vec{x}))$ an der Stelle $\vec{a}$ ist $[\![f]\!](\vec{a})$, der für $\exists y\, (y = c)$ ist $[\![c]\!]$. Also ist $\mathfrak{N}$ eine Teilstruktur.

Die Gleichung zeigt man durch Induktion über den Aufbau von φ, Klausel für Klausel (Definition 2.37). Für atomare Formeln stimmt sie, weil $\mathfrak{N}$ die Zeichen wie $\mathfrak{M}$ deutet; Komplement, Schnitt und Vereinigung vertragen sich mit dem Schneiden mit N^n. Heikel sind nur die Quantoren. Bei $\exists$, der Projektion, könnte ein Punkt $\vec{a} \in N^n$ in M einen Zeugen haben, in N aber keinen; genau das schließt die Konstruktion aus (Kriterium von Tarski und Vaught). Bei $\forall$, der Faser, ist es dasselbe von der anderen Seite: Ein Gegenbeispiel in M ist ein Zeuge für $\exists y\, \neg\varphi$, und ein solcher liegt schon in N. □

Beispiel 2.49 (Ein abzählbarer Doppelgänger von $\mathbb{R}$)**.** Die Signatur sei $\{<\}$. $(\mathbb{R}, <)$ und $(\mathbb{Q}, <)$ sind *dichte lineare Ordnungen ohne Endpunkte*: Zwischen je zwei Elementen liegt ein drittes, und es gibt weder ein größtes noch ein kleinstes. Das sind Aussagen erster Stufe über $<$, etwa $\forall x\, \forall y\, (x < y \to \exists z\, (x < z \wedge z < y))$.

$\mathbb{Q}$ ist abzählbar, $\mathbb{R}$ nicht. Wendet man Satz 2.48 auf $(\mathbb{R}, <)$ an, so erhält man eine abzählbare Ordnung, in der dieselben Aussagen gelten wie in $\mathbb{R}$ – also wieder eine dichte lineare Ordnung ohne Endpunkte. Nach einem Satz von Cantor sind je zwei abzählbare solche Ordnungen isomorph. Der abzählbare Doppelgänger von $(\mathbb{R}, <)$ ist also, bis auf Umbenennen der Elemente, $(\mathbb{Q}, <)$, und keine Aussage über $<$ unterscheidet die beiden. Dasselbe Argument zeigt: In allen dichten linearen Ordnungen ohne Endpunkte gelten dieselben Aussagen. Nach dem Vollständigkeitssatz ist deshalb jede Aussage über $<$ oder ihre Negation aus den Axiomen dieser Ordnungen herleitbar; man nennt eine solche Theorie *vollständig* – ein zweiter Sinn des Wortes, siehe Bemerkung 2.50.

Was $\mathbb{R}$ von $\mathbb{Q}$ unterscheidet, ist die *Supremumseigenschaft*: Jede nichtleere, nach oben beschränkte *Teilmenge* hat ein Supremum. Diese Aussage quantifiziert über Teilmengen von M, nicht über Elemente. Sie ist nicht erster Stufe (Definition 2.41), und nach dem eben Gesagten lässt sie sich durch keine Aussage erster Stufe über $<$ ersetzen.

In der Körpersignatur sieht es anders aus: Dort unterscheidet $\exists x\,(x \cdot x = 1 + 1)$ aus Beispiel 2.39 die beiden sehr wohl. Ein abzählbarer Doppelgänger von $\mathbb{R}$ ist dort nicht $\mathbb{Q}$, sondern etwa der Körper der reellen algebraischen Zahlen – er enthält $\sqrt{2}$.

Das Skolem-Paradoxon. Man wende Satz 2.48 auf ZFC an. Ist ZFC widerspruchsfrei – und davon geht aus, wer in ZFC arbeitet –, so hat ZFC nach Satz 2.43 ein Modell. Jedes Modell von ZFC ist unendlich, schon weil die Zahlen $0, 1, 2, \ldots$ darin verschieden sein müssen. Nach Löwenheim–Skolem hat ZFC dann auch ein *abzählbares* Modell $\mathfrak{M} = (M, E)$. Zugleich beweist ZFC, dass es überabzählbare Mengen gibt, etwa $\mathbb{R}$. Beides zusammen klingt unmöglich. Mit der Semantik lässt es sich nachrechnen.

- In M gibt es ein Element r, das $\mathfrak{M}$ für $\mathbb{R}$ hält: Es erfüllt in $\mathfrak{M}$ die Formel, mit der ZFC die reellen Zahlen definiert.
- Sei $\mathrm{Bij}(f, x)$ die Formel „f ist eine Bijektion von ω auf x“; ω steht dabei für die Menge der natürlichen Zahlen. Weil ZFC beweist, dass $\mathbb{R}$ überabzählbar ist, gilt nach der Korrektheit $r \in [\![\neg\exists f\,\mathrm{Bij}(f, x)]\!]_x$. Nach der Klausel für $\exists$ heißt das nur: Es gibt *in* M kein f, das $\mathfrak{M}$ für eine solche Bijektion hält.
- Von außen gesehen bilden die E-Elemente von r, also $\{\, a \in M \mid (a, r) \in E \,\}$, eine Teilmenge der abzählbaren Menge M – eine unendliche, denn $\mathfrak{M}$ hält r für unendlich. Von außen gibt es also eine Bijektion von $\mathbb{N}$ auf sie. Sie wird aber von keinem Element von M dargestellt: Für kein $f \in M$ ist $(f, r) \in [\![\mathrm{Bij}(f, x)]\!]_{f,x}$ – und nur solche f sieht der Quantor $\exists f$.

Dasselbe wie $\sqrt{2}$ in $\mathbb{Q}$. Das Muster steht schon in der Tabelle zur Parabel, Zeile $a = 2$:

	$\sqrt{2}$ in $\mathbb{Q}$	**Bijektion im abzählbaren Modell**
Formel	$\neg\exists y\,(y \cdot y = x)$, an der Stelle 2	$\neg\exists f\,\mathrm{Bij}(f, x)$, an der Stelle r
in der Struktur	wahr: kein $y \in \mathbb{Q}$ passt	wahr: kein $f \in M$ passt
von außen	$\sqrt{2}$ gibt es, aber $\sqrt{2} \notin \mathbb{Q}$	eine Bijektion gibt es, aber kein Element von M stellt sie dar

Niemand findet es paradox, dass in $\mathbb{Q}$ die Gleichung $y \cdot y = 2$ keine Lösung hat, obwohl es $\sqrt{2}$ gibt: $\sqrt{2}$ liegt eben nicht in $\mathbb{Q}$. Beim Skolem-Paradoxon fehlt statt $\sqrt{2}$ die Bijektion.

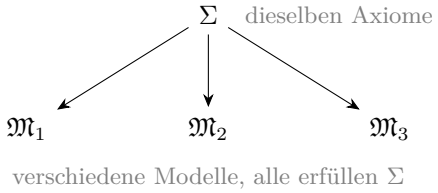
Quantoren laufen nur über den Träger. Das ist die ganze Auflösung, in $\mathbb{Q}$ wie in $\mathfrak{M}$: Ein Quantor in einer Struktur fragt nur nach Elementen des Trägers – das „es gibt $b \in M$“ der Klausel für $\exists$.

„Abzählbar“ heißt „endlich oder in Bijektion mit $\mathbb{N}$“, für das unendliche r also „es gibt eine Bijektion mit $\mathbb{N}$“. Ob es die gibt, hängt davon ab, welche Abbildungen die Struktur enthält – innen gibt es weniger als außen. „Abzählbar“ ist also keine absolute Eigenschaft einer Menge. Axiome erster Stufe reden nur über das, was im Modell liegt; deshalb können sie „wirklich überabzählbar“ nicht erzwingen. Das Modell kann seine eigene Abzählbarkeit nicht sehen.

Warum M Abbildungen verpassen muss. Dass innen weniger Abbildungen liegen als außen, ist kein Zufall, sondern unvermeidlich. M ist abzählbar, enthält also nur abzählbar viele Objekte, die $\mathfrak{M}$ für Abbildungen hält. Abbildungen $\mathbb{N} \to \mathbb{N}$ gibt es aber überabzählbar viele: Schon die Abbildungen $\mathbb{N} \to \{0, 1\}$ entsprechen den Teilmengen von $\mathbb{N}$, und die sind nach Satz 2.6 nicht

abzählbar. Ein abzählbares Modell lässt also fast alle Abbildungen aus. *Welche* es auslässt, sagt das Abzählen nicht: Die E-Elemente von ω sind von außen genauso abzählbar wie die von r, und hier hält $\mathfrak{M}$ sehr wohl ein Element von M für eine Bijektion von ω auf ω. Dass es keines für eine Bijektion von ω auf r hält, erzwingt ZFC selbst: ZFC beweist, dass $\mathbb{R}$ überabzählbar ist, und nach der Korrektheit gilt das auch in $\mathfrak{M}$ (zweiter Punkt oben).

Bei abzählbarer Signatur gibt es auch die *aufsteigende* Richtung: Hat eine Theorie ein Modell mit unendlichem Träger, so hat sie Modelle jeder größeren Mächtigkeit. Der Beweis benutzt die Kompaktheit, Satz 2.47. Eine Theorie mit einem unendlichen Modell legt also nicht einmal fest, wie groß ihre Modelle sind.



Bauplan und Gebäude. Ein Axiomensystem ist ein Bauplan, ein Modell ein danach errichtetes Gebäude. Derselbe Plan lässt viele Gebäude zu, und kein Plan legt jede Einzelheit fest.

Bei den Gruppenaxiomen stört das niemanden – man *will* viele Gruppen. Bei der Mengenlehre irritiert es, weil man meint, über *die* Mengen zu reden. Tut man aber nicht: Man redet über alles, was die Axiome erfüllt.

Wie weit das reicht, zeigt die *Kontinuumshypothese* CH: Jede Teilmenge von $\mathbb{R}$ ist abzählbar oder in Bijektion mit $\mathbb{R}$ – dazwischen gibt es nichts (Cantor 1878). Wie beim Auswahlaxiom (Bemerkung 2.46) gilt: Hat ZFC überhaupt ein Modell, so auch eines, in dem CH gilt (Gödel 1938), und eines, in dem CH falsch ist (Cohen 1963). Nach Satz 2.43 ist das dasselbe wie: Ist ZFC widerspruchsfrei, so auch ZFC + CH und ZFC + ¬CH. Nach Korollar 2.45 ist CH dann von ZFC unabhängig, weder herleitbar noch widerlegbar – wie $\exists x\,(x \cdot x = 1 + 1)$ von den Körperaxiomen, nur mit zwei Modellen von ZFC statt $\mathbb{R}$ und $\mathbb{Q}$. Ein handfestes Beispiel für die Unvollständigkeit aus Satz 2.51.

Was man von einem Axiomensystem verlangt

Widerspruchsfreiheit
Nicht beides, φ und $\neg\varphi$, ist herleitbar; gleichwertig: Es gibt ein Modell (Satz 2.43). Ohne sie ist alles herleitbar und das System wertlos.

Vollständigkeit einer Theorie
Für jede Aussage φ ist φ oder $\neg\varphi$ herleitbar – die Axiome entscheiden jede Frage.

Entscheidbarkeit
Es gibt ein Verfahren, das zu jeder Formel sagt, ob sie herleitbar ist.

Unabhängigkeit
Kein Axiom folgt aus den übrigen. Man zeigt das mit einem Modell der übrigen Axiome, in dem das eine falsch ist; nach der Korrektheit ist es dann nicht aus ihnen herleitbar. Das ist die eine Hälfte von Korollar 2.45. Die andere, ein Modell, in dem es gilt, liefert jedes Modell des ganzen Systems.

Bemerkung 2.50 (Zwei verschiedene Vollständigkeiten). Das Wort meint hier zwei verschiedene Dinge, und sie werden oft verwechselt. Oben ging es um die Vollständigkeit der *Logik*: Was

folgt, ist herleitbar. Hier geht es um die Vollständigkeit einer *Theorie*: Die Axiome entscheiden jede Aussage über ihren Gegenstand. Gödel hat das erste bewiesen und das zweite widerlegt, jedenfalls für Theorien wie in Satz 2.51, die genügend Arithmetik enthalten. Die Theorie der dichten linearen Ordnungen ohne Endpunkte aus Beispiel 2.49 enthält keine Arithmetik und ist vollständig.

Die Grenze

Satz 2.51 (Gödel, 1931)**.** *Jedes widerspruchsfreie formale System, dessen Axiome sich mechanisch aufzählen lassen und das genügend Arithmetik enthält, ist unvollständig: Es gibt eine Aussage φ, für die weder φ noch $\neg\varphi$ herleitbar ist. Überdies lässt sich die Widerspruchsfreiheit des Systems in ihm selbst nicht beweisen.*

Das betrifft ZFC aus Abschnitt 2.5 unmittelbar, ebenso NBG aus dem nächsten Abschnitt: Beide sind Axiomensysteme dieser Art. Man kann also nicht darauf hoffen, dass sie jede mengentheoretische Frage entscheiden – und man kann ihre Widerspruchsfreiheit nicht mit ihren eigenen Mitteln sichern.

Mit Satz 2.43, dessen Beweis sich in ZFC selbst führen lässt, heißt der zweite Teil für ZFC: Ist ZFC widerspruchsfrei, so beweist ZFC nicht, dass ZFC ein Modell hat. Ein Modell von ZFC lässt sich in ZFC nicht bauen; dazu braucht man ein stärkeres System, etwa ZFC mit einem Grothendieck-Universum (Satz 2.57 im nächsten Abschnitt, fortgesetzt zu einer ganzen Leiter in Satz 2.61). Auf demselben Weg sichert man die Widerspruchsfreiheit von Leans Typentheorie: durch ein Modell in den Mengen (Abschnitt 3.6, „Bedeutung in den Mengen“), gebaut in ZFC mit unerreichbaren Kardinalzahlen (Bemerkung 2.59; Carneiro 2019). Das ist die Antwort auf die zweite Frage vom Anfang des Abschnitts: Mehr als eine solche *relative* Widerspruchsfreiheit – widerspruchsfrei, falls das stärkere System es ist – ist nach Gödel nicht zu haben.

Was ein Axiomensystem leistet – und was nicht. Die Axiome sind kein Beweis dafür, dass es Mengen gibt. Sie sind eine Festlegung, was man mit ihnen tun darf, zusammen mit der Erfahrung, dass sich daraus in hundert Jahren kein Widerspruch ergeben hat.

Das klingt nach wenig, ist aber genau das, was nach Russell übrig bleibt: keine Gewissheit über das Wesen der Mengen, sondern ein nachprüfbares Regelwerk. Wer sichergehen will, muss nicht mehr die Anschauung befragen, sondern die Herleitung durchsehen.

2.7 NBG: Mengen und Klassen

Die von-Neumann–Bernays–Gödel-Mengenlehre (NBG) ist die zweite Antwort auf Russell (Abschnitt 2.3). Sie geht anders vor als ZFC (Abschnitt 2.5) und schränkt die Komprehension *nicht* ein. Jede Formel darf zusammenfassen – nur ist das Ergebnis nicht unbedingt eine Menge.

Definition 2.52 (Klasse und Menge)**.** Zu jeder Formel φ gibt es die *Klasse*

$$\{\, x \mid \varphi(x) \,\},$$

also die Zusammenfassung aller Mengen x mit $\varphi(x)$. Eine Klasse heißt *Menge*, wenn sie selbst Element einer Klasse ist. Klassen, die keine Mengen sind, heißen *echte Klassen*.

Welche Regel gilt wofür

NBG hat zwei Sorten von Objekten, und die Axiome zerfallen entsprechend in zwei Gruppen. Man sollte sie auseinanderhalten.

Regeln für Klassen. Davon gibt es nur zwei.

(K1) Klassenextensionalität
Zwei Klassen mit denselben Elementen sind gleich.

(K2) Klassenkomprehension
Zu jeder Formel φ, deren Quantoren nur über *Mengen* laufen, gibt es die Klasse $\{\, x \mid \varphi(x) \,\}$. Die Variable x läuft dabei ebenfalls über Mengen.

Regeln für Mengen. Das sind im Wesentlichen die ZFC-Axiome, jetzt als Aussagen über Mengen gelesen: Paarmenge, Vereinigung, Potenzmenge, Unendlichkeit, Ersetzung, Fundierung, Auswahl. Dazu die Bestimmung, wann eine Klasse eine Menge ist – nämlich genau dann, wenn sie Element einer Klasse ist.

	Klassen	**Mengen**
Wie entstehen sie?	aus jeder zulässigen Formel, also frei (K2)	nur über die Axiome, also kontrolliert
Dürfen sie Element sein?	nein – außer sie sind Mengen	ja
Worüber quantifiziert die Komprehension?	über Mengen, *nicht* über Klassen	—
Wann sind zwei gleich?	gleiche Elemente (K1)	gleiche Elemente

Zwei Punkte daran sind leicht zu übersehen.

Erstens sammelt $\{x \mid \varphi(x)\}$ in (K2) nur *Mengen* ein, nicht Klassen. Eine Klasse von Klassen gibt es in NBG nicht.

Zweitens ist die Bedingung „Quantoren nur über Mengen“ in (K2) wesentlich. Lässt man auch Quantoren über Klassen zu, erhält man die Morse–Kelley-Mengenlehre MK – und die ist echt stärker als ZFC, beweist etwa die Widerspruchsfreiheit von ZFC. NBG mit der Einschränkung tut das nicht.

Bemerkung 2.53 (Endlich viele Axiome). Hier nützt die Unterscheidung. In ZFC braucht man Schemata, weil man erststufig nicht über Formeln quantifizieren kann (Definition 2.41). In NBG sind Klassen *Objekte* der Theorie, über die man ganz normal quantifizieren darf. Gödel zeigte, dass sich (K2) deshalb auf endlich viele klassenbildende Operationen zurückführen lässt. NBG kommt damit mit endlich vielen Axiomen aus, ZFC nicht.

Was daraus folgt

Damit lässt sich die Überschrift von Abschnitt 2.3 genau fassen. In NBG gilt:

Jede Formel liefert eine **Klasse**.	uneingeschränkte Komprehension
Nicht jede Klasse ist eine **Menge**.	nur die, die Element sein können
Also: *nicht jede Formel ist eine Menge.*	

Der Unterschied zu ZFC liegt also darin, an welcher Stelle eingeschränkt wird. ZFC beschränkt, *was man bilden darf* – ausgesondert wird nur aus vorhandenen Mengen. NBG erlaubt jede Bildung und beschränkt stattdessen, *was als Element auftreten darf.*

Russells Konstruktion wird damit von einem Widerspruch zu einem Satz.

Satz 2.54 (Die Russell-Klasse ist eine echte Klasse)**.** *Die Klasse* $R := \{\, x \mid x \notin x \,\}$ *existiert, ist aber keine Menge.*

Beweis. Dass R als Klasse existiert, ist Definition 2.52: $x \notin x$ ist eine Formel. Wäre R eine Menge, so dürfte man $x := R$ einsetzen und erhielte $R \in R \iff R \notin R$, einen Widerspruch. Also ist R keine Menge. □

Das Argument aus Satz 2.4 ist dasselbe geblieben. Nur seine Rolle hat sich geändert: Es widerlegt nicht mehr die Theorie, sondern zeigt von einer bestimmten Klasse, dass sie keine Menge ist. Ebenso gibt es die Allklasse $V := \{\, x \mid x = x \,\}$; auch sie ist eine echte Klasse.

Bemerkung 2.55. NBG ist gegenüber ZFC konservativ: Beide beweisen genau dieselben Aussagen über Mengen. NBG ist dabei endlich axiomatisierbar, ZFC nicht – Aussonderung und Ersetzung sind dort Schemata mit unendlich vielen Instanzen. Der Preis ist die zweite Sorte von Objekten, über die man mitreden muss.

Universen

Nach Russell und Cantor gibt es keine Menge aller Mengen. Trotzdem will man manchmal über „alle Mengen“ reden – etwa in der Kategorientheorie (Abschnitt 2.8), in der die Kategorie *aller* Mengen ein Grundbeispiel ist. Es gibt drei Antworten.

(1) *Klassen*, wie eben in NBG: Die Allklasse V existiert, ist aber keine Menge. Man darf über sie reden, nur tritt sie nie als Element auf.

(2) *Grothendieck-Universen*: Man bleibt bei Mengen, nimmt aber zu ZFC ein Axiom hinzu, das eine Menge liefert, die so groß ist, dass man von ihrer Grenze nichts merkt (Korollar 2.58).

(3) *Typuniversen*: die Antwort der Typentheorie (Abschnitt 3.1).

Definition 2.56 (Grothendieck-Universum)**.** Eine Menge U heißt *Universum*, wenn gilt:

(i) Ist $x \in U$ und $y \in x$, so ist $y \in U$.

(ii) Sind $x, y \in U$, so ist $\{x, y\} \in U$.

(iii) Ist $x \in U$, so ist $\mathcal{P}(x) \in U$.

(iv) Ist $I \in U$ und $x_i \in U$ für alle $i \in I$, so ist $\bigcup_{i \in I} x_i \in U$.

Ein Universum ist also abgeschlossen unter allem, was man mit Mengen gewöhnlich tut: Elemente nehmen, Paare und Potenzmengen bilden, über eine Familie vereinigen. Daraus folgen Teilmengen, Produkte und Funktionenmengen. Das kleinste nichtleere Universum ist die Menge V_ω der *erblich endlichen* Mengen: der endlichen Mengen, deren Elemente wieder erblich endlich sind. Sie enthält aber $\mathbb{N}$ nicht. Interessant wird es erst mit $\mathbb{N} \in U$.

Satz 2.57. *Ist U ein Universum mit $\mathbb{N} \in U$, so ist $(U, \in)$ ein Modell von ZFC: In der Struktur mit Träger U, in der das Zeichen $\in$ das gewöhnliche Enthaltensein bedeutet, sind alle Axiome von ZFC wahr.*

Beweisskizze. Nach (i) ist U transitiv. Deshalb bedeuten „Element“, „Teilmenge“, „Paar“ und „Vereinigung“ in U dasselbe wie außerhalb, und es genügt zu prüfen, dass die Mengen, deren Existenz die Axiome verlangen, in U liegen. Die Leermenge liegt wegen $\emptyset \in \mathbb{N} \in U$ und (i) in U. Paarmenge und Potenzmenge sind (ii) und (iii); weil mit $\mathcal{P}(x)$ nach (i) alle Teilmengen von x in U liegen, ist es auch in U die volle Potenzmenge. Die Vereinigung ist (iv) mit $I := x$, die Unendlichkeit ist $\mathbb{N} \in U$. Aussonderungen aus $x \in U$ sind Teilmengen von x, also in U. Für die Ersetzung schreibt man das Bild von x als Vereinigung der Einermengen $\{F(y)\}$, $y \in x$, und benutzt (ii) und (iv). Eine Auswahlfunktion gibt es nach dem Auswahlaxiom der umgebenden Mengenwelt; sie ist eine Teilmenge eines Produkts, das in U liegt. Extensionalität und Fundierung übertragen sich von der umgebenden Mengenwelt, weil U transitiv ist. □

Korollar 2.58. *Ist ZFC widerspruchsfrei, so lässt sich in ZFC nicht beweisen, dass es ein Universum mit $\mathbb{N} \in U$ gibt.*

Beweis. Ein solches U liefert nach Satz 2.57 ein Modell $(U, \in)$ von ZFC, nach Satz 2.43 also die Widerspruchsfreiheit von ZFC. Beide Beweise lassen sich in ZFC selbst führen. Bewiese ZFC die Existenz eines solchen U, so bewiese ZFC also die eigene Widerspruchsfreiheit. Das schließt Gödels Satz 2.51 aus. □

Bemerkung 2.59 (Unerreichbare Kardinalzahlen). Baut man die Mengen in Stufen auf – $V_0 := \emptyset$, $V_{\alpha+1} := \mathcal{P}(V_\alpha)$, an Limesstellen die Vereinigung aller früheren Stufen –, so sind die Universen mit $\mathbb{N} \in U$ genau die Stufen V_κ für eine (stark) *unerreichbare* Kardinalzahl κ: überabzählbar, nicht als Vereinigung von weniger als κ kleineren Mengen zu erreichen, und $2^\lambda < \kappa$ für alle $\lambda < \kappa$. Ein Universum zu fordern heißt also, eine solche Kardinalzahl zu fordern.

Eine Hierarchie von Universen

Ein Universum allein reicht selten. Über alle Mengen aus U kann man zwar reden – sie bilden die Menge U. Was man aber aus U selbst baut, etwa die Kategorie aller Mengen aus U, liegt nicht mehr in U. Wer damit so frei rechnen will wie mit den Mengen in U, braucht ein Universum, das U als Element enthält. Grothendieck hat deshalb für die algebraische Geometrie nicht ein Universum gefordert, sondern beliebig viele.

Definition 2.60 (Universenaxiom, Grothendieck)**.** Jede Menge ist Element eines Universums.

Das Axiom steht in Grothendiecks Seminar SGA 4 (1963/64, gedruckt 1972). Mit Bemerkung 2.59 lässt es sich auch so sagen: Über jeder Kardinalzahl liegt eine unerreichbare. ZFC mit dem Universenaxiom heißt *Tarski–Grothendieck-Mengenlehre*. Tarski hatte schon 1938 ein Axiom dieser Art angegeben; zu ZF hinzugenommen, liefert es die Universen und sogar das Auswahlaxiom. Auf dieser Mengenlehre baut der Beweisassistent Mizar auf.

Mit dem Axiom erhält man nicht nur ein Universum, sondern eine ganze Kette. Man wählt ein Universum U_0 mit $\mathbb{N} \in U_0$, dann eines U_1 mit $U_0 \in U_1$, dann U_2 mit $U_1 \in U_2$ und so fort:

$$\mathbb{N} \in U_0 \in U_1 \in U_2 \in \cdots$$

Weil Universen transitiv sind – das ist Bedingung (i) –, folgt aus $U_0 \in U_1$ auch $U_0 \subseteq U_1$: Was in U_0 liegt, liegt auch in U_1. Die Hierarchie ist also *kumulativ*, $U_0 \subseteq U_1 \subseteq U_2 \subseteq \cdots$, und jedes U_n ist ein Modell von ZFC (Satz 2.57). Nach Bemerkung 2.59 ist $U_n = V_{\kappa_n}$ mit unerreichbaren Kardinalzahlen $\kappa_0 < \kappa_1 < \kappa_2 < \cdots$.

Klein und groß. Wozu man so viele Universen braucht, zeigt die Kategorientheorie (Abschnitt 2.8). In ihr legt man ein Universum fest, etwa U_0, und nennt eine Menge *klein*, wenn sie in U_0 liegt. Die Kategorie $\mathbf{Set}_{U_0}$ der kleinen Mengen – Objekte sind die Elemente von U_0, Morphismen die Abbildungen zwischen ihnen – hat U_0 selbst als Menge ihrer Objekte. Sie ist also nicht klein, denn $U_0 \notin U_0$. Aber sie liegt in U_1, denn sie entsteht aus $U_0 \in U_1$ durch Paare, Teilmengen und Potenzmengen: *groß* bezüglich U_0, klein bezüglich U_1. Ebenso die Kategorie aller kleinen Kategorien. Wer $\mathbf{Set}_{U_0}$ selbst wieder als Objekt braucht, etwa in der Kategorie aller bezüglich U_1 kleinen Kategorien, landet in U_2, und so fort. Das Universenaxiom garantiert, dass man immer eine Stufe höher gehen kann.

Grothendiecks Methoden, in SGA mit Universen formuliert, stecken auch im Beweis von Fermats letztem Satz durch Wiles (1995). Nötig sind die Universen dort nach Einschätzung der Fachleute aber nicht: Sie lassen sich im Prinzip zugunsten von ZFC beseitigen (McLarty 2010, nach Deligne). Ausgeführt hat das McLarty 2020, und zwar weit unterhalb von ZFC: Er begründet alle Sätze aus SGA ohne Universen, in der Arithmetik endlicher Stufe.

Die Kette hat noch eine zweite Folge, für die Widerspruchsfreiheit. Jedes Universum sieht die darunter von außen: Von U_1 aus ist U_0 eine Menge und ein Modell von ZFC; in U_1 gilt also „ZFC hat ein Modell“ und nach Satz 2.43 „ZFC ist widerspruchsfrei“. Dasselbe wiederholt sich auf jeder Stufe. Als Aussage über Beweisbarkeit lautet das so:

Satz 2.61 (Eine Leiter der Widerspruchsfreiheit)**.** *Sei T_n die Theorie ZFC + „es gibt n verschiedene Universen mit $\mathbb{N} \in U$“; insbesondere ist T_0 einfach ZFC. Dann gilt für jedes n:*

(a) T_{n+1} beweist, dass T_n widerspruchsfrei ist.

(b) Ist T_n widerspruchsfrei, so lässt sich in T_n nicht beweisen, dass es $n+1$ solche Universen gibt.

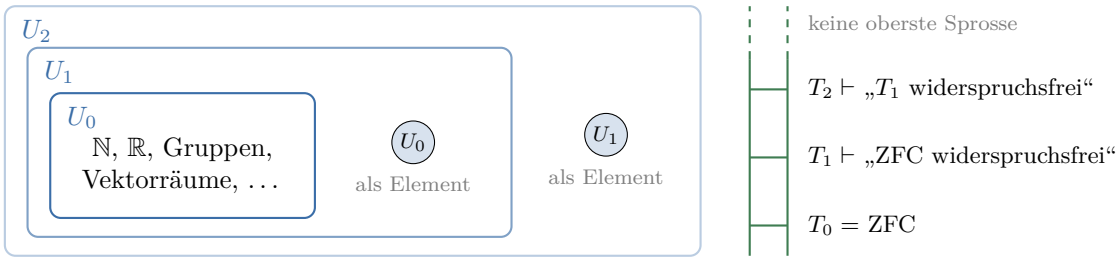
Beweisskizze. (a) Man arbeitet in T_{n+1}. Nach Bemerkung 2.59 sind die $n+1$ Universen Stufen V_κ, und von zwei Stufen liegt die frühere in der späteren. Sie bilden also eine Kette $U_0 \in U_1 \in \cdots \in U_n$. Nach Satz 2.57 ist $(U_n, \in)$ ein Modell von ZFC. Die übrigen $U_0, \ldots, U_{n-1}$ liegen in U_n und sind auch *in* U_n Universen: Definition 2.56 redet von Elementen, Paaren, Potenzmengen und Vereinigungen von Familien, und die liegen in U_n und bedeuten dort dasselbe wie außerhalb (wie im Beweis von Satz 2.57). Also ist $(U_n, \in)$ ein Modell von T_n, und nach Satz 2.43 ist T_n widerspruchsfrei. Beide Sätze lassen sich in ZFC beweisen, erst recht in T_{n+1}.

(b) Bewiese T_n, dass es $n+1$ solche Universen gibt, so bewiese T_n nach (a) seine eigene Widerspruchsfreiheit. Das schließt Gödels Satz 2.51 aus. □

Für $n = 0$ ist (b) gerade Korollar 2.58. Die Theorien $T_0, T_1, T_2, \ldots$ bilden also eine Leiter: Jede Sprosse beweist die Widerspruchsfreiheit der Sprosse darunter, die eigene aber keine, sofern sie widerspruchsfrei ist (Satz 2.51). Eine oberste Sprosse gibt es nicht: Auch ZFC mit dem Universenaxiom, das alle Sprossen auf einmal liefert, beweist seine eigene Widerspruchsfreiheit nicht, sofern es widerspruchsfrei ist. Mehr als relative Widerspruchsfreiheit ist auch hier nicht zu haben.

Stockwerke. Kein Stockwerk enthält das ganze Haus. Wer im Erdgeschoss U_0 arbeitet, merkt davon nichts: Zahlen, Gruppen, Vektorräume – alles liegt dort. Erst wer mit allen Mengen des Erdgeschosses zugleich rechnen will – wie die Kategorientheorie mit der Kategorie aller Mengen –, steigt ein Stockwerk höher, in dem U_0 selbst ein Element ist.

Von oben sieht man auch, dass das Stockwerk darunter trägt: In U_1 ist U_0 ein Modell von ZFC, also gilt dort „ZFC ist widerspruchsfrei“. Das eigene Stockwerk sieht man so nicht, denn es liegt nicht in sich selbst. Beweisen lässt sich die Widerspruchsfreiheit von T_n erst mit dem Axiom für ein weiteres Universum, in T_{n+1} (Satz 2.61); das zeigt die Leiter rechts im Bild. Ein Dach hat das Haus nicht.



Typuniversen. Die Typentheorie braucht für diese Hierarchie kein Axiom; sie ist in die Sprache eingebaut. Jeder Typ hat selbst einen Typ, und die Typen stehen in einer Hierarchie von Universen: ℕ liegt im untersten Universum für Daten, dieses im nächsten und so fort (Abschnitt 3.1). Kein Universum enthält sich selbst – ließe man das zu, wäre das System widersprüchlich (Girard 1972). Jedes Universum spielt die Rolle eines Grothendieck-Universums:

abgeschlossen unter den üblichen Konstruktionen und doch nicht das Ganze. Leans Universen für Daten, die Stufen $0, 1, 2, \ldots$, bilden also dieselbe Kette wie $U_0 \in U_1 \in U_2 \in \cdots$. Ein Mengenmodell für Lean deutet die Stufe u durch ein Universum U_u; es braucht also eine unerreichbare Kardinalzahl pro Stufe, abzählbar viele insgesamt (Carneiro 2019). Anders als Grothendieck-Universen sind Leans Universen aber nicht kumulativ: Ein Typ des untersten Universums ist nicht von selbst auch einer des nächsten; man hebt ihn ausdrücklich an. Klein und groß gibt es dort genauso: In Mathlib bilden die Typen der Stufe u eine Kategorie, deren Objekte zusammen erst ein Typ der Stufe $u+1$ sind. Damit man Sätze nicht für jede Stufe neu beweisen muss, gelten Definitionen für alle Stufen zugleich. Und weil jede Stufe wie ein Universum wirkt, lässt sich in Lean ein Modell von ZFC bauen (Abschnitt 3.6) – genau das, was ZFC nach Korollar 2.58 für sich selbst nicht kann.

2.8 Kategorientheorie

Die Mengenlehre beschreibt ein Objekt über seine *Elemente*. Die Kategorientheorie lässt das weg und beschreibt es über seine *Beziehungen*.

Wer programmiert, kennt diesen Standpunkt schon. Ein Typ ist keine Menge, in die man hineinschaut, sondern etwas, wohin und woher Funktionen laufen. Die Beispiele dieses Abschnitts kommen deshalb aus der Informatik, und zwar in Haskell – dort sind die Begriffe der Kategorientheorie nicht Anwendung, sondern Sprachbestandteil.

Die Axiome

Definition 2.62 (Kategorie)**.** Eine Kategorie $\mathcal{C}$ besteht aus vier Angaben:

- einer Gesamtheit von *Objekten* $X, Y, Z, \ldots$;
- zu je zwei Objekten X, Y einer Gesamtheit von *Morphismen* $\mathrm{Hom}(X, Y)$; für $f \in \mathrm{Hom}(X, Y)$ schreibt man $f : X \to Y$;
- zu jedem Objekt X einem Morphismus $\mathrm{id}_X : X \to X$;
- einer *Komposition*: zu $f : X \to Y$ und $g : Y \to Z$ ein Morphismus $g \circ f : X \to Z$.

Dazu drei Axiome:

(K1) Assoziativität
$(h \circ g) \circ f = h \circ (g \circ f)$ für alle $f : W \to X$, $g : X \to Y$, $h : Y \to Z$.

(K2) Linksneutralität
$\mathrm{id}_Y \circ f = f$ für alle $f : X \to Y$.

(K3) Rechtsneutralität
$f \circ \mathrm{id}_X = f$ für alle $f : X \to Y$.

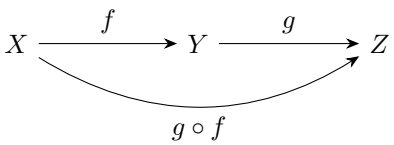
Das Grundbeispiel: Objekte sind Typen, Morphismen sind Funktionen. Dass die Komposition nur erklärt ist, wenn Ziel und Quelle zusammenpassen, erzwingt dort schon die Typsignatur:

Haskell code/Kategorien.hs

```haskell
-- Objekte sind Typen, Morphismen sind Funktionen.
-- Die Komposition ist nur erklaert, wenn Ziel und Quelle zusammenpassen --
-- genau das erzwingt die Typsignatur:
komposition :: (b -> c) -> (a -> b) -> (a -> c)
komposition g f = \x -> g (f x)          -- in Haskell schon da: (.)

identitaet :: a -> a
identitaet x = x                         -- in Haskell schon da: id
```

Aus $f : X \to Y$ und $g : Y \to Z$ wird ein einziger neuer Pfeil $X \to Z$:

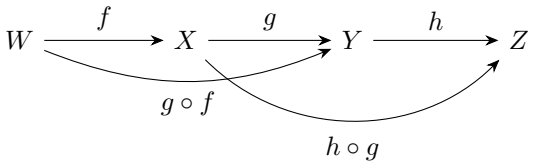


Ein Diagramm heißt *kommutativ*, wenn je zwei Wege mit gleichem Anfang und Ende denselben Morphismus ergeben. Das obige kommutiert also genau dann, wenn der untere Pfeil wirklich $g \circ f$ ist – und das ist hier die Definition.

(K1) Assoziativität. Bei einer Kette von drei Morphismen gibt es zwei Arten zu klammern, und sie liefern dasselbe:

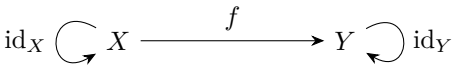
$$(h \circ g) \circ f \;=\; h \circ (g \circ f).$$

Deshalb darf man die Klammern weglassen und $h \circ g \circ f$ schreiben.



(K2) und (K3) Identitäten. Die Identität ändert nichts, egal von welcher Seite man sie anhängt:

$$\mathrm{id}_Y \circ f \;=\; f \;=\; f \circ \mathrm{id}_X.$$



In Haskell sind das drei Gleichungen über Funktionen, die man punktweise nachrechnen kann:

Haskell code/Kategorien.hs

```haskell
-- Die drei Axiome sind Gleichungen zwischen Funktionen:
--
--   (K1)  (h . g) . f  ==  h . (g . f)
--   (K2)  id . f       ==  f
--   (K3)  f . id       ==  f
--
-- Punktweise nachgerechnet -- diese Funktionen liefern immer True:
assoziativ :: Eq d => (c -> d) -> (b -> c) -> (a -> b) -> a -> Bool
assoziativ h g f x = ((h . g) . f) x == (h . (g . f)) x

neutral :: Eq b => (a -> b) -> a -> Bool
neutral f x = (id . f) x == f x && (f . id) x == f x
```

Bemerkung 2.63 (Was *nicht* gefordert wird). Auffällig ist, was in Definition 2.62 fehlt. Es wird nicht verlangt, dass Objekte Elemente haben, dass Morphismen Abbildungen sind, dass $\mathrm{Hom}(X, Y)$ nichtleer ist oder dass es zu f ein Inverses gibt. Alles, was man über Kategorien im Allgemeinen beweist, benutzt nur die drei Gleichungen (K1)–(K3).

Beispiele

Kategorie	Objekte	Morphismen
Typen und Funktionen	Datentypen	Funktionen `a -> b`
Set	Mengen	Abbildungen
$\mathbf{Vect}_K$	K-Vektorräume	lineare Abbildungen
Halbordnung $(P, \leq)$	die Elemente von P	genau ein Pfeil $x \to y$, falls $x \leq y$
Monoid M	ein einziges Objekt $*$	die Elemente von M
Zustandsautomat	Zustände	Eingabewörter, die überführen

Die unteren Zeilen sind die lehrreichen. In einer Halbordnung ist ein Morphismus kein Objekt mit Struktur, sondern nur die Tatsache $x \leq y$. Assoziativität ist dort nichts weiter als die Transitivität, die Identität die Reflexivität. Beim Automaten ist ein Morphismus ein Eingabewort, die Komposition die Hintereinanderausführung, die Identität das leere Wort.

Und ein Monoid ist eine Kategorie mit genau einem Objekt – die Morphismen sind dann die Elemente selbst:

Haskell code/Kategorien.hs

```haskell
-- Ein Monoid ist eine Kategorie mit genau einem Objekt: die Morphismen sind
-- die Elemente, die Komposition ist die Verknuepfung, die Identitaet das
-- neutrale Element. Fuer Strings:
verketten :: String -> String -> String
verketten = (++)

leer :: String
leer = ""
```

Eine Kategorie ist also nicht notwendig „Strukturen und Abbildungen“. Sie ist ein Graph mit einer assoziativen Verknüpfung von Wegen.

Bemerkung 2.64 (Eine Ungenauigkeit, ehrlichkeitshalber). „Die Kategorie der Haskell-Typen“ ist eine Idealisierung. Wegen nichtterminierender Ausdrücke und `seq` gelten die Gesetze nur bis auf solche Feinheiten. Für alles Folgende spielt das keine Rolle; gemeint ist stets die idealisierte Fassung.

Auch **Set** hat eine Feinheit, die Größe: Alle Mengen zusammen bilden keine Menge (Satz 2.24). Deshalb spricht Definition 2.62 von einer *Gesamtheit* von Objekten. Wer **Set** selbst wie eine Menge behandeln will, nimmt die Mengen eines Universums, $\mathbf{Set}_{U_0}$ (Abschnitt 2.7, „Klein und groß“).

Funktoren

Definition 2.65 (Funktor)**.** Ein *Funktor* $F : \mathcal{C} \to \mathcal{D}$ ordnet jedem Objekt X von $\mathcal{C}$ ein Objekt FX von $\mathcal{D}$ zu und jedem Morphismus $f : X \to Y$ einen Morphismus $Ff : FX \to FY$, sodass

$$F(\mathrm{id}_X) = \mathrm{id}_{FX}, \qquad F(g \circ f) = Fg \circ Ff.$$

Ein Funktor bildet also nicht nur Objekte ab, sondern auch Pfeile – und zwar verträglich mit der Komposition:

$$X \xrightarrow{f} Y \xrightarrow{g} Z \qquad \longmapsto \qquad FX \xrightarrow{Ff} FY \xrightarrow{Fg} FZ$$

In Haskell ist das keine Analogie, sondern die Definition. Die Typklasse `Functor` verlangt genau die Abbildung auf Pfeilen, und die beiden Funktorgesetze heißen dort *functor laws*:

Haskell code/Kategorien.hs

```haskell
-- Ein Funktor ist in Haskell eine Typklasse -- woertlich die Definition
-- aus dem Skript:
--
--   class Functor f where
--     fmap :: (a -> b) -> f a -> f b
--
-- mit den beiden Gesetzen
--   fmap id       ==  id
--   fmap (g . f)  ==  fmap g . fmap f

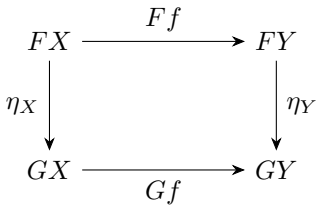
-- Ein eigener Funktor: binaere Baeume.
data Baum a = Blatt | Knoten (Baum a) a (Baum a)

instance Functor Baum where
  fmap _ Blatt          = Blatt
  fmap h (Knoten l x r) = Knoten (fmap h l) (h x) (fmap h r)
```

F ist hier ein Typkonstruktor wie `Maybe`, `[]` oder `Baum`, und Ff ist `fmap f`. Ein Funktor ist damit keine exotische Konstruktion, sondern das, was jeder Container anbietet: „wende eine Funktion auf den Inhalt an, lass die Form unverändert“.

Natürliche Transformationen

Definition 2.66 (Natürliche Transformation)**.** Seien $F, G : \mathcal{C} \to \mathcal{D}$ Funktoren. Eine *natürliche Transformation* $\eta : F \Rightarrow G$ ist eine Familie von Morphismen $\eta_X : FX \to GX$, eine für jedes Objekt X, sodass für jedes $f : X \to Y$ das folgende Quadrat kommutiert:



In Formeln: $\eta_Y \circ Ff = Gf \circ \eta_X$. Die Bedingung sagt, dass die Familie η nicht für jedes Objekt willkürlich gewählt werden darf, sondern mit allen Morphismen verträglich sein muss. „Natürlich“ heißt: ohne Wahl, gleichmäßig in X.

Das Musterbeispiel ist eine Funktion, die jeder Programmierer kennt – sie geht vom Funktor `[]` zum Funktor `Maybe`:

Haskell code/Kategorien.hs

```haskell
-- Eine natuerliche Transformation zwischen den Funktoren [] und Maybe:
sicherKopf :: [a] -> Maybe a
sicherKopf []      = Nothing
sicherKopf (x : _) = Just x

-- Das Naturalitaetsquadrat besagt
--   sicherKopf . fmap f  ==  fmap f . sicherKopf
-- Beide Seiten haben den Typ [a] -> Maybe b:
natuerlich :: Eq b => (a -> b) -> [a] -> Bool
natuerlich f xs = (sicherKopf . fmap f) xs == (fmap f . sicherKopf) xs
```

Bemerkung 2.67 (Natürlichkeit geschenkt). In Haskell muss man die Naturalität nicht nachweisen. Eine Funktion vom Typ `[a] -> Maybe a` ist parametrisch polymorph: Sie kann über die Elemente nichts wissen und darf sie deshalb nur umschichten, wegwerfen oder auswählen. Das Naturalitätsquadrat folgt daraus automatisch – ein *free theorem* im Sinne von Wadler (1989). Was in der Mathematik eine Bedingung ist, ist hier eine Folge des Typs.

Elemente kommen in alldem nicht vor. Man kann sie aber zurückgewinnen: In **Set** entsprechen die Elemente von X genau den Abbildungen von der einpunktigen Menge nach X. Was in der Mengenlehre ein Grundbegriff ist, wird hier zu einem abgeleiteten.

Warum Beziehungen genügen: das Yoneda-Lemma

Damit stellt sich eine Frage. Wenn man nur noch Morphismen anschaut und die Elemente weglässt – verliert man dann nicht Information?

Die Antwort ist nein, und der Satz, der das sagt, ist das Yoneda-Lemma. Es ist der Hauptsatz der Kategorientheorie.

Die Beziehungen eines Objekts X fasst man zu einem Funktor zusammen:

$$h_X := \mathrm{Hom}(-, X) \;:\; \mathcal{C}^{\mathrm{op}} \longrightarrow \mathbf{Set}, \qquad Z \longmapsto \mathrm{Hom}(Z, X).$$

Er sagt für jedes Objekt Z, wie man von Z nach X kommt – also genau das, was X von außen ausmacht.

Satz 2.68 (Yoneda). *Sei $\mathcal{C}$ eine Kategorie, X ein Objekt und $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$ ein Funktor. Dann gibt es eine Bijektion*

$$\mathrm{Nat}(h_X, F) \;\cong\; F(X),$$

und zwar natürlich in X und F.

Beweisskizze. Einer natürlichen Transformation $\eta : h_X \Rightarrow F$ ordne man $\eta_X(\mathrm{id}_X) \in F(X)$ zu. Umgekehrt definiere man zu $u \in F(X)$ die Transformation

$$\eta^u_Z : \mathrm{Hom}(Z, X) \to F(Z), \qquad f \longmapsto F(f)(u).$$

Beide Zuordnungen sind zueinander invers. In der einen Richtung ist $\eta^u_X(\mathrm{id}_X) = F(\mathrm{id}_X)(u) = u$. In der anderen liefert die Natürlichkeit von η für $f : Z \to X$ $F(f)(\eta_X(\mathrm{id}_X)) = \eta_Z(\mathrm{id}_X \circ f) = \eta_Z(f)$. □

Der Beweis ist kurz, die Aussage aber folgenreich. Der springende Punkt ist id_X: Die ganze natürliche Transformation steckt schon darin, wohin sie die Identität schickt. id_X ist sozusagen das generische Element von h_X.

Korollar 2.69 (Yoneda-Einbettung). *Die Zuordnung $X \mapsto h_X$ ist voll und treu. Insbesondere gilt*

$$h_X \cong h_Y \quad \Longrightarrow \quad X \cong Y.$$

Beweis. Man wende Satz 2.68 auf $F := h_Y$ an. Er liefert $\mathrm{Nat}(h_X, h_Y) \cong h_Y(X) = \mathrm{Hom}(X, Y)$, also eine Bijektion zwischen den natürlichen Transformationen und den Morphismen. Unter dieser Bijektion entsprechen Isomorphismen einander. □

Ein Objekt ist, was man mit ihm messen kann. Man denke an ein Bauteil, das man nicht öffnen darf. Prüfen kann man es nur von außen: Man schickt Testsignale hinein und beobachtet, was herauskommt. $h_X = \mathrm{Hom}(-, X)$ ist genau die Sammlung aller dieser Tests.

Das Yoneda-Lemma sagt zwei Dinge. Erstens: Eine Familie von Antworten, die mit allen Tests verträglich ist, wird schon durch *eine* Antwort festgelegt, nämlich durch die auf den trivialen Test id_X. Zweitens: Zwei Bauteile, die auf alle Tests gleich reagieren, sind ununterscheidbar – also isomorph.

Kurz: Was man von außen nicht unterscheiden kann, ist auch nicht verschieden. Deshalb darf man aufhören, hineinzuschauen.

Deshalb darf man sich auf Beziehungen beschränken. Ein Objekt ist bis auf Isomorphie dadurch festgelegt, wie alle anderen Objekte auf es abbilden. Wer h_X kennt, kennt X. Man darf ein Objekt also durch seine universelle Eigenschaft definieren – durch Produkte, Limites, adjungierte Funktoren – und weiß, dass dadurch höchstens ein Objekt beschrieben wird.

Die Beobachtung von eben ist der einfachste Fall davon: Die Elemente von X sind die Morphismen aus dem terminalen Objekt, also ein einzelner Wert des Funktors h_X.

In Haskell lässt sich das Lemma sogar als Typ hinschreiben. Satz 2.68 besagt dort

`(forall b. (a -> b) -> f b)` ist dasselbe wie `f a`,

und die Hinrichtung ist wörtlich der Beweis von oben: auf die Identität anwenden.

Haskell code/Kategorien.hs

```haskell
-- Das Yoneda-Lemma, als Typ lesbar:
--
--   (forall b. (a -> b) -> f b)   ist dasselbe wie   f a
--
newtype Yoneda f a = Yoneda (forall b. (a -> b) -> f b)

-- Hin: auf die Identitaet anwenden -- genau der Beweis aus dem Skript.
aus :: Yoneda f a -> f a
aus (Yoneda k) = k identitaet

-- Zurueck: mit fmap nachziehen.
nach :: Functor f => f a -> Yoneda f a
nach fa = Yoneda (\k -> fmap k fa)
```

Wer eine Funktion hat, die für *jedes* `b` aus `a -> b` ein `f b` macht, hat damit nicht mehr und nicht weniger als ein `f a`. Die ganze Familie steckt in dem einen Wert, den sie der Identität zuordnet.

Bemerkung 2.70 (Warum das für Mathlib wichtig ist). Kategorientheorie ist in Mathlib kein Nebenthema. Die algebraischen Kategorien, Limites und Kolimites, adjungierte Funktoren und Garben sind durchweg kategoriell gebaut. Universelle Eigenschaften ersetzen dabei das Rechnen mit Elementen. Das macht Beweise oft kürzer und allgemeiner.

Zusammenfassung

- Uneingeschränkte Komprehension ist widersprüchlich (Satz 2.4). Der Grund ist rein logisch.
- Nach Cantor gibt es keine Menge aller Mengen (Satz 2.6).
- Ein formales System ersetzt Einsicht durch Nachprüfen (Definition 2.7): $\Sigma \vdash \varphi$ heißt, es gibt eine Herleitung – im Hilbert-Kalkül eine Folge, im natürlichen Schließen ein Baum. Der λ-Kalkül ist ein formales System ohne Logik.

- ZFC repariert die naive Mengenlehre, indem es die Komprehension einschränkt: Man darf nur aus vorhandenen Mengen aussondern.
- Das Auswahlaxiom behauptet eine Auswahlfunktion, ohne sie anzugeben (Definition 2.25); gleichwertig: Jede Surjektion hat ein Rechtsinverses (Satz 2.26).
- ZFC ist klassisch. Intuitionistisch ist das tertium non datur kein Gesetz, aber auch nicht widerlegbar (Satz 2.29). Konstruktive Beweise liefern ihre Objekte mit. Aus der Auswahl folgt, zusammen mit Extensionalität, Leermenge, Paarmenge und Aussonderung, das tertium non datur (Satz 2.32); in Lean übernehmen `propext` und `funext` die Rolle der Extensionalität.
- Semantik ist eine Abbildung $[\![\cdot]\!]$ von der Syntax in die Mengen: Eine Formel bedeutet eine Teilmenge von M^n, eine Aussage *wahr* oder *falsch* (Definition 2.37). Korrektheit und Vollständigkeit verbinden $\vdash$ und $\models$; eine Theorie ist genau dann widerspruchsfrei, wenn sie ein Modell hat (Satz 2.43). Unabhängig heißt: zwei Modelle (Korollar 2.45) – so beim Auswahlaxiom und bei der Kontinuumshypothese, falls die Mengenlehre widerspruchsfrei ist. Hat eine Theorie erster Stufe in abzählbarer Signatur ein unendliches Modell, so auch ein abzählbares (Satz 2.48) und beliebig große: Ihre Axiome legen die Größe der Modelle nicht fest. ZFC entscheidet nicht jede Frage und beweist, falls widerspruchsfrei, die eigene Widerspruchsfreiheit nicht (Satz 2.51); mehr als relative Widerspruchsfreiheit ist nicht zu haben.
- NBG lässt jede Formel zu, aber sie liefert nur eine *Klasse*. Mengen sind die Klassen, die selbst Element sein können. Die Russell-Klasse existiert und ist eine echte Klasse (Satz 2.54).
- Zu große Gesamtheiten fangen Klassen, Grothendieck-Universen oder Typuniversen auf. Ein Universum mit $\mathbb{N}$ ist ein Modell von ZFC; ist ZFC widerspruchsfrei, so ist seine Existenz in ZFC nicht beweisbar (Korollar 2.58). Das Universenaxiom liefert eine kumulative Kette $\mathbb{N} \in U_0 \in U_1 \in U_2 \in \cdots$. Die Kategorie $\mathbf{Set}_{U_0}$ der Mengen aus U_0 ist bezüglich U_0 groß, bezüglich U_1 klein. ZFC mit $n+1$ Universen, die $\mathbb{N}$ enthalten, beweist die Widerspruchsfreiheit von ZFC mit n solchen Universen (Satz 2.61); die eigene beweist keine dieser Theorien, sofern sie widerspruchsfrei ist (Satz 2.51).
- Die Kategorientheorie beschreibt Objekte über Morphismen statt über Elemente.
- Wie das alles in Lean aussieht, steht in Abschnitt 3.6.

Kapitel 3

Typentheorie

Kapitel 2 hat zwei klassische Antworten auf das Komprehensionsproblem gezeigt. ZFC schränkt ein, *was man bilden darf*; NBG schränkt ein, *was Element sein darf*. Es gibt eine dritte Antwort, und auf ihr beruht Mathlib: die *Typentheorie*. Sie schränkt ein, *worüber eine Formel überhaupt reden kann*.

Der Unterschied zu ZFC ist der Zeitpunkt:

ZFC	**Typentheorie**
Ersetzt Komprehension durch *Aussonderung*: Man darf nur aus einer Menge aussondern, die es schon gibt.	Verlangt, dass jede Komprehension einen *Typ* angibt, über den sie läuft.
Ein Axiomenschema löst das Problem. Unzulässige Bildungen sind gültige, aber unbeweisbare Aussagen.	Die Syntax löst das Problem. Unzulässige Bildungen sind gar keine Ausdrücke.

Dieses Kapitel erklärt, wie das funktioniert und warum es als Grundlage taugt.

Ergänzend. Eine ausführliche Einführung in die Taktiksprache mit vielen Beispielen steht im Verzeichnis `LeanMath/basics/lean_typetheory_curry_howard_intro.lean`. Hier geht es um die Typentheorie als *Fundament*.

3.1 Terme, Typen, Universen

Die Idee ist einfach: Es gibt *Terme*, und jeder Term hat genau einen *Typ*. Man schreibt $t : T$ für „t ist ein Term vom Typ T“. Anders als in der Mengenlehre ist $t : T$ keine Aussage, über die man streiten kann, sondern eine Eigenschaft der Syntax. Entweder der Typcheck geht durch, oder der Ausdruck ist keiner.

Typen sind selbst Terme und haben deshalb selbst Typen. Sie stehen dabei in einer Hierarchie von *Universen*: `Nat : Type`, `Type : Type 1`, `Type 1 : Type 2` und so weiter. `Type` ohne Index ist dabei `Type 0`.

Eine Stufe fällt aus der Reihe: `Prop`, die Stufe der *Aussagen*. Aussagen sind Typen wie andere auch – aber was in ihnen steht, verhält sich anders.

Lean `code/Kapitel3.lean`

```
/- Jeder Ausdruck hat einen Typ; es gibt kein untypisiertes "Universum". -/
#check (3 : ℕ)
#check ℕ
#check Type
#check Prop
```

```
/- Typen leben selbst wieder in Typen, gestaffelt in Universen. -/
#check Type       -- Type : Type 1
#check Type 1     -- Type 1 : Type 2
example : Type = Type 0 := rfl

/- Eine Stufe ist besonders: ‘Prop‘, die Stufe der Aussagen.
   Aussagen sind Typen wie andere auch -- man kann ueber sie quantifizieren. -/
#check Prop
example (p q : Prop) : Prop := p → q
```

Beweiseindeutigkeit. Der Unterschied lässt sich in einem Satz sagen: *Von einem Beweis zählt nur, dass es ihn gibt, nicht welcher er ist.*

Eine Aussage kann man auf viele Arten beweisen. In Lean sind alle diese Beweise *derselbe* Term – und zwar definitorisch, sodass `rfl` genügt:

Lean code/Kapitel3.lean

```
/- In ‘Prop‘ zaehlt nur, *dass* es einen Beweis gibt, nicht welchen.
   Je zwei Beweise derselben Aussage sind definitorisch gleich: -/
example (p : Prop) (h₁ h₂ : p) : h₁ = h₂ := rfl

/- Das gilt auch, wenn man sie sichtbar verschieden herstellt. -/
example : 0 < 2 := by norm_num
example : 0 < 2 := by omega
example (h₁ h₂ : 0 < 2) : h₁ = h₂ := rfl

/- Das greift weiter, als es zunaechst aussieht: ‘a = b‘ ist selbst eine
   ‘Prop‘. Also sind auch je zwei *Gleichheitsbeweise* gleich -- das nennt
   man UIP, uniqueness of identity proofs. -/
example {α : Type} {a b : α} (p q : a = b) : p = q := rfl

/- Bei Daten ist es umgekehrt: verschiedene Werte bleiben verschieden. -/
example : (0 : ℕ) ≠ 1 := by decide
example (b : Bool) : b = true ∨ b = false := by cases b <;> simp
```

Bei Daten ist es umgekehrt. `0` und `1` sind verschieden, und darauf beruht jede Rechnung. Genau deshalb trennt Lean die beiden Stufen: `Prop` für Dinge, bei denen nur die Existenz zählt, `Type` für Dinge, die man unterscheiden können muss.

Quittung und Geldschein. Zwei Geldscheine über zehn Euro sind zwei Scheine. Man kann sie auseinanderhalten, sie haben verschiedene Seriennummern, und es macht einen Unterschied, welchen man hat.

Zwei Quittungen über dieselbe Zahlung sind dagegen nur *ein* Vorgang. Ob der Beleg getippt oder handgeschrieben ist, spielt keine Rolle – bezahlt ist bezahlt. Genau so verhält sich `Prop`.

Bemerkung 3.1 (Der andere Weg: Univalenz und HoTT). Dass Beweise eindeutig sind, ist eine *Entwurfsentscheidung*, keine Notwendigkeit. Man kann es auch genau andersherum machen.

In der *homotopischen Typentheorie* (HoTT, nach Voevodsky) ist $a = b$ selbst ein Typ, der Struktur haben darf: Es kann mehrere, voneinander verschiedene Gleichheiten geben. Man liest Typen als Räume und Gleichheiten als Wege zwischen Punkten. Beweiseindeutigkeit ist dort keine Vorschrift, sondern eine *Eigenschaft*, die ein Typ haben kann oder nicht: Ein Typ heißt *Proposition*, wenn je zwei seiner Elemente gleich sind.

Das *Univalenzaxiom* sagt dann

$$(A \simeq B) \;\simeq\; (A = B),$$

also: Äquivalente Typen *sind* gleich. Damit wird das, was Mathematiker ohnehin tun – isomorphe Strukturen miteinander identifizieren – zu einer Rechenregel statt zu Handarbeit.

Beides zusammen geht nicht. In Lean ist $a = b$ eine `Prop`, also sind je zwei Gleichheitsbeweise gleich (UIP, siehe oben). Univalenz widerspricht dem: Es gibt zwei verschiedene Äquivalenzen Bool $\simeq$ Bool, die Identität und die Negation – also müsste Bool $=$ Bool zwei verschiedene Elemente haben.

Mathlib erledigt den Transport entlang von Isomorphien deshalb von Hand, über `Equiv` und passende Lemmas. Das ist Arbeit, die Univalenz geschenkt gäbe, und einer der meistdiskutierten Nachteile dieses Entwurfs. Wer es ausprobieren will: In *Cubical Agda* hat Univalenz sogar Rechenverhalten.

Bemerkung 3.2 (Was das praktisch bedeutet). Zwei Dinge folgen daraus. Erstens darf man beim Übersetzen alle Beweise löschen: Kein Programm kann sie unterscheiden, also trägt keines Information über sie. Zweitens muss man nie fragen, *welchen* Beweis man hat – eine Sorge weniger, die in anderen Systemen echte Arbeit macht.

Der Preis: Aus einem Beweis von $\exists$ `n, p n` lässt sich das n nicht herausrechnen, denn dazu müsste man den Beweis ansehen. Wer das braucht, formuliert die Aussage in `Type` statt in `Prop`.

Die Staffelung ist kein Schönheitsfehler, sondern nötig. Sie ist das genaue Gegenstück zu Satz 2.6: Wie es keine Menge aller Mengen gibt, so ist auch die Stufe aller Typen nie ein Objekt ihrer eigenen Stufe. Jede Stufe `Type u` spielt dabei die Rolle eines Grothendieck-Universums (Definition 2.56): abgeschlossen unter den üblichen Konstruktionen, aber selbst erst eine Stufe höher ein Objekt.

Bemerkung 3.3 (Girards Paradoxon). Ließe man zu, dass die Stufe aller Typen ein Objekt ihrer selbst ist, wäre das System widersprüchlich – man könnte `False` herleiten. Girard zeigte das 1972. Die Universenhierarchie ist also nicht Vorsicht, sondern Notwendigkeit.

Damit man einen Satz nicht für jede Universenstufe neu beweisen muss, sind Definitionen *universenpolymorph*:

Lean code/Kapitel3.lean

```
universe u v

/-- Definitionen koennen ueber Universen polymorph sein: ‘u‘ ist eine
Universenvariable, kein Wert, den man ausrechnen koennte. -/
def meineId {α : Type u} (a : α) : α := a

#check @meineId.{u}
#check @meineId.{0}
#check @meineId.{1}

example : ℕ := meineId 3
example : Type := meineId ℕ
```

Abhängige Typen

Ein Typ darf von einem *Wert* abhängen. Das ist der Hauptvorteil. Der Typ `Fin n` der Zahlen $< n$ hängt von n ab. Eine Funktion, die zu n ein Element von `Fin n` liefert, hat deshalb einen *abhängigen Funktionstyp* `(n : ℕ) → Fin n`.

Lean code/Kapitel3.lean

```
/-- Ein **abhaengiger** Funktionstyp: der Ergebnistyp haengt vom Argument ab. -/
def diagonale : (n : ℕ) → (Fin n → Fin n) := fun _ i => i

/-- Laenge im Typ: ‘Vektor α n‘ kennt seine eigene Laenge. -/
inductive Vektor (α : Type u) : ℕ → Type u where
  | nil : Vektor α 0
```

```
  | cons : α → {n : ℕ} → Vektor α n → Vektor α (n + 1)

/-- 'head' ist nur fuer nichtleere Vektoren definiert -- garantiert vom Typsystem,
nicht durch eine Laufzeitpruefung. -/
def Vektor.head {α : Type u} {n : ℕ} : Vektor α (n + 1) → α
  | .cons a _ => a


#eval Vektor.head (Vektor.cons 7 Vektor.nil)
example : Vektor.head (Vektor.cons 7 Vektor.nil) = 7 := rfl
```

An `Vektor.head` sieht man, wozu das gut ist. Die Funktion ist nur für nichtleere Vektoren definiert, und das steht im *Typ*. Ein Aufruf auf dem leeren Vektor ist kein Laufzeitfehler, sondern ein Typfehler. Er kommt gar nicht zustande.

Bemerkung 3.4. Mengentheoretisch ist ein abhängiger Funktionstyp ein Produkt $\prod_{x \in A} B(x)$ und ein abhängiges Paar eine disjunkte Vereinigung $\sum_{x \in A} B(x)$. Daher die Namen Π-Typ und Σ-Typ.

3.2 Induktive Typen, Strukturen, Typklassen

Fast alle Typen in Lean sind *induktiv*. Man gibt die Konstruktoren an, und Lean baut daraus den Rekursor. Der ist Rekursions- und Induktionsprinzip in einem.

Lean code/Kapitel3.lean

```
/-- Ein induktiver Typ wird durch seine Konstruktoren erzeugt. -/
inductive MeinNat where
  | zero : MeinNat
  | succ : MeinNat → MeinNat
  deriving Repr

def MeinNat.add : MeinNat → MeinNat → MeinNat
  | n, .zero => n
  | n, .succ m => .succ (MeinNat.add n m)

def zro : MeinNat := MeinNat.zero
#check MeinNat.succ zro

#eval MeinNat.succ zro
/- Der automatisch erzeugte Rekursor ist zugleich das Induktionsprinzip. -/
#check @MeinNat.rec

theorem MeinNat.add_zero (n : MeinNat) : n.add .zero = n := rfl

theorem MeinNat.zero_add (n : MeinNat) : MeinNat.zero.add n = n := by
  induction n with
  | zero => rfl
  | succ k ih =>                                  -- kurz: simp [MeinNat.add, ih]
    change (MeinNat.zero.add k).succ = k.succ     -- per Definition von add
    rw [ih]                                       -- Induktionsannahme ih : 0+k = k
```

Derselbe Rekursor erzeugt Programme und Beweise. Das ist kein Zufall, sondern genau der Inhalt des nächsten Abschnitts.

Eine *Struktur* ist ein induktiver Typ mit einem Konstruktor. Eine *Typklasse* ist eine Struktur, deren Instanzen Lean selbst sucht. Darauf beruht der Aufbau von Mathlib: Die ganze algebraische Hierarchie, von `Semigroup` über `Ring` bis `Field`, besteht aus Typklassen, die auseinander hervorgehen.

Lean code/Kapitel3.lean

```
/-- Typklassen sind Strukturen plus automatische Instanzsuche.
Mathlibs gesamte algebraische Hierarchie ist so gebaut. -/
class MeinMonoid (α : Type) where
  op : α → α → α
  e : α
  assoc : ∀ a b c : α, op (op a b) c = op a (op b c)
  e_op : ∀ a : α, op e a = a
  op_e : ∀ a : α, op a e = a

instance : MeinMonoid ℕ where
  op := (· + ·)
  e := 0
  assoc := Nat.add_assoc
  e_op := Nat.zero_add
  op_e := Nat.add_zero

/-- Ein Satz, der fuer *jedes* Monoid gilt -- Instanzsuche liefert die Struktur. -/
theorem op_e_op {α : Type} [M : MeinMonoid α] (a : α) :
    M.op (M.op a M.e) M.e = a := by
  rw [M.op_e, M.op_e]

/- So findet Lean die echten Mathlib-Strukturen: -/
#check (inferInstance : AddGroup ℤ)
#check (inferInstance : CommRing ℤ)
#check (inferInstance : Field ℝ)
/- Auch '/' gehoert zu einer Typklasse. Welche Instanz gilt, entscheidet der Typ:
    -/
example : (7 : ℕ) / 2 = 3 := by decide        -- Division mit Rest
example : (7 : ℚ) / 2 = 3.5 := by norm_num    -- Bruchrechnung
```

Mathlib. Die eckigen Klammern in `[M : MeinMonoid α]` markieren ein Instanzargument: Lean sucht es selbst. Deshalb muss man beim Rechnen in $\mathbb{R}$ nie dazusagen, woher `+` und `*` kommen. Auch `/` ist eine Typklasse: In $\mathbb{N}$ findet Lean die Division mit Rest, in $\mathbb{Q}$ die Bruchrechnung.

3.3 Proof as Proposition: die Curry–Howard-Korrespondenz

Jetzt die Hauptidee. Logik und Typentheorie sind nicht zwei Dinge, sondern eines:

Logik	Typentheorie	in Lean
Aussage A	Typ A	`A : Prop`
Beweis von A	Term vom Typ A	`h : A`
$A \to B$	Funktionstyp	`A → B`
$A \wedge B$	Produkt	`A ∧ B`
$A \vee B$	Summe	`A ∨ B`
$\neg A$	Funktion in den leeren Typ	`A → False`
$\forall x, P(x)$	abhängiger Funktionstyp	`∀ x, P x`
$\exists x, P(x)$	abhängiges Paar	`∃ x, P x`

Ein Beweis ist also ein Programm. Einen Satz beweisen heißt, einen Term des passenden Typs angeben. Und einen Beweis prüfen heißt: den Term typchecken.

Lean code/Kapitel3.lean

```
/- Derselbe Term, zwei Lesarten: einmal als Programm, einmal als Beweis. -/
def konstante {α β : Type} : α → β → α := fun a _ => a
theorem abschwaechung {A B : Prop} : A → B → A := fun a _ => a

/- Die Junktoren sind Typkonstruktionen: -/
example (P : Prop) : (¬ P) = (P → False) := rfl          -- Negation = Funktion
    nach False
example (A B : Prop) (h : A ∧ B) : A := h.1              -- Konjunktion = Produkt
example (A B : Prop) (h : A ∨ B) : B ∨ A :=              -- Disjunktion = Summe
  h.elim Or.inr Or.inl
example (p : ℕ → Prop) (h : ∃ n, p n) : ∃ m, p m := h    -- Existenz = abh. Paar

/- Ein Existenzbeweis *ist* ein Paar aus Zeuge und Nachweis. -/
example : ∃ n : ℕ, 3 < n := ⟨4, by norm_num⟩
```

Der erste Block zeigt das am deutlichsten. `konstante` und `abschwaechung` haben denselben Term `fun a _ => a`. Einmal liest man ihn als Funktion, die das erste Argument zurückgibt. Einmal als Beweis, dass aus A und B wieder A folgt.

Warum `Prop` von `Type` getrennt ist

Wenn Beweise Terme sind – wozu dann die eigene Stufe `Prop`? Weil Beweise etwas haben, was Daten nicht haben: Sie sind irrelevant.

Satz 3.5 (Beweisirrelevanz)**.** *Sind h_1, h_2 Beweise derselben Aussage $P : \mathrm{Prop}$, so gilt $h_1 = h_2$, und zwar definitorisch.*

Lean code/Kapitel3.lean

```
/- **Beweisirrelevanz**: je zwei Beweise derselben Aussage sind
   *definitorisch* gleich. Beweise tragen keine Information. -/
example (P : Prop) (h₁ h₂ : P) : h₁ = h₂ := rfl

/- Fuer Daten gilt das natuerlich nicht: -/
example : (0 : ℕ) ≠ 1 := by decide

/- Deshalb trennt Lean ‘Prop‘ (beweisirrelevant, loeschbar)
   von ‘Type‘ (Daten, rechenrelevant). -/
```

Das hat zwei Folgen. Mathematisch: Es ist egal, *wie* man etwas bewiesen hat, nur *dass*. Praktisch: Beweise kann man beim Übersetzen löschen, denn kein Programm kann sie unterscheiden. Bei Daten geht das nicht – `0` und `1` sind verschieden, und darauf beruht jede Rechnung.

3.4 Gleichheit und die Axiome

Lean hat zwei Gleichheitsbegriffe, und der Unterschied ist wichtig.

Definitorische Gleichheit
Zwei Terme sind gleich, weil der Kern sie durch Rechnen ineinander überführt. Der Beweis ist `rfl` und kostet nichts. Beispiel: `2 + 2 = 4`.

Propositionale Gleichheit
Zwei Terme sind gleich, weil es einen Beweis gibt. Beispiel: `0 + n = n` für variables `n`. Hier muss man induzieren, denn `Nat.add` rekursiert über das *zweite* Argument.

Lean code/Kapitel3.lean

```
/- Definitorische Gleichheit: der Kern rechnet selbst. -/
example : 2 + 2 = 4 := rfl
example (n : ℕ) : n + 0 = n := rfl

/- Propositionale Gleichheit: braucht einen Beweis per Induktion. -/
example (n : ℕ) : 0 + n = n := Nat.zero_add n

/- Funktionale und propositionale Extensionalitaet sind Saetze bzw. Axiome. -/
#check @funext
#check @propext
#check @Quot.sound
```

Über die reine Typentheorie hinaus braucht Mathlib sehr wenig – genau drei Axiome:

`propext`	äquivalente Aussagen sind gleich
`Classical.choice`	Auswahl (liefert klassische Logik)
`Quot.sound`	Quotienten identifizieren äquivalente Elemente

Lean code/Kapitel3.lean

```
/- Ganz Mathlib ruht auf genau drei Axiomen ueber der Typentheorie hinaus:
   'propext', 'Classical.choice', 'Quot.sound'. -/
#print axioms Nat.zero_add
#print axioms Classical.em
#print axioms Function.cantor_surjective

/- Aus der Auswahl folgt klassisch das Prinzip vom ausgeschlossenen Dritten
   (Satz von Diaconescu) -- konstruktiv ist es nicht beweisbar. -/
#check @Classical.em
example (P : Prop) : P ∨ ¬ P := Classical.em P
```

Bemerkung 3.6 (Konstruktiv oder klassisch?). Die Typentheorie selbst ist konstruktiv. $P \vee \neg P$ ist nicht beweisbar, denn dafür müsste man für jedes P entscheiden, welche Seite gilt. Leans Standardbibliothek deklariert `Classical.choice`, und Mathlib benutzt es durchgehend. Daraus folgt der Satz vom ausgeschlossenen Dritten (`Classical.em`), mit Diaconescus Argument aus Satz 2.32. Dafür braucht man die beiden anderen Axiome:

- `propext`, die *Aussagenextensionalität*: Aus `P` $\leftrightarrow$ `Q` folgt `P = Q`.
- `Quot.sound`, das Quotientenaxiom. Aus ihm beweist Lean die *Funktionsextensionalität* `funext`: Stimmen `f` und `g` an jeder Stelle überein, so ist `f = g`.

Zusammen übernehmen `propext` und `funext` genau den Schritt, für den die Mengenlehre die Extensionalität braucht: Gilt P, so sind die beiden Prädikate U und V aus dem Beweis gleich. Die übrigen Voraussetzungen des Satzes – Leermenge, Paarmenge, Aussonderung – sind hier umsonst. Aussondern heißt, ein Prädikat hinzuschreiben, und ein Prädikat ist einfach eine Funktion nach `Prop`. Die Rolle der Menge $\{0,1\}$ spielt `Prop` selbst; statt $0 \neq 1$ genügt `True` $\neq$ `False`.

Nachgebaut sieht das so aus. Der Beweis benutzt nur `Classical.choose` (gebaut aus `Classical.choice`), `propext` und `funext`, dazu die Beweisirrelevanz (Satz 3.5). Sie gilt definitorisch, ist also kein Axiom, und sorgt dafür, dass `Classical.choose` aus gleichen Prädikaten dasselbe Element wählt – das Gegenstück zu $f(U) = f(V)$ aus $U = V$:

Lean code/Kapitel3.lean

```
/- Diaconescus Argument, nachgebaut wie 'Classical.em' in Leans
   Standardbibliothek. 'Prop' spielt die Rolle der Menge {0, 1};
   statt 0 ≠ 1 genuegt 'True ≠ False'. -/
```

```
theorem tertium_non_datur (p : Prop) : p ∨ ¬ p :=
  -- Aussonderung kostet nichts: ein Praedikat ist eine Funktion nach Prop.
  let U : Prop → Prop := fun x => x = True ∨ p
  let V : Prop → Prop := fun x => x = False ∨ p
  have hU : ∃ x, U x := ⟨True, Or.inl rfl⟩
  have hV : ∃ x, V x := ⟨False, Or.inl rfl⟩
  -- Auswahl (Classical.choice): aus U und aus V je ein Element waehlen.
  let u := Classical.choose hU
  let v := Classical.choose hV
  have hu : U u := Classical.choose_spec hU
  have hv : V v := Classical.choose_spec hV
  -- Extensionalitaet: gilt p, so ist U = V (erst propext, dann funext) ...
  have gleich : p → u = v := fun hp =>
    have hUV : U = V :=
      funext fun _ => propext ⟨fun _ => Or.inr hp, fun _ => Or.inr hp⟩
    -- ... und aus gleichen Praedikaten waehlt choose dasselbe Element;
    -- die beiden Existenzbeweise sind gleich (Beweisirrelevanz).
    have h : ∀ (e₁ : ∃ x, U x) (e₂ : ∃ x, V x),
        Classical.choose e₁ = Classical.choose e₂ := by
      rw [hUV]; intros; rfl
    h hU hV
  -- Fallunterscheidung wie im Beweis des Satzes von Diaconescu.
  match hu, hv with
  | Or.inr hp, _ => Or.inl hp
  | _, Or.inr hp => Or.inl hp
  | Or.inl hut, Or.inl hvf => Or.inr fun hp =>
      -- u = True und v = False; aus p folgte True = u = v = False.
      true_ne_false (hut.symm.trans ((gleich hp).trans hvf))

#print axioms tertium_non_datur
```

`#print axioms` meldet dafür genau `propext`, `Classical.choice` und `Quot.sound` – das letzte über `funext`. So sieht man für jeden Satz nach, worauf er wirklich beruht. Viele elementare Sätze brauchen gar kein Axiom.

3.5 Inferenzregeln und Typprüfung

Bisher wurde über Typen nur ungefähr geredet. Dieser Abschnitt sagt genau, nach welchen Regeln sich entscheidet, ob ein Ausdruck einen Typ hat – und wie Lean das tatsächlich prüft.

Urteile und Kontexte

Eine Typentheorie wird durch *Urteile* beschrieben. Das wichtigste ist

$$\Gamma \vdash t : A,$$

gelesen: „Im Kontext Γ hat der Term t den Typ A.“ Dabei ist der *Kontext* eine Liste von Annahmen

$$\Gamma = x_1 : A_1,\ x_2 : A_2,\ \ldots,\ x_n : A_n,$$

in der jedes A_i nur die vorherigen Variablen benutzen darf. Daneben gibt es das Urteil $\Gamma \vdash A \equiv B$ für definitorische Gleichheit.

Genau nach dem ersten Urteil fragt `#check`:

Lean code/Kapitel3.lean

```
/- Ein Urteil hat die Form ‘Γ ⊢ t : A‘: Im Kontext ‘Γ‘ hat der Term ‘t‘
   den Typ ‘A‘. Genau danach fragt ‘#check‘. -/
```

```
#check (3 : ℕ)
#check (fun x : ℕ => x + 1)
#check ℕ
#check Type

/- Der Kontext sind die Annahmen links vom Doppelpunkt. -/
example (x : ℕ) (y : ℕ) : ℕ := x + y
```

Die Regeln

Die Regeln haben die Form „aus den Prämissen oben folgt die Konklusion unten“. Es sind nur wenige.

In den Regeln steht $\mathrm{Sort}\,u$ als gemeinsamer Name für alle Universenstufen: $\mathrm{Sort}\,0$ ist Prop, und $\mathrm{Sort}\,(u+1)$ ist $\mathrm{Type}\,u$. So lassen sich die Regeln einmal hinschreiben, statt zweimal – für Aussagen und für Daten.

Eine Variable hat den Typ, unter dem sie im Kontext steht, und Universen liegen jeweils eine Stufe höher:

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A}\ (\textsc{Var}) \qquad \frac{}{\Gamma \vdash \mathrm{Sort}\,u : \mathrm{Sort}\,(u+1)}\ (\textsc{Sort})$$

Der abhängige Funktionstyp bekommt drei Regeln – eine zum Bilden, eine zum Erzeugen, eine zum Anwenden:

$$\frac{\Gamma \vdash A : \mathrm{Sort}\,u \qquad \Gamma,\, x : A \vdash B : \mathrm{Sort}\,v}{\Gamma \vdash (x : A) \to B : \mathrm{Sort}\,(\mathrm{imax}\,u\,v)}\ (\Pi\text{-}\textsc{Form})$$

$$\frac{\Gamma,\, x : A \vdash t : B}{\Gamma \vdash (\lambda x.\, t) : (x : A) \to B}\ (\lambda) \qquad \frac{\Gamma \vdash f : (x : A) \to B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\,a : B[a/x]}\ (\textsc{App})$$

Dazu kommen die Regeln für induktive Typen: zu jedem induktiven Typ seine Konstruktoren und sein Rekursor (Abschnitt 3.2).

Bemerkung 3.7 (Wo die Impredikativität steckt). In der Π-Form-Regel steht imax, nicht max. Der Unterschied: $\mathrm{imax}\,u\,0 = 0$. Ein Funktionstyp mit Ziel in Prop landet also wieder in Prop, ganz gleich, wie groß die Quelle ist. Genau deshalb darf man in Lean über *alle* Aussagen quantifizieren und bleibt trotzdem eine Aussage. Für Type gilt das nicht – sonst hätte man Girards Paradoxon.

Die Konversionsregel

Eine Regel fehlt noch, und sie ist die wichtigste:

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash A \equiv B}{\Gamma \vdash t : B}\ (\textsc{Conv})$$

Sie verbindet Rechnen und Typisierung. Weil Typen von Werten abhängen dürfen, können in Typen Rechnungen stehen – und die Konversionsregel sagt, dass es auf das Ergebnis ankommt, nicht auf die Schreibweise:

Lean code/Kapitel3.lean

```
/- Die **Konversionsregel** verbindet Rechnen und Typisierung: Sind ‘A‘ und
   ‘B‘ definitorisch gleich, so hat jeder Term vom Typ ‘A‘ auch den Typ ‘B‘.
   Hier auf Typebene -- ‘2 + 1‘ und ‘3‘ rechnen zum selben Wert, also sind
   die Vektortypen derselbe Typ. -/
example (v : Vektor ℕ (2 + 1)) : Vektor ℕ 3 := v
```

Ohne diese Regel wäre `Vektor ℕ (2 + 1)` ein anderer Typ als `Vektor ℕ 3`, und man müsste von Hand umrechnen. Mit ihr ist es derselbe Typ. Der Preis: Der Typchecker muss rechnen können.

Woher die definitorische Gleichheit kommt

$A \equiv B$ ist kein Grundbegriff, sondern entsteht aus Reduktionsregeln:

Regel	was reduziert wird	Beispiel
β	Anwendung einer Abstraktion	$(\lambda x.\, t)\, a \rightsquigarrow t[a/x]$
δ	Auffalten einer Definition	`zwei` $\rightsquigarrow 2$
ι	Rekursor auf Konstruktor	rec … $(n{+}1) \rightsquigarrow \ldots$
ζ	`let`-Bindung einsetzen	**let** $x := a;\ t \rightsquigarrow t[a/x]$
η	Funktionen ausdehnen	$f \equiv \lambda x.\, f\, x$

Dazu kommt die Beweisirrelevanz: Je zwei Beweise derselben Aussage sind definitorisch gleich. Alle diese Gleichheiten prüft Lean durch Ausrechnen:

Lean code/Kapitel3.lean

```
/- Definitorische Gleichheit entsteht aus Reduktionsregeln. -/

/- beta: eine Anwendung wird eingesetzt. -/
example : (fun x : ℕ => x + 1) 2 = 3 := rfl

/- delta: eine Definition wird aufgefaltet. -/
def zwei : ℕ := 2
example : zwei + zwei = 4 := rfl

/- iota: eine Fallunterscheidung trifft auf einen Konstruktor und rechnet. -/
def doppelt : ℕ → ℕ
  | 0 => 0
  | k + 1 => doppelt k + 2
example : doppelt 3 = 6 := rfl

/- eta: jede Funktion ist ihre eigene Abstraktion. -/
example (f : ℕ → ℕ) : f = fun x => f x := rfl

/- Dazu die Beweisirrelevanz in ‘Prop‘. -/
example (p : Prop) (h₁ h₂ : p) : h₁ = h₂ := rfl

/- Die Konversionspruefung rechnet wirklich. -/
example : 2 ^ 12 = 4096 := rfl
```

Zu den einzelnen Zeilen: Bei β wird 2 für x eingesetzt; aus $(\lambda x.\, x{+}1)\, 2$ wird $2{+}1$, ausgerechnet 3. Bei δ wird `zwei` durch seine Definition ersetzt; aus `zwei + zwei` wird $2+2$, ausgerechnet 4. Die Zeile mit `doppelt` zeigt ι, und zwar über mehrere Schritte (gleich genauer). Bei η gibt es nichts auszurechnen, denn f ist eine Variable: Der Kern bringt f in die Form $\lambda x.\, f\, x$ und vergleicht die Rümpfe. Die letzten beiden Zeilen zeigen die Beweisirrelevanz und dass der Kern wirklich rechnet.

ι Schritt für Schritt. Der Rekursor von $\mathbb{N}$ bekommt einen Wert z für 0 und einen Schritt s, der aus k und dem Ergebnis r für k das Ergebnis $s\,k\,r$ für $k+1$ macht. Seine ι-Regeln sind

$$\mathrm{rec}\, z\, s\, 0 \rightsquigarrow z, \qquad \mathrm{rec}\, z\, s\, (k+1) \rightsquigarrow s\, k\, (\mathrm{rec}\, z\, s\, k).$$

`doppelt` ist durch Fallunterscheidung definiert und rechnet wie $f := \mathrm{rec}\, z\, s$ mit $z := 0$ und $s\,k\,r := r + 2$, also

$$f\, 0 \rightsquigarrow 0, \qquad f\, (k+1) \rightsquigarrow s\, k\, (f\, k) \rightsquigarrow_\beta f\, k + 2.$$

Genau genommen führt Lean die Fallunterscheidung auf eine Hilfskonstruktion zurück, die aus dem Rekursor gebaut ist (`#print doppelt` zeigt sie); am Rechenverhalten ändert das nichts. So rechnet sich `doppelt 3` aus:

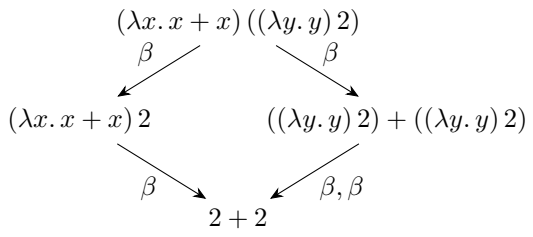
$$f\,3 \rightsquigarrow f\,2 + 2 \rightsquigarrow (f\,1 + 2) + 2 \rightsquigarrow ((f\,0 + 2) + 2) + 2 \rightsquigarrow ((0 + 2) + 2) + 2 = 6.$$

Die ersten drei Schritte sind je eine ι-Reduktion ($3 = 2 + 1$ usw.) mit anschließendem β-Schritt für s, der vierte ist $f\,0 \rightsquigarrow 0$; am Ende wird addiert. Eine durch Fallunterscheidung definierte Funktion rechnet also nur, wenn sich ihr Argument zu einem Konstruktor ausrechnen lässt, wie 3 zu $\mathrm{succ}\,2$ oder $k+1$ zu $\mathrm{succ}\,k$; bei einer Variablen n bleibt $f\,n$ stehen. Die Addition ist ebenso über ihr *zweites* Argument definiert. Deshalb gilt $n + 0 = n$ definitorisch, $0 + n = n$ dagegen nicht (Abschnitt 3.4).

Warum der Typcheck entscheidbar ist

Bei jeder Anwendung der Konversionsregel muss der Typchecker $A \equiv B$ entscheiden. Zwei Eigenschaften der Reduktion machen das möglich.

Konfluenz
Für das Ergebnis ist die Reihenfolge der Schritte egal: Führen zwei Wege von einem Term weg, so treffen sie sich wieder. In $(\lambda x.\, x + x)\,((\lambda y.\, y)\,2)$ kann man innen oder außen anfangen und kommt beide Male bei $2 + 2$ an:



Starke Normalisierung
Bei jedem Term, der einen Typ hat, endet jede Reduktionsfolge, gleich wie man rechnet.

Zusammen ergibt das: Jeder Term mit Typ hat genau eine Normalform, und man erreicht sie auf jedem Weg. $A \equiv B$ entscheidet man also, indem man beide Seiten zu Ende rechnet und die Normalformen vergleicht.

Konfluenz allein genügt dafür nicht. Auch im ungetypten λ-Kalkül ist die Reduktion konfluent, aber $\Omega := (\lambda x.\, x\,x)\,(\lambda x.\, x\,x)$ reduziert in einem Schritt auf sich selbst und rechnet ewig (Bemerkung 2.19). In der Typentheorie hat Ω keinen Typ: x müsste zugleich einen Typ A und einen Funktionstyp $A \to B$ haben. Erst die Typen bringen das Rechnen zum Halten.

Entscheidbar heißt nicht billig. Die Normalform kann enorm groß werden und der Weg dorthin lang. `doppelt` etwa rechnet in einzelnen ι-Schritten: `doppelt 100 = 200` geht per `rfl` noch durch, bei `doppelt 1000 = 2000` bricht Lean mit „maximum recursion depth has been reached“ ab. Lean rechnet deshalb nicht blind aus, sondern nur so weit wie nötig, bis zur *schwachen Kopfnormalform*: bis außen ein Konstruktor, eine Abstraktion, ein Typ oder eine Variable steht. Stimmen die Köpfe zweier Terme überein, vergleicht der Kern die Argumente einzeln; Definitionen faltet er erst auf, wenn es nicht anders geht. Für natürliche Zahlen hat er eine eingebaute Arithmetik: `2 ^ 100` rechnet er in einem Schritt aus, nicht in hundert.

Genauer für Lean. Konfluenz und starke Normalisierung sind das Bild der Lehrbücher, und für die Reduktionen der Tabelle stimmt es. Die definitorische Beweisirrelevanz (Satz 3.5) ist aber keine Reduktion, sondern eine zusätzliche Gleichung: Zwei Beweise derselben Aussage sind gleich, auch wenn ihre Normalformen verschieden sind. Im Zusammenspiel mit der Impredikativität von `Prop` (Bemerkung 3.7) gilt das Bild deshalb nicht mehr ganz: Abel und Coquand haben gezeigt, dass die Reduktion dann nicht mehr immer endet, und Carneiro hat bewiesen, dass die definitorische Gleichheit in Leans voller Theorie unentscheidbar ist. Leans Typprüfer ist deshalb nicht vollständig: In eigens konstruierten Fällen lehnt er einen nach den Regeln korrekten Term ab oder rechnet nicht zu Ende. In der Praxis begegnet man diesen Fällen nicht.[1]

Wie der Typchecker vorgeht

Naiv könnte man meinen, der Checker rechne zu jedem Term den Typ aus. Das geht nicht: Bei $\lambda x.\, x$ ist unklar, welchen Typ x haben soll. Also arbeitet er *bidirektional*, mit zwei ineinandergreifenden Funktionen:

INFER(Γ, t) – **den Typ ausrechnen**
Für eine Variable: im Kontext nachschlagen. Für eine Abstraktion mit Typangabe $\lambda(x : A).\, t$: den Typ B von t im Kontext $\Gamma,\ x : A$ inferieren und $(x : A) \to B$ zurückgeben – das ist die Regel (λ). Für eine Anwendung $f\ a$: den Typ von f inferieren, er muss ein Π-Typ $(x : A) \to B$ sein; dann a gegen A CHECKen und $B[a/x]$ zurückgeben.

CHECK(Γ, t, A) – **gegen einen Typ prüfen**
Für eine Abstraktion $\lambda x.\, t$: A in schwache Kopfnormalform bringen, sie muss $(x : A_1) \to A_2$ sein; dann t im erweiterten Kontext gegen A_2 prüfen. In allen anderen Fällen: Typ INFERieren und mit A auf Konversion testen – das ist die Regel (CONV) in Aktion.

Die Information fließt also in beide Richtungen: aus dem Term heraus und vom erwarteten Typ herein. Deshalb braucht `fun x => x` eine Typangabe von außen, `fun x : ℕ => x + 1` aber nicht:

Lean code/Kapitel3.lean

```
/- Die Pruefung laeuft in zwei Richtungen. Mit Typangabe am Binder wird der
   Typ *inferiert*, ohne sie gegen einen erwarteten Typ *geprueft*. -/
#check fun x : ℕ => x + 1       -- inferiert
#check (fun x => x : ℕ → ℕ)     -- geprueft: der Typ kommt von aussen
```

Im Einzelnen:

(1) Der erste `#check`, `fun x : ℕ => x + 1`: Die Typangabe legt $x : \mathbb{N}$ fest, und der Rumpf $x + 1$ hat im Kontext $x : \mathbb{N}$ den Typ $\mathbb{N}$. INFER liefert $\mathbb{N} \to \mathbb{N}$, ohne Hilfe von außen.

(2) Im zweiten `#check` steht `fun x => x`. Welchen Typ x hat, sagt dieser Term nicht; INFER kann dafür nur eine Unbekannte ansetzen. Allein meldet `#check fun x => x` deshalb nur den Typ `?m.1 → ?m.1`, und als Definition, `def g := fun x => x`, lehnt Lean den Term ab.

(3) Mit der Typangabe von außen prüft Lean dagegen im leeren Kontext CHECK$(\emptyset,\ \lambda x.\, x,\ \mathbb{N} \to \mathbb{N})$. Der erwartete Typ ist schon ein Π-Typ mit $A_1 = A_2 = \mathbb{N}$; zu prüfen bleibt x gegen $\mathbb{N}$, im Kontext $x : \mathbb{N}$.

[1] A. Abel, Th. Coquand, *Failure of Normalization in Impredicative Type Theory with Proof-Irrelevant Propositional Equality*, Logical Methods in Computer Science 16 (2020), `https://arxiv.org/abs/1911.08174`;
M. Carneiro, *The Type Theory of Lean*, Masterarbeit, Carnegie Mellon University 2019, Abschnitt 3.1, `https://github.com/digama0/lean-type-theory/releases/tag/v1.0`;
ders., *Lean4Lean: Verifying a Typechecker for Lean, in Lean*, `https://arxiv.org/abs/2403.14064`.

(4) Für die Variable x greift der zweite Fall von CHECK: INFER schlägt $x : \mathbb{N}$ im Kontext nach, und der Vergleich $\mathbb{N} \equiv \mathbb{N}$ gelingt – die Regel (CONV).

In Lean erledigt diese Arbeit genau genommen der Elaborator (nächster Abschnitt): Er trägt den Typ, der von außen kommt, am Binder ein. Der Kern bekommt `fun (x : ℕ) => x` zu sehen; dort hat jede Abstraktion eine Typangabe, und er kommt mit INFER und der Konversionsprüfung aus.

Elaboration und Kern

Was man schreibt, ist nicht das, was geprüft wird. Dazwischen liegt die *Elaboration*: Sie füllt implizite Argumente ein, löst Instanzargumente auf, fügt Koerzierungen ein, wertet Notation aus und führt Taktiken aus. Implizite Argumente bestimmt der Elaborator, indem er Gleichungen zwischen Termen mit Unbekannten löst (*Unifikation*); Instanzargumente findet er durch Suche unter den Instanzen der Typklasse (Abschnitt 3.2). Das Ergebnis ist ein vollständiger Term ohne jede Lücke.

Lean code/Kapitel3.lean

```
/- Was man schreibt, ist nicht das, was der Kern sieht. Die Elaboration
   fuellt implizite Argumente, Instanzen und Koerzierungen ein. -/
#check (rfl : 2 + 2 = 4)

set_option pp.explicit true in
#check (rfl : 2 + 2 = 4)

/- Geprueft wird am Ende nur der fertige Term, und nur vom Kern. -/
#print axioms Nat.add_comm
```

Schon die erste Ausgabe zeigt die Unifikation: Lean meldet `rfl : 2 + 2 = 2 + 2`. `rfl` hat den Typ `a = a`, mit impliziten Argumenten für den Typ und für `a`. Der Abgleich mit `2 + 2 = 4` setzt $a := 2+2$, und dass auch $4 \equiv 2+2$ gilt, rechnet der Kern nach – (CONV). Mit `pp.explicit` zeigt Lean den vollständigen Term, `@rfl ℕ (2 + 2)`, und auch in `2 + 2` stehen dann Typ und Instanzen. Die letzte Zeile fragt mit `#print axioms`, worauf der fertige Term von `Nat.add_comm` beruht: auf keinem Axiom. Erst dieser fertige Term geht an den *Kern*, und der kann nur eines: die Regeln von oben anwenden.



Die beiden Teile im Vergleich:

Elaborator	**Kern**
groß, komplex, entwickelt sich	klein, stabil
rät, sucht, probiert	rechnet nur nach
darf Fehler machen	darf keine machen

Ein Fehler im Elaborator führt dazu, dass ein richtiger Beweis nicht durchgeht. Ein falscher käme nur durch, wenn der Kern selbst fehlerhaft wäre – und der ist klein genug, um ihn zu überblicken. Das ist der Grund, warum man einem Beweisassistenten überhaupt trauen kann.

Zum Weiterlesen. Wie Leans Kern rechnet und prüft – Reduktion, definitorische Gleichheit, Typinferenz und die Frage, wem man trauen muss –, beschreibt *Type Checking in Lean 4*:

`https://ammkrn.github.io/type_checking_in_lean4/`
Elaborator und Kern im Zusammenspiel erklärt *The Lean Language Reference* im Kapitel „Elaboration and Compilation“:
`https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/`

Bemerkung 3.8 (Beweise, die eine Maschine geschrieben hat). Genau diese Arbeitsteilung wird gerade praktisch wichtig. Sprachmodelle schlagen inzwischen Beweise vor. Manche sind gut, manche falsch, und die falschen sehen den guten zum Verwechseln ähnlich – sie sind flüssig formuliert, zitieren plausible Sätze und haben eine Lücke an einer Stelle, die niemand vermutet.

Für den Kern ist das ohne Belang. Er fragt nicht, woher ein Term stammt, ob ihn ein Mensch, ein Modell oder ein Zufallsgenerator erzeugt hat. Er wendet die Regeln aus diesem Abschnitt an, und entweder sie passen oder nicht.

Damit verschiebt sich, worauf man achten muss. Man muss der Maschine nicht glauben, und man muss ihren Beweis nicht einmal lesen – es genügt, dass er durch den Typcheck geht. Zu prüfen bleibt nur noch die *Aussage*: ob der Satz, der da bewiesen wurde, auch der gemeinte ist. Das ist eine deutlich kleinere Aufgabe als einen Beweis nachzuvollziehen, und sie lässt sich nicht delegieren.

3.6 Mengenlehre in Lean

Damit sind alle Begriffe da. Dieser Abschnitt geht Kapitel 2 noch einmal durch, diesmal in Lean. Zwei Dinge werden dabei sichtbar: wie die mengentheoretischen Begriffe in der Typentheorie aussehen, und an welcher Stelle die Paradoxien abgefangen werden.

Mengen sind Prädikate

In Lean ist `Set` α dasselbe wie ein Prädikat $\alpha \to$ `Prop` – und zwar definitorisch, nicht bloß ähnlich:

Lean code/Kapitel3.lean

```
/- In Lean ist ‘Set α‘ *definitorisch* dasselbe wie ein Praedikat ‘α → Prop‘.
   Eine Menge ist also immer Teilmenge eines bereits vorhandenen Typs. -/
example (α : Type) : Set α = (α → Prop) := rfl

/- ‘x ∈ s‘ bedeutet schlicht: das Praedikat ‘s‘ trifft auf ‘x‘ zu. -/
example (α : Type) (s : Set α) (x : α) : (x ∈ s) = s x := rfl

/- Komprehension ‘{x | p x}‘ ist genau das Praedikat ‘p‘. -/
example (α : Type) (p : α → Prop) (x : α) : (x ∈ {y | p y}) = p x := rfl
```

Das ist der Unterschied zur uneingeschränkten Komprehension (Definition 2.3). Die Komprehension gibt es, aber sie liefert immer eine Teilmenge eines Typs α, den es schon gibt. „Die Menge aller x mit $\varphi(x)$“ kann man nicht bilden, ohne vorher zu sagen, woher die x kommen.

Mathlib. `Set` α ist `def Set (α : Type u) := α → Prop` in `Mathlib/Data/Set/Defs.lean`. Das Zeichen `x ∈ s` ist Notation für die Anwendung `s x`. Mengenbeweise sind in Lean deshalb punktweise Aussagenlogik. Die Taktik `ext` geht von der Mengengleichheit zur elementweisen Äquivalenz.

Extensionalität und die Booleschen Operationen sehen aus wie gewohnt:

Lean code/Kapitel3.lean

```
/- Extensionalitaet: zwei Mengen sind gleich, wenn sie dieselben Elemente haben. -/
example (α : Type) (s t : Set α) (h : ∀ x, x ∈ s ↔ x ∈ t) : s = t := Set.ext h
```

```
/- Aussagen als Typen. Eine Aussage `P : Prop` ist ein Typ, und ihre Beweise
   sind die Elemente dieses Typs. `P` gilt, wenn der Typ *bewohnt* ist
   (englisch inhabited), wenn es also einen Term `p : P` gibt. Zwei Aussagen
   bilden die Grenzfaelle:
   - `True` hat genau einen Beweis, `trivial`. Der Typ ist immer bewohnt.
   - `False` hat keinen Beweis, denn als induktiver Typ hat `False` keinen
     Konstruktor. Der Typ ist leer.

   Ein Beweis von `P → Q` ist eine Abbildung: Sie macht aus jedem Beweis
   von `P` einen Beweis von `Q`. Fuer `True` und `False` folgt daraus:
   - Eine Abbildung `P → True` gibt es immer, naemlich `fun _ => trivial`.
     Sie sagt deshalb nichts ueber `P`.
   - Eine Abbildung `P → False` gibt es nur, wenn `P` leer ist. Sonst wuerde
     sie einen Beweis von `P` auf einen Beweis von `False` abbilden. Deshalb
     ist `¬ P` definiert als `P → False`: `P` hat keinen Beweis.
   - Eine Abbildung `False → P` gibt es immer (`False.elim`), denn auf dem
     leeren Typ ist nichts festzulegen. Aus `False` folgt alles.

   Bei Mengen ist es genauso: In eine einpunktige Menge fuehrt von jeder
   Menge eine Abbildung, in die leere Menge nur von der leeren Menge aus,
   und von der leeren Menge fuehrt eine Abbildung in jede Menge. `True`
   entspricht der einpunktigen Menge, `False` der leeren. -/
example : True := trivial                              -- True ist bewohnt
example (P : Prop) : P → True := fun _ => trivial      -- geht fuer jedes P
example (P : Prop) : False → P := False.elim           -- aus False folgt alles
example (P : Prop) (f : ¬ P) (p : P) : False := f p    -- ¬ P ist P → False

/- Eine Aequivalenz `P ↔ Q` besteht aus zwei Abbildungen, `P → Q` und
   `Q → P`. Mit den Regeln oben folgt:
   - `P ↔ True`: `P → True` gibt es immer. `True → P` gibt es genau dann,
     wenn `P` bewohnt ist: Aus `p : P` wird `fun _ => p`, und umgekehrt
     liefert die Abbildung, angewendet auf `trivial`, einen Beweis von `P`.
     Also ist `P` genau dann bewohnt, wenn `P ↔ True` gilt.
   - `P ↔ False`: `False → P` gibt es immer. `P → False` gibt es genau dann,
     wenn `P` leer ist. Also ist `P` genau dann leer, wenn `P ↔ False` gilt.
   Das Axiom `propext` macht aequivalente Aussagen gleich. Mit ihm ist eine
   bewohnte Aussage sogar gleich `True` und eine leere gleich `False`. Dass
   jede Aussage bewohnt oder leer ist, braucht zusaetzlich das
   ausgeschlossene Dritte (`Classical.em`). -/
example (P : Prop) (p : P) : P ↔ True := ⟨fun _ => trivial, fun _ => p⟩
example (P : Prop) (h : True → P) : P := h trivial
example (P : Prop) (f : ¬ P) : P ↔ False := ⟨f, False.elim⟩
example (P : Prop) (p : P) : P = True := propext ⟨fun _ => trivial, fun _ => p⟩

/- Hinter dem Komplement steckt ein Pfeil. `s^c` ist definiert als die Menge
   `{a | a ∉ s}`, `x ∉ s` ist eine Abkuerzung fuer `¬ (x ∈ s)`, und `¬ P` ist
   `P → False`. Zusammen gilt per Definition

     x ∈ s^c  =  x ∈ {a | a ∉ s}  =  x ∉ s
              =  ¬ (x ∈ s)  =  (x ∈ s → False).

   Das hat zwei Folgen fuer Beweise:
   - Ein Ziel `x ∈ s^c` ist eine Implikation. `intro hs` macht ihre linke
     Seite zur Annahme `hs : x ∈ s`; zu zeigen bleibt `False`.
   - Eine Annahme `hc : x ∈ s^c` ist eine Funktion. Angewendet auf einen
     Beweis von `x ∈ s` liefert sie einen Beweis von `False`.

   `change` schreibt das Ziel in einer gleichwertigen Form hin; Lean prueft
```

```
   dabei, dass es per Definition dasselbe Ziel ist. Noetig ist ‘change‘ nicht,
   ‘intro‘ setzt die Definitionen auch selbst ein.

   Beispiel: Was in ‘s‘ liegt, liegt nicht in ‘sᶜ‘, also in ‘sᶜᶜ‘. -/
example (α : Type) (s : Set α) (x : α) (hs : x ∈ s) : x ∈ sᶜᶜ := by
  change x ∈ sᶜ → False        -- dasselbe Ziel, der Pfeil ausgeschrieben
  intro hc                     -- hc : x ∈ sᶜ,  Ziel: False
  exact hc hs                  -- hc : x ∈ s → False, angewendet auf hs

/- De Morgan zu Fuss, ohne ‘simp‘. Nach ‘ext x‘ ist eine Aequivalenz zu
   zeigen; ‘constructor‘ teilt sie in zwei Richtungen. -/
example (α : Type) (s t : Set α) : (s ∪ t)ᶜ = sᶜ ∩ tᶜ := by
  ext x
  constructor
  · intro h                    -- h : x ∈ (s ∪ t)ᶜ,  Ziel: x ∈ sᶜ ∩ tᶜ
    constructor
    · intro hs                 -- hs : x ∈ s,  Ziel: False
      exact h (Or.inl hs)      -- aus x ∈ s folgt x ∈ s ∪ t
    · intro ht                 -- ht : x ∈ t,  Ziel: False
      exact h (Or.inr ht)
  · intro h hst                -- h : x ∈ sᶜ ∩ tᶜ,  hst : x ∈ s ∪ t,  Ziel: False
    cases hst with
    | inl hs => exact h.1 hs   -- h.1 : x ∈ sᶜ widerspricht hs
    | inr ht => exact h.2 ht

/- De Morgan -- der Beweis ist punktweise reine Aussagenlogik. -/
example (α : Type) (s t : Set α) : (s ∪ t)ᶜ = sᶜ ∩ tᶜ := by
  ext x
  simp

example (α : Type) (s t u : Set α) (h : s ⊆ t) : s ∩ u ⊆ t ∩ u := by
  intro x hx
  exact ⟨h hx.1, hx.2⟩
```

Russell scheitert am Typcheck

Satz 2.4 zerfiel in zwei Teile. Der erste war reine Aussagenlogik, ohne jede Mengenlehre:

Lean code/Kapitel3.lean

```
/-- Der logische Kern des Paradoxons: keine Aussage ist zu ihrer eigenen
Negation aequivalent. Der Beweis benutzt keinerlei Mengenlehre. -/
theorem iff_not_self_false {P : Prop} (h : P ↔ ¬ P) : False :=
  have hn : ¬ P := fun hp => h.mp hp hp
  hn (h.mpr hn)
```

Der zweite Teil sagt, dass uneingeschränkte Komprehension genau so eine Aussage liefert. Dafür schreiben wir „naive Mengenlehre“ als Struktur auf: ein Typ von Mengen, eine Elementbeziehung, das Komprehensionsprinzip.

Lean code/Kapitel3.lean

```
/-- Eine *naive Mengenlehre*: ein Typ ‘S‘ von Mengen, eine Elementbeziehung
und -- das ist der fatale Punkt -- **uneingeschraenkte Komprehension**:
zu jeder Formel ‘p‘ gibt es die Menge ‘compr p‘ ihrer Erfueller. -/
structure NaiveMengenlehre where
  S : Type
  mem : S → S → Prop
  compr : (S → Prop) → S
  mem_compr : ∀ (p : S → Prop) (x : S), mem x (compr p) ↔ p x
```

```
/-- **Russells Paradoxon.** Eine solche Theorie ist widerspruechlich.
Man bilde ‘R = {x | x ∉ x}‘ und frage, ob ‘R ∈ R‘. -/
theorem russell (N : NaiveMengenlehre) : False :=
  iff_not_self_false
    (N.mem_compr (fun x => ¬ N.mem x x) (N.compr (fun x => ¬ N.mem x x)))
```

Der Satz sagt: So eine Struktur gibt es nicht. Wer sie annimmt, kann `False` beweisen und damit alles.

Warum trifft das Lean selbst nicht? Weil man $\{x \mid x \notin x\}$ dort nicht hinschreiben kann. Für `x ∉ x` müsste `x` zugleich ein Element vom Typ α und eine Menge vom Typ `Set α` sein. Das scheitert am Typcheck, nicht erst am Beweis:

Lean code/Kapitel3.lean

```
/- In Lean laesst sich die Russell-Menge gar nicht erst *hinschreiben*:
   ‘x ∉ x‘ verlangt, dass ‘x‘ zugleich Element und Menge desselben Typs ist.
   Der Typcheck scheitert, nicht erst der Beweis. -/
#check_failure ({x : Set ℕ | x ∉ x})
```

Bemerkung 3.9. `#check_failure` ist eine Lean-Anweisung, die *erwartet*, dass ihr Argument nicht typisiert. Geht der Typcheck doch durch, meldet Lean einen Fehler. Die Datei oben übersetzt also gerade deshalb fehlerfrei, weil die Russell-Menge unzulässig ist.

Cantor und die Universen

Der Satz von Cantor (Satz 2.6) steht in Mathlib in zwei Fassungen:

Lean code/Kapitel3.lean

```
/- Satz von Cantor: keine Abbildung ‘α → Set α‘ ist surjektiv, -/
#check @Function.cantor_surjective
/- und ‘Set α‘ laesst sich nicht injektiv nach ‘α‘ einbetten. -/
#check @Function.cantor_injective

/- Es kann also keinen Typ ‘U‘ geben, der "alle seine Teilmengen enthaelt". -/
example (U : Type) (f : Set U → U) : ¬ Function.Injective f :=
  Function.cantor_injective f
```

Die Universenhierarchie aus Abschnitt 3.1 ist die typentheoretische Antwort darauf. Hier noch einmal der Punkt, auf den es ankommt: `Type` ist nicht Element von `Type`.

Lean code/Kapitel3.lean

```
/- Lean loest das Problem durch eine Hierarchie von Universen. -/
#check Type       -- Type : Type 1
#check Type 1     -- Type 1 : Type 2

/- ‘Type : Type‘ waere inkonsistent (Girards Paradoxon) und wird abgelehnt. -/
#check_failure (Type : Type)
```

ZFC als Gegenstand, nicht als Grundlage

ZFC steckt in Mathlib – aber nicht als Fundament, sondern als *Gegenstand*. Der Typ `ZFSet` entsteht als Quotient des Typs `PSet` der „Vor-Mengen“ nach Extensionalität. Die Axiome aus Kapitel 2 sind dann Sätze:

Lean code/Kapitel3.lean

```
/- Mathlib *definiert* ZFC innerhalb der Typentheorie. ‘PSet‘ sind
   "Vor-Mengen", ‘ZFSet‘ deren Quotient nach Extensionalitaet. -/
#check @PSet
#check @ZFSet
```

```
/- Die Axiome erscheinen als Saetze ueber 'ZFSet': -/
#check @ZFSet.ext            -- Extensionalitaet
#check @ZFSet.notMem_empty   -- Leermenge
#check @ZFSet.mem_pair       -- Paarmenge
#check @ZFSet.mem_sUnion     -- Vereinigung
#check @ZFSet.mem_powerset   -- Potenzmenge
#check @ZFSet.mem_sep        -- Aussonderung (NICHT freie Komprehension!)
#check @ZFSet.omega_zero     -- Unendlichkeit
#check @ZFSet.omega_succ
#check @ZFSet.regularity     -- Fundierung
#check @ZFSet.choice         -- Auswahl
```

Mathlib. `PSet` ist induktiv definiert: Eine Vor-Menge besteht aus einem Indextyp und einer Familie von Vor-Mengen. `ZFSet` ist der Quotient nach extensionaler Gleichheit – hier zahlt sich `Quot.sound` aus. Die ZFC-Axiome sind damit *beweisbare Sätze*. Die Typentheorie liefert also ein Modell von ZFC.

Damit lässt sich nachrechnen, was in Kapitel 2 behauptet wurde: Es gibt keine Allmenge, und die Russellsche Aussonderung ist harmlos.

Lean code/Kapitel3.lean

```
/-- In ZFC gibt es keine Allmenge: sie waere Element ihrer selbst. -/
theorem keine_allmenge : ¬ ∃ u : ZFSet, ∀ x : ZFSet, x ∈ u := by
  rintro ⟨u, hu⟩
  exact ZFSet.mem_irrefl u (hu u)

/-- Aussonderung entschaerft Russell: aus einer *vorhandenen* Menge 'x'
darf man aussondern, und die Russell-Teilmenge von 'x' ist einfach 'x'. -/
theorem russell_teilmenge (x : ZFSet) : ZFSet.sep (fun y => y ∉ y) x = x := by
  apply ZFSet.ext
  intro z
  simp [ZFSet.mem_sep, ZFSet.mem_irrefl]
```

Klassen

Auch NBG bildet Mathlib ab: `Class` ist `Set ZFSet`, also die Klassen über dem ZFC-Universum. Der Punkt aus Definition 2.52 wird hier wörtlich sichtbar: Eine Klasse *ist* ein Prädikat auf `ZFSet`, jede Formel liefert also eine. Ob sie eine Menge ist, ist eine ganz andere Frage – und `Class.mem_univ` sagt, wann.

Lean code/Kapitel3.lean

```
/- Eine **Klasse** ist in Mathlib eine Menge von 'ZFSet's. Klassen duerfen
   frei komprehendiert werden; nur "kleine" Klassen sind Mengen. -/
#check @Class

/- Die Allklasse existiert ... -/
#check Class.univ
/- ... ist aber keine Menge: -/
theorem allklasse_keine_menge : Class.univ ∉ Class.univ := Class.univ_notMem_univ

/- Genau die Klassen, die von einer Menge kommen, sind Elemente von Klassen. -/
#check @Class.mem_univ

/- Die Russell-Klasse existiert als Klasse -- sie ist die Allklasse,
   denn keine Menge ist Element ihrer selbst. -/
theorem russell_klasse : {x : ZFSet | x ∉ x} = Class.univ := by
```

```
  ext x
  simp [Class.univ, ZFSet.mem_irrefl]
```

Bemerkung 3.10. `russell_klasse` sagt etwas Nettes: Da in ZFC keine Menge Element ihrer selbst ist, *ist* die Russell-Klasse einfach die Allklasse. Dass diese keine Menge ist, steht in `Class.univ_notMem_univ`.

Kategorien

Die Definition einer Kategorie ist als Lean-Struktur wörtlich dieselbe wie in Kapitel 2:

Lean code/Kapitel3.lean

```
universe u v

/-- Eine Kategorie: Objekte, Morphismen, Identitaeten, Komposition --
und *keine* Elemente. -/
structure MeineKategorie where
  Obj : Type u
  Hom : Obj → Obj → Type v
  id : ∀ X : Obj, Hom X X
  comp : ∀ {X Y Z : Obj}, Hom X Y → Hom Y Z → Hom X Z
  id_comp : ∀ {X Y : Obj} (f : Hom X Y), comp (id X) f = f
  comp_id : ∀ {X Y : Obj} (f : Hom X Y), comp f (id Y) = f
  assoc : ∀ {W X Y Z : Obj} (f : Hom W X) (g : Hom X Y) (h : Hom Y Z),
    comp (comp f g) h = comp f (comp g h)

/-- Die Kategorie der Typen und Funktionen. -/
def TypKategorie : MeineKategorie where
  Obj := Type
  Hom X Y := X → Y
  id _ := fun x => x
  comp f g := g ∘ f
  id_comp _ := rfl
  comp_id _ := rfl
  assoc _ _ _ := rfl
```

Mathlib nimmt statt einer Struktur eine *Typklasse* `CategoryTheory.Category` (Abschnitt 3.2), damit Lean die Kategorienstruktur selbst findet:

Lean code/Kapitel3.lean

```
open CategoryTheory

/- In Mathlib ist ‘Category‘ eine Typklasse ueber einem ‘Quiver‘. -/
#check @CategoryTheory.Category
#check @CategoryTheory.Category.assoc

/- ‘Type‘ ist eine Kategorie; Morphismen sind genau Funktionen. -/
example : Category.{0, 1} Type := inferInstance
example (X Y : Type) : (X ⟶ Y) = (X → Y) := rfl

/-- Ein Funktor ‘Type ⥤ Type‘. Die Funktorgesetze erledigt ‘aesop_cat‘. -/
def ListFunktor : Type ⥤ Type where
  obj α := List α
  map f := List.map f

example (α β : Type) (f : α → β) (l : List α) : ListFunktor.map f l = l.map f :=
    rfl
```

Das Yoneda-Lemma (Satz 2.68) und die Yoneda-Einbettung (Korollar 2.69) stehen in Mathlib bereit. `yonedaEquiv` ist die Bijektion aus dem Satz; `Yoneda.ext` ist genau die Aussage, dass natürlich übereinstimmende Morphismenmengen isomorphe Objekte bedeuten:

Lean code/Kapitel3.lean

```
/- **Yoneda.** Der Einbettungsfunktor schickt ein Objekt ‘X‘ auf die Familie
   aller Morphismen nach ‘X‘, also auf ‘Hom(-, X)‘. -/
#check @CategoryTheory.yoneda

/- Das Yoneda-Lemma in der brauchbaren Form: natuerliche Transformationen
   von ‘Hom(-, X)‘ nach ‘F‘ entsprechen genau den Elementen von ‘F X‘. -/
#check @CategoryTheory.yonedaEquiv

/- Die Einbettung ist voll und treu -- sie verliert also nichts. -/
#check @CategoryTheory.Yoneda.fullyFaithful

/- Und das ist der Grund, warum Beziehungen genuegen: Passen die
   Morphismenmengen natuerlich zusammen, so sind die Objekte isomorph. -/
#check @CategoryTheory.Yoneda.ext
```

Und die Beobachtung von vorhin, dass Elemente den Morphismen aus dem terminalen Objekt entsprechen, lässt sich direkt hinschreiben:

Lean code/Kapitel3.lean

```
/-- Der strukturelle Standpunkt: "Elemente" von ‘X‘ sind dasselbe wie
Morphismen aus dem terminalen Objekt ‘Unit‘. -/
def elementeAlsMorphismen (X : Type) : (Unit ⟶ X) ≃ X where
  toFun f := f ()
  invFun x := fun _ => x
  left_inv f := by funext u; cases u; rfl
  right_inv _ := rfl
```

Umgekehrt: die Typentheorie *ist* eine Kategorie

Bisher haben wir Kategorien *in* Lean definiert. Es geht auch andersherum, denn die Typentheorie selbst erfüllt die Axiome aus Definition 2.62. Man setzt:

Kategorie	**Typentheorie**
Objekte	Typen $X, Y, \ldots$
Morphismen $X \to Y$	Terme $x : X \vdash t : Y$, modulo $\equiv$
id_X	die Variable x selbst
$g \circ f$	Einsetzen: $g[f/x]$

Die drei Axiome sind dann genau die Gesetze des Einsetzens. (K1) sagt, dass Substitution assoziativ ist – es ist gleichgültig, in welcher Reihenfolge man nacheinander einsetzt. (K2) und (K3) sagen, dass das Einsetzen der Variablen für sich selbst nichts ändert. Beides sieht man den Regeln aus Abschnitt 3.5 an: In der Regel (App) steht die Substitution $B[a/x]$, und sie ist der ganze Inhalt der Komposition.

Das Ergebnis heißt die *syntaktische Kategorie* einer Typentheorie, auch *Termmodell*. Weil man modulo $\equiv$ rechnet, erledigt die Konversionsregel dabei die Arbeit: Zwei Terme, die derselbe Morphismus sein sollen, sind definitorisch gleich.

Der Zusammenhang geht in beide Richtungen und ist präzise:

Typentheorie	**Kategorie**
Produkttypen $X \times Y$	Produkte
Funktionstypen $X \to Y$	Exponentiale
einfach getypter λ-Kalkül	kartesisch abgeschlossene Kategorie
abhängige Typen $(x : X) \to B$	lokal kartesisch abgeschlossene Kategorie

Die dritte Zeile ist der Satz von Lambek: Einfach getypte λ-Kalküle und kartesisch abgeschlossene Kategorien entsprechen einander. Man sagt, der λ-Kalkül sei die *interne Sprache* einer solchen Kategorie. Für abhängige Typen stammt die Entsprechung von Seely.

Bemerkung 3.11 (Eine Feinheit). Die letzte Zeile stimmt nicht ganz so glatt, wie sie aussieht. In der Syntax ist Substitution *strikt* assoziativ, in einer lokal kartesisch abgeschlossenen Kategorie nur bis auf kanonische Isomorphie. Dieses Kohärenzproblem haben Curien und Hofmann aufgeklärt; heute benutzt man meist *categories with families* statt der naiven Fassung.

Für dieses Skript ist daran eines wichtig: Kapitel 2 und Kapitel 3 handeln nicht von zwei getrennten Dingen. Die Axiome (K1)–(K3) aus Abschnitt 2.8 und die Inferenzregeln aus Abschnitt 3.5 beschreiben denselben Sachverhalt, einmal von außen und einmal von innen.

Bedeutung in den Mengen. Warum heißt die syntaktische Kategorie *Termmodell*? Weil sie selbst eine Deutung der Typentheorie ist: Jeder Typ wird ein Objekt, jeder Term ein Morphismus – gebaut aus nichts als den Termen. Die vertrautere Deutung ist die in den Mengen, ein *Mengenmodell*. In Abschnitt 2.6 war Semantik eine Abbildung $[\![\cdot]\!]$ von der Syntax in die Mengen (Definition 2.37). Dieselbe Idee funktioniert für Typen: Ein Typ bedeutet eine Menge, ein Term im Kontext $x : A$ eine Abbildung.

Typentheorie	**Bedeutung in den Mengen**
Typ A	Menge $[\![A]\!]$
$\mathbb{N}$	$\mathbb{N}$
$A \times B$	$[\![A]\!] \times [\![B]\!]$
$A \to B$	Menge aller Abbildungen $[\![A]\!] \to [\![B]\!]$
Term $x : A \vdash t : B$	Abbildung $[\![t]\!] : [\![A]\!] \to [\![B]\!]$
Aussage P : Prop	höchstens einelementige Menge: $\emptyset$ (falsch) oder ein Punkt (wahr)

λ wird „Abbildung bilden“: Ist $x : A$, $y : C \vdash s : D$, so ist $[\![\lambda y.\, s]\!](a)$ die Abbildung $c \mapsto [\![s]\!](a, c)$. Anwendung wird „auswerten“: $[\![f\, r]\!](a) = [\![f]\!](a)([\![r]\!](a))$ – man nimmt die Abbildung $[\![f]\!](a)$ an der Stelle $[\![r]\!](a)$. Wie in Abschnitt 2.6 ist das eine Rekursion über den Aufbau, und sie ist *korrekt*: Definitorisch gleiche Terme bedeuten dasselbe. Für die β-Regel ist das ein Lemma über das Einsetzen, $[\![s[r/y]]\!](a) = [\![s]\!](a, [\![r]\!](a))$; also ist $[\![(\lambda y.\, s)\, r]\!] = [\![s[r/y]]\!]$. Für η ist es die Extensionalität der Mengenabbildungen: Eine Abbildung ist durch ihre Werte bestimmt.

Deshalb ist $[\![\cdot]\!]$ auf dem Termmodell wohldefiniert, obwohl man dort modulo $\equiv$ rechnet. Es ist ein Funktor (Definition 2.65) von der syntaktischen Kategorie in die Kategorie **Set** der Mengen: Die Variable x geht auf die Identität, Einsetzen auf Komposition, und Produkte und Funktionstypen bleiben erhalten. So passen die Modelle aus Abschnitt 2.6 und das Termmodell zusammen: Das Termmodell ist die Syntax selbst als Kategorie, und jedes Mengenmodell ist ein Funktor aus ihm heraus. Aussagen bedeuten dabei höchstens einelementige Mengen; je zwei Beweise derselben Aussage bedeuten also dasselbe Element. Das ist die Beweisirrelevanz, in Mengen gelesen, und es passt zu den Wahrheitswerten $\emptyset$ und $\{()\}$ aus Abschnitt 2.6.

Abhängige Typen und Universen gehen genauso: Für einen Typ B bedeutet $(x : A) \to B$ das Produkt $\prod_{a \in [\![A]\!]} [\![B]\!](a)$. Ist B eine Aussage, so bedeutet $\forall x : A$, B wieder $\emptyset$ oder einen

Punkt, und zwar einen Punkt, wenn alle $[\![B]\!](a)$ einen haben; denn `Prop` ist impredikativ (Bemerkung 3.7). `Type u` bedeutet ein Grothendieck-Universum (Definition 2.56). Ein solches Mengenmodell für Leans Typentheorie braucht deshalb unerreichbare Kardinalzahlen; ausgeführt hat es M. Carneiro (*The Type Theory of Lean*, 2019). Wie in Satz 2.43, Richtung ($\Leftarrow$), sichert das Modell die Widerspruchsfreiheit: Ein Beweis von `False` müsste ein Element von $[\![\text{False}]\!] = \emptyset$ sein. Lean ist also widerspruchsfrei, wenn ZFC mit diesen Kardinalzahlen es ist.

3.7 Lean: Term- und Taktikmodus

Einen Beweis kann man auf zwei Arten angeben. Im *Termmodus* schreibt man den Beweisterm direkt hin. Im *Taktikmodus* (eingeleitet mit `by`) beschreibt man, wie der Term gebaut werden soll. Heraus kommt beide Male ein Term, den derselbe Kern prüft.

Lean code/Kapitel3.lean

```
/-- Termmodus: der Beweis wird direkt hingeschrieben. -/
theorem symm_term (a b : ℕ) (h : a = b) : b = a := h.symm

/-- Taktikmodus: der Beweisterm wird schrittweise konstruiert.
Auch hier entsteht ein Kern-Term, nur ein anderer. -/
theorem symm_taktik (a b : ℕ) (h : a = b) : b = a := by
  rw [h]

example (a b : ℕ) (h : a = b) : b = a := by
  subst h
  rfl
```

Daran sieht man, wie Lean aufgebaut ist. Ein kleiner *Kern* prüft Terme. Alles andere – Taktiken, Elaboration, Instanzsuche, Notation – darf beliebig kompliziert sein, denn es kann nichts Falsches durchlassen. Ein Fehler in einer Taktik führt dazu, dass ein Beweis nicht gelingt. Nie dazu, dass ein falscher akzeptiert wird.

3.8 Die Strukturen der Mathematik in Lean

Abschnitt 3.2 hat gezeigt, was eine Typklasse ist. Jetzt kommt die Anwendung: Mathlib baut daraus die gesamte Mathematik. Wer die folgenden Muster kennt, findet sich in der Bibliothek zurecht.

Zahlen und Rechnen

Am Anfang steht die Arithmetik. $\mathbb{N}$ ist der induktive Typ aus Abschnitt 3.2 – Null und Nachfolger, sonst nichts. Alles Weitere baut darauf auf: $\mathbb{Z}$ als Differenzen, $\mathbb{Q}$ als Brüche, $\mathbb{R}$ über Cauchy-Folgen.

Lean code/Kapitel3.lean

```
/- Die Zahlbereiche sind gewoehnliche Typen. `ℕ` ist induktiv definiert,
   alles Weitere wird darauf aufgebaut. -/
#check @Nat
#check @Int
#check @Rat
#check @Real

/- Konkrete Zahlen: der Kern rechnet selbst. -/
example : 2 + 2 = 4 := rfl
example : 17 * 23 = 391 := by norm_num
example : (5 : ℤ) - 8 = -3 := by decide
#eval (2 : ℕ) ^ 10
```

```
/- Gesetze fuer *alle* Zahlen sind kein Rechnen mehr, sondern Induktion.
   Man sieht das an der Asymmetrie: `n + 0` reduziert, `0 + n` nicht. -/
example (n : ℕ) : n + 0 = n := rfl
example (n : ℕ) : 0 + n = n := Nat.zero_add n

/- Drei Taktiken decken das meiste ab:
     ring      -- Gleichungen in kommutativen Ringen
     omega     -- lineare Arithmetik ueber ℕ und ℤ, mit Ungleichungen
     norm_num -- konkrete Zahlen, auch in ℚ und ℝ                    -/
example (a b : ℤ) : (a + b) ^ 2 = a ^ 2 + 2 * a * b + b ^ 2 := by ring
example (n : ℕ) (h : 3 < n) : 2 < n := by omega
example : (1 : ℝ) / 3 + 1 / 6 = 1 / 2 := by norm_num

/-- Ein Induktionsbeweis: die Gaussche Summenformel. -/
theorem gauss (n : ℕ) : 2 * (∑ i ∈ Finset.range (n + 1), i) = n * (n + 1) := by
  induction n with
  | zero => simp
  | succ k ih =>
      rw [Finset.sum_range_succ, Nat.mul_add, ih]
      ring

/- `ℝ` ist nicht berechenbar -- `#eval` gibt es dort nicht, und `decide`
   fuehrt zu nichts. Stattdessen `norm_num`. -/
#check (inferInstance : LinearOrder ℝ)
```

Beim Rechnen gibt es zwei ganz verschiedene Situationen, und man sollte sie auseinanderhalten.

Konkrete Zahlen. $17 \cdot 23 = 391$ ist keine Behauptung über alle Zahlen, sondern eine Rechnung. Der Kern führt sie aus, und der Beweis ist im Grunde das Ergebnis selbst.

Aussagen über alle Zahlen. $0 + n = n$ für jedes n lässt sich nicht ausrechnen – es sind unendlich viele Fälle. Hier braucht man Induktion. Die Asymmetrie zu $n + 0 = n$, das per `rfl` durchgeht, kommt daher, dass `Nat.add` über das zweite Argument rekursiert.

Für die Praxis genügen meist drei Taktiken:

Taktik	wofür	Beispiel
`ring`	Gleichungen in kommutativen Ringen	$(a+b)^2 = a^2 + 2ab + b^2$
`omega`	lineare Arithmetik über $\mathbb{N}$, $\mathbb{Z}$, mit Ungleichungen	aus $3 < n$ folgt $2 < n$
`norm_num`	konkrete Zahlen, auch in $\mathbb{Q}$ und $\mathbb{R}$	$\frac{1}{3} + \frac{1}{6} = \frac{1}{2}$

Bemerkung 3.12 ($\mathbb{R}$ rechnet nicht). $\mathbb{N}$ ist berechenbar: `#eval (2:ℕ)^10` liefert 1024. Bei $\mathbb{R}$ geht das nicht. Eine reelle Zahl ist dort durch eine unendliche Folge von Näherungen gegeben, und ob zwei solche Folgen dieselbe Zahl beschreiben, kann kein Programm entscheiden.

Deshalb sind die Rechenoperationen auf $\mathbb{R}$ `noncomputable`, und statt `decide` nimmt man `norm_num`. Für das Beweisen ändert das nichts – nur ausrechnen lassen kann man sich nichts.

Die algebraische Hierarchie

Die Grundstrukturen bauen aufeinander auf, jede mit `class ... extends ...`:

$$\text{Semigroup} \to \text{Monoid} \to \text{Group} \to \text{Ring} \to \text{Field}$$

Lean code/Kapitel3.lean

```
/- Die algebraische Hierarchie. Jede Stufe erweitert die vorige,
   aufgebaut mit ‘class ... extends ...‘. -/
#check @Semigroup
#check @Monoid
#check @Group
#check @CommRing
#check @Field

/-- Ein Satz fuer *alle* Gruppen: die Struktur kommt als Instanzargument
‘[Group G]‘ herein, nicht als Datenargument. -/
theorem inverse_von_rechts {G : Type*} [Group G] (a : G) : a * a⁻¹ = 1 :=
  mul_inv_cancel a

/- Damit gilt er sofort fuer jede konkrete Gruppe -- Lean sucht die Instanz. -/
example (a : Equiv.Perm (Fin 3)) : a * a⁻¹ = 1 := inverse_von_rechts a
```

Wichtig ist die Rollenverteilung. Die Gruppenstruktur ist ein *Instanzargument* `[Group G]`, kein normales Argument. Man beweist den Satz einmal für alle Gruppen, und Lean setzt ihn für jede konkrete Gruppe ein, ohne dass man etwas übergeben muss.

Lineare Algebra

Einen Typ `VectorSpace` gibt es in Mathlib nicht. Ein Vektorraum ist ein Modul über einem Körper, also `[Field K] [AddCommGroup V] [Module K V]`. Das spart eine Stufe und macht jeden Satz über Moduln automatisch zu einem Satz über Vektorräume.

Lean code/Kapitel3.lean

```
/- Lineare Algebra: ein Vektorraum ist ein Modul ueber einem Koerper.
   Einen eigenen Typ ‘VectorSpace‘ gibt es nicht -- er waere ueberfluessig. -/
#check @Module
#check @Module.Basis
#check @Module.finrank
#check @LinearMap

example {K V : Type*} [Field K] [AddCommGroup V] [Module K V] (a : K) (v : V) : V
    :=
  a • v
```

Lineare Algebra in $\mathbb{R}^n$

Gerechnet wird in diesem Skript mit $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ und mit $\mathbb{R}^n$. In Lean ist $\mathbb{R}^n$ der Typ `Fin n → ℝ`: Ein Vektor ist eine Funktion von den Indizes $0, \ldots, n-1$ in die Zahlen. Summe, Vielfaches und Skalarprodukt gehen komponentenweise. In $\mathbb{Z}$ rechnet `#eval` sie aus; in $\mathbb{R}$ rechnet es nicht (siehe die Bemerkung oben), dort beweist man dieselben Gleichungen.

Lean code/Kapitel3.lean

```
/- ℝ^n ist in Lean der Typ ‘Fin n → ℝ‘: Ein Vektor ordnet jedem Index
   0, ..., n-1 eine Zahl zu. ‘![a, b, c]‘ schreibt einen Vektor hin.
   Summe, Vielfaches und Skalarprodukt rechnet ‘#eval‘ in ℤ aus. -/
#eval (![1, 2, 3] + ![4, 5, 6] : Fin 3 → ℤ)      -- ![5, 7, 9]
#eval ((2 : ℤ) • ![1, 2, 3] : Fin 3 → ℤ)         -- ![2, 4, 6]
#eval (![1, 2, 3] ⬝ᵥ ![4, 5, 6] : ℤ)            -- 32

/- In ℝ rechnet ‘#eval‘ nicht; dort beweist man solche Gleichungen.
   Zwei Vektoren sind gleich, wenn alle Komponenten gleich sind. -/
example : (2 : ℝ) • (![1, 2, 3] : Fin 3 → ℝ) + ![4, 5, 6] = ![6, 9, 12] := by
  ext i                          -- Komponente i vergleichen
```

```
  fin_cases i <;> norm_num     -- die Faelle i = 0, 1, 2
example : (![1, 2, 3] : Fin 3 → ℝ) ⬝ᵥ ![4, 5, 6] = 32 := by norm_num
```

Eine Matrix ist ebenso eine Funktion, nur von zwei Indizes. Das Produkt ist assoziativ, aber nicht kommutativ:

Lean code/Kapitel3.lean

```
open Matrix   -- fuer die Schreibweisen '*ᵥ' und 'ᵀ'

/- Eine Matrix ist eine Funktion zweier Indizes; '!![a, b; c, d]' schreibt
   sie zeilenweise hin. -/
def A : Matrix (Fin 2) (Fin 2) ℤ := !![1, 2; 3, 4]
def B : Matrix (Fin 2) (Fin 2) ℤ := !![0, 1; 1, 0]

#eval A * B          -- !![2, 1; 4, 3]   Matrixprodukt
#eval B * A          -- !![3, 4; 1, 2]   nicht kommutativ
#eval A *ᵥ ![1, 1]   -- ![3, 7]          Matrix mal Vektor
#eval Aᵀ             -- !![1, 3; 2, 4]   transponiert

/- In ℝ: ein Produkt ausrechnen und ein Gesetz fuer alle Matrizen. -/
example : (!![1, 2; 3, 4] * !![0, 1; 1, 0] : Matrix (Fin 2) (Fin 2) ℝ) = !![2, 1;
    4, 3] := by
  norm_num [Matrix.mul_fin_two]
example (M N P : Matrix (Fin 3) (Fin 3) ℝ) : M * N * P = M * (N * P) :=
  Matrix.mul_assoc M N P
```

Eine Matrix M definiert die lineare Abbildung $v \mapsto Mv$. In Mathlib ist eine lineare Abbildung ein eigener Typ: `V →ₗ[ℝ] W` bündelt die Funktion mit dem Beweis, dass sie Summen und Vielfache erhält. Kern und Bild sind Unterräume.

Lean code/Kapitel3.lean

```
/- Eine Matrix macht aus Vektoren Vektoren: 'Matrix.mulVecLin M' ist v ↦ M v.
   Der Typ '(Fin 2 → ℝ) →ₗ[ℝ] (Fin 2 → ℝ)' buendelt die Abbildung mit dem
   Beweis, dass sie linear ist. Hier: die Drehung um 90 Grad. -/
def drehung : (Fin 2 → ℝ) →ₗ[ℝ] (Fin 2 → ℝ) := Matrix.mulVecLin !![0, -1; 1, 0]

example : drehung ![1, 0] = ![0, 1] := by
  ext i; fin_cases i <;> simp [drehung]

/- Linear heisst: Summen und Vielfache bleiben erhalten. -/
example (v w : Fin 2 → ℝ) : drehung (v + w) = drehung v + drehung w := map_add
    drehung v w
example (c : ℝ) (v : Fin 2 → ℝ) : drehung (c • v) = c • drehung v := map_smul
    drehung c v

/- Kern und Bild sind Unterraeume, beschrieben durch ihre Elemente. -/
example (v : Fin 2 → ℝ) : v ∈ LinearMap.ker drehung ↔ drehung v = 0 :=
    LinearMap.mem_ker
example (w : Fin 2 → ℝ) : w ∈ LinearMap.range drehung ↔ ∃ v, drehung v = w :=
  LinearMap.mem_range
```

Die Standardbasis, die Dimension und die Dimensionsformel stehen in der Bibliothek bereit:

Lean code/Kapitel3.lean

```
/- Die Standardbasis von ℝ³: Der i-te Vektor hat an der Stelle i eine 1,
   sonst 0. -/
#check (Pi.basisFun ℝ (Fin 3) : Module.Basis (Fin 3) ℝ (Fin 3 → ℝ))
example (i : Fin 3) : Pi.basisFun ℝ (Fin 3) i = Pi.single i 1 :=
    Pi.basisFun_apply ℝ (Fin 3) i
```

```
/- Die Dimension von ℝ³ ist 3. -/
example : Module.finrank ℝ (Fin 3 → ℝ) = 3 := Module.finrank_fin_fun ℝ

/- Dimensionsformel: Fuer jede lineare Abbildung f : ℝ³ → ℝ² ist
   dim (Bild f) + dim (Kern f) = 3. -/
example (f : (Fin 3 → ℝ) →ₗ[ℝ] (Fin 2 → ℝ)) :
    Module.finrank ℝ (LinearMap.range f) + Module.finrank ℝ (LinearMap.ker f) = 3
    := by
  rw [LinearMap.finrank_range_add_finrank_ker, Module.finrank_fin_fun]
```

Die Determinante rechnet `#eval` in $\mathbb{Z}$ aus. Sie ist multiplikativ, und eine Matrix ist genau dann invertierbar, wenn ihre Determinante nicht verschwindet:

Lean code/Kapitel3.lean

```
/- Die Determinante; in ℤ rechnet `#eval` sie aus. -/
#eval A.det          -- -2

/- Fuer 2×2-Matrizen ist sie ad - bc. -/
example (a b c d : ℝ) : !![a, b; c, d].det = a * d - b * c :=
    Matrix.det_fin_two_of a b c d

/- Sie ist multiplikativ, und eine Matrix ist genau dann invertierbar
   (`IsUnit`), wenn ihre Determinante nicht 0 ist. -/
example (M N : Matrix (Fin 3) (Fin 3) ℝ) : (M * N).det = M.det * N.det :=
    Matrix.det_mul M N
example (M : Matrix (Fin 3) (Fin 3) ℝ) : IsUnit M ↔ M.det ≠ 0 := by
  rw [Matrix.isUnit_iff_isUnit_det, isUnit_iff_ne_zero]
```

Ordnungen

Auch Ordnungen sind Typklassen, die mit `extends` aufeinander aufbauen:

$$\text{Preorder} \to \text{PartialOrder} \to \text{LinearOrder}, \qquad \text{Lattice} \to \text{CompleteLattice}$$

Lean code/Kapitel3.lean

```
/- Ordnungen sind ebenso gestuft. -/
#check @Preorder
#check @PartialOrder
#check @LinearOrder
#check @Lattice
#check @CompleteLattice

/- Die Teilmengen eines Typs bilden einen vollstaendigen Verband. -/
example : CompleteLattice (Set ℕ) := inferInstance
```

Mixins: kombinieren statt vererben

Ein geordnetes Monoid ist ein Monoid *und* eine Ordnung *und* die Verträglichkeit von beidem: Aus $a \le b$ folgt $c \cdot a \le c \cdot b$. Man könnte dafür eine Klasse `OrderedMonoid` bauen, die von beiden erbt. Mathlib macht es nicht so, sondern schreibt drei getrennte Instanzargumente:

Lean code/Kapitel3.lean

```
/- **Mixins**: Struktur wird kombiniert, nicht vererbt. Ein geordnetes
   Monoid ist ein Monoid *und* eine Ordnung *und* die Vertraeglichkeit
   beider: aus a <= b folgt c*a <= c*b. Das sind drei getrennte
   Instanzargumente. `IsOrderedMonoid` traegt selbst keine Daten, es ist
   eine `Prop`. -/
```

```
#check @IsOrderedMonoid

example {M : Type*} [CommMonoid M] [PartialOrder M] [IsOrderedMonoid M]
    (a b c : M) (h : a ≤ b) : c * a ≤ c * b := by gcongr
```

`IsOrderedMonoid` trägt selbst keine Daten – es ist eine `Prop` und sagt nur, dass Multiplikation und Ordnung zusammenpassen. So eine datenfreie Zusatzklasse heißt *Mixin*.

Der Grund für diesen Entwurf ist das *Diamantproblem*. Erbte `OrderedMonoid` sowohl von `CommMonoid` als auch von `PartialOrder`, dann gäbe es bei einer Struktur, die auf zwei Wegen entsteht, zwei Kopien derselben Multiplikation. Lean müsste entscheiden, welche gilt – und die beiden wären nicht definitorisch gleich. Mit Mixins tritt das nicht auf: Die Multiplikation kommt genau einmal, nämlich aus `[CommMonoid M]`.

Mathlib. Daran erkennt man Mathlib-Code. Ein Satz beginnt typischerweise mit einer Reihe von Instanzargumenten, zum Beispiel `{K : Type*} [Field K] {V : Type*} [AddCommGroup V] [Module K V]`. Das ist keine Umständlichkeit, sondern sagt genau, welche Struktur der Beweis braucht – und nicht mehr.

Unterobjekte und Morphismen

Zwei weitere Muster, die überall auftauchen.

Unterobjekte sind gebündelt. `Subgroup G` ist ein eigener Typ, kein Prädikat auf `Set G`. Ein Element davon enthält die Trägermenge und die Beweise, dass sie das Neutrale enthält und unter Produkt und Inversem abgeschlossen ist – alles in einem Objekt. Deshalb kann man mit Untergruppen rechnen, etwa Schnitte und Erzeugnisse bilden.

Morphismen sind eigene Typen. Ein Gruppenhomomorphismus ist keine Funktion mit einer Zusatzbedingung daneben, sondern ein Element von `G →* H`, das die Funktion und den Beweis zusammenfasst. Angewendet wird er trotzdem wie eine Funktion, über eine Koerzierung.

Lean code/Kapitel3.lean

```
/- Unterobjekte sind **gebuendelt**: `Subgroup G` ist ein eigener Typ und
   kein Praedikat auf `Set G`. Traegermenge und Abgeschlossenheitsbeweise
   stecken zusammen in einem Objekt. -/
#check @Subgroup
#check @Submodule

example {G : Type*} [Group G] (H : Subgroup G) (a : G) : Prop := a ∈ H

/- Morphismen ebenso: eigene Typen mit Koerzierung zur Funktion.
   `MonoidHom` schreibt man `G →* H`, dazu gibt es `LinearMap` und
   `ContinuousLinearMap`. -/
#check @MonoidHom
#check @LinearMap
#check @ContinuousLinearMap

example {G H : Type*} [Group G] [Group H] (f : G →* H) (a b : G) :
    f (a * b) = f a * f b := map_mul f a b
```

Bemerkung 3.13 (Namen finden). Mathlib benennt Sätze nach ihrer Aussage, in Kleinschreibung mit Unterstrichen: `mul_comm`, `add_assoc`, `mul_inv_cancel`, `map_mul`. Wer die Konvention kennt, rät den Namen meist richtig. Sonst helfen `exact?` und `apply?`, die passende Lemmata suchen.

Verifizierte Programme

Lean ist auch eine Programmiersprache. Ein Programm ist ein Term wie jeder andere, und eine Aussage über das Programm ist ein Typ. Damit lässt sich beweisen, dass ein Programm seine Spezifikation erfüllt – für alle Eingaben, nicht nur für die getesteten.

Ein eigener Datentyp entsteht wie $\mathbb{N}$ aus seinen Konstruktoren. Funktionen darauf sind rekursiv, mit einem Fall pro Konstruktor, und `#eval` führt sie aus:

Lean code/Kapitel3.lean

```
/-- Ein eigener Datentyp: Binaerbaeume mit natuerlichen Zahlen in den Knoten. -/
inductive Baum where
  | blatt : Baum
  | knoten : Baum → ℕ → Baum → Baum

/- Rekursive Funktionen: ein Fall pro Konstruktor. -/
def Baum.groesse : Baum → ℕ
  | .blatt => 0
  | .knoten l _ r => l.groesse + 1 + r.groesse

def Baum.spiegeln : Baum → Baum
  | .blatt => .blatt
  | .knoten l w r => .knoten r.spiegeln w l.spiegeln

def Baum.inorder : Baum → List ℕ
  | .blatt => []
  | .knoten l w r => l.inorder ++ [w] ++ r.inorder

def beispielBaum : Baum := .knoten (.knoten .blatt 1 .blatt) 2 (.knoten .blatt 3
    .blatt)

#eval beispielBaum.groesse              -- 3
#eval beispielBaum.inorder              -- [1, 2, 3]
#eval beispielBaum.spiegeln.inorder     -- [3, 2, 1]
```

Aussagen über alle Bäume beweist man per Induktion über den Aufbau. Im Schritt gibt es zwei Induktionsannahmen, eine für jeden Teilbaum:

Lean code/Kapitel3.lean

```
/-- Zweimal spiegeln ergibt den alten Baum -- fuer jeden Baum, per Induktion.
Im Schritt gibt es zwei Induktionsannahmen, eine fuer jeden Teilbaum. -/
theorem Baum.spiegeln_spiegeln (t : Baum) : t.spiegeln.spiegeln = t := by
  induction t with
  | blatt => rfl
  | knoten l w r ihl ihr => simp only [Baum.spiegeln, ihl, ihr]

/-- Spiegeln kehrt die Reihenfolge der Inorder-Liste um. -/
theorem Baum.inorder_spiegeln (t : Baum) : t.spiegeln.inorder = t.inorder.reverse
    := by
  induction t with
  | blatt => rfl
  | knoten l w r ihl ihr => simp [Baum.spiegeln, Baum.inorder, ihl, ihr]

/-- Spiegeln aendert die Groesse nicht. -/
theorem Baum.groesse_spiegeln (t : Baum) : t.spiegeln.groesse = t.groesse := by
  induction t with
  | blatt => rfl
  | knoten l w r ihl ihr =>
      simp only [Baum.spiegeln, Baum.groesse, ihl, ihr]
      omega
```

Ein Suchbaum ordnet die Zahlen: links die kleineren, rechts die größeren. Einfügen und Suchen gehen denselben Weg. Deshalb findet man wieder, was man eingefügt hat – in jedem Baum, sogar in einem unsortierten. `split_ifs` zerlegt dabei die `if`-Abfragen in drei Fälle.

Lean code/Kapitel3.lean

```
/-- Ein Suchbaum: links die kleineren Zahlen, rechts die groesseren.
Einfuegen und Suchen gehen denselben Weg. -/
def Baum.einfuegen (x : ℕ) : Baum → Baum
  | .blatt => .knoten .blatt x .blatt
  | .knoten l w r =>
      if x < w then .knoten (l.einfuegen x) w r
      else if w < x then .knoten l w (r.einfuegen x) else .knoten l w r

def Baum.enthaelt (x : ℕ) : Baum → Bool
  | .blatt => false
  | .knoten l w r =>
      if x < w then l.enthaelt x else if w < x then r.enthaelt x else true

/-- Sortieren durch Einfuegen in einen Suchbaum. -/
def baumAusListe (xs : List ℕ) : Baum := xs.foldl (fun t x => t.einfuegen x)
    .blatt

#eval (baumAusListe [5, 3, 8, 1, 4]).inorder     -- [1, 3, 4, 5, 8]
#eval (baumAusListe [5, 3, 8]).enthaelt 7        -- false

/-- Was man einfuegt, findet man wieder -- fuer jeden Baum. -/
theorem Baum.enthaelt_einfuegen (x : ℕ) (t : Baum) :
    (t.einfuegen x).enthaelt x = true := by
  induction t with
  | blatt => simp [Baum.einfuegen, Baum.enthaelt]
  | knoten l w r ihl ihr =>
      unfold Baum.einfuegen
      split_ifs with h₁ h₂
      · simp [Baum.enthaelt, h₁, ihl]        -- links eingefuegt
      · simp [Baum.enthaelt, h₁, h₂, ihr]    -- rechts eingefuegt
      · simp [Baum.enthaelt, h₁, h₂]         -- x war schon da
```

Zusammenfassung

- Jeder Term hat einen Typ. Typen stehen in einer Universenhierarchie.
- Abhängige Typen erlauben es, Voraussetzungen in den Typ zu schreiben.
- Curry–Howard: Aussagen sind Typen, Beweise sind Terme. Beweisen heißt, einen Term angeben.
- `Prop` ist beweisirrelevant und deshalb von `Type` getrennt.
- Über die Typentheorie hinaus braucht Mathlib nur `propext`, `Classical.choice` und `Quot.sound`. Mit `propext` und `funext` (aus `Quot.sound`) macht die Auswahl die Logik klassisch – Diaconescus Argument.
- Auch Typen haben eine Mengensemantik: Typen bedeuten Mengen, Terme Abbildungen; sie ist ein Funktor vom Termmodell in die Mengen.
- Mathlib baut die Mathematik aus Typklassen: Zahlbereiche, algebraische Hierarchie, Ordnungen. Nach demselben Muster sind die Räume der Analysis gebaut (Kapitel 4).

- Zusatzstruktur kommt als *Mixin* dazu, statt vererbt zu werden. Das vermeidet das Diamantproblem.
- Unterobjekte und Morphismen sind gebündelte Typen, keine Prädikate neben einer Funktion.
- Lineare Algebra in $\mathbb{R}^n$: Vektoren und Matrizen sind Funktionen auf `Fin n`; lineare Abbildungen, Dimension und Determinante stehen in Mathlib bereit.
- Programme sind Terme wie alle anderen: eigene Datentypen, rekursive Funktionen, und ihre Korrektheit beweist man per Induktion.
- Abschnitt 3.6 formalisiert Kapitel 2: Mengen sind Prädikate, Russell scheitert am Typcheck, ZFC und NBG kommen als `ZFSet` und `Class` vor.

Kapitel 4

Infinitesimalrechnung

In der Analysis weicht Mathlib am stärksten vom Lehrbuch ab. Der Grund ist ein einziger Begriff: der *Filter*. Er ersetzt die vielen Grenzwertdefinitionen durch eine, und er macht die Ableitung zu einem Spezialfall statt zu einem Neuanfang. Der Begleitcode baut die tragenden Begriffe deshalb zuerst selbst – Topologie, offene Menge, Filter, Konvergenz, Cauchy-Folge, Kontraktion – und vergleicht sie dann mit Mathlib, so wie `MeinNat` in Abschnitt 3.2 mit `Nat`. Landau-Symbole und Ableitung nimmt er direkt aus Mathlib.

Das Kapitel baut aufeinander auf. Zuerst die Räume, in denen überhaupt von Nähe die Rede sein kann. Dann die Konvergenz. Dann die Vollständigkeit, die ohne Konvergenz nicht zu erklären wäre. Erst danach Ableitung, Optimierung und Nebenbedingungen.

4.1 Räume

Wozu das gut ist

Fast jeder Satz der Analysis handelt von Nähe. „Liegt x nah genug bei a, dann liegt $f(x)$ nah bei $f(a)$." „Die Folge kommt a beliebig nahe." „Der Fehler wird beliebig klein." Nimmt man den Begriff „nah" weg, bleibt von der Analysis nichts übrig.

Auf der Zahlengeraden ist er leicht zu haben: $|x - y|$. Schwierig wird es, sobald die Punkte keine Zahlen mehr sind. Drei Fälle, die in diesem Skript vorkommen:

- **Punkte im $\mathbb{R}^n$.** Der Gradientenabstieg (Abschnitt 4.5) macht einen Schritt und fragt, ob er dem Minimum näher gekommen ist.
- **Funktionen.** Die Simpson-Regel (Kapitel 5) ersetzt f durch eine Parabel und behauptet, die beiden lägen nah beieinander. Ein „Punkt" ist hier eine ganze Funktion.
- **Lineare Abbildungen.** Differenzierbarkeit (Abschnitt 4.4) heißt: Es gibt eine lineare Abbildung, die f in der Nähe eines Punktes gut genug annähert.

Man könnte für jeden dieser Fälle eine eigene ε-Definition aufschreiben und jeden Satz einzeln beweisen. Genau das ist die Arbeit, die man sich spart.

Der Ausweg ist, nicht auf die Objekte zu sehen, sondern auf das, was man über sie voraussetzt. Für „stetige Bilder kompakter Mengen sind kompakt" braucht man keine Zahlen, nur offene Mengen. Für „jede Cauchy-Folge konvergiert" braucht man einen Abstand. Für die Ableitung braucht man zusätzlich Addition, denn eine Näherung ist eine Differenz. Für „senkrecht" braucht man ein Skalarprodukt.

Jeder Satz hat also ein Minimum, mit dem er auskommt. Je kleiner dieses Minimum, desto mehr Fälle deckt der Satz ab. Wer eine Aussage über metrische Räume beweist, hat sie damit für die Zahlengerade, für den $\mathbb{R}^n$, für Funktionenräume und für alles bewiesen, was später dazukommt. Daraus werden vier Stufen.

Vier Werkzeugkästen. Im ersten Kasten liegt nichts. Man weiß nur, welche Punkte zusammengehören, und darf fragen, ob etwas in der Nähe liegt – nicht, wie weit.

Im zweiten liegt ein Maßband. Abstände sind jetzt Zahlen, und Zahlen kann man vergleichen.

Im dritten kommt die Erlaubnis dazu, Punkte zu verschieben und zu strecken. Aus zwei Punkten wird ihre Summe, aus einem sein Doppeltes.

Im vierten liegt ein Winkelmesser. Damit lässt sich sagen, ob zwei Richtungen senkrecht aufeinander stehen.

Die vier Stufen

Am leichtesten merkt man sie sich über die Frage, die auf ihnen erlaubt ist:

Stufe	**Welche Frage ist erlaubt?**	**in Mathlib**
topologischer Raum	Liegt x *nahe* bei y?	`TopologicalSpace`
metrischer Raum	*Wie weit* ist x von y?	`MetricSpace`
normierter Raum	Wie *lang* ist x? Was ist $x + y$?	`NormedAddCommGroup`
Innenproduktraum	Welchen *Winkel* schließen x und y ein?	`InnerProductSpace`

Die Länge allein ist `NormedAddCommGroup`; Länge und Skalare zusammen ($\|\lambda x\| = |\lambda|\,\|x\|$) sind `NormedSpace`.

Jede Stufe liefert die darunterliegenden mit: Ein Skalarprodukt bestimmt eine Norm, eine Norm einen Abstand, ein Abstand eine Topologie. Umgekehrt gilt nichts davon. Deshalb gilt jeder Satz über metrische Räume auch in einem normierten Raum, und man muss in Lean nichts dafür tun – die Typklassensuche (Abschnitt 3.2) findet die Instanzen selbst. Die Schritte von der Norm zum Abstand und vom Abstand zur Topologie rechnet das Ende des Abschnitts nach; zuerst kommen die Definitionen.

Topologischer Raum

Definition 4.1 (Topologischer Raum)**.** Eine *Topologie* auf einer Menge X ist ein System $\mathcal{O}$ von Teilmengen – den *offenen* Mengen – mit

(i) $\emptyset \in \mathcal{O}$ und $X \in \mathcal{O}$,

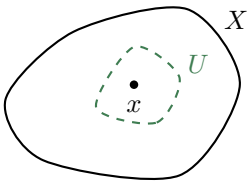
(ii) beliebige Vereinigungen offener Mengen sind offen,

(iii) endliche Schnitte offener Mengen sind offen.

Eine *Umgebung* von x ist eine Menge, die eine offene Menge mit x darin enthält.

Nähe ohne Zahlen. Man weiß nur, welche Mengen offen sind – also für welche gilt: Wer drin ist, hat nach allen Seiten noch Platz. Mehr nicht. Es gibt keine Zahlen, keine Radien, kein „näher als“.

Der Punkt x hat eine Umgebung U, und das heißt nur: x steckt nicht am Rand von U fest, sondern hat ringsum Luft. Wie viel, sagt niemand.



Beispiel 4.2 (Alles offen, fast nichts offen)**.** Auf jeder Menge X gibt es zwei Topologien, die man sofort hinschreiben kann. In der *diskreten* ist jede Teilmenge offen, auf $\mathbb{N}$ also auch $\{7\}$. Kein Punkt liegt nahe bei einem anderen, und Konvergenz heißt dort: irgendwann konstant. In der *indiskreten* sind nur $\emptyset$ und X offen, $\{7\}$ also nicht. Dort liegt jeder Punkt nahe bei jedem.

Dazwischen liegt $\mathbb{R}$. Das Intervall $(0,1)$ ist offen: Wer darin liegt, hat nach beiden Seiten noch Platz. $[0,1]$ ist nicht offen, denn bei 0 ist Schluss; abgeschlossen ist es aber schon. Die Topologie von $\mathbb{R}$ kommt aus dem Abstand. Deshalb folgt der Beweis, dass $(0,1)$ offen ist und $[0,1]$ nicht, erst beim metrischen Raum.

In Lean wird die Definition wörtlich eine Struktur, wie später der Filter: ein Mengensystem `offen` und Beweisfelder für (i) bis (iii). Bei der diskreten Topologie ist jedes Feld trivial, denn jede Menge ist offen.

Lean code/Kapitel4.lean

```
/-- Eine Topologie: ein System offener Mengen mit den Axiomen (i)-(iii).
‘Set.sUnion S‘ ist die Vereinigung aller Mengen der Familie ‘S‘. -/
structure Topologie (X : Type*) where
  offen : Set (Set X)
  leer_offen : ∅ ∈ offen                                              -- (i)
  univ_offen : Set.univ ∈ offen                                       -- (i)
  vereinigung {S : Set (Set X)} : S ⊆ offen → Set.sUnion S ∈ offen  -- (ii)
  schnitt {U V : Set X} : U ∈ offen → V ∈ offen → U ∩ V ∈ offen     -- (iii)

/-- Diskret: Alle Mengen sind offen, jedes Axiom gilt trivial. -/
def diskret (X : Type*) : Topologie X where
  offen := Set.univ                          -- alle Teilmengen von X
  leer_offen := trivial
  univ_offen := trivial
  vereinigung _ := trivial
  schnitt _ _ := trivial
```

Bei der indiskreten braucht nur die Vereinigung eine Idee, eine Fallunterscheidung. Jede Menge der Familie ist $\emptyset$ oder X. Kommt X darin vor, ist die Vereinigung X. Sonst sind alle Mengen leer, und die Vereinigung ist $\emptyset$. Beim Schnitt sind es vier Fälle, die `simp` erledigt. Zum Schluss wird $\{7\}$ in beiden Topologien auf $\mathbb{N}$ geprüft:

Lean code/Kapitel4.lean

```
/-- Indiskret: Nur ∅ und X sind offen. -/
def indiskret (X : Type*) : Topologie X where
  offen := {U | U = ∅ ∨ U = Set.univ}
  leer_offen := Or.inl rfl
  univ_offen := Or.inr rfl
  vereinigung := by
    intro S hS                              -- jede Menge in S ist ∅ oder X
    rcases Classical.em (Set.univ ∈ S) with h | h  -- X in S oder nicht
    · -- X liegt in S: Jedes x liegt in X, also in der Vereinigung.
      right
      exact Set.eq_univ_of_forall fun x => ⟨Set.univ, h, trivial⟩
    · -- X fehlt: Dann ist jede Menge in S leer, also auch die Vereinigung.
      left
      rw [Set.sUnion_eq_empty]
      intro U hU
      rcases hS hU with h0 | rfl          -- U = ∅ oder U = X
      · exact h0
      · contradiction                     -- X liegt aber nicht in S
  schnitt := by                           -- U, V je ∅ oder X: vier Faelle
    rintro U V (rfl | rfl) (rfl | rfl) <;> simp
```

```
/- Die Menge {7} ist offen in ‘diskret ℕ‘, aber nicht in ‘indiskret ℕ‘:
   Sie ist weder leer noch ganz ℕ. -/
example : {7} ∈ (diskret ℕ).offen := trivial
example : {7} ∉ (indiskret ℕ).offen := by
  rintro (h | h)                          -- {7} = ∅ oder {7} = ℕ
  · exact Set.singleton_ne_empty 7 h      -- nein: 7 liegt in {7}
  · have h8 : 8 ∈ ({7} : Set ℕ) := by rw [h]; trivial  -- 8 ∈ ℕ = {7}
    simp at h8                            -- also 8 = 7: Widerspruch
```

Zum Abgleich: Mathlibs `TopologicalSpace` hat dieselben Felder, nur keines für $\emptyset$. Es wäre überflüssig, denn $\emptyset$ ist die Vereinigung der leeren Familie und nach Axiom (ii) schon offen. Feld für Feld wird so jede `Topologie` eine Mathlib-Topologie. Mathlib nennt eine Topologie mit mehr offenen Mengen *kleiner*. Die diskrete ist deshalb das kleinste Element $\bot$, und zwar per Definition (`rfl`).

Lean code/Kapitel4.lean

```
/- Mathlibs ‘TopologicalSpace‘ hat die Felder ‘IsOpen‘, ‘isOpen_univ‘,
   ‘isOpen_inter‘ und ‘isOpen_sUnion‘, aber keines fuer ∅. Das folgt aus
   (ii): ∅ ist die Vereinigung der leeren Familie. -/
example {X : Type*} (T : Topologie X) : ∅ ∈ T.offen := by
  have h := T.vereinigung (Set.empty_subset T.offen)  -- (ii), leere Familie
  rwa [Set.sUnion_empty] at h                         -- ihre Vereinigung: ∅

/-- Feld fuer Feld wird eine ‘Topologie‘ eine Mathlib-Topologie;
‘leer_offen‘ wird dafuer nicht gebraucht. -/
def Topologie.toTopologicalSpace {X : Type*} (T : Topologie X) :
    TopologicalSpace X where
  IsOpen U := U ∈ T.offen
  isOpen_univ := T.univ_offen
  isOpen_inter _ _ := T.schnitt
  isOpen_sUnion _ := T.vereinigung

/- Mathlib ordnet Topologien so, dass mehr offene Mengen kleiner heisst. Die
   diskrete hat alle Mengen offen, ist also ‘⊥‘ -- sogar per Definition: -/
example (X : Type*) : (diskret X).toTopologicalSpace = ⊥ := rfl
```

Metrischer Raum

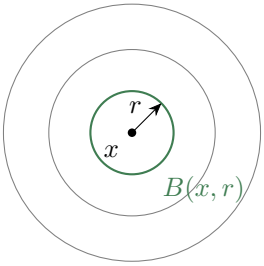
Definition 4.3 (Metrischer Raum)**.** Eine *Metrik* auf X ist eine Abbildung $d : X \times X \to \mathbb{R}$ mit

$$d(x,y) \geq 0, \qquad d(x,y) = 0 \iff x = y, \qquad d(x,y) = d(y,x), \qquad d(x,z) \leq d(x,y) + d(y,z).$$

Die *Kugel* $B(x,r)$ mit Radius $r > 0$ besteht aus allen y mit $d(x,y) < r$. Eine Menge U heißt *offen*, wenn um jeden Punkt von U eine Kugel ganz in U passt; so erzeugen die Kugeln eine Topologie.

Jetzt gibt es eine Zahl. Umgebungen werden zu Kugeln mit einem Radius, den man kleiner machen kann. Erst hier lässt sich sagen, *wie* nah etwas ist. Jeder metrische Raum ist damit auch ein topologischer – die Umkehrung gilt nicht.

Was noch fehlt: Man kann Punkte nicht addieren und nicht strecken. $x + y$ ergibt keinen Sinn.



Beispiel 4.4 (Betrag und Pythagoras)**.** Auf $\mathbb{R}$ ist $d(x,y) = |x-y|$, also $d(3,7) = 4$. In der Ebene ist

$$d(x,y) = \sqrt{(x_1-y_1)^2 + (x_2-y_2)^2},$$

der Satz des Pythagoras, hier als Definition.

Jetzt lässt sich beweisen, was oben nur behauptet war. $(0,1)$ ist offen: Um $x \in (0,1)$ passt die Kugel vom Radius $r = \min(x, 1-x)$, dem Abstand zum näheren Rand. Wegen $0 < x < 1$ ist $r > 0$, und aus $|y-x| < r$ folgt

$$0 \le x - r < y < x + r \le 1.$$

$[0,1]$ ist nicht offen: Läge eine Kugel $B(0,\varepsilon)$ ganz in $[0,1]$, dann läge $-\varepsilon/2$ darin – und das ist negativ.

Koordinaten. Lean zählt ab 0: Was hier x_1 und x_2 heißt, steht im Code als `x 0` und `x 1`.

In Lean gilt $d(x,y) = |x-y|$ auf $\mathbb{R}$ per Definition (`rfl`). „Offen“ heißt im Code `Offen`, wörtlich wie in Definition 4.3. Die Intervalle $(0,1)$ und $[0,1]$ heißen in Mathlib `Set.Ioo 0 1` und `Set.Icc 0 1` (o für „open“, c für „closed“). Der Beweis für $[0,1]$ nimmt gleich Mathlibs `IsOpen`; `Metric.isOpen_iff` übersetzt es in die Bedingung von `Offen`.

Lean code/Kapitel4.lean

```
/- Auf ℝ ist der Abstand per Definition |x - y| (‘Real.dist_eq‘). -/
example (x y : ℝ) : dist x y = |x - y| := rfl
example : dist (3:ℝ) 7 = 4 := by rw [Real.dist_eq]; norm_num

/- In der Ebene ist es der Satz des Pythagoras. -/
example (x y : EuclideanSpace ℝ (Fin 2)) :
    dist x y = √((x 0 - y 0) ^ 2 + (x 1 - y 1) ^ 2) := by
  rw [EuclideanSpace.dist_eq]
  simp [Fin.sum_univ_two, Real.dist_eq, sq_abs]

/-- U heisst offen, wenn um jeden Punkt von U eine Kugel ganz in U passt. -/
def Offen {X : Type*} [MetricSpace X] (U : Set X) : Prop :=
  ∀ x ∈ U, ∃ ε > 0, Metric.ball x ε ⊆ U

/- (0,1) ist offen: Um x passt die Kugel vom Radius r = min(x, 1 - x),
   dem Abstand zum naeheren Rand. -/
example : Offen (Set.Ioo (0:ℝ) 1) := by
  rintro x ⟨h0, h1⟩                                  -- 0 < x < 1
  use min x (1 - x), lt_min h0 (by linarith)          -- r = min(x, 1 - x) > 0
  intro y hy
  rw [Metric.mem_ball, Real.dist_eq, abs_lt] at hy   -- -r < y - x < r
  constructor
  · linarith [hy.1, min_le_left x (1 - x)]           -- 0 ≤ x - r < y
  · linarith [hy.2, min_le_right x (1 - x)]          -- y < x + r ≤ 1
```

```
/- Erst mit der Metrik laesst sich sagen, warum [0,1] nicht offen ist:
   um 0 passt keine Kugel hinein. Diesmal mit Mathlibs 'IsOpen';
   'Metric.isOpen_iff' uebersetzt es in die Bedingung von 'Offen'. -/
example : ¬ IsOpen (Set.Icc (0:ℝ) 1) := by
  intro h
  obtain ⟨ε, hε, hsub⟩ := Metric.isOpen_iff.1 h 0 (by norm_num)
  have h1 : (-(ε / 2) : ℝ) ∈ Metric.ball (0:ℝ) ε := by
    rw [Metric.mem_ball, Real.dist_eq, abs_of_nonpos (by linarith)]
    linarith
  have h2 := hsub h1
  rw [Set.mem_Icc] at h2
  linarith [h2.1]
```

Normierter Raum

Definition 4.5 (Normierter Raum)**.** Ein Vektorraum V über $\mathbb{R}$ mit einer *Norm* $\|\cdot\| : V \to \mathbb{R}$, sodass

$$\|x\| = 0 \iff x = 0, \qquad \|\lambda x\| = |\lambda|\,\|x\|, \qquad \|x + y\| \le \|x\| + \|y\|.$$

Der Abstand ist dann $d(x, y) = \|x - y\|$.

Anschaulich. Jetzt darf man addieren und strecken. Weil der Abstand aus der Länge kommt, sieht der Raum überall gleich aus: Eine Kugel um 0, verschoben, ist eine Kugel um jeden anderen Punkt.

Beispiel 4.6 (Zahlen, Pfeile, Funktionen)**.** Auf $\mathbb{R}$ ist $\|x\| = |x|$. In der Ebene ist $\|x\| = \sqrt{x_1^2 + x_2^2}$, also $\|(3, 4)\| = 5$.

Der dritte Fall ist der wichtigste. Die stetigen Funktionen auf $[0, 1]$ mit

$$\|f\| = \max_{t \in [0,1]} |f(t)|$$

bilden ebenfalls einen normierten Raum. Ein Punkt dieses Raumes ist eine ganze Funktion; addiert und gestreckt wird punktweise, und $\|f\|$ misst, wie weit f höchstens von der Nulllinie abweicht. Darauf beruht später jede Aussage über Näherungsverfahren: „Fehler klein“ heißt „Abstand in einem Funktionenraum klein“.

Auch $\|x\| = |x|$ gilt in Lean per Definition (`rfl`). Die Supremumsnorm wird nur zitiert: Dass das Maximum existiert, braucht die Kompaktheit von $[0, 1]$.

Lean code/Kapitel4.lean

```
/- Auf ℝ ist die Laenge per Definition der Betrag ('Real.norm_eq_abs'). -/
example (x : ℝ) : ‖x‖ = |x| := rfl

/- In der Ebene: Laenge ueber die Koordinaten, und ein Zahlenbeispiel. -/
example (x : EuclideanSpace ℝ (Fin 2)) : ‖x‖ = √(x 0 ^ 2 + x 1 ^ 2) := by
  rw [EuclideanSpace.norm_eq]
  simp [Fin.sum_univ_two]

example : ‖(!₂[(3:ℝ), 4] : EuclideanSpace ℝ (Fin 2))‖ = 5 := by
  rw [EuclideanSpace.norm_eq]
  simp [Fin.sum_univ_two]
  rw [show (3:ℝ) ^ 2 + 4 ^ 2 = 5 ^ 2 by norm_num]
  exact Real.sqrt_sq (by norm_num)
```

```
/- Auch Funktionen sind Vektoren: die stetigen Funktionen auf [0,1] mit der
   Supremumsnorm. Nur zitiert -- das Maximum braucht Kompaktheit. -/
#check (inferInstance : NormedAddCommGroup C(Set.Icc (0:ℝ) 1, ℝ))
example (f : C(Set.Icc (0:ℝ) 1, ℝ)) (x) : ‖f x‖ ≤ ‖f‖ := f.norm_coe_le_norm x
```

Innenproduktraum

Definition 4.7 (Innenproduktraum)**.** Ein Vektorraum mit einer Abbildung $\langle \cdot, \cdot \rangle : V \times V \to \mathbb{R}$, die symmetrisch, in jedem Argument linear und positiv definit ist. Die Norm ist dann $\|x\| = \sqrt{\langle x, x \rangle}$.

Beispiel 4.8 (Senkrecht in Koordinaten)**.** In der Ebene ist $\langle x, y \rangle = x_1 y_1 + x_2 y_2$. Die beiden Einheitsvektoren $(1, 0)$ und $(0, 1)$ stehen senkrecht aufeinander, denn das Produkt ist 0.

Zwei Sätze gelten ab hier und vorher nicht:

$$|\langle x, y \rangle| \le \|x\| \, \|y\| \qquad \text{und} \qquad \langle x, y \rangle = 0 \implies \|x + y\|^2 = \|x\|^2 + \|y\|^2.$$

Das erste ist die Ungleichung von Cauchy–Schwarz, das zweite der Satz des Pythagoras – jetzt nicht mehr als Definition, sondern als Folgerung.

Die beiden Sätze beweist der Code in jedem reellen Innenproduktraum, Cauchy–Schwarz von Hand: $\|x + t\,y\|^2 = \|y\|^2 t^2 + 2\langle x, y \rangle t + \|x\|^2$ ist ein Polynom in t ohne negative Werte. Also ist seine Diskriminante höchstens 0, das heißt $\langle x, y \rangle^2 \le \|x\|^2 \, \|y\|^2$. Wurzelziehen liefert die Fassung mit Betrag.

Lean code/Kapitel4.lean

```
/- Das Skalarprodukt in Koordinaten. -/
example (x y : EuclideanSpace ℝ (Fin 2)) :
    inner ℝ x y = x 0 * y 0 + x 1 * y 1 := by
  simp [EuclideanSpace.inner_eq_star_dotProduct, mul_comm]

/- Die beiden Einheitsvektoren stehen senkrecht aufeinander. -/
example : inner ℝ (EuclideanSpace.single 0 (1:ℝ)) : EuclideanSpace ℝ (Fin 2))
    (EuclideanSpace.single 1 (1:ℝ)) = (0:ℝ) := by
  simp [EuclideanSpace.inner_single_left, EuclideanSpace.single_apply]

/- Ab hier ist V ein beliebiger reeller Innenproduktraum. -/
variable {V : Type*} [NormedAddCommGroup V] [InnerProductSpace ℝ V]

/- Pythagoras fuer senkrechte Vektoren. ‘norm_add_sq_real‘ ist die Rechnung
   aus den Axiomen: ‖x + y‖^2 = ‖x‖^2 + 2⟨x,y⟩ + ‖y‖^2. -/
example (x y : V) (h : inner ℝ x y = 0) :
    ‖x + y‖ ^ 2 = ‖x‖ ^ 2 + ‖y‖ ^ 2 := by
  rw [norm_add_sq_real, h]; ring

/-- Cauchy-Schwarz, von Hand: ‖x + t • y‖^2 ist ein Polynom in t, das nie
negativ wird. Also ist seine Diskriminante ≤ 0 (‘discrim_le_zero‘). -/
theorem cauchy_schwarz (x y : V) : inner ℝ x y ^ 2 ≤ ‖x‖ ^ 2 * ‖y‖ ^ 2 := by
  have h : ∀ t : ℝ, 0 ≤ ‖y‖ ^ 2 * (t * t) + 2 * inner ℝ x y * t + ‖x‖ ^ 2 := by
    intro t                                  -- a = ‖y‖^2, b = 2⟨x,y⟩, c = ‖x‖^2
    have h0 := sq_nonneg ‖x + t • y‖         -- 0 ≤ ‖x + t • y‖^2
    rw [norm_add_sq_real, real_inner_smul_right, norm_smul, mul_pow,
      Real.norm_eq_abs, sq_abs] at h0        -- 0 ≤ ‖x‖^2 + 2t⟨x,y⟩ + t^2‖y‖^2
    linarith                                 -- nur umgeordnet
  have hd : (2 * inner ℝ x y) ^ 2 - 4 * ‖y‖ ^ 2 * ‖x‖ ^ 2 ≤ 0 :=
    discrim_le_zero h                        -- b^2 - 4ac ≤ 0
  linarith                                   -- durch 4 teilen
```

```
/- Die Fassung mit Betrag; in Mathlib heisst sie 'abs_real_inner_le_norm'. -/
example (x y : V) : |inner ℝ x y| ≤ ‖x‖ * ‖y‖ := by
  have h := cauchy_schwarz x y
  rw [← mul_pow] at h                          -- ⟨x,y⟩² ≤ (‖x‖ ‖y‖)²
  exact abs_le_of_sq_le_sq h (by positivity) -- Wurzel ziehen, ‖x‖ ‖y‖ ≥ 0
```

Das Skalarprodukt bringt Winkel. Und damit den Begriff, auf dem alles Weitere beruht: *senkrecht*. Zwei Vektoren stehen senkrecht aufeinander, wenn $\langle x, y\rangle = 0$.

Was daraus folgt – die Projektion auf einen Teilraum – kommt in Abschnitt 4.3. Dafür fehlt noch die Vollständigkeit, und die braucht Konvergenz.

Wie eine Stufe die nächste liefert

Jetzt stehen alle vier Begriffe da. Dass die Norm mit $d(x, y) = \|x - y\|$ einen Abstand liefert, sind kurze Rechnungen. Die Dreiecksungleichung etwa folgt aus der der Norm:

$$d(x, z) = \|(x - y) + (y - z)\| \leq \|x - y\| + \|y - z\| = d(x, y) + d(y, z).$$

Eine Idee braucht erst der Schritt vom Abstand zur Topologie. Offen heißt im metrischen Raum: Um jeden Punkt passt eine Kugel (Definition 4.3). Die Axiome (i) und (ii) der Topologie (Definition 4.1) folgen sofort. Für (iii) braucht man eine kleine Rechnung: Passt um x eine Kugel vom Radius ε_1 in U und eine vom Radius ε_2 in V, dann passt die mit $\min(\varepsilon_1, \varepsilon_2)$ in $U \cap V$. Für unendlich viele Mengen können die Radien gegen 0 gehen – $\bigcap_n (-\frac{1}{n}, \frac{1}{n}) = \{0\}$ ist nicht offen. Deshalb verlangt Axiom (iii) nur endliche Schnitte.

Zuletzt der Abgleich mit Mathlib: `IsOpen` ist auf einem metrischen Raum genau `Offen`. Per Definition (`Iff.rfl`) gilt das nicht, denn Mathlib gewinnt die Topologie eines metrischen Raums auf einem Umweg, über die uniforme Struktur.

Der Beweis geht deshalb über Umgebungen. Eine Menge ist genau dann offen, wenn sie Umgebung jedes ihrer Punkte ist. Im metrischen Raum ist U genau dann Umgebung von x, wenn eine Kugel um x in U passt. Zusammen steht da wörtlich `Offen U`; in Lean sind das zwei Umschreibungen und ein `rfl`.

Lean code/Kapitel4.lean

```
/- Norm → Abstand: d(x,y) = ‖x - y‖. Die Dreiecksungleichung fuer d folgt
   aus der fuer die Norm, denn x - z = (x - y) + (y - z). -/
example {E : Type*} [NormedAddCommGroup E] (x y z : E) :
    dist x z ≤ dist x y + dist y z := by
  simp only [dist_eq_norm]                       -- d(x,y) = ‖x - y‖
  calc ‖x - z‖
      = ‖(x - y) + (y - z)‖ := by congr 1; abel  -- Umweg ueber y
    _ ≤ ‖x - y‖ + ‖y - z‖ := norm_add_le _ _     -- Dreiecksungleichung der Norm

/-- Abstand → Topologie, Axiom (iii): Der kleinere Radius passt in beide
Mengen. Bei unendlich vielen koennen die Radien gegen 0 gehen (etwa 1/n). -/
theorem Offen.inter (hU : Offen U) (hV : Offen V) : Offen (U ∩ V) := by
  rintro x ⟨hxU, hxV⟩
  obtain ⟨ε₁, h₁, hU'⟩ := hU x hxU           -- B(x, ε₁) ⊆ U
  obtain ⟨ε₂, h₂, hV'⟩ := hV x hxV           -- B(x, ε₂) ⊆ V
  use min ε₁ ε₂, lt_min h₁ h₂                -- der kleinere Radius
  intro y hy
  exact ⟨hU' (Metric.ball_subset_ball (min_le_left ε₁ ε₂) hy),
    hV' (Metric.ball_subset_ball (min_le_right ε₁ ε₂) hy)⟩
```

```
/- Abgleich mit Mathlib. Kein 'Iff.rfl': Mathlib gewinnt die Topologie
   eines metrischen Raums ueber die uniforme Struktur. Der Weg geht ueber
   Umgebungen ('U ∈ 𝓝 x': U ist Umgebung von x): -/
example : Offen U ↔ IsOpen U := by
  rw [isOpen_iff_mem_nhds]          -- offen: Umgebung jedes seiner Punkte
  simp only [Metric.mem_nhds_iff]   -- Umgebung: enthaelt eine Kugel um x
  rfl                               -- genau die Definition von 'Offen'
```

Eine eigene Instanz für `MetricSpace` ℝ oder `TopologicalSpace` ℝ muss man nirgends angeben – Lean findet sie selbst. Dasselbe min wie in `Offen.inter` kehrt im nächsten Abschnitt wieder, beim Filter der Umgebungen.

4.2 Konvergenz durch Filter

Das Problem

Ein Analysiskurs definiert Konvergenz mehrfach:

$$\lim_{n\to\infty} u_n = a, \quad \lim_{x\to a} f(x) = L, \quad \lim_{x\to a^+} f(x) = L, \quad \lim_{x\to\infty} f(x) = L, \quad \lim_{x\to a} f(x) = \infty, \dots$$

Jedes Mal eine eigene ε-δ- oder ε-N-Formel. Und jedes Mal dieselben Sätze von vorn: Summenregel, Produktregel, Eindeutigkeit, Verkettung. Wer das n-mal beweist, hat n-mal denselben Beweis geschrieben.

Dahinter steckt eine Beobachtung. Alle diese Aussagen haben die Form „für alle x in einem hinreichend kleinen bzw. großen Bereich gilt …“. Was sich ändert, ist nur, was „hinreichend“ heißt. Genau das fasst ein Filter.

Definition 4.9 (Filter)**.** Ein *Filter* $\mathcal{F}$ auf einer Menge X ist ein System von Teilmengen von X mit

(i) $X \in \mathcal{F}$,

(ii) ist $U \in \mathcal{F}$ und $U \subseteq V$, so ist $V \in \mathcal{F}$,

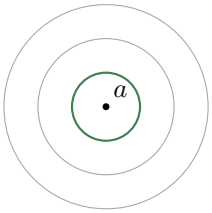
(iii) sind $U, V \in \mathcal{F}$, so ist $U \cap V \in \mathcal{F}$.

Man liest $U \in \mathcal{F}$ als „U ist groß“ oder „U gilt schließlich“. Die Axiome sagen dann: Das Ganze ist groß, Obermengen großer Mengen sind groß, und endlich viele große Mengen haben großen Schnitt.

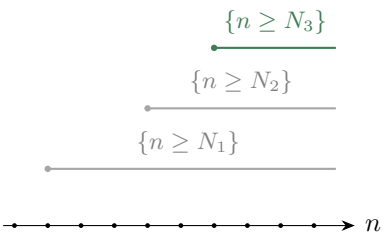
Was „groß“ hier heißt. „Groß“ heißt nicht „viel“, sondern „genug“. Ein Filter legt fest, welche Mengen man als groß ansehen will – und das dritte Axiom ist das entscheidende: Sind U und V groß, so auch $U \cap V$. Übersetzt heißt das: Zwei Aussagen, die jede für sich *schließlich* gelten, gelten auch gemeinsam schließlich. Ohne diese Eigenschaft könnte man mit „schließlich“ nicht rechnen.

Zwei Filter, dieselbe Idee – nur schrumpfen die großen Mengen einmal auf einen Punkt zusammen und einmal auf nichts:

$\mathcal{N}(a)$: die Umgebungen von a



atTop: die Endstücke von $\mathbb{N}$



Der Punkt, den es nicht gibt. Links schrumpfen die großen Mengen auf den Punkt a zusammen: Der Schnitt aller Umgebungen ist $\{a\}$ – im metrischen Raum liegt $b \neq a$ nicht in $B(a, d(a,b))$. Rechts schrumpfen sie auf *nichts* – der Schnitt aller Endstücke ist leer.

Trotzdem verhalten sich beide gleich. atTop benimmt sich wie die Umgebungen eines Punktes, den es in $\mathbb{N}$ gar nicht gibt: „$+\infty$“. Genau deshalb lassen sich Folgengrenzwerte und Funktionsgrenzwerte mit *einem* Begriff behandeln.

Die beiden wichtigsten Beispiele sind

$$\text{atTop} = \{\, U \mid \exists N,\ \forall n \geq N,\ n \in U \,\}, \qquad \mathcal{N}(x) = \{\, U \mid U \text{ ist Umgebung von } x \,\}.$$

In Lean ist ein Filter eine Struktur wie `Topologie`: ein Mengensystem `gross` und drei Beweisfelder, eines je Axiom. Für die Endstücke rechnet man die Axiome nach. Im dritten steckt das $\max(N_1, N_2)$, das man in ε-N-Beweisen immer wieder sucht – hier ein für alle Mal:

Lean code/Kapitel4.lean

```
/-- Ein Filter: ein System "grosser" Mengen mit den Axiomen (i)-(iii). -/
structure MeinFilter (α : Type*) where
  gross : Set (Set α)
  univ_gross : Set.univ ∈ gross                                    -- (i)
  obermenge {U V : Set α} : U ∈ gross → U ⊆ V → V ∈ gross          -- (ii)
  schnitt {U V : Set α} : U ∈ gross → V ∈ gross → U ∩ V ∈ gross    -- (iii)

/-- Die Endstuecke: gross ist, was ein Endstueck {n | n ≥ N} enthaelt. -/
def endstuecke : MeinFilter ℕ where
  gross := {U | ∃ N, ∀ n ≥ N, n ∈ U}
  univ_gross := ⟨0, fun n _ => Set.mem_univ n⟩
  obermenge := by
    rintro U V ⟨N, hN⟩ hUV
    exact ⟨N, fun n hn => hUV (hN n hn)⟩
  schnitt := by
    rintro U V ⟨N₁, h₁⟩ ⟨N₂, h₂⟩            -- ab N₁ in U, ab N₂ in V
    use max N₁ N₂                            -- (iii): das groessere N
    intro n hn
    exact ⟨h₁ n (le_of_max_le_left hn), h₂ n (le_of_max_le_right hn)⟩
```

Im metrischen Raum ist U genau dann Umgebung von a, wenn eine Kugel um a in U passt. Das dritte Axiom geht dann mit $\min(\varepsilon_1, \varepsilon_2)$, wie beim Schnitt offener Mengen:

Lean code/Kapitel4.lean

```
/-- Die Umgebungen von a: gross ist, was eine Kugel um a enthaelt. -/
def umgebungen {X : Type*} [MetricSpace X] (a : X) : MeinFilter X where
  gross := {U | ∃ ε > 0, Metric.ball a ε ⊆ U}
  univ_gross := ⟨1, one_pos, Set.subset_univ _⟩
  obermenge := by
    rintro U V ⟨ε, hε, h⟩ hUV
    exact ⟨ε, hε, h.trans hUV⟩
  schnitt := by
    rintro U V ⟨ε₁, h₁, hU⟩ ⟨ε₂, h₂, hV⟩
    use min ε₁ ε₂, lt_min h₁ h₂              -- (iii): das kleinere ε
    intro y hy
    exact ⟨hU (Metric.ball_subset_ball (min_le_left ε₁ ε₂) hy),
      hV (Metric.ball_subset_ball (min_le_right ε₁ ε₂) hy)⟩
```

Bemerkung 4.10. Man kann einen Filter als „verallgemeinerten Punkt“ lesen. $\mathcal{N}(x)$ ist der Punkt x selbst, atTop ist der Punkt „$+\infty$“, den es in $\mathbb{N}$ nicht gibt. So bekommen auch uneigentliche Grenzwerte einen ganz normalen Punkt, gegen den konvergiert wird.

Konvergenz

Definition 4.11 (Tendsto)**.** Für $f : X \to Y$ und Filter $\mathcal{F}$ auf X, $\mathcal{G}$ auf Y sagt man $f \to \mathcal{G}$ längs $\mathcal{F}$, geschrieben Tendsto $f\ \mathcal{F}\ \mathcal{G}$, wenn

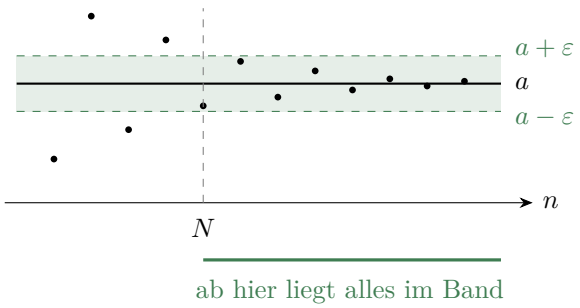
$$f_*\mathcal{F} \leq \mathcal{G},$$

also: Für jedes „große“ $V \in \mathcal{G}$ ist $f^{-1}(V)$ groß in $\mathcal{F}$.

Urbilder großer Mengen bleiben groß. Tendsto $f\ \mathcal{F}\ \mathcal{G}$ heißt wörtlich: Nimm irgendeine große Menge des Ziels und ziehe sie mit f zurück – was herauskommt, muss groß in der Quelle sein.

Für eine Folge u mit $\mathcal{F} = \text{atTop}$ und $\mathcal{G} = \mathcal{N}(a)$ liest sich das so: Nimm ein Band um a. Sein Urbild muss ein Endstück $\{n \geq N\}$ enthalten – also liegen ab irgendeinem N *alle* Folgenglieder im Band. Je enger das Band, desto weiter hinten beginnt das Endstück. Aber es gibt immer eines.

Das ist genau die ε-N-Definition, nur ohne ε und ohne N hinzuschreiben.



In Lean ist die zweite Fassung der Definition eine Zeile. Auch die Verkettung ist kurz: Aus Tendsto $f\ \mathcal{F}\ \mathcal{G}$ und Tendsto $g\ \mathcal{G}\ \mathcal{H}$ folgt Tendsto $(g \circ f)\ \mathcal{F}\ \mathcal{H}$. Denn ist W groß in $\mathcal{H}$, so ist $g^{-1}(W)$ groß in $\mathcal{G}$, also $f^{-1}(g^{-1}(W)) = (g \circ f)^{-1}(W)$ groß in $\mathcal{F}$.

Lean code/Kapitel4.lean

```
/-- Konvergenz laengs Filtern: Urbilder grosser Mengen sind gross. -/
def MeinTendsto (f : α → β) (F : MeinFilter α) (G : MeinFilter β) : Prop :=
  ∀ V ∈ G.gross, f ⁻¹' V ∈ F.gross

/-- Verkettung. Wie in Mathlib steht die aeussere Funktion vorn:
`hg.comp hf` fuer `g ∘ f`. -/
theorem MeinTendsto.comp {f : α → β} {g : β → γ} {F : MeinFilter α}
    {G : MeinFilter β} {H : MeinFilter γ} (hg : MeinTendsto g G H)
    (hf : MeinTendsto f F G) : MeinTendsto (g ∘ f) F H := by
  intro W hW                    -- W gross in H
  exact hf _ (hg W hW)          -- g⁻¹(W) gross in G, also f⁻¹(g⁻¹(W)) in F
```

Jetzt der Abgleich: Mathlibs `Filter` hat dieselben vier Felder, und ein `MeinFilter` wird Feld für Feld ein Mathlib-Filter. `Tendsto` ist dort dieselbe Definition (`Iff.rfl`), und die Endstücke und Umgebungen sind `atTop` und $\mathcal{N}$ `a`.

Lean code/Kapitel4.lean

```
/- Mathlibs `Filter` hat dieselben vier Felder; ein `MeinFilter` wird Feld
   fuer Feld ein Mathlib-Filter. `Tendsto` ist dort dieselbe Definition: -/
def MeinFilter.toFilter (F : MeinFilter α) : Filter α :=
  ⟨F.gross, F.univ_gross, F.obermenge, F.schnitt⟩

example (f : α → β) (F : MeinFilter α) (G : MeinFilter β) :
    Tendsto f F.toFilter G.toFilter ↔ MeinTendsto f F G := Iff.rfl
```

```
/- Die beiden Beispiele sind Mathlibs ‘atTop‘ und ‘𝓝 a‘: -/
theorem endstuecke_eq : endstuecke.toFilter = atTop :=
  Filter.ext fun _ => mem_atTop_sets.symm

theorem umgebungen_eq {X : Type*} [MetricSpace X] (a : X) :
    (umgebungen a).toFilter = 𝓝 a :=
  Filter.ext fun _ => Metric.mem_nhds_iff.symm
```

Das ist *alles*. Alle Grenzwertbegriffe von oben sind Spezialfälle:

klassisch	**als Tendsto**
$u_n \to a$	`Tendsto u atTop (𝓝 a)`
$f(x) \to L$ für $x \to a$, $x \neq a$	`Tendsto f (𝓝[≠] a) (𝓝 L)`
$f(x) \to L$ für $x \to a^+$	`Tendsto f (𝓝[>] a) (𝓝 L)`
$f(x) \to L$ für $x \to \infty$	`Tendsto f atTop (𝓝 L)`
$f(x) \to \infty$ für $x \to a$, $x \neq a$	`Tendsto f (𝓝[≠] a) atTop`

Einen Satz wie „Grenzwerte sind eindeutig“ oder „die Verkettung konvergenter Abbildungen konvergiert“ beweist man damit einmal – die Verkettung oben, in Mathlib `Tendsto.comp`; die Eindeutigkeit folgt unten. Er gilt dann für alle Zeilen der Tabelle.

Dabei geht nichts verloren. Der Vergleich mit der gewohnten Definition:

Satz 4.12. *Für $u : \mathbb{N} \to \mathbb{R}$ und $a \in \mathbb{R}$ gilt*

$$\text{Tendsto } u \text{ atTop } \mathcal{N}(a) \iff \forall \varepsilon > 0\ \exists N\ \forall n \geq N : |u_n - a| < \varepsilon.$$

Der Beweis übersetzt nur die Definitionen. Ist Tendsto u atTop $\mathcal{N}(a)$, so ist die Kugel $B(a,\varepsilon)$ groß in $\mathcal{N}(a)$, ihr Urbild also groß in atTop: Es enthält ein Endstück $\{n \geq N\}$. Umgekehrt passt in jede große Menge V um a eine Kugel $B(a,\varepsilon)$; ab dem N zu diesem ε liegen alle u_n in V. In Lean, wörtlich:

Lean code/Kapitel4.lean

```
/-- Die ε-N-Form, von Hand. Hin: Die Kugel um a ist gross, ihr Urbild
enthaelt ein Endstueck. Rueck: In jede grosse Menge um a passt eine Kugel. -/
theorem meinTendsto_iff_eps_N (u : ℕ → ℝ) (a : ℝ) :
    MeinTendsto u endstuecke (umgebungen a) ↔
      ∀ ε > 0, ∃ N, ∀ n ≥ N, |u n - a| < ε := by
  constructor
  · intro h ε hε
    obtain ⟨N, hN⟩ := h (Metric.ball a ε) ⟨ε, hε, subset_rfl⟩
    exact ⟨N, hN⟩                  -- u n ∈ B(a, ε) heisst |u n - a| < ε
  · rintro h V ⟨ε, hε, hV⟩
    obtain ⟨N, hN⟩ := h ε hε
    exact ⟨N, fun n hn => hV (hN n hn)⟩
```

Weil die Endstücke `atTop` und die Umgebungen $\mathcal{N}$ `a` sind, gilt der Satz auch in Mathlib-Sprache: Man schreibt beide mit den Gleichungen von oben zurück und wendet den Satz von Hand an. Mathlib selbst hat ihn als `Metric.tendsto_atTop`, für Folgen in jedem metrischen Raum. Dort steht `dist` statt des Betrags; `Real.dist_eq` übersetzt:

Lean code/Kapitel4.lean

```
/-- Dieselbe Aussage fuer Mathlibs ‘Tendsto‘: zurueckuebersetzen, fertig. -/
theorem tendsto_iff_eps_N (u : ℕ → ℝ) (a : ℝ) :
    Tendsto u atTop (𝓝 a) ↔ ∀ ε > 0, ∃ N, ∀ n ≥ N, |u n - a| < ε := by
  rw [← endstuecke_eq, ← umgebungen_eq]
  exact meinTendsto_iff_eps_N u a
```

```
/- Mathlib hat die ε-N-Form schon, fuer Folgen in jedem metrischen Raum: -/
example (u : ℕ → ℝ) (a : ℝ) :
    Tendsto u atTop (𝓝 a) ↔ ∀ ε > 0, ∃ N, ∀ n ≥ N, |u n - a| < ε := by
  rw [Metric.tendsto_atTop]
  simp [Real.dist_eq]
```

Ab hier benutzen wir für Filter und Konvergenz nur noch Mathlibs Namen.

Beim Rechnen merkt man den Vorteil. Zuerst die Schreibweise:

Mathlib. `∀ᶠ x in F, P x` steht für `Filter.Eventually`: „P gilt schließlich“, ausgeschrieben $\{x \mid P(x)\} \in \mathcal{F}$. Für atTop heißt das: ab einem N.

Lean code/Kapitel4.lean

```
/- "P gilt schliesslich" (‘∀ᶠ‘) heisst: Die Menge, auf der P gilt, ist
   gross -- per Definition. -/
example {α : Type*} (F : Filter α) (P : α → Prop) :
    (∀ᶠ x in F, P x) ↔ {x | P x} ∈ F := Iff.rfl

/- Fuer ‘atTop‘ also: ab einem N. Im Beispiel reicht N = 11. -/
example : ∀ᶠ n : ℕ in atTop, 10 < n :=
  eventually_atTop.2 ⟨11, fun n hn => by omega⟩
```

Klassisch braucht die Summenregel zwei Kunstgriffe: ε halbieren und N maximieren. Mit Filtern bleibt das Halbieren; das Maximieren ist Axiom (iii) geworden. Und der Beweis gilt für jeden Filter, also für alle Zeilen der Tabelle mit endlichem Grenzwert auf einmal – unten für eine Folge und für eine Funktion:

Lean code/Kapitel4.lean

```
/-- Summenregel, einmal fuer jeden Filter: ε halbieren wie immer -- aber statt
N₁ und N₂ zu maximieren, schneidet Axiom (iii) die beiden grossen Mengen. -/
theorem summenregel {α : Type*} {F : Filter α} {u v : α → ℝ} {a b : ℝ}
    (hu : Tendsto u F (𝓝 a)) (hv : Tendsto v F (𝓝 b)) :
    Tendsto (fun x => u x + v x) F (𝓝 (a + b)) := by
  rw [Metric.tendsto_nhds] at *
  intro ε hε
  have h1 := hu (ε / 2) (by linarith)     -- schliesslich |u x - a| < ε/2
  have h2 := hv (ε / 2) (by linarith)     -- schliesslich |v x - b| < ε/2
  have h := inter_mem h1 h2               -- (iii): beides zugleich
  apply mem_of_superset h                 -- (ii): eine Obermenge genuegt
  rintro x ⟨hx₁, hx₂⟩
  rw [Set.mem_setOf_eq, Real.dist_eq] at hx₁ hx₂ ⊢
  calc |u x + v x - (a + b)| = |(u x - a) + (v x - b)| := by congr 1; ring
    _ ≤ |u x - a| + |v x - b| := abs_add_le _ _
    _ < ε := by linarith

/- Derselbe Satz fuer Folgen (‘atTop‘) und fuer Funktionen (‘𝓝 1‘): -/
example : Tendsto (fun n : ℕ => 2 + 1 / (n : ℝ)) atTop (𝓝 (2 + 0)) :=
  summenregel tendsto_const_nhds tendsto_one_div_atTop_nhds_zero_nat
example : Tendsto (fun x : ℝ => x + 3) (𝓝 1) (𝓝 (1 + 3)) :=
  summenregel tendsto_id tendsto_const_nhds
```

Ebenso die Eindeutigkeit, für jeden Filter, in dem die leere Menge nicht groß ist (`NeBot`) – etwa atTop und $\mathcal{N}(a)$:

Lean code/Kapitel4.lean

```
/-- Grenzwerte sind eindeutig, fuer jeden Filter, in dem ∅ nicht gross ist
('NeBot'): Die Kugeln vom Radius d(a,b)/2 um a und b sind disjunkt. -/
theorem grenzwert_eindeutig {α X : Type*} [MetricSpace X] {F : Filter α}
    [F.NeBot] {u : α → X} {a b : X}
    (ha : Tendsto u F (𝓝 a)) (hb : Tendsto u F (𝓝 b)) : a = b := by
  by_contra hne
  have hd : 0 < dist a b / 2 := half_pos (dist_pos.2 hne)
  rw [Metric.tendsto_nhds] at ha hb
  obtain ⟨x, hxa, hxb⟩ := ((ha _ hd).and (hb _ hd)).exists   -- (iii), NeBot
  have h := dist_triangle_left a b (u x)   -- d(a,b) ≤ d(u x,a) + d(u x,b)
  linarith
```

4.3 Vollständigkeit

Jetzt ist Konvergenz da, und damit lässt sich der Begriff nachholen, der in Abschnitt 4.1 gefehlt hat.

Definition 4.13 (Cauchy-Folge, Vollständigkeit)**.** Eine Folge (x_n) in einem metrischen Raum heißt *Cauchy-Folge*, wenn

$$\forall \varepsilon > 0\ \exists N\ \forall m, n \geq N : d(x_m, x_n) < \varepsilon.$$

Der Raum heißt *vollständig*, wenn jede Cauchy-Folge konvergiert.

Der Unterschied zur Konvergenz. Bei der Konvergenz vergleicht man die Folgenglieder mit dem *Grenzwert*. Bei einer Cauchy-Folge vergleicht man sie nur *untereinander*. Man kann also feststellen, dass sich eine Folge zusammenzieht, ohne zu wissen, worauf.

Vollständigkeit besagt: Dort, wo sich etwas zusammenzieht, ist auch wirklich ein Punkt. $\mathbb{Q}$ ist nicht vollständig – die Näherungen an $\sqrt{2}$ ziehen sich zusammen, aber der Grenzwert fehlt. Genau um dieses Loch zu stopfen, baut man $\mathbb{R}$.

In Lean steht die Definition wörtlich da. Mathlibs `CauchySeq` ist über Filter erklärt und dasselbe; `CompleteSpace` heißt: Jede Cauchy-Folge hat einen Grenzwert.

Lean code/Kapitel4.lean

```
/-- Cauchy-Folge: Die Glieder kommen sich *untereinander* beliebig nahe --
ein Grenzwert kommt nicht vor. -/
def IstCauchy (u : ℕ → X) : Prop :=
  ∀ ε > 0, ∃ N, ∀ m ≥ N, ∀ n ≥ N, dist (u m) (u n) < ε

/- Mathlib definiert 'CauchySeq' ueber Filter; fuer Folgen ist es dasselbe: -/
example (u : ℕ → X) : IstCauchy u ↔ CauchySeq u := Metric.cauchySeq_iff.symm

/- Vollstaendig ('CompleteSpace') heisst: Jede Cauchy-Folge hat einen
   Grenzwert. Mehr liefert die Vollstaendigkeit nicht: -/
example [CompleteSpace X] (u : ℕ → X) (h : CauchySeq u) :
    ∃ x, Tendsto u atTop (𝓝 x) :=
  cauchySeq_tendsto_of_complete h
```

Die Umkehrung gilt in jedem metrischen Raum: Konvergente Folgen sind Cauchy-Folgen. Man halbiert ε und geht über den Grenzwert:

Lean code/Kapitel4.lean

```
/-- Konvergent ⇒ Cauchy: ε halbieren, dann Dreiecksungleichung ueber a. -/
theorem IstCauchy.of_tendsto {u : ℕ → X} {a : X}
    (h : Tendsto u atTop (𝓝 a)) : IstCauchy u := by
  intro ε hε
  obtain ⟨N, hN⟩ := Metric.tendsto_atTop.1 h (ε / 2) (by linarith)
  use N
  intro m hm n hn
  have h1 := dist_triangle_right (u m) (u n) a  -- ≤ d(u m, a) + d(u n, a)
  linarith [hN m hm, hN n hn]
```

Damit bekommen die beiden oberen Stufen ihre Namen:

Raum	ist	in Mathlib
Banachraum	normierter Raum + vollständig	`NormedSpace` + `CompleteSpace`
Hilbertraum	Innenproduktraum + vollständig	`InnerProductSpace` + `CompleteSpace`

Einen Typ `Banachraum` gibt es in Mathlib nicht, einen Typ `Hilbertraum` auch nicht. Man schreibt den normierten bzw. den Innenproduktraum hin und hängt `[CompleteSpace E]` an. `CompleteSpace` bringt keine neuen Operationen mit, nur eine Eigenschaft: Es ist eine `Prop`. Eine solche datenfreie Zusatzklasse heißt *Mixin* (Abschnitt 3.8). Zusatzstruktur wird so kombiniert, nicht vererbt; damit entsteht dieselbe Struktur nie auf zwei Wegen doppelt.

Wozu man die Vollständigkeit braucht

Satz 4.14 (Banachscher Fixpunktsatz)**.** *Sei X ein nichtleerer vollständiger metrischer Raum und $T : X \to X$ eine* Kontraktion*: Es gebe ein $K < 1$ mit*

$$d(Tx, Ty) \;\le\; K\, d(x, y) \qquad \text{für alle } x, y \in X.$$

Dann hat T genau einen Fixpunkt x^*. Für jeden Startwert x_0 konvergiert die Folge $x_{n+1} = Tx_n$ gegen ihn, und es gilt

$$d(x_n, x^*) \;\le\; \frac{K^n}{1-K}\, d(x_0, Tx_0).$$

Jeder Schritt schrumpft den Rest. T zieht Abstände um mindestens den Faktor K zusammen. Wendet man T immer wieder an, schrumpft der Abstand zweier Bahnen wie K^n – und die Folge x_n zieht sich zusammen. Dass der Grenzwert dann auch wirklich *da* ist, ist die Vollständigkeit. In $\mathbb{Q}$ scheitert derselbe Beweis an genau einer Stelle, Schritt 3 unten.

Die Abschätzung am Ende ist praktisch: Sie schätzt den Abstand zum Grenzwert allein aus dem *ersten* Schritt. Man kann also nach n Iterationen sagen, wie weit man höchstens noch entfernt ist, ohne die Lösung zu kennen.

Der Beweis hat sechs Schritte. Man wählt einen Startwert x_0 – dafür muss X nichtleer sein – und setzt $x_{n+1} = Tx_n$.

1. **Ein Schritt.** $d(x_{n+1}, x_n) \le K^n\, d(x_1, x_0)$: die Kontraktionsbedingung, n-mal angewandt.
2. **Cauchy.** Mit Dreiecksungleichung und geometrischer Reihe ist $d(x_m, x_n) \le \frac{K^n}{1-K}\, d(x_1, x_0)$ für $m > n$.
3. **Grenzwert.** X ist vollständig, also gibt es x^* mit $x_n \to x^*$.

4. **Fixpunkt.** Mit $Tx_n = x_{n+1}$ und der Dreiecksungleichung ist $d(Tx^*, x^*) \le K\,d(x^*, x_n) + d(x_{n+1}, x^*) \to 0$.

5. **Eindeutigkeit.** Aus $Tx = x$ und $Ty = y$ folgt $d(x, y) \le K\,d(x, y)$, wegen $K < 1$ also $d(x, y) = 0$.

6. **Fehlerschranke.** In Schritt 2 $m \to \infty$ gehen lassen.

In Lean ist eine Kontraktion eine Struktur mit drei Feldern: $0 \le K$, $K < 1$ und die Ungleichung. Das erste steht nicht im Satz, schränkt aber nichts ein: Für $K < 0$ hätte X höchstens einen Punkt, und dann tut es auch $K = 0$. Die Iteration ist eine Rekursion über n, gebaut wie später `gdIter` in Abschnitt 4.5:

Lean code/Kapitel4.lean

```
/-- Kontraktion: Abstaende schrumpfen mindestens um den Faktor K < 1. -/
structure Kontraktion (K : ℝ) (T : X → X) : Prop where
  nichtneg : 0 ≤ K
  klein : K < 1
  schrumpft : ∀ x y, dist (T x) (T y) ≤ K * dist x y

/-- Die Iteration x₀, x₁ = T x₀, x₂ = T (T x₀), ... -- gebaut wie 'gdIter'. -/
def iter {Y : Type*} (T : Y → Y) (x₀ : Y) : ℕ → Y
  | 0 => x₀
  | n + 1 => T (iter T x₀ n)
```

Die Schritte 1, 4 und 5 sind eigene Lemmas; keines braucht die Vollständigkeit:

Lean code/Kapitel4.lean

```
/-- Schritt 1: d(x_n, x_(n+1)) ≤ d(x₀, x₁) · K^n, wobei x_n = iter T x₀ n. -/
theorem Kontraktion.schritt1 (hT : Kontraktion K T) (x₀ : X) (n : ℕ) :
    dist (iter T x₀ n) (iter T x₀ (n + 1)) ≤ dist x₀ (T x₀) * K ^ n := by
  induction n with
  | zero => simp [iter]
  | succ k ih =>
      calc dist (iter T x₀ (k + 1)) (iter T x₀ (k + 2))
          ≤ K * dist (iter T x₀ k) (iter T x₀ (k + 1)) := hT.schrumpft _ _
        _ ≤ K * (dist x₀ (T x₀) * K ^ k) := by gcongr; exact hT.nichtneg
        _ = dist x₀ (T x₀) * K ^ (k + 1) := by ring

/-- Schritt 4: Der Grenzwert x ist ein Fixpunkt. Das Glied x_(N+1) = T x_N
liegt nahe bei x und nahe bei T x. -/
theorem Kontraktion.schritt4 (hT : Kontraktion K T) (x₀ : X) {x : X}
    (hx : Tendsto (iter T x₀) atTop (𝓝 x)) : T x = x := by
  apply eq_of_forall_dist_le
  intro ε hε
  obtain ⟨N, hN⟩ := Metric.tendsto_atTop.1 hx (ε / 2) (by linarith)
  have h1 := hN N le_rfl                    -- d(x_N, x) < ε/2
  have h2 := hN (N + 1) (by omega)          -- d(x_(N+1), x) < ε/2
  have h3 : dist (T x) (iter T x₀ (N + 1)) ≤ K * dist x (iter T x₀ N) :=
    hT.schrumpft x (iter T x₀ N)            -- x_(N+1) = T x_N
  have h4 := dist_triangle (T x) (iter T x₀ (N + 1)) x
  rw [dist_comm] at h1
  nlinarith [hT.klein, hT.nichtneg, dist_nonneg (x := x) (y := iter T x₀ N)]

/-- Schritt 5: hoechstens ein Fixpunkt -- aus d ≤ K·d und K < 1. -/
theorem Kontraktion.schritt5 (hT : Kontraktion K T) {x y : X}
    (hx : T x = x) (hy : T y = y) : x = y := by
  have h := hT.schrumpft x y
  rw [hx, hy] at h                          -- d(x,y) ≤ K·d(x,y)
  have h0 : dist x y ≤ 0 := by
```

```
    nlinarith [hT.klein, dist_nonneg (x := x) (y := y)]
  exact dist_le_zero.1 h0
```

Der Satz setzt sie zusammen: Schritt 1 und 4 liefern den Fixpunkt, Schritt 5 die Eindeutigkeit. Schritt 2 und 6 sind die geometrische Reihe, die Mathlib beisteuert. Die Vollständigkeit braucht nur Schritt 3: Streicht man `[CompleteSpace X]`, wird in Lean nur die Zeile von Schritt 3 rot – alle übrigen Zeilen prüft Lean weiter, und sie gehen ohne durch.

Lean code/Kapitel4.lean

```
/-- Banachscher Fixpunktsatz: genau ein Fixpunkt, Konvergenz, Schranke. Der
Startwert x₀ ist "X nichtleer"; nur Schritt 3 braucht ‘[CompleteSpace X]‘. -/
theorem banach [CompleteSpace X] (hT : Kontraktion K T) (x₀ : X) :
    ∃ x, T x = x ∧ (∀ y, T y = y → y = x) ∧ Tendsto (iter T x₀) atTop (𝓝 x) ∧
      ∀ n, dist (iter T x₀ n) x ≤ dist x₀ (T x₀) * K ^ n / (1 - K) := by
  have h2 : CauchySeq (iter T x₀) :=                          -- Schritt 2
    cauchySeq_of_le_geometric K _ hT.klein (hT.schritt1 x₀)
  have h3 : ∃ x, Tendsto (iter T x₀) atTop (𝓝 x) :=           -- Schritt 3
    cauchySeq_tendsto_of_complete h2
  obtain ⟨x, hx⟩ := h3
  have h4 : T x = x := hT.schritt4 x₀ hx                      -- Schritt 4
  refine ⟨x, h4, fun y hy => hT.schritt5 hy h4, hx, ?_⟩       -- Schritt 5
  intro n                                                     -- Schritt 6
  exact dist_le_of_le_geometric_of_tendsto K _ hT.klein (hT.schritt1 x₀) hx n
```

Ohne Vollständigkeit fehlt der Grenzwert für Schritt 4 und 6: $x \mapsto x/2$ zieht $(0,1]$ zusammen, die Iteration strebt gegen 0 – aber 0 fehlt. Mathlib nennt die Kontraktion `ContractingWith`, mit K vom Typ `NNReal`; Mathlibs Satz sagt dasselbe, nur ohne die Eindeutigkeit:

Lean code/Kapitel4.lean

```
/- Mathlibs ‘ContractingWith K T‘ (K : NNReal) ist derselbe Begriff: -/
example (K : NNReal) (T : X → X) : Kontraktion K T ↔ ContractingWith K T := by
  rw [ContractingWith, lipschitzWith_iff_dist_le_mul]
  exact ⟨fun h => ⟨by exact_mod_cast h.klein, h.schrumpft⟩,
    fun h => ⟨K.2, by exact_mod_cast h.1, h.2⟩⟩

/- Mathlibs Satz, mit ‘f^[n] x‘ statt ‘iter T x₀ n‘. Er gilt fuer erweiterte
   Metriken (‘edist‘ darf ∞ sein, in Lean ‘⊤‘) und setzt daher
   ‘edist x (f x) ≠ ⊤‘ voraus; im metrischen Raum ist das immer erfuellt: -/
#check @ContractingWith.exists_fixedPoint

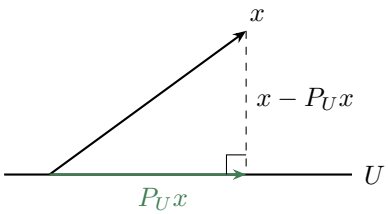
/- Picard-Lindeloef sucht den Fixpunkt unter den stetigen Funktionen mit der
   Supremumsnorm -- ein vollstaendiger Raum, weil der Zielraum es ist: -/
#check (inferInstance : CompleteSpace C(Set.Icc (0:ℝ) 1, ℝ))
```

Der Fixpunktsatz ist das Arbeitspferd für Existenzaussagen: Man formuliert das Problem als „gesucht ist ein x mit $Tx = x$“ und weist nach, dass T zusammenzieht. Kapitel 6 macht genau das für Differentialgleichungen. Der Raum X ist dort ein Funktionenraum wie in Beispiel 4.6, und seine Vollständigkeit ist der Grund, warum der Beweis überhaupt funktioniert.

Satz 4.15 (Projektionssatz)**.** *Sei H ein Hilbertraum und $U \subseteq H$ ein abgeschlossener Teilraum. Dann gibt es zu jedem $x \in H$ genau einen nächstgelegenen Punkt $P_U x \in U$, und $x - P_U x$ steht senkrecht auf U.*

Senkrecht heißt kürzester Weg. Man sucht in U den Punkt, der x am nächsten liegt. Nähert man sich in U dem kleinsten Abstand zu x, ziehen sich die Punkte zusammen – aber dass es den Grenzpunkt wirklich *gibt*, liefert erst die Vollständigkeit.

Und wenn es ihn gibt, steht die Verbindung senkrecht auf U. Denn wäre sie schief, könnte man in U ein Stück in ihre Richtung gehen und wäre näher dran.



Darauf beruht eine ganze Reihe von Verfahren: Fourierreihen, Ausgleichsrechnung, die Methode der kleinsten Quadrate. Alle sind „bester Ersatz in einem kleineren Raum“.

Die Richtung „senkrecht ⇒ kürzester Weg“ ist Pythagoras und braucht keine Vollständigkeit: Steht $x - p$ senkrecht auf $p - u$, so ist $\|x-u\|^2 = \|x-p\|^2 + \|p-u\|^2 \geq \|x-p\|^2$. Steht $x - p$ für ein $p \in U$ senkrecht auf ganz U, gilt das für jedes $u \in U$, denn $p - u$ liegt in U: Dann ist p der nächstgelegene Punkt.

Lean code/Kapitel4.lean

```
/-- Projektionssatz, senkrecht ⇒ kuerzester Weg: Steht x - p senkrecht auf
p - u, so liegt p mindestens so nah an x wie u -- wieder Pythagoras. -/
theorem senkrecht_kuerzer {V : Type*} [NormedAddCommGroup V]
    [InnerProductSpace ℝ V] (x p u : V)
    (h : inner ℝ (x - p) (p - u) = (0 : ℝ)) :
    ‖x - p‖ ^ 2 ≤ ‖x - u‖ ^ 2 := by
  have e : x - u = (x - p) + (p - u) := by abel
  rw [e, norm_add_sq_real, h]
  nlinarith [sq_nonneg ‖p - u‖]
```

4.4 Ableitung

Landau-Symbole

Die Ableitung vergleicht, wie schnell zwei Größen gegen null gehen. Dafür gibt es eine eigene Schreibweise, die Landau-Symbole. Wie die Konvergenz funktionieren sie mit jedem Filter.

Definition 4.16 (Landau-Symbole)**.** Seien f und g Abbildungen von X in normierte Räume und $\mathcal{F}$ ein Filter auf X. Man schreibt

$$\begin{aligned} f = O(g) \text{ längs } \mathcal{F} \quad &:\Longleftrightarrow \quad \exists c \,:\, \{\, x \mid \|f(x)\| \leq c\,\|g(x)\| \,\} \in \mathcal{F}, \\ f = o(g) \text{ längs } \mathcal{F} \quad &:\Longleftrightarrow \quad \forall c > 0 \,:\, \{\, x \mid \|f(x)\| \leq c\,\|g(x)\| \,\} \in \mathcal{F}. \end{aligned}$$

Höchstens so groß, vernachlässigbar klein. $f = O(g)$ heißt: f ist schließlich höchstens so groß wie g, bis auf einen festen Faktor. $f = o(g)$ verlangt mehr: *Jeder* noch so kleine Faktor reicht, wenn man nur nahe genug herangeht. Gegen g ist f dann vernachlässigbar. Für reelle Funktionen mit g schließlich ungleich null heißt das einfach $f/g \to 0$.

Was „nahe genug“ heißt, sagt der Filter. Bei 0 ist $x^2 = o(x)$, denn $x^2 \leq c\,|x|$ gilt für $|x| \leq c$. Bei $+\infty$ ist es umgekehrt: $x = o(x^2)$. Dieselben Funktionen, die umgekehrte Aussage. Und $f = o(1)$ heißt nichts anderes als $f \to 0$.

Ein Symbol, keine Gleichung. Das Gleichheitszeichen in $f = o(g)$ ist historisch und irreführend. Aus $x^2 = o(x)$ und $x^3 = o(x)$ folgt nicht $x^2 = x^3$. Gemeint ist $f \in o(g)$. In Lean ist `=o[l]` deshalb *ein* Symbol und steht für die Aussage `Asymptotics.IsLittleO l f g`, entsprechend `=O[l]` für `Asymptotics.IsBigO l f g`.

Die Definition von o ist in Lean ein $\leftrightarrow$ mit `=o[l]`. Der Beweis von $(y-x)^2 = o(y-x)$ längs $\mathcal{N}(x)$ zeigt, was „schließlich“ hier leistet: Zu jedem c wählt man eine eigene Kugel um x, vom Radius c. `filter_upwards` ist dabei Axiom (ii): Jede Obermenge der Kugel ist groß.

Lean code/Kapitel4.lean

```
/- o(g) laengs l, ausgeschrieben: fuer jedes c > 0 schliesslich |f| ≤ c·|g|. -/
example (f g : ℝ → ℝ) (l : Filter ℝ) :
    f =o[l] g ↔ ∀ c > 0, ∀ᶠ x in l, ‖f x‖ ≤ c * ‖g x‖ :=
  Asymptotics.isLittleO_iff

/-- Bei x ist (y - x)^2 klein gegen y - x: Zu jedem c reicht die Kugel vom
Radius c um x. Wie nah "nahe genug" ist, haengt von c ab. -/
theorem quadrat_klein_o (x : ℝ) :
    (fun y : ℝ => (y - x) ^ 2) =o[𝓝 x] (fun y => y - x) := by
  rw [Asymptotics.isLittleO_iff]
  intro c hc
  filter_upwards [Metric.ball_mem_nhds x hc] with y hy  -- B(x, c) ist gross
  rw [Metric.mem_ball, Real.dist_eq] at hy              -- hy : |y - x| < c
  rw [norm_pow, Real.norm_eq_abs, sq]                   -- |y-x|·|y-x| ≤ c·|y-x|
  exact mul_le_mul_of_nonneg_right hy.le (abs_nonneg _)
```

Die Definition

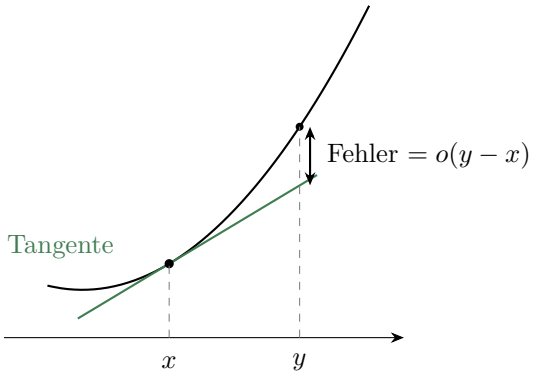
Der Differenzenquotient lässt sich schlecht verallgemeinern: In mehreren Veränderlichen kann man nicht durch einen Vektor teilen. Was man dort hat, ist eine Norm (Definition 4.5) – also ein Maß dafür, wie groß ein Fehler ist. Stattdessen nimmt man die *lineare Approximation*.

Definition 4.17 (Ableitung)**.** f hat in x die Ableitung f', wenn

$$f(y) - f(x) - (y-x)\,f' = o(y-x) \qquad \text{längs } \mathcal{N}(x).$$

Die Gerade, die am besten passt. Man legt an die Kurve im Punkt x eine Gerade. Der Fehler ist der senkrechte Abstand zwischen Kurve und Gerade. Für *jede* Gerade durch den Punkt geht dieser Fehler gegen null, wenn man y gegen x schickt – das ist nichts Besonderes.

Bei der Tangente ist es anders: Ihr Fehler geht *schneller* gegen null als der Abstand $y - x$ selbst. Genau das heißt $o(y-x)$: Teilt man den Fehler noch durch $y-x$, bleibt immer noch null übrig. Unter allen Geraden schafft das nur eine, und ihre Steigung ist die Ableitung.



Genau so ist die Ableitung in Mathlib definiert, mit dem Landau-Symbol längs $\mathcal{N}(x)$ – also wieder über Filter. Der Vorteil: Ersetzt man $(y-x)\,f'$ durch $f'(y-x)$ mit einer stetigen linearen Abbildung f', steht da wörtlich die Fréchet-Ableitung in normierten Räumen. `HasDerivAt` ist in Mathlib der eindimensionale Spezialfall von `HasFDerivAt`, kein eigener Begriff.

Wie man mit der Definition rechnet, zeigt $f(y)=y^2$. Der Fehler der Tangente ist $y^2-x^2-(y-x)\,2x=(y-x)^2$, und das ist nach dem Lemma oben $o(y-x)$. In Lean: Mathlibs Definition aufrufen, den Fehler ausrechnen, das Lemma einsetzen. Mit den fertigen Ableitungsregeln ist es eine Zeile:

Lean code/Kapitel4.lean

```
/- y² hat in x die Ableitung 2x, aus der Definition: Der Fehler der Tangente
   ist (y - x)², und das ist o(y - x) nach ‘quadrat_klein_o‘. -/
example (x : ℝ) : HasDerivAt (fun y : ℝ => y ^ 2) (2 * x) x := by
  rw [hasDerivAt_iff_isLittleO]          -- Mathlibs Definition der Ableitung
  have fehler : (fun y : ℝ => y ^ 2 - x ^ 2 - (y - x) • (2 * x)) =
      fun y => (y - x) ^ 2 := by
    ext y; rw [smul_eq_mul]; ring
  rw [fehler]                            -- der Fehler ist (y - x)²
  exact quadrat_klein_o x

/- Mit den fertigen Ableitungsregeln ist es eine Zeile: -/
example (x : ℝ) : HasDerivAt (fun y : ℝ => y ^ 2) (2 * x) x := by
  simpa using hasDerivAt_pow 2 x
```

Dass nur die Tangente das schafft, folgt aus der Eindeutigkeit der Grenzwerte: Die Ableitung ist auch der Grenzwert der Sekantensteigungen längs `𝓝[≠] x`, der Umgebungen von x ohne x selbst.

Lean code/Kapitel4.lean

```
/-- Nur eine Gerade passt am besten. Die Ableitung ist der Grenzwert der
Sekantensteigungen laengs 𝓝[≠] x (‘hasDerivAt_iff_tendsto_slope‘), und
Grenzwerte sind eindeutig: Neben x liegen beliebig nahe andere Punkte. -/
theorem ableitung_eindeutig {f : ℝ → ℝ} {x s t : ℝ}
    (hs : HasDerivAt f s x) (ht : HasDerivAt f t x) : s = t :=
  grenzwert_eindeutig (hasDerivAt_iff_tendsto_slope.1 hs)
    (hasDerivAt_iff_tendsto_slope.1 ht)
```

Über die Sekantensteigungen geht auch der Satz von Fermat: Hat f in a ein lokales Minimum und die Ableitung f', so ist $f'=0$. Denn nahe a ist $f(x)\ge f(a)$. Die Sekantensteigung ist also links von a höchstens 0 und rechts mindestens 0, und Grenzwerte erhalten $\le$: $f'\le 0\le f'$.

Lean code/Kapitel4.lean

```
/-- Fermat: An einem lokalen Minimum ist f' = 0. Nahe a ist f a ≤ f x, also
die Sekantensteigung ‘slope f a x‘ = (f x - f a)/(x - a) links von a ≤ 0,
rechts ≥ 0. Grenzwerte erhalten ≤, also f' ≤ 0 ≤ f'. -/
theorem fermat {f : ℝ → ℝ} {a f' : ℝ} (h : IsLocalMin f a)
    (hf : HasDerivAt f f' a) : f' = 0 := by
  have hs : Tendsto (slope f a) (𝓝[≠] a) (𝓝 f') :=   -- Steigungen → f'
    hasDerivAt_iff_tendsto_slope.1 hf
  apply le_antisymm                                  -- f' ≤ 0 und 0 ≤ f'
  · apply le_of_tendsto (hs.mono_left (nhdsLT_le_nhdsNE a))  -- nur links von a
    filter_upwards [nhdsWithin_le_nhds h, self_mem_nhdsWithin] with x hfx hxa
    rw [slope_comm]                                  -- Steigung von x nach a
    exact (slope_nonpos_iff_of_le hxa.le).2 hfx      -- f a ≤ f x, x < a: ≤ 0
  · apply ge_of_tendsto (hs.mono_left (nhdsGT_le_nhdsNE a))  -- nur rechts von a
    filter_upwards [nhdsWithin_le_nhds h, self_mem_nhdsWithin] with x hfx hax
    exact (slope_nonneg_iff_of_le hax.le).2 hfx      -- f a ≤ f x, a < x: ≥ 0
```

Junk-Werte. `deriv f x` ist *total* definiert. Ist f in x nicht differenzierbar, ist der Wert 0. Das ist Absicht, keine Schlamperei: In der Typentheorie sind alle Funktionen total. Die Differenzierbarkeit wandert dafür in die Voraussetzungen der Sätze. Beim Arbeiten heißt das: `HasDerivAt f f' x` ist die inhaltliche Aussage, `deriv` nur die bequeme Schreibweise.

Im Alltag rechnet man mit Regeln statt mit der Definition. Rechenregeln wie die Produktregel zitiert der Code aus Mathlib, ebenso den Junk-Wert: So ist `deriv` dort festgelegt. Aus `fermat` und dem Junk-Wert folgt Fermat für `deriv`, sogar ohne Differenzierbarkeit: Ist f in a differenzierbar, greift `fermat`; sonst ist `deriv f a` ohnehin 0.

Lean code/Kapitel4.lean

```
/- Produktregel: eine Rechenregel, aus Mathlib (‘HasDerivAt.mul‘). -/
example (f g : ℝ → ℝ) (x f' g' : ℝ)
    (hf : HasDerivAt f f' x) (hg : HasDerivAt g g' x) :
    HasDerivAt (fun y => f y * g y) (f' * g x + f x * g') x := hf.mul hg

/- ‘deriv‘ ist total: Wo f nicht differenzierbar ist, liefert es den
   Junk-Wert 0 -- so legt Mathlib ‘deriv‘ fest, daher zitiert. -/
example (f : ℝ → ℝ) (x : ℝ) (h : ¬ DifferentiableAt ℝ f x) : deriv f x = 0 :=
  deriv_zero_of_not_differentiableAt h

/- Deshalb braucht Fermat fuer ‘deriv‘ keine Differenzierbarkeit: -/
example (f : ℝ → ℝ) (a : ℝ) (h : IsLocalMin f a) : deriv f a = 0 := by
  rcases Classical.em (DifferentiableAt ℝ f a) with hd | hd
  · exact fermat h hd.hasDerivAt                  -- differenzierbar
  · exact deriv_zero_of_not_differentiableAt hd   -- sonst Junk-Wert 0
```

4.5 Gradientenabstieg

Gradient und Differential

In einem Hilbertraum H (etwa $\mathbb{R}^n$) ist das Differential $Df(x)$ eine stetige Linearform. Der *Gradient* ist ihr Riesz-Repräsentant, also der eindeutige Vektor $\nabla f(x)$ mit

$$Df(x)[h] = \langle \nabla f(x), h \rangle \quad \text{für alle } h \in H.$$

Das Differential ist der Grundbegriff, der Gradient kommt daraus. Genau so ist es in Mathlib umgesetzt, dort mit `CompleteSpace`. Über $\mathbb{R}$ ist der Gradient in einer Variablen die Ableitung; deshalb steht im Verfahren unten `deriv`.

Lean code/Kapitel4.lean

```
/- Der Gradient ist der Riesz-Repraesentant des Differentials in einem
   Hilbertraum (‘[CompleteSpace F]‘): Das Differential ist primaer. Ueber ℝ
   faellt er in einer Variablen mit der Ableitung zusammen: -/
example (f : ℝ → ℝ) (x : ℝ) : gradient f x = deriv f x := gradient_eq_deriv'
```

Das Verfahren

Definition 4.18 (Gradientenabstieg)**.** Zu f, Schrittweite $t > 0$ und Startwert x_0 setze

$$x_{k+1} := x_k - t\,\nabla f(x_k).$$

Lean code/Kapitel4.lean

```
/-- Ein Schritt des Gradientenabstiegs in einer Variablen. -/
noncomputable def gdSchritt (f : ℝ → ℝ) (t x : ℝ) : ℝ := x - t * deriv f x
```

```
/-- Die Iteration ‘x₀, x₁, x₂, …‘ mit fester Schrittweite ‘t‘. -/
noncomputable def gdIter (f : ℝ → ℝ) (t x₀ : ℝ) : ℕ → ℝ
  | 0 => x₀
  | n + 1 => gdSchritt f t (gdIter f t x₀ n)
```

Ein Modellproblem, ganz durchgerechnet

Für die quadratische Zielfunktion

$$q(x) = a\,(x-m)^2 + c, \qquad a > 0,$$

kann man das Verfahren exakt analysieren. Der Beweis geht in Lean vollständig durch.

Lean code/Kapitel4.lean

```
/-- Modellproblem ‘q(x) = a·(x - m)² + c‘; fuer ‘a > 0‘ Minimum bei ‘m‘. -/
noncomputable def q (x : ℝ) : ℝ := a * (x - m) ^ 2 + c

lemma hasDerivAt_q (x : ℝ) : HasDerivAt (q a m c) (2 * a * (x - m)) x := by
  show HasDerivAt (fun x : ℝ => a * (x - m) ^ 2 + c) (2 * a * (x - m)) x
  have h1 : HasDerivAt (fun y : ℝ => y - m) 1 x := (hasDerivAt_id x).sub_const m
  have h2 := ((h1.pow 2).const_mul a).add_const c
  convert h2 using 1
  push_cast
  ring

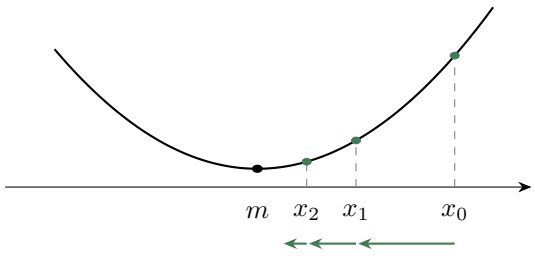
lemma deriv_q (x : ℝ) : deriv (q a m c) x = 2 * a * (x - m) :=
  (hasDerivAt_q a m c x).deriv

/-- ‘m‘ ist globales Minimum, sobald ‘a ≥ 0‘. -/
lemma q_min (ha : 0 ≤ a) (x : ℝ) : q a m c m ≤ q a m c x := by
  show a * (m - m) ^ 2 + c ≤ a * (x - m) ^ 2 + c
  nlinarith [mul_nonneg ha (sq_nonneg (x - m))]
```

Bergab gehen mit fester Schrittlänge. Man steht auf einem Hang und will ins Tal. Sehen kann man nur den Boden direkt unter den Füßen, also die *Steigung*. Die Regel lautet: in Richtung des stärksten Gefälles gehen, und zwar um ein festes Vielfaches t der Steigung.

Weit oben am Hang ist die Steigung groß, also sind die Schritte groß. Je näher man dem Tal kommt, desto flacher wird es und desto kürzer werden die Schritte. Deshalb erreicht man das Minimum im Allgemeinen nicht exakt, kommt ihm aber geometrisch schnell nahe: Jeder Schritt schrumpft den Abstand um denselben Faktor.

Ist t zu groß, schießt man über das Tal hinaus – auf der anderen Seite weiter oben als vorher. Dann wird der nächste Schritt noch größer, und das Verfahren läuft davon.



jeder Schritt halbiert den Abstand zu m

Wegen $q'(x) = 2a(x-m)$ ist ein Schritt

$$x_{k+1} - m = (x_k - t\,q'(x_k)) - m = (1 - 2at)\,(x_k - m).$$

Der Fehler wird also in jedem Schritt mit demselben Faktor multipliziert.

Satz 4.19 (Fehlerformel und Konvergenz)**.** *Für alle n gilt*

$$x_n - m = (1 - 2at)^n \, (x_0 - m).$$

Ist $|1 - 2at| < 1$, so konvergiert $x_n \to m$.

Lean code/Kapitel4.lean

```
/-- Ein Schritt multipliziert den Fehler mit dem Faktor `1 - 2at`. -/
lemma gd_q_schritt (t x : ℝ) :
    gdSchritt (q a m c) t x - m = (1 - 2 * a * t) * (x - m) := by
  show x - t * deriv (q a m c) x - m = (1 - 2 * a * t) * (x - m)
  rw [deriv_q]
  ring

/-- Exakte Fehlerformel nach `n` Schritten. -/
lemma gd_q_fehler (t x₀ : ℝ) (n : ℕ) :
    gdIter (q a m c) t x₀ n - m = (1 - 2 * a * t) ^ n * (x₀ - m) := by
  induction n with
  | zero => simp [gdIter]
  | succ k ih =>
      simp only [gdIter]
      rw [gd_q_schritt, ih]
      ring

/-- **Konvergenzsatz.** Ist `|1 - 2at| < 1`, so konvergiert der
Gradientenabstieg gegen `m` (fuer `a > 0` das Minimum). -/
theorem gd_q_tendsto (t : ℝ) (h : |1 - 2 * a * t| < 1) (x₀ : ℝ) :
    Tendsto (fun n => gdIter (q a m c) t x₀ n) atTop (𝓝 m) := by
  have key : ∀ n,
      gdIter (q a m c) t x₀ n = (1 - 2 * a * t) ^ n * (x₀ - m) + m := by
    intro n
    have := gd_q_fehler a m c t x₀ n
    linarith
  have hpow : Tendsto (fun n : ℕ => (1 - 2 * a * t) ^ n) atTop (𝓝 0) :=
    tendsto_pow_atTop_nhds_zero_of_norm_lt_one (by rwa [Real.norm_eq_abs])
  simpa [key] using (hpow.mul_const (x₀ - m)).add_const m

/-- Die zulaessigen Schrittweiten: genau `0 < t < 1/a`. Zu grosse Schritte
divergieren, zu kleine konvergieren langsam. -/
theorem schrittweite (ha : 0 < a) (t : ℝ) :
    |1 - 2 * a * t| < 1 ↔ (0 < t ∧ t < 1 / a) := by
  rw [abs_lt, lt_div_iff₀ ha]
  constructor
  · rintro ⟨h1, h2⟩
    exact ⟨by nlinarith, by nlinarith⟩
  · rintro ⟨h1, h2⟩
    exact ⟨by nlinarith, by nlinarith⟩
```

Für $a > 0$ ist $|1 - 2at| < 1$ gleichbedeutend (`schrittweite`) mit

$$0 < t < \frac{1}{a}.$$

Das ist genau die bekannte Schrittweitenbedingung $0 < t < 2/L$ für L-glatte Funktionen: Hier ist $q'' \equiv 2a$, also $L = 2a$ und $2/L = 1/a$. Der Satz enthält damit drei Dinge, die man sonst einzeln lernt:

- **Zu große Schritte divergieren.** Für $t > 1/a$ ist $|1-2at| > 1$, der Fehler wächst geometrisch.
- **Zu kleine Schritte sind langsam.** Für $t \to 0$ geht $|1 - 2at| \to 1$; die Konvergenz wird beliebig zäh.
- **Es gibt eine beste Schrittweite.** Für $t = 1/(2a)$ ist $1 - 2at = 0$: Das Verfahren trifft das Minimum in einem Schritt.

Der Konvergenzteil ist ein Fall des Banachschen Fixpunktsatzes (Satz 4.14): Für $|1 - 2at| < 1$ ist ein Schritt eine Kontraktion mit $K = |1 - 2at|$, und ihr Fixpunkt ist das Minimum m.

Lean code/Kapitel4.lean

```
/-- Fuer |1 - 2at| < 1 ist ein Schritt eine Kontraktion mit K = |1 - 2at|:
Der Konvergenzsatz ist ein Fall des Fixpunktsatzes. -/
lemma gd_q_kontraktion (t : ℝ) (h : |1 - 2 * a * t| < 1) :
    Kontraktion |1 - 2 * a * t| (gdSchritt (q a m c) t) where
  nichtneg := abs_nonneg _
  klein := h
  schrumpft x y := by
    have e : gdSchritt (q a m c) t x - gdSchritt (q a m c) t y =
        (1 - 2 * a * t) * (x - y) := by
      linarith [gd_q_schritt a m c t x, gd_q_schritt a m c t y]
    rw [Real.dist_eq, Real.dist_eq, e, abs_mul]

/- Ihr Fixpunkt ist m, fuer a > 0 das Minimum: -/
example (t : ℝ) : gdSchritt (q a m c) t m = m := by
  linarith [gd_q_schritt a m c t m]
```

Bemerkung 4.20 (Was hier fehlt). Satz 4.19 ist ein Modellfall. Die allgemeine Theorie steht nicht in Mathlib: das Abstiegslemma $f(x - t\nabla f(x)) \le f(x) - t(1 - \frac{Lt}{2})\|\nabla f(x)\|^2$ für L-glatte f, die Rate $O(1/k)$ für konvexe und $O(\rho^k)$ für stark konvexe Zielfunktionen. Einen Baustein kann man aber schon beweisen.

Für konvexe f ist jeder Punkt c mit $f'(c) = 0$ ein globales Minimum. Rechts von c sind die Sekanten mindestens so steil wie die Tangente, also ≥ 0; links höchstens 0. Mit `deriv` allein stimmt das nicht – `deriv f c = 0` gilt auch an einem Knick, als Junk-Wert. So ist $f(x) = \max(x, 2x)$ konvex mit `deriv f 0 = 0` (Knick in 0: Steigung 1 links, 2 rechts), aber $f(-1) = -1 < 0 = f(0)$.

Lean code/Kapitel4.lean

```
/-- Fuer konvexe f ist ein Punkt mit Ableitung 0 ein globales Minimum: Rechts
von c sind die Sekanten mindestens so steil wie f'(c) = 0, links hoechstens.
Mit 'deriv f c = 0' allein waere es falsch (Junk-Wert). -/
theorem konvex_kritisch {f : ℝ → ℝ} {c : ℝ} (hf : ConvexOn ℝ Set.univ f)
    (hc : HasDerivAt f 0 c) (y : ℝ) : f c ≤ f y := by
  rcases lt_trichotomy y c with hy | rfl | hy
  · -- y < c: Die Sekante von y nach c steigt hoechstens mit 0.
    have h := hf.slope_le_of_hasDerivAt (Set.mem_univ y) (Set.mem_univ c) hy hc
    exact (slope_nonpos_iff_of_le hy.le).1 h
  · exact le_rfl
  · -- c < y: Die Sekante von c nach y steigt mindestens mit 0.
    have h := hf.le_slope_of_hasDerivAt (Set.mem_univ c) (Set.mem_univ y) hy hc
    exact (slope_nonneg_iff_of_le hy.le).1 h
```

4.6 Optima unter Nebenbedingungen

Die klassische Aussage

Satz 4.21 (Lagrange, klassisch)**.** *Seien f, φ in x_0 streng differenzierbar (etwa: stetig differenzierbar). Hat φ unter der Nebenbedingung $f(x) = f(x_0)$ in x_0 ein lokales Extremum und ist $Df(x_0) \neq 0$, so gibt es ein $\lambda \in \mathbb{R}$ mit*

$$D\varphi(x_0) = \lambda\, Df(x_0).$$

Die Voraussetzung $Df(x_0) \neq 0$ (allgemeiner: eine Regularitätsbedingung an die Nebenbedingungen) wird oft weggelassen. Ohne sie stimmt der Satz nicht: Für $f(x) = x^2$, $\varphi(x) = x$ und $x_0 = 0$ lässt die Nebenbedingung nur $x = 0$ zu, dort liegt also ein Extremum – aber $D\varphi(0) = 1$ ist kein Vielfaches von $Df(0) = 0$.

Die Mathlib-Fassung

Mathlib formuliert den Satz *projektiv* und kommt damit ohne Regularitätsvoraussetzung aus:

Satz 4.22 (Lagrange, Mathlib)**.** *Hat φ unter $f(x) = f(x_0)$ in x_0 ein lokales Extremum und sind f, φ in x_0 streng differenzierbar, so gibt es $(\Lambda, \Lambda_0) \neq (0, 0)$ mit*

$$\Lambda \cdot Df(x_0) + \Lambda_0 \cdot D\varphi(x_0) = 0.$$

Die klassische Fassung folgt daraus, unter denselben Differenzierbarkeitsvoraussetzungen. Ist $\Lambda_0 = 0$, so ist $\Lambda \neq 0$ und $\Lambda\, Df(x_0) = 0$, also $Df(x_0) = 0$ – genau der Fall, den die klassische Fassung ausschließt. Also ist $\Lambda_0 \neq 0$, man darf teilen und erhält $D\varphi(x_0) = \lambda\, Df(x_0)$ mit $\lambda = -\Lambda/\Lambda_0$. Die projektive Form deckt beide Fälle ab. In Lean ist dieser Schritt reine lineare Algebra:

Lean code/Kapitel4.lean

```
/-- Projektiv → klassisch, sobald Df ≠ 0: Waere Λ₀ = 0, so waere Λ ≠ 0 und
Λ • Df = 0, also Df = 0. Sonst darf man durch Λ₀ teilen: μ = -Λ/Λ₀ ist das
λ der klassischen Fassung ('λ' ist in Lean fuer 'fun' reserviert). -/
theorem klassisch_aus_projektiv {E : Type*} [AddCommGroup E] [Module ℝ E]
    {Df Dφ : E} {Λ Λ₀ : ℝ} (hne : (Λ, Λ₀) ≠ 0) (h : Λ • Df + Λ₀ • Dφ = 0)
    (hDf : Df ≠ 0) : ∃ μ : ℝ, Dφ = μ • Df := by
  have hΛ₀ : Λ₀ ≠ 0 := by
    rintro rfl                                        -- Fall Λ₀ = 0
    have hΛ : Λ ≠ 0 := fun h0 => hne (by simp [h0])
    rw [zero_smul, add_zero, smul_eq_zero] at h  -- Λ = 0 oder Df = 0
    exact h.elim hΛ hDf
  have h2 : Λ₀ • Dφ = -(Λ • Df) := eq_neg_of_add_eq_zero_right h
  use -Λ / Λ₀                                         -- Fall Λ₀ ≠ 0: teilen
  calc Dφ = Λ₀⁻¹ • (Λ₀ • Dφ) := by rw [smul_smul, inv_mul_cancel₀ hΛ₀, one_smul]
    _ = Λ₀⁻¹ • -(Λ • Df) := by rw [h2]
    _ = (-Λ / Λ₀) • Df := by rw [smul_neg, smul_smul, ← neg_smul]; congr 1; ring
```

Mit Mathlibs projektivem Satz ist damit die klassische Fassung bewiesen, mit genau ihren Voraussetzungen:

Lean code/Kapitel4.lean

```
/-- Lagrange, klassisch, mit allen Voraussetzungen: f und φ streng
differenzierbar und Df ≠ 0. Beweis: Mathlibs projektiver Satz, dann der
Zusammenhang von oben. -/
theorem lagrange_klassisch {E : Type*} [NormedAddCommGroup E]
    [NormedSpace ℝ E] [CompleteSpace E] {f φ : E → ℝ} {x₀ : E}
    {Df Dφ : E →L[ℝ] ℝ} (hext : IsLocalExtrOn φ {x | f x = f x₀} x₀)
```

```
    (hf : HasStrictFDerivAt f Df x₀) (hφ : HasStrictFDerivAt φ Dφ x₀)
    (hDf : Df ≠ 0) : ∃ μ : ℝ, Dφ = μ • Df := by
  obtain ⟨Λ, Λ₀, hne, h⟩ :=
    hext.exists_multipliers_of_hasStrictFDerivAt_1d hf hφ
  exact klassisch_aus_projektiv hne h hDf
```

Ein Beispiel

Gesucht ist das Maximum von $\varphi(x,y) = xy$ unter $f(x,y) = x + y = 1$. Die Gradienten sind $\nabla\varphi = (y,x)$ und $\nabla f = (1,1)$. Die Lagrange-Bedingung $\nabla\varphi = \lambda\nabla f$ gibt

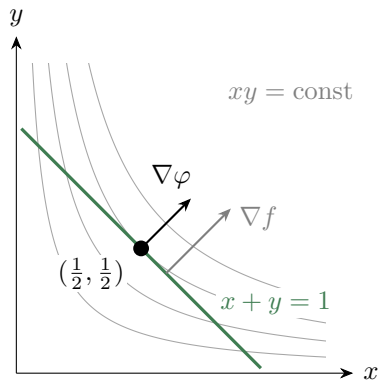
$$y = \lambda, \quad x = \lambda \implies x = y,$$

zusammen mit $x + y = 1$ also $x = y = \frac{1}{2}$ und den Wert $\frac{1}{4}$.

Wo die Höhenlinie die Straße nur streift. Man wandert auf einer Straße durch ein Gebirge. Die Straße ist die Nebenbedingung $f = \text{const}$, die Höhenlinien sind die Niveaulinien von φ.

Solange die Straße eine Höhenlinie *kreuzt*, geht es auf der einen Seite bergauf und auf der anderen bergab – man ist also nicht am höchsten Punkt der Straße. Erst wo die Straße eine Höhenlinie nur noch *berührt*, kann ein Extremum liegen.

Berühren heißt: gleiche Tangente, also parallele Normalen. Und die Normale einer Höhenlinie ist der Gradient. Deshalb steht dort $\nabla\varphi \parallel \nabla f$ – das ist die Lagrange-Bedingung.



Lagrange liefert nur den *Kandidaten*. Dass das wirklich das Maximum ist, muss man extra zeigen. Hier geht es elementar:

Lean code/Kapitel4.lean

```
/-- Beispiel: maximiere ‘x·y‘ unter ‘x + y = 1‘.
Lagrange liefert den Kandidaten ‘x = y = 1/2‘; hier die elementare
Bestaetigung, dass dieser Kandidat wirklich das Maximum ist. -/
theorem xy_le_quarter (x y : ℝ) (h : x + y = 1) : x * y ≤ 1 / 4 := by
  nlinarith [sq_nonneg (x - y)]

/- Der Wert wird im Lagrange-Punkt angenommen. -/
example : (1 / 2 : ℝ) * (1 / 2) = 1 / 4 := by norm_num
```

Bemerkung 4.23. Der Beweis ist die Ungleichung $xy \le \frac{1}{4}(x+y)^2$, also $(x-y)^2 \ge 0$. Die Taktik `nlinarith` findet sie mit dem Hinweis `sq_nonneg (x - y)` selbst. Man sieht hier gut die Arbeitsteilung: Lagrange verengt die Suche auf endlich viele Punkte, das Optimum bestätigen muss man von Hand.

Zusammenfassung

- Die Räume bauen aufeinander auf: topologisch (offene Mengen), metrisch (Abstand), normiert (Länge), Innenprodukt (Winkel). Jede Stufe liefert die darunter mit.
- Ein Filter ist ein System „großer“ Mengen. `Tendsto f F G` ersetzt alle Grenzwertdefinitionen durch eine; Summenregel und Eindeutigkeit beweist man einmal, für jeden Filter – die Eindeutigkeit, sobald die leere Menge nicht groß ist (`NeBot`).
- Vollständigkeit lässt sich erst nach der Konvergenz erklären: Jede Cauchy-Folge konvergiert. Banach- und Hilbertraum sind normierter bzw. Innenproduktraum *plus* `CompleteSpace`. Im Beweis des Fixpunktsatzes braucht nur ein Schritt die Vollständigkeit.
- Darauf beruht der Projektionssatz (Satz 4.15) – und damit Fourierreihen und kleinste Quadrate.
- Die Landau-Symbole O und o vergleichen Funktionen längs eines Filters, und der Filter entscheidet mit: Bei 0 ist $x^2 = o(x)$, bei $+\infty$ ist $x = o(x^2)$. Die Ableitung ist darüber definiert und damit selbst ein Filterbegriff. Die Tangente ist eindeutig, weil Grenzwerte es sind; an einem lokalen Minimum ist die Ableitung 0, weil Grenzwerte $\leq$ erhalten (Fermat). `HasDerivAt` ist der Spezialfall der Fréchet-Ableitung.
- `deriv` ist total. Voraussetzungen stehen in den Sätzen, nicht im Definitionsbereich.
- Für die quadratische Zielfunktion lässt sich der Gradientenabstieg vollständig analysieren (Satz 4.19). Die Schrittweitenbedingung $0 < t < 1/a$ ist genau $0 < t < 2/L$.
- Mathlibs Lagrange-Satz ist projektiv formuliert und braucht keine Regularitätsvoraussetzung; die klassische Fassung folgt daraus, sobald $Df(x_0) \neq 0$.

Kapitel 5

Numerische Integration

Die meisten Integrale kann man nicht geschlossen ausrechnen. Die numerische Integration ersetzt das Integral durch eine endliche Linearkombination von Funktionswerten – eine *Quadraturformel* – und fragt, wie gut das ist.

Die Grundfrage dabei ist der *Exaktheitsgrad*. Der lässt sich vollständig in Lean beweisen, und genau das macht dieses Kapitel.

5.1 Das Integral in Mathlib

Mathlib baut das Integral in Stufen auf: Maßtheorie, dann das Bochner-Integral $\int_X f \, d\mu$ für Funktionen in Banachräume, darauf das orientierte *Intervallintegral*

$$\int_a^b f(x)\,dx := \int_{(a,b]} f \, d\lambda - \int_{(b,a]} f \, d\lambda.$$

Die Orientierung sorgt dafür, dass $\int_a^b = -\int_b^a$ ohne Fallunterscheidung gilt.

Lean code/Kapitel5.lean

```
/- Das orientierte Intervallintegral ist ueber das Bochner-Integral definiert:
   ‘∫ x in a..b, f x = ∫ x in Ioc a b, f x - ∫ x in Ioc b a, f x‘. -/
#check @intervalIntegral

/- Wie ‘deriv‘ ist auch ‘∫‘ total: fuer nicht integrierbare ‘f‘ ist der Wert 0.
   Saetze tragen deshalb ‘IntervalIntegrable‘-Hypothesen. -/
#check @IntervalIntegrable
#check @intervalIntegral.integral_undef

/- Hauptsatz der Differential- und Integralrechnung: -/
#check @intervalIntegral.integral_deriv_eq_sub'

example : (∫ x in (0:ℝ)..1, x) = 1 / 2 := by simp
example : (∫ x in (0:ℝ)..1, x ^ 2) = 1 / 3 := by norm_num [integral_pow]
```

Junk-Werte, wieder. Wie `deriv` ist auch das Integral total. Ist f nicht integrierbar, ist der Wert 0 (`intervalIntegral.integral_undef`). Sätze über das Integral tragen deshalb eine `IntervalIntegrable`-Hypothese. Besonders wichtig: Linearität gilt *nicht* ohne Weiteres – man braucht die Integrierbarkeit der Summanden.

Für alles Weitere brauchen wir die Integrale von Polynomen. Sie folgen aus `integral_pow` und der Linearität; die Integrierbarkeit liefert die Stetigkeit.

Lean code/Kapitel5.lean

```
/-- Integral einer affinen Funktion. -/
theorem integral_affin (p r a b : ℝ) :
    (∫ x in a..b, (p * x + r)) = p * (b ^ 2 - a ^ 2) / 2 + r * (b - a) := by
  have h1 : IntervalIntegrable (fun x : ℝ => p * x) volume a b :=
    (by fun_prop : Continuous fun x : ℝ => p * x).intervalIntegrable a b
  have h2 : IntervalIntegrable (fun _ : ℝ => r) volume a b :=
    _root_.intervalIntegrable_const
  rw [intervalIntegral.integral_add h1 h2, intervalIntegral.integral_const_mul,
    integral_id, intervalIntegral.integral_const, smul_eq_mul]
  ring

/-- Integral eines Polynoms vom Grad ≤ 3. -/
theorem integral_kubisch (c₃ c₂ c₁ c₀ a b : ℝ) :
    (∫ x in a..b, (c₃ * x ^ 3 + c₂ * x ^ 2 + c₁ * x + c₀)) =
      c₃ * (b ^ 4 - a ^ 4) / 4 + c₂ * (b ^ 3 - a ^ 3) / 3
        + c₁ * (b ^ 2 - a ^ 2) / 2 + c₀ * (b - a) := by
  have i3 : IntervalIntegrable (fun x : ℝ => c₃ * x ^ 3) volume a b :=
    (by fun_prop : Continuous fun x : ℝ => c₃ * x ^ 3).intervalIntegrable a b
  have i2 : IntervalIntegrable (fun x : ℝ => c₂ * x ^ 2) volume a b :=
    (by fun_prop : Continuous fun x : ℝ => c₂ * x ^ 2).intervalIntegrable a b
  have i1 : IntervalIntegrable (fun x : ℝ => c₁ * x) volume a b :=
    (by fun_prop : Continuous fun x : ℝ => c₁ * x).intervalIntegrable a b
  have i0 : IntervalIntegrable (fun _ : ℝ => c₀) volume a b :=
    _root_.intervalIntegrable_const
  rw [intervalIntegral.integral_add ((i3.add i2).add i1) i0,
    intervalIntegral.integral_add (i3.add i2) i1,
    intervalIntegral.integral_add i3 i2,
    intervalIntegral.integral_const_mul, intervalIntegral.integral_const_mul,
    intervalIntegral.integral_const_mul, intervalIntegral.integral_const,
    integral_pow, integral_pow, integral_id, smul_eq_mul]
  push_cast
  ring
```

5.2 Quadraturformeln

Definition 5.1 (Quadraturformel)**.** Eine Quadraturformel auf $[a,b]$ mit Knoten $x_0,\dots,x_s \in [a,b]$ und Gewichten $w_0,\dots,w_s$ ist

$$Q(f) = (b-a)\sum_{i=0}^{s} w_i\, f(x_i) \;\approx\; \int_a^b f(x)\,dx.$$

Die drei klassischen Formeln:

$$Q_{\mathrm{M}}(f) = (b-a)\, f\Big(\tfrac{a+b}{2}\Big) \qquad \text{(Mittelpunktregel)}$$

$$Q_{\mathrm{T}}(f) = (b-a)\,\frac{f(a)+f(b)}{2} \qquad \text{(Trapezregel)}$$

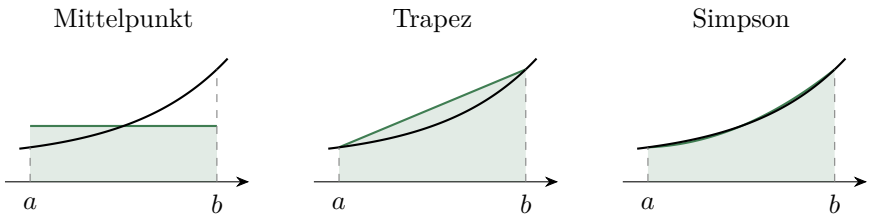
$$Q_{\mathrm{S}}(f) = \frac{b-a}{6}\Big(f(a) + 4f\Big(\tfrac{a+b}{2}\Big) + f(b)\Big) \qquad \text{(Simpsonregel)}$$

Die Fläche durch eine einfachere ersetzen. Gesucht ist die Fläche unter einer Kurve. Statt sie auszurechnen, legt man eine einfache Figur darüber, deren Fläche man kennt.

Mittelpunktregel: ein Rechteck, dessen Höhe man in der Mitte abliest. Links wird zu wenig, rechts zu viel gerechnet – und beides hebt sich weitgehend auf. Deshalb ist die Regel besser, als ein einzelner Messpunkt vermuten lässt.

Trapezregel: die Kurve wird durch die Sehne zwischen den Endpunkten ersetzt. Bei einer nach oben gekrümmten Kurve liegt die Sehne darüber, man rechnet also systematisch zu viel.

Simpsonregel: statt einer Geraden legt man eine Parabel durch die beiden Endpunkte und den Mittelpunkt. Eine Parabel schmiegt sich deutlich besser an – und trifft sogar kubische Kurven noch exakt.



Die schattierte Fläche ist jeweils das, was die Formel berechnet, die farbige Kante ihre Oberkante. Schwarz ist die wahre Kurve. Man sieht, warum die Trapezregel hier systematisch zu viel liefert und warum Simpson kaum noch danebenliegt.

Lean code/Kapitel5.lean

```
/-- Mittelpunktregel: ein Knoten, Gewicht `b - a`. -/
noncomputable def mittelpunkt (f : ℝ → ℝ) (a b : ℝ) : ℝ := (b - a) * f ((a + b)
    / 2)

/-- Trapezregel: zwei Knoten, Gewichte `(b-a)/2`. -/
noncomputable def trapez (f : ℝ → ℝ) (a b : ℝ) : ℝ := (b - a) * (f a + f b) / 2

/-- Simpsonregel: drei Knoten, Gewichte `(b-a)/6 · (1, 4, 1)`. -/
noncomputable def simpson (f : ℝ → ℝ) (a b : ℝ) : ℝ :=
  (b - a) / 6 * (f a + 4 * f ((a + b) / 2) + f b)
```

Exaktheitsgrad

Definition 5.2 (Exaktheitsgrad)**.** Eine Quadraturformel Q hat *Exaktheitsgrad* p, wenn $Q(f) = \int_a^b f$ für alle Polynome f vom Grad $\leq p$ gilt, aber nicht für alle vom Grad $p+1$.

Dieser Begriff steuert alles: Je höher der Exaktheitsgrad, desto schneller fällt der Fehler beim Verfeinern. Und er ist vollständig beweisbar.

Satz 5.3 (Exaktheitsgrade)**.** *Mittelpunkt- und Trapezregel haben Exaktheitsgrad* 1*, die Simpsonregel hat Exaktheitsgrad* 3*.*

Lean code/Kapitel5.lean

```
/-- Die Trapezregel ist exakt fuer Polynome vom Grad ≤ 1. -/
theorem trapez_exakt_affin (p r a b : ℝ) :
    trapez (fun x => p * x + r) a b = ∫ x in a..b, (p * x + r) := by
  rw [integral_affin]
  show (b - a) * ((p * a + r) + (p * b + r)) / 2 = _
  ring

/-- Die Mittelpunktregel ist ebenfalls exakt bis Grad 1 -- mit nur *einem*
Funktionsaufruf statt zweien. -/
theorem mittelpunkt_exakt_affin (p r a b : ℝ) :
    mittelpunkt (fun x => p * x + r) a b = ∫ x in a..b, (p * x + r) := by
  rw [integral_affin]
  show (b - a) * (p * ((a + b) / 2) + r) = _
  ring
```

```
/-- Die Simpsonregel ist exakt bis Grad 3 -- ein Grad mehr, als die drei
Knoten erwarten lassen. -/
theorem simpson_exakt_kubisch (c₃ c₂ c₁ c₀ a b : ℝ) :
    simpson (fun x => c₃ * x ^ 3 + c₂ * x ^ 2 + c₁ * x + c₀) a b
      = ∫ x in a..b, (c₃ * x ^ 3 + c₂ * x ^ 2 + c₁ * x + c₀) := by
  rw [integral_kubisch]
  show (b - a) / 6 * ((c₃ * a ^ 3 + c₂ * a ^ 2 + c₁ * a + c₀)
      + 4 * (c₃ * ((a + b) / 2) ^ 3 + c₂ * ((a + b) / 2) ^ 2 + c₁ * ((a + b) / 2)
    + c₀)
      + (c₃ * b ^ 3 + c₂ * b ^ 2 + c₁ * b + c₀)) = _
  ring

/-- Die Schranke ist scharf: fuer ‘x^2‘ ist die Trapezregel *nicht* exakt. -/
theorem trapez_nicht_exakt_quadratisch :
    trapez (fun x => x ^ 2) 0 1 ≠ ∫ x in (0:ℝ)..1, x ^ 2 := by
  rw [show (∫ x in (0:ℝ)..1, x ^ 2) = 1 / 3 by norm_num [integral_pow]]
  show (1 - 0) * ((0:ℝ) ^ 2 + 1 ^ 2) / 2 ≠ 1 / 3
  norm_num

/-- Ebenso die Mittelpunktregel: Grad 1 ist auch hier scharf. -/
theorem mittelpunkt_nicht_exakt_quadratisch :
    mittelpunkt (fun x => x ^ 2) 0 1 ≠ ∫ x in (0:ℝ)..1, x ^ 2 := by
  rw [show (∫ x in (0:ℝ)..1, x ^ 2) = 1 / 3 by norm_num [integral_pow]]
  show (1 - 0) * (((0:ℝ) + 1) / 2) ^ 2 ≠ 1 / 3
  norm_num

/-- Und Simpson ist fuer ‘x^4‘ nicht mehr exakt: Grad 3 ist scharf. -/
theorem simpson_nicht_exakt_grad4 :
    simpson (fun x => x ^ 4) 0 1 ≠ ∫ x in (0:ℝ)..1, x ^ 4 := by
  rw [show (∫ x in (0:ℝ)..1, x ^ 4) = 1 / 5 by norm_num [integral_pow]]
  show ((1:ℝ) - 0) / 6 * ((0:ℝ) ^ 4 + 4 * (((0:ℝ) + 1) / 2) ^ 4 + 1 ^ 4) ≠ 1 / 5
  norm_num
```

Drei Dinge fallen auf.

Erstens ist die Simpsonregel besser, als ihre drei Knoten erwarten lassen. Eine Formel mit $s+1$ Knoten hat normalerweise Exaktheitsgrad s; Simpson erreicht 3 statt 2. Der Grund ist die Symmetrie der Knoten um den Mittelpunkt: Der Fehler bei x^3 hebt sich weg. Genau das rechnet `ring` im Beweis nach. Nachvollziehen kann man es, indem man $a = -h$ und $b = h$ setzt.

Zweitens ist die Mittelpunktregel interessant. Sie erreicht denselben Exaktheitsgrad wie die Trapezregel, braucht dafür aber nur *eine* statt zwei Funktionsauswertungen. Bei teuren Integranden ist das der halbe Aufwand.

Drittens sind die Grade wirklich scharf. Das zeigen die drei Gegenbeispiele am Ende des Blocks. Für $\int_0^1 x^2\,dx = \frac{1}{3}$ liefert die Trapezregel $\frac{1}{2}$ und die Mittelpunktregel $\frac{1}{4}$; für $\int_0^1 x^4\,dx = \frac{1}{5}$ liefert Simpson $\frac{5}{24}$. Jeweils knapp daneben – und deshalb ist der Exaktheitsgrad eine scharfe Aussage und nicht nur eine untere Schranke.

Bemerkung 5.4 (Fehlerterme). Für $f \in C^2$ bzw. C^4 gelten die klassischen Fehlerdarstellungen

$$\int_a^b f - Q_{\mathrm{T}}(f) = -\frac{(b-a)^3}{12} f''(\xi), \qquad \int_a^b f - Q_{\mathrm{S}}(f) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$$

mit einem $\xi \in (a,b)$. Diese Aussagen stehen nicht in Mathlib; ihr Beweis läuft über den Mittelwertsatz der Integralrechnung und die Peano-Kerndarstellung. Die hier bewiesenen Exaktheitsgrade sind der Teil, der ohne diesen Apparat auskommt – und sie legen die Fehlerordnung schon fest.

5.3 Zusammengesetzte Formeln

Auf einem großen Intervall wird es nicht genau. Man zerlegt $[a, b]$ deshalb gleichmäßig in n Teile mit den Knoten

$$x_i = a + i\,\frac{b-a}{n}, \qquad i = 0, \dots, n,$$

und wendet die Formel auf jedem Teilintervall an.

Lean code/Kapitel5.lean

```
/-- Die Knoten der gleichmaessigen Zerlegung von `[a, b]` in `n` Teile. -/
noncomputable def knoten (a b : ℝ) (n i : ℕ) : ℝ := a + i * (b - a) / n

lemma knoten_zero (a b : ℝ) (n : ℕ) : knoten a b n 0 = a := by
  simp [knoten]

lemma knoten_last (a b : ℝ) (n : ℕ) (hn : n ≠ 0) : knoten a b n n = b := by
  have hn' : (n : ℝ) ≠ 0 := Nat.cast_ne_zero.mpr hn
  show a + (n : ℝ) * (b - a) / n = b
  have h : (n : ℝ) * (b - a) / n = b - a := by
    rw [div_eq_iff hn']
    ring
  rw [h]
  ring

/-- Zusammengesetzte Trapezregel. -/
noncomputable def trapezSumme (f : ℝ → ℝ) (a b : ℝ) (n : ℕ) : ℝ :=
  ∑ i ∈ Finset.range n, trapez f (knoten a b n i) (knoten a b n (i + 1))

/-- Auch die zusammengesetzte Regel ist exakt bis Grad 1 -- die Teilintegrale
teleskopieren zum Gesamtintegral. -/
theorem trapezSumme_exakt_affin (p r a b : ℝ) (n : ℕ) (hn : n ≠ 0) :
    trapezSumme (fun x => p * x + r) a b n = ∫ x in a..b, (p * x + r) := by
  have hstep : ∀ i : ℕ,
      trapez (fun x => p * x + r) (knoten a b n i) (knoten a b n (i + 1))
        = ∫ x in (knoten a b n i)..(knoten a b n (i + 1)), (p * x + r) :=
    fun i => trapez_exakt_affin p r _ _
  have hsum := intervalIntegral.sum_integral_adjacent_intervals
    (μ := volume) (a := fun i => knoten a b n i) (f := fun x : ℝ => p * x + r)
    (n := n)
    (fun k _ => (by fun_prop : Continuous fun x : ℝ => p * x +
    r).intervalIntegrable _ _)
  calc trapezSumme (fun x => p * x + r) a b n
      = ∑ i ∈ Finset.range n,
          ∫ x in (knoten a b n i)..(knoten a b n (i + 1)), (p * x + r) :=
        Finset.sum_congr rfl (fun i _ => hstep i)
    _ = ∫ x in (knoten a b n 0)..(knoten a b n n), (p * x + r) := hsum
    _ = ∫ x in a..b, (p * x + r) := by rw [knoten_zero, knoten_last a b n hn]
```

Der Beweis von `trapezSumme_exakt_affin` zeigt das übliche Muster:

1. Auf jedem Teilintervall ist die Regel exakt (Satz 5.3).
2. Die Teilintegrale *teleskopieren* zum Gesamtintegral; dafür gibt es in Mathlib das Lemma `sum_integral_adjacent_intervals` (siehe unten).
3. Die Randknoten sind $x_0 = a$ und $x_n = b$.

Bei Schritt 2 zahlt sich die Orientierung des Intervallintegrals aus: Die Additivität $\int_a^b + \int_b^c = \int_a^c$ gilt ohne jede Voraussetzung an die Lage von a, b, c.

Bemerkung 5.5 (Fehlerordnung). Summiert man die Fehlerterme über n Teilintervalle der Länge $h = (b-a)/n$, bekommt man für die zusammengesetzten Regeln

$$\text{Trapez: } O(h^2) = O(n^{-2}), \qquad \text{Simpson: } O(h^4) = O(n^{-4}).$$

Zehnmal so viele Stützstellen verkleinern den Trapezfehler also um den Faktor 100 und den Simpsonfehler um 10 000.

5.4 Mehrere Dimensionen

Bis hierher war der Integrationsbereich ein Intervall. In der Praxis ist er das selten. Ein Erwartungswert über drei Zufallsgrößen ist ein Integral über $\mathbb{R}^3$, der Schwerpunkt eines Körpers eines über ein Gebiet im Raum, und beim Bewerten von Finanzprodukten oder in der Statistik sind zwanzig oder hundert Dimensionen nichts Besonderes.

Das Integral über einen Quader

In Mathlib ist das kein neuer Begriff. Das Bochner-Integral ist für jedes Maß definiert; für $\mathbb{R}^n$ nimmt man das Lebesgue-Maß. Ein Quader $[a_1, b_1] \times \cdots \times [a_n, b_n]$ hat das Volumen $\prod_i (b_i - a_i)$, und das Maß auf einem Produktraum ist das Produktmaß.

Satz 5.6 (Fubini). *Ist f integrierbar bezüglich $\mu \otimes \nu$, dann gilt*

$$\int_{X\times Y} f \,\mathrm{d}(\mu\otimes\nu) \;=\; \int_X \left(\int_Y f(x,y)\,\mathrm{d}\nu(y)\right)\mathrm{d}\mu(x) \;=\; \int_Y \left(\int_X f(x,y)\,\mathrm{d}\mu(x)\right)\mathrm{d}\nu(y).$$

Schichtweise. Man schneidet den Körper in Scheiben. Für jedes feste x berechnet man den Inhalt der Scheibe, und danach integriert man über die Scheiben. Dass es auf die Reihenfolge nicht ankommt, ist die eigentliche Aussage – und sie gilt nur, wenn f integrierbar ist. Ohne diese Voraussetzung gibt es Gegenbeispiele.

Lean code/Kapitel5.lean

```
/- In mehreren Dimensionen ist das Integral kein neuer Begriff: dasselbe
   Bochner-Integral, nur mit dem Lebesgue-Mass auf einem Produktraum. -/

/- Das Volumen eines Quaders ist das Produkt der Kantenlaengen. -/
example (a b : Fin 3 → ℝ) :
    volume (Set.Icc a b) = ∏ i, ENNReal.ofReal (b i - a i) :=
  Real.volume_Icc_pi

/- Auf einem Produktraum ist das Volumen das Produktmass ... -/
#check @MeasureTheory.Measure.volume_eq_prod

/- ... und Satz von Fubini macht daraus ein geschachteltes Integral. -/
#check @MeasureTheory.integral_prod
#check @MeasureTheory.integral_integral_swap

/- Damit rechnet man in zwei Dimensionen wie in einer. -/
example : (∫ x in (0:ℝ)..1, ∫ y in (0:ℝ)..1, x * y) = 1 / 4 := by
  have hy : ∀ x : ℝ, (∫ y in (0:ℝ)..1, x * y) = x * (1 / 2) := by
    intro x
    rw [intervalIntegral.integral_const_mul, integral_id]
    norm_num
  simp only [hy]
  rw [intervalIntegral.integral_mul_const, integral_id]
  norm_num
```

Damit ist der Weg frei: Ein Integral über einen Quader wird zu geschachtelten Integralen über Intervalle, und darauf passen die Formeln aus Abschnitt 5.2.

Produktregeln

Definition 5.7 (Produktregel)**.** Sei Q eine Quadraturformel in einer Variablen. Die zugehörige *Produktregel* auf dem Rechteck $[a,b]\times[c,d]$ ist

$$Q^{\otimes 2}(f) \;=\; Q_x\Big(x \mapsto Q_y(y \mapsto f(x,y))\Big).$$

Ihre Knoten sind das Gitter aus den Knoten von Q, ihre Gewichte die Produkte der Gewichte. In d Dimensionen entsprechend.

Für die Trapezregel sind das die vier Ecken des Rechtecks, jede mit dem Gewicht $\frac{1}{4}$; für Simpson ein 3×3-Gitter aus neun Punkten.

Lean code/Kapitel5.lean

```
/-- Produktregel: die eindimensionale Regel in jeder Richtung angewandt. -/
noncomputable def trapez2 (f : ℝ → ℝ → ℝ) (a b c d : ℝ) : ℝ :=
  trapez (fun x => trapez (f x) c d) a b

/-- Ausgeschrieben sind das die vier Ecken, jede mit demselben Gewicht. -/
theorem trapez2_ecken (f : ℝ → ℝ → ℝ) (a b c d : ℝ) :
    trapez2 f a b c d
      = (b - a) * (d - c) / 4 * (f a c + f a d + f b c + f b d) := by
  unfold trapez2 trapez; ring

/-- Auf einem Produkt `g(x)·h(y)` zerfaellt die Produktregel in das Produkt
der eindimensionalen Regeln. -/
theorem trapez2_separabel (g h : ℝ → ℝ) (a b c d : ℝ) :
    trapez2 (fun x y => g x * h y) a b c d = trapez g a b * trapez h c d := by
  unfold trapez2 trapez; ring

/-- Daraus folgt: Ist die eindimensionale Regel auf `g` und auf `h` exakt,
dann ist die Produktregel auf `g(x)·h(y)` exakt. So uebertraegt sich der
Exaktheitsgrad koordinatenweise. -/
theorem trapez2_exakt_separabel (g h : ℝ → ℝ) (a b c d : ℝ)
    (hg : trapez g a b = ∫ x in a..b, g x)
    (hh : trapez h c d = ∫ y in c..d, h y) :
    trapez2 (fun x y => g x * h y) a b c d
      = ∫ x in a..b, ∫ y in c..d, g x * h y := by
  have hy : ∀ x : ℝ, (∫ y in c..d, g x * h y) = g x * (∫ y in c..d, h y) :=
    fun x => intervalIntegral.integral_const_mul _ _
  simp only [hy]
  rw [intervalIntegral.integral_mul_const, trapez2_separabel, hg, hh]

/-- Zum Beispiel ist die Produkt-Trapezregel exakt fuer `f(x,y) = x·y`. -/
example (a b c d : ℝ) :
    trapez2 (fun x y => x * y) a b c d = ∫ x in a..b, ∫ y in c..d, x * y :=
  trapez2_exakt_separabel (fun x => x) (fun y => y) a b c d
    (by simpa using trapez_exakt_affin 1 0 a b)
    (by simpa using trapez_exakt_affin 1 0 c d)
```

Der Satz `trapez2_exakt_separabel` ist der Grund, warum sich der Exaktheitsgrad überträgt: Auf einem Produkt $g(x)\,h(y)$ zerfällt die Produktregel in das Produkt der eindimensionalen Regeln, und dasselbe tut das Integral. Ist Q exakt für Polynome vom Grad $\le p$, dann ist $Q^{\otimes d}$ exakt für alle Polynome, die *in jeder Variablen einzeln* höchstens Grad p haben.

5.5 Der Fluch der Dimension

Die Fehlerordnung bleibt also erhalten. Der Preis steht woanders.

Eine zusammengesetzte Produktregel mit n Teilintervallen je Richtung braucht $N = (n+1)^d$ Funktionsauswertungen. Der Fehler ist nach wie vor $O(h^p) = O(n^{-p})$. Drückt man ihn in der Anzahl der Auswertungen aus, steht dort

$$\text{Fehler} \;=\; O\big(N^{-p/d}\big).$$

Der Exponent wird durch die Dimension geteilt. Für die zusammengesetzte Trapezregel ($p = 2$) bedeutet das:

Dimension	Punkte je Richtung	Auswertungen
$d = 1$	1000	10^3
$d = 2$	1000	10^6
$d = 3$	1000	10^9
$d = 10$	1000	10^{30}

Die Zeile für $d = 10$ ist kein Rechenaufwand mehr, sondern eine Absage. 10^{30} Auswertungen schafft kein Rechner, und zwar nicht heute und nicht später. Das Phänomen heißt *Fluch der Dimension*.

Woran es liegt. Ein Gitter mit n Punkten je Richtung hat n^d Punkte. Halbiert man den Abstand zwischen ihnen, braucht man in einer Dimension doppelt so viele, in drei Dimensionen achtmal so viele und in zehn Dimensionen tausendmal so viele. Der Aufwand hängt am Abstand, und der Abstand ist das, was die Genauigkeit bestimmt.

Umgekehrt gerechnet: Verteilt man eine Million Punkte gleichmäßig im zehndimensionalen Einheitswürfel, dann liegen Nachbarn immer noch etwa 0,25 auseinander – ein Viertel der Kantenlänge. Von „fein“ kann keine Rede sein.

Monte-Carlo

Der Ausweg gibt das Gitter auf. Man zieht N Punkte $X_1, \dots, X_N$ zufällig und gleichverteilt aus dem Gebiet Ω und nimmt

$$\hat{I}_N \;=\; \frac{|\Omega|}{N} \sum_{i=1}^{N} f(X_i).$$

Dass das funktioniert, ist das starke Gesetz der großen Zahlen: $\hat{I}_N \to \int_\Omega f$ fast sicher. Wie schnell, sagt die Varianz. Die Summanden sind unabhängig, also addieren sich die Varianzen, und

$$\operatorname{Var}(\hat{I}_N) = \frac{|\Omega|^2 \sigma^2}{N}, \qquad \text{also} \qquad \text{Fehler} \;\sim\; \frac{\sigma}{\sqrt{N}}.$$

Lean `code/Kapitel5.lean`

```
/- Monte-Carlo beruht auf dem starken Gesetz der grossen Zahlen: Der
   Mittelwert unabhaengiger, gleich verteilter Zufallsgroessen konvergiert
   fast sicher gegen den Erwartungswert. Die Aussage kennt keine Dimension --
   deshalb wird das Verfahren von ihr auch nicht schlechter. -/
#check @ProbabilityTheory.strong_law_ae
```

Warum die Dimension verschwindet. In der Schranke $\sigma/\sqrt{N}$ kommt d nicht vor. Das ist kein Trick, sondern eine Folge davon, dass das Gesetz der großen Zahlen nichts über den Raum weiß, aus dem gezogen wird – es sieht nur die Zahlen $f(X_i)$. Die Dimension steckt allenfalls in σ, und die hängt von f ab, nicht vom Verfahren.

Dafür ist der Exponent mit $\frac{1}{2}$ schlecht. Eine Dezimalstelle mehr kostet hundertmal so viele Punkte – in einer Dimension ist die Trapezregel klar besser. Der Wechsel lohnt sich ab etwa $d = 4$, und zwar unabhängig davon, wie gut die eine oder die andere Seite programmiert ist: Die Exponenten entscheiden.

	Produktregel (Trapez)	**Monte-Carlo**
Fehler	$O(N^{-2/d})$	$O(N^{-1/2})$
gut für	kleine d, glattes f	große d
Ergebnis	deterministisch	zufällig
Schranke	Fehlerabschätzung	Konfidenzintervall

Die letzte Zeile ist der eigentliche Preis. Monte-Carlo liefert keine Garantie, sondern eine Aussage über Wahrscheinlichkeiten. Wer eine echte Schranke braucht, bekommt sie hier nicht.

Bemerkung 5.8 (Quasi-Monte-Carlo). Zwischen beidem liegt ein drittes Verfahren. Statt zufälliger Punkte nimmt man Folgen, die gleichmäßiger liegen als zufällige – Halton, Sobol. Die Ungleichung von Koksma–Hlawka schätzt den Fehler durch $V(f) \cdot D_N^*$ ab, wobei $V(f)$ die Variation von f ist und D_N^* die *Diskrepanz* der Punktfolge misst, also wie weit sie von der Gleichverteilung abweicht. Für gute Folgen ist $D_N^* = O((\log N)^d/N)$. Das ist fast N^{-1} und damit besser als Monte-Carlo – solange d nicht zu groß und f von beschränkter Variation ist.

5.6 Numerisches Experiment

Reelle Zahlen sind in Lean nicht berechenbar. $\mathbb{R}$ ist über Cauchy-Folgen konstruiert, und Gleichheit darauf ist unentscheidbar. Für konkrete Zahlen nimmt man deshalb `Float`. Die Formeln bleiben dieselben, nur der Typ ist ein anderer.

Lean code/Kapitel5.lean

```
/- ‘ℝ‘ ist in Lean nicht berechenbar. Fuer konkrete Zahlen benutzen wir
   dieselben Formeln ueber ‘Float‘. -/
def trapezF (f : Float → Float) (a b : Float) : Float := (b - a) * (f a + f b) / 2

def simpsonF (f : Float → Float) (a b : Float) : Float :=
  (b - a) / 6 * (f a + 4 * f ((a + b) / 2) + f b)

def trapezSummeF (f : Float → Float) (a b : Float) (n : Nat) : Float :=
  let h := (b - a) / n.toFloat
  (List.range n).foldl
    (fun acc i => acc + trapezF f (a + i.toFloat * h) (a + (i.toFloat + 1) * h))
    0.0

/- ∫₀¹ x² dx = 1/3 = 0.3333... -/
#eval trapezF   (fun x => x * x) 0 1        -- 0.5      (Fehler 1/6)
#eval simpsonF  (fun x => x * x) 0 1        -- 0.333333 (exakt)
#eval trapezSummeF (fun x => x * x) 0 1 10  -- Fehler ~ 1/1200
#eval trapezSummeF (fun x => x * x) 0 1 100 -- Fehler ~ 1/120000

/- Die Fehler der zusammengesetzten Trapezregel fallen wie n⁻²: -/
#eval trapezSummeF (fun x => x * x) 0 1 10  - 1.0/3.0
#eval trapezSummeF (fun x => x * x) 0 1 100 - 1.0/3.0
```

Die Werte, die `#eval` für $\int_0^1 x^2\,dx = \frac{1}{3}$ liefert:

Verfahren	Wert	Fehler
Trapez, $n = 1$	0.500000	$1.67 \cdot 10^{-1}$
Simpson, $n = 1$	0.333333	0 (exakt)
Trapez, $n = 10$	0.335000	$1.67 \cdot 10^{-3}$
Trapez, $n = 100$	0.333350	$1.67 \cdot 10^{-5}$

Von $n = 10$ auf $n = 100$ wird der Fehler hundertmal kleiner – genau die vorhergesagte Ordnung $O(n^{-2})$. Simpson ist hier exakt, weil x^2 ein Polynom vom Grad $2 \le 3$ ist: Satz 5.3, in Zahlen.

Zwei Dimensionen, und dann Monte-Carlo

Dasselbe für $\int_0^1\int_0^1 e^{xy}\,\mathrm{d}y\,\mathrm{d}x = 1.317902\ldots$

Lean code/Kapitel5.lean

```
/- Dieselbe Regel in zwei Dimensionen: die vier Ecken eines Rechtecks. -/
def trapez2F (f : Float → Float → Float) (a b c d : Float) : Float :=
  (b - a) * (d - c) / 4 * (f a c + f a d + f b c + f b d)

def trapezSumme2F (f : Float → Float → Float) (a b c d : Float) (n : Nat) :
    Float :=
  let hx := (b - a) / n.toFloat
  let hy := (d - c) / n.toFloat
  (List.range n).foldl (fun acc i =>
    (List.range n).foldl (fun acc2 j =>
      acc2 + trapez2F f (a + i.toFloat * hx) (a + (i.toFloat + 1) * hx)
                        (c + j.toFloat * hy) (c + (j.toFloat + 1) * hy)) acc) 0.0

/- ∫₀¹∫₀¹ exp(x·y) dy dx = 1.31790... -/
#eval trapezSumme2F (fun x y => Float.exp (x * y)) 0 1 0 1 10   -- 100 Auswertungen
#eval trapezSumme2F (fun x y => Float.exp (x * y)) 0 1 0 1 40   -- 1600
    Auswertungen

/- Ein Spielzeug-Zufallsgenerator (linear kongruent). Fuer echte Rechnungen
   nimmt man bessere; fuer die Groessenordnung reicht er. -/
def lcg (s : Nat) : Nat := (1103515245 * s + 12345) % 2147483648

def skala : Float := 2147483648.0

/-- Mittelwert ueber ‘n‘ Punkte, endrekursiv -- ohne Listen aufzubauen. -/
def monteCarlo (f : Float → Float → Float) (n : Nat) : Float :=
  let rec go : Nat → Nat → Nat → Float → Float
    | 0,     _,  _,  acc => acc
    | k + 1, sx, sy, acc =>
        let sx' := lcg sx
        let sy' := lcg sy
        go k sx' sy' (acc + f (sx'.toFloat / skala) (sy'.toFloat / skala))
  go n 42 4711 0.0 / n.toFloat

#eval monteCarlo (fun x y => Float.exp (x * y)) 100
#eval monteCarlo (fun x y => Float.exp (x * y)) 10000
```

Verfahren	**Auswertungen**	**Wert**	**Fehler**
Produkt-Trapez, $n = 10$	121	1.318738	$8.4 \cdot 10^{-4}$
Produkt-Trapez, $n = 40$	1681	1.317954	$5.2 \cdot 10^{-5}$
Monte-Carlo, $N = 100$	100	1.239437	$7.8 \cdot 10^{-2}$
Monte-Carlo, $N = 10000$	10000	1.312882	$5.0 \cdot 10^{-3}$

Die ersten beiden Zeilen zeigen wieder die Ordnung: Vervierfachen der Knoten je Richtung verkleinert den Fehler um den Faktor $16 = 4^2$. Die letzten beiden zeigen, warum man in zwei Dimensionen nicht würfeln sollte – 10 000 Zufallspunkte sind schlechter als 121 Gitterpunkte. In zehn Dimensionen steht dieselbe Tabelle andersherum.

Bemerkung 5.9 (Grenzen der Ordnung). Die Fehlerordnungen setzen Glattheit voraus. Für $f(x) = \sqrt{x}$ auf $[0, 1]$ ist f'' bei 0 unbeschränkt, und die zusammengesetzte Trapezregel schafft nur noch $O(n^{-3/2})$. Wer höhere Ordnung braucht, muss entweder glattere Integranden haben, andere Knoten wählen (Gauß-Quadratur) oder adaptiv verfeinern.

Zusammenfassung

- Das Intervallintegral ist orientiert und total. Integrierbarkeit steht in den Voraussetzungen.
- Der Exaktheitsgrad einer Quadraturformel ist vollständig formalisierbar: Mittelpunkt und Trapez haben Grad 1, Simpson Grad 3 (Satz 5.3).
- Zusammengesetzte Formeln bekommt man aus der Exaktheit auf den Teilintervallen plus Teleskopieren der Teilintegrale.
- Die Fehlerordnungen $O(n^{-2})$ und $O(n^{-4})$ bestätigt das numerische Experiment.
- In mehreren Dimensionen ist das Integral derselbe Begriff; Fubini (Satz 5.6) macht daraus geschachtelte Intervallintegrale, und die Produktregel wendet die eindimensionale Formel je Richtung an.
- Der Exaktheitsgrad überträgt sich, die Kosten nicht: Für den Fehler $O(N^{-p/d})$ braucht man $N = n^d$ Auswertungen – der Fluch der Dimension (Abschnitt 5.5).
- Monte-Carlo hat den Fehler $O(N^{-1/2})$, unabhängig von d. Das ist in einer Dimension schlecht und ab etwa $d = 4$ konkurrenzlos.

Kapitel 6

Gewöhnliche Differentialgleichungen

6.1 Wozu das gut ist

Bisher waren Gleichungen Gleichungen für Zahlen: $x^2 = 2$, gesucht ist x. Eine Differentialgleichung ist eine Gleichung für eine *Funktion*, und sie verknüpft die Funktion mit ihrer Ableitung.

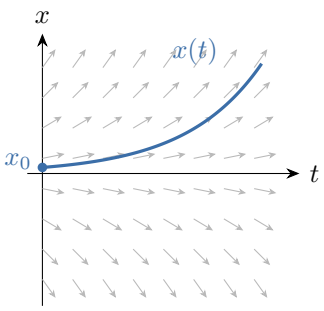
So sehen Naturgesetze aus. Man weiß selten, wo etwas in zehn Minuten sein wird. Man weiß, wie schnell es sich gerade ändert – und zwar in Abhängigkeit davon, wo es jetzt ist.

Gleichung	**beschreibt**	**Dimension**
$x' = a\,x$	Zinsen, Zerfall, Wachstum	1
$m\,y'' = -k\,y$	Feder	2
$\varphi'' = -\frac{g}{l}\sin\varphi$	Pendel	2
$u' = u(\alpha - \beta v),\ v' = v(\delta u - \gamma)$	Räuber und Beute	2
$\ddot{q}_i = \sum_j G m_j \frac{q_j - q_i}{\|q_j - q_i\|^3}$	N Himmelskörper	$6N$

Die rechte Spalte ist der Grund, warum dieses Kapitel nicht mit $x : \mathbb{R} \to \mathbb{R}$ auskommt.

Ein Feld von Pfeilen. $f(t, x)$ heftet an jeden Punkt einen Pfeil: die Richtung, in die es von dort aus weitergeht. Eine Lösung ist eine Kurve, die in jedem ihrer Punkte in Pfeilrichtung läuft.

Damit sind die beiden Fragen dieses Kapitels anschaulich. *Existenz:* Gibt es so eine Kurve überhaupt? *Eindeutigkeit:* Kann man unterwegs an einer Weggabelung stehen und sich aussuchen, wie es weitergeht?



Warum beliebige Dimension

Zwei Beobachtungen machen den allgemeinen Fall billig.

Erstens ist ein System von n Gleichungen eine einzige Gleichung in $\mathbb{R}^n$. Man schreibt die n gesuchten Funktionen in einen Vektor und die n rechten Seiten ebenso.

Zweitens lässt sich jede Gleichung höherer Ordnung auf erste Ordnung zurückführen. Aus $y'' = g(t, y, y')$ wird mit $x = (y, y')$

$$x' = \begin{pmatrix} x_2 \\ g(t, x_1, x_2) \end{pmatrix}.$$

Die Ableitung wird also zu einer zusätzlichen Koordinate. Das Pendel, eine Gleichung zweiter Ordnung für einen Winkel, ist damit ein System erster Ordnung in der Ebene aus Winkel und Winkelgeschwindigkeit.

Es genügt deshalb, $x' = f(t, x)$ mit x in $\mathbb{R}^n$ zu betrachten. Und wenn man einmal so weit ist, kostet es nichts mehr, gleich einen beliebigen Banachraum E zu nehmen – alle Beweise dieses Kapitels brauchen von $\mathbb{R}^n$ nur Norm und Vollständigkeit. Mathlib macht es genau so.

6.2 Das Anfangswertproblem

Definition 6.1 (Anfangswertproblem)**.** Sei E ein Banachraum, $I \subseteq \mathbb{R}$ ein Intervall, $f : I \times E \to E$, $t_0 \in I$ und $x_0 \in E$. Eine *Lösung* des Anfangswertproblems

$$x'(t) = f(t, x(t)), \qquad x(t_0) = x_0$$

ist eine differenzierbare Funktion $x : J \to E$ auf einem Teilintervall $J \subseteq I$ mit $t_0 \in J$, die beide Bedingungen erfüllt.

Lean code/Kapitel6.lean

```
/- Ein Anfangswertproblem in einem Banachraum ‘E‘:
       x'(t) = f(t, x(t)),   x(t₀) = x₀.
   Die Dimension kommt in der Formulierung nicht vor. ‘E‘ darf ℝ sein,
   ℝⁿ, ein Matrizenraum oder ein Funktionenraum. -/
def LoestAWP {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : ℝ → E → E) (t₀ : ℝ) (x₀ : E) (α : ℝ → E) : Prop :=
  α t₀ = x₀ ∧ ∀ t, HasDerivAt α (f t (α t)) t

/- Dieselbe Bedingung als Integralgleichung -- die Picard-Abbildung
   bildet α auf  t ↦ x₀ + (Integral von t₀ bis t ueber f(s, α s))  ab.
   Eine Loesung ist genau ein Fixpunkt davon. -/
#check @ODE.picard
```

Satz 6.2 (Integralform)**.** *x ist genau dann Lösung, wenn x stetig ist und*

$$x(t) \;=\; x_0 + \int_{t_0}^{t} f(s, x(s))\,\mathrm{d}s \qquad \text{für alle } t \in J.$$

Der Beweis ist der Hauptsatz der Differential- und Integralrechnung, einmal in jede Richtung. Wichtig ist, was die Umformulierung bringt: Rechts steht kein Ableitungsbegriff mehr, sondern eine Abbildung, die aus einer Funktion eine Funktion macht. Nennt man sie T, dann heißt die Gleichung

$$Tx = x.$$

Aus der Differentialgleichung ist ein *Fixpunktproblem* geworden. Damit ist Abschnitt 4.3 zuständig.

6.3 Stetigkeit reicht nicht

Man könnte hoffen, dass eine stetige rechte Seite genügt. Tut sie nicht.

Erstes Problem: mehrere Lösungen

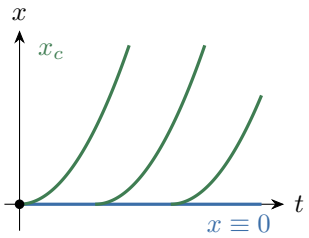
Betrachte

$$x' = 2\sqrt{|x|}, \qquad x(0) = 0.$$

Die rechte Seite ist stetig auf ganz $\mathbb{R}$. Eine Lösung ist $x \equiv 0$. Eine zweite ist $x(t) = t^2$ für $t \geq 0$, denn dann ist $x' = 2t$ und $2\sqrt{|t^2|} = 2t$. Und es gibt noch mehr: Für jedes $c \geq 0$ ist

$$x_c(t) = \begin{cases} 0 & t \leq c \\ (t-c)^2 & t > c \end{cases}$$

eine Lösung. Es sind also unendlich viele.



Beliebig lange liegenbleiben. Bei $x = 0$ ist die Geschwindigkeit 0. Das Teilchen darf dort also stehen bleiben – und weil die Geschwindigkeit in der Nähe von 0 nur wie $\sqrt{|x|}$ wächst, kostet das Losfahren keine Zeit. Es kann sich zu jedem Zeitpunkt c entscheiden, doch noch loszulaufen. Genau das sind die Lösungen x_c.

Zweites Problem: Weglaufen in endlicher Zeit

Betrachte

$$x' = x^2, \qquad x(0) = 1.$$

Die rechte Seite ist ein Polynom, also beliebig glatt. Die Lösung ist $x(t) = \frac{1}{1-t}$, und sie existiert nur für $t < 1$. Bei $t = 1$ ist sie weg.

Lean code/Kapitel6.lean

```
/- x' = 2·√|x| mit x(0) = 0 hat mehr als eine Loesung.
   Erste Loesung: die Nullfunktion. -/
theorem loesung_null (t : ℝ) :
    HasDerivAt (fun _ : ℝ => (0:ℝ)) (2 * Real.sqrt |(0:ℝ)|) t := by
  simpa using hasDerivAt_const t (0:ℝ)

/- Zweite Loesung auf [0, ∞): t ↦ t². Sie startet ebenfalls bei 0. -/
theorem loesung_quadrat {t : ℝ} (ht : 0 ≤ t) :
    HasDerivAt (fun s : ℝ => s ^ 2) (2 * Real.sqrt |t ^ 2|) t := by
  have h : Real.sqrt |t ^ 2| = t := by
    rw [abs_of_nonneg (sq_nonneg t), Real.sqrt_sq ht]
  rw [h]
  simpa using hasDerivAt_pow 2 t

/- Zweites Problem: die Loesung kann in endlicher Zeit weglaufen.
```

```
    x' = x^2 mit x(0) = 1 hat die Loesung 1/(1-t). Sie existiert nur auf
    (-∞, 1), obwohl die rechte Seite auf ganz ℝ beliebig glatt ist. -/
theorem blowup {t : ℝ} (ht : t < 1) :
    HasDerivAt (fun s : ℝ => (1 - s)⁻¹) (((1 - t)⁻¹) ^ 2) t := by
  have hc : HasDerivAt (fun s : ℝ => 1 - s) (-1) t := by
    simpa using (hasDerivAt_const t (1:ℝ)).sub (hasDerivAt_id t)
  have hne : (1 - t) ≠ 0 := by linarith
  have h := hc.inv hne
  convert h using 1
  field_simp
```

Daraus folgen die beiden Einschränkungen, mit denen der ganze Rest des Kapitels lebt:

- Für Eindeutigkeit braucht man mehr als Stetigkeit.
- Existenz gibt es nur *lokal*, also auf einem Intervall um t_0, dessen Länge man nicht frei wählen kann.

Bemerkung 6.3 (Peano und Picard–Lindelöf). Stetigkeit allein liefert immerhin Existenz: Das ist der Satz von Peano. Er gilt aber nur in endlicher Dimension, denn sein Beweis benutzt, dass beschränkte abgeschlossene Mengen kompakt sind. In unendlichdimensionalen Banachräumen ist er falsch – es gibt stetige rechte Seiten ganz ohne Lösung.

Der Satz von Picard–Lindelöf verlangt stattdessen etwas mehr von f und bekommt dafür beides, Existenz *und* Eindeutigkeit, und das in jedem Banachraum. Deshalb steht in Mathlib er und nicht Peano.

6.4 Die Lipschitz-Bedingung

Definition 6.4 (Lipschitz-stetig). $f : X \to Y$ heißt *Lipschitz-stetig mit Konstante* K, wenn

$$d(f(x), f(y)) \;\le\; K\, d(x, y) \qquad \text{für alle } x, y \in X.$$

Gilt das nur auf einer Teilmenge $U \subseteq X$, heißt f dort *lokal* Lipschitz-stetig.

Eine Schranke für die Steilheit. Stetigkeit sagt: Kleine Änderungen am Eingang machen kleine Änderungen am Ausgang. Lipschitz sagt zusätzlich, *wie* klein – höchstens K-mal so groß. Die Funktion darf also nirgends beliebig steil werden.

Genau daran scheitert $2\sqrt{|x|}$ bei 0: Der Anstieg von 0 auf x beträgt $2\sqrt{x}/x = 2/\sqrt{x}$, und das wächst über jede Schranke.

Lean code/Kapitel6.lean

```
#check @LipschitzWith
#check @LipschitzOnWith

/- Eine stetige lineare Abbildung ist global Lipschitz, mit ihrer Norm als
   Konstante -- in jeder Dimension und in jedem Banachraum. -/
example {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] (A : E →L[ℝ] E) :
    LipschitzWith ‖A‖₊ A := A.lipschitz

/- Stetig differenzierbar ⇒ lokal Lipschitz. Das ist der uebliche Weg, die
   Voraussetzung nachzuweisen. -/
#check @HasStrictFDerivAt.exists_lipschitzOnWith
```

Nachweisen muss man die Bedingung selten von Hand. Ist f stetig differenzierbar, so ist es auf jeder kompakten Menge Lipschitz-stetig, mit dem Maximum der Ableitungsnorm als Konstante. Alle Beispiele aus der Tabelle am Kapitelanfang sind glatt.

6.5 Picard–Lindelöf

Satz 6.5 (Picard–Lindelöf)**.** *Sei E ein Banachraum, $t_0 \in \mathbb{R}$, $x_0 \in E$, und seien $a, L, K > 0$. Die Funktion f erfülle auf $[t_0 - \alpha, t_0 + \alpha] \times \overline{B}(x_0, a)$:*

(i) $f(t, \cdot)$ *ist Lipschitz-stetig mit Konstante K,*

(ii) $f(\cdot, x)$ *ist stetig,*

(iii) $\|f(t,x)\| \le L$,

(iv) $L \cdot \alpha \le a$.

Dann gibt es auf $[t_0 - \alpha, t_0 + \alpha]$ genau eine Lösung des Anfangswertproblems $x' = f(t,x)$, $x(t_0) = x_0$.

Bedingung (iv) sieht technisch aus, sagt aber etwas Einfaches: In der Zeit α kann sich die Lösung bei Geschwindigkeit höchstens L um höchstens $L\alpha$ vom Startpunkt entfernen. Ist das kleiner als a, bleibt sie in der Kugel, auf der die Voraussetzungen überhaupt gelten. Man kauft das Intervall also mit seiner Länge.

Der Beweis

Er besteht aus drei Schritten, und alle drei stehen schon irgendwo in diesem Skript.

1. **Umschreiben.** Nach Satz 6.2 ist eine Lösung dasselbe wie ein Fixpunkt der *Picard-Abbildung*
$$(Tx)(t) \;=\; x_0 + \int_{t_0}^{t} f\big(s, x(s)\big)\,\mathrm{d}s.$$

2. **Den Raum wählen.** T bildet $X = \{x \in C([t_0 - \alpha, t_0 + \alpha], E) : \|x - x_0\|_\infty \le a\}$ in sich ab – dafür ist (iv) da. Mit der Supremumsnorm ist X ein vollständiger metrischer Raum, weil E vollständig ist (Abschnitt 4.3).

3. **Zusammenziehen.** Für zwei Funktionen $x, y \in X$ ist
$$\|(Tx)(t) - (Ty)(t)\| \;\le\; \int_{t_0}^{t} \big\|f(s,x(s)) - f(s,y(s))\big\|\,\mathrm{d}s \;\le\; K\,\alpha\,\|x - y\|_\infty,$$
also ist T eine Kontraktion, sobald $K\alpha < 1$. Der Banachsche Fixpunktsatz 4.14 liefert genau einen Fixpunkt.

Die Picard-Iteration. Der Fixpunktsatz sagt mehr als „es gibt eine Lösung“. Er sagt, wie man sie bekommt: Man fängt mit irgendeiner Funktion an, etwa der konstanten $x^{(0)} \equiv x_0$, und setzt $x^{(k+1)} = Tx^{(k)}$. Jeder Schritt verkleinert den Abstand zur Lösung um den Faktor $K\alpha$.

Für $x' = x$, $x(0) = 1$ ergibt das
$$x^{(0)} = 1, \quad x^{(1)} = 1 + t, \quad x^{(2)} = 1 + t + \tfrac{t^2}{2}, \quad \dots$$
also die Partialsummen von e^t. Die Iteration rechnet die Exponentialreihe aus, ohne dass jemand sie hingeschrieben hätte.

Lean code/Kapitel6.lean

```
/- Die Voraussetzungen des Satzes, als Struktur gebuendelt: ‘f‘ ist
   Lipschitz in x, stetig in t, beschraenkt, und das Zeitintervall ist
   klein genug, dass die Loesung die Kugel nicht verlaesst. -/
#check @IsPicardLindelof

/- Existenz und Anfangswert -- die Differentialform des Satzes. -/
#check @IsPicardLindelof.exists_eq_forall_mem_Icc_hasDerivWithinAt₀

/- Dieselbe Loesung als Fixpunkt der Picard-Abbildung. -/
#check @IsPicardLindelof.exists_eq_forall_mem_Icc_eq_picard

/- Der Beweis laeuft ueber den Banachschen Fixpunktsatz. -/
#check @ContractingWith
#check @ContractingWith.exists_fixedPoint

/- Ist ‘f‘ stetig differenzierbar, bekommt man lokale Loesungen geschenkt,
   zu allen Anfangswerten nahe x₀ zugleich (ein lokaler Fluss). -/
#check @ContDiffAt.exists_eventually_eq_hasDerivAt
```

Mathlib. Mathlib bündelt die vier Voraussetzungen in `IsPicardLindelof` und führt den Beweis nach demselben Plan, über `ODE.picard` und `ContractingWith`, mit einem Kniff: Statt $K\alpha < 1$ zu verlangen, zieht eine Potenz T^m zusammen, mit Faktor $(K\alpha)^m/m!$. Der Funktionenraum `ODE.FunSpace` besteht aus Lipschitz-stetigen Kurven. Die Existenzaussage heißt `IsPicardLindelof.exists_eq_forall_mem_Icc_hasDerivWithinAt₀`.

6.6 Eindeutigkeit und Grönwall

Die Eindeutigkeit aus Satz 6.5 gilt zunächst nur auf dem kleinen Intervall, das der Fixpunktsatz liefert. Dass sie überall gilt, wo beide Lösungen existieren, folgt aus einer Abschätzung, die für sich genommen nützlicher ist als der Satz.

Lemma 6.6 (Grönwall)**.** *Ist $u : [a,b] \to \mathbb{R}$ stetig, $u(a) \le \delta$ und $u'(t) \le K\,u(t) + \varepsilon$, dann gilt für alle $t \in [a,b]$*

$$u(t) \;\le\; \delta\, e^{K(t-a)} + \frac{\varepsilon}{K}\bigl(e^{K(t-a)} - 1\bigr).$$

Wer langsamer wächst, bleibt darunter. Rechts steht die Lösung der Gleichung, die man bekäme, wenn überall Gleichheit stünde. Das Lemma sagt: Die Ungleichung überträgt sich von den Ableitungen auf die Funktionen. Mehr ist es nicht – aber fast jede Abschätzung in der Theorie der Differentialgleichungen läuft darüber.

Für zwei Lösungen x, y desselben Systems setzt man $u(t) = \|x(t) - y(t)\|$. Dann ist

$$u'(t) \;\le\; \|f(t,x(t)) - f(t,y(t))\| \;\le\; K\,u(t),$$

also nach Grönwall mit $\varepsilon = 0$

$$\|x(t) - y(t)\| \;\le\; \|x(t_0) - y(t_0)\|\; e^{K|t-t_0|}.$$

Daraus liest man beides ab:

Satz 6.7 (Eindeutigkeit und stetige Abhängigkeit)**.** *Ist f Lipschitz-stetig in x mit Konstante K, dann gilt:*

(a) Zwei Lösungen mit $x(t_0) = y(t_0)$ sind gleich.

(b) Zwei Lösungen mit verschiedenen Anfangswerten entfernen sich höchstens exponentiell voneinander.

Lean code/Kapitel6.lean

```
/- Zwei Loesungen mit demselben Anfangswert stimmen ueberein. -/
#check @ODE_solution_unique
#check @ODE_solution_unique_univ

/- Dahinter steht das Lemma von Gronwall: Wer langsamer waechst als eine
   Exponentialfunktion, bleibt unter ihr. -/
#check @gronwallBound
#check @norm_le_gronwallBound_of_norm_deriv_right_le

/- Dieselbe Abschaetzung liefert die stetige Abhaengigkeit von den
   Anfangsdaten: zwei Loesungen laufen hoechstens exponentiell auseinander. -/
#check @dist_le_of_trajectories_ODE
```

Bemerkung 6.8 (Deterministisch und trotzdem unvorhersagbar)*.* Teil (b) ist eine Schranke, keine Versprechung. e^{Kt} darf tatsächlich erreicht werden, und dann verzehnfacht sich ein Anfangsfehler alle $(\ln 10)/K$ Zeiteinheiten.

Beim Wetter liegt diese Zeitspanne bei etwa anderthalb Tagen. Eine Messungenauigkeit von 10^{-6} ist nach neun Tagen auf 1 angewachsen – die Gleichung ist eindeutig lösbar, und die Vorhersage trotzdem wertlos. Eindeutigkeit ist eine Aussage über die Mathematik, nicht über das, was man ausrechnen kann.

6.7 Lineare Systeme

Ein Fall geht besser als der allgemeine, und zwar so viel besser, dass man ihn vollständig lösen kann. *Linear* heißt: Die rechte Seite ist in x linear,

$$x' = A(t)\,x + b(t),$$

mit $A(t)$ einer stetigen linearen Abbildung $E \to E$ und b einer stetigen Funktion. Ist $b \equiv 0$, heißt die Gleichung *homogen*, sonst *inhomogen*.

Homogen: der Lösungsraum

Satz 6.9 (Lösungen linearer Gleichungen laufen nicht weg)**.** *Sind A und b auf einem Intervall I stetig, so existiert zu jedem $t_0 \in I$ und $x_0 \in E$ genau eine Lösung, und sie ist auf* ganz *I definiert.*

Das ist der Unterschied zu $x' = x^2$ aus Abschnitt 6.3. Dort war die rechte Seite nur lokal Lipschitz-stetig, hier ist sie es auf jedem kompakten Teilintervall gleichmäßig – mit $K = \max_t \|A(t)\|$. Und Grönwall (Lemma 6.6) liefert sofort die Schranke

$$\|x(t)\| \;\le\; \Big(\|x_0\| + \textstyle\int_{t_0}^{t} \|b\|\Big)\, e^{K|t-t_0|},$$

also kann die Lösung in endlicher Zeit nicht unendlich werden. Lokale Existenz plus Schranke ergibt globale Existenz – dasselbe Argument wie bei Lotka–Volterra.

Satz 6.10 (Der homogene Lösungsraum)**.** *Die Lösungen von $x' = A(t)x$ auf I bilden einen Vektorraum. Für $E = \mathbb{R}^n$ hat er die Dimension n: Die Abbildung $x \mapsto x(t_0)$ ist ein Isomorphismus auf $\mathbb{R}^n$.*

Dass Summen und Vielfache von Lösungen wieder Lösungen sind, ist eine Zeile – unten steht sie in Lean. Dass die Dimension genau n ist, steckt in Satz 6.9: Zu jedem Anfangswert gehört genau eine Lösung, also ist die Auswertung bei t_0 injektiv *und* surjektiv.

Eine Basis $x_1, \dots, x_n$ des Lösungsraums heißt *Fundamentalsystem*. Schreibt man die x_i als Spalten einer Matrix, bekommt man die *Fundamentalmatrix* $\Phi(t)$, und jede Lösung ist $x(t) = \Phi(t)\,c$ mit einem festen Vektor c.

Satz 6.11 (Formel von Liouville)**.** *Für die* Wronski-Determinante $W(t) = \det \Phi(t)$ *gilt*

$$W(t) \;=\; W(t_0)\,\exp \int_{t_0}^{t} \operatorname{tr} A(s)\,\mathrm{d}s.$$

Entweder immer oder nie. Die rechte Seite ist nie null. W ist also entweder auf ganz I von null verschieden oder überall null – dazwischen gibt es nichts. Um zu prüfen, ob n Lösungen ein Fundamentalsystem bilden, genügt deshalb *ein* Zeitpunkt.

Die Spur taucht auf, weil sie die Ableitung der Determinante in der Eins ist: Ein kleiner Schritt $\Phi \mapsto (\mathrm{id} + hA)\Phi$ ändert das Volumen um den Faktor $\det(\mathrm{id} + hA) \approx 1 + h \operatorname{tr} A$.

Inhomogen: allgemeine und spezielle Lösung

Satz 6.12 (Struktur der Lösungsmenge)**.** *Sei x_p irgendeine Lösung von $x' = A(t)x + b(t)$. Dann ist die Menge* aller *Lösungen*

$$\{\, x_p + x_h \;:\; x_h \text{ löst } x' = A(t)x \,\}.$$

Beweis. Ist x_h homogen, so löst $x_p + x_h$ die inhomogene Gleichung. Sind umgekehrt x_1, x_2 beides Lösungen der inhomogenen Gleichung, so löst $x_1 - x_2$ die homogene. Beides ist eine Rechnung mit der Linearität von $A(t)$. □

Man nennt x_p eine *spezielle* oder *partikuläre* Lösung und x_h die *allgemeine Lösung der homogenen Gleichung*. Zusammen:

$$\underbrace{x}_{\text{allgemeine Lösung}} \;=\; \underbrace{x_p}_{\text{eine spezielle}} \;+\; \underbrace{\Phi(t)\,c}_{\text{allgemeine homogene}}, \qquad c \in \mathbb{R}^n \text{ frei.}$$

Dasselbe wie bei $Ax = b$. Die Struktur ist aus der linearen Algebra bekannt: Die Lösungsmenge von $Ax = b$ ist eine spezielle Lösung plus der Kern von A. Hier heißt der Operator $Lx = x' - A(t)x$, und er ist ebenso linear. Der Kern ist der homogene Lösungsraum, und die Lösungsmenge von $Lx = b$ ist ein affiner Unterraum.

Was in der linearen Algebra die Dimension des Kerns ist, ist hier die Dimension n aus Satz 6.10: Es bleiben genau n Konstanten frei, und die legt der Anfangswert fest.

Es fehlt noch die spezielle Lösung. Man bekommt sie umsonst, sobald man die homogene Gleichung gelöst hat.

Satz 6.13 (Variation der Konstanten)**.** *Ist Φ eine Fundamentalmatrix der homogenen Gleichung, so löst*

$$x(t) \;=\; \Phi(t)\,\Phi(t_0)^{-1}x_0 \;+\; \int_{t_0}^{t} \Phi(t)\,\Phi(s)^{-1}\,b(s)\,\mathrm{d}s$$

das Anfangswertproblem $x' = A(t)x + b(t)$, $x(t_0) = x_0$. Für konstantes A steht dort

$$x(t) \;=\; e^{(t-t_0)A}x_0 \;+\; \int_{t_0}^{t} e^{(t-s)A}\,b(s)\,\mathrm{d}s.$$

Woher der Name kommt, und was die Formel sagt. Die homogene Lösung ist $\Phi(t)c$ mit konstantem c. Der Ansatz lässt c mitlaufen: $x(t) = \Phi(t)\,c(t)$. Einsetzen gibt $\Phi' c + \Phi c' = A\Phi c + b$, und weil $\Phi' = A\Phi$ ist, bleibt $\Phi c' = b$ übrig – also $c' = \Phi^{-1}b$, und das kann man integrieren. Die Konstante wird variiert; daher der Name.

Lesen kann man die Formel so: Zur Zeit s stößt die Störung $b(s)\,\mathrm{d}s$ einen neuen Anfangswert an, der danach ungestört mit $e^{(t-s)A}$ weiterläuft. Das Integral addiert alle diese Beiträge auf. In der Physik heißt dasselbe *Duhamel-Prinzip*.

Lean code/Kapitel6.lean

```
section Struktur
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  {A : ℝ → E →L[ℝ] E} {f x y xp xh : ℝ → E} {t : ℝ}

/-- **Superposition.** Linearkombinationen von Loesungen der homogenen
Gleichung sind wieder Loesungen -- der Loesungsraum ist ein Vektorraum. -/
theorem superposition (a b : ℝ)
    (hx : HasDerivAt x (A t (x t)) t) (hy : HasDerivAt y (A t (y t)) t) :
    HasDerivAt (fun s => a • x s + b • y s) (A t (a • x t + b • y t)) t := by
  simpa [map_add, map_smul] using (hx.const_smul a).add (hy.const_smul b)

/-- **Speziell plus homogen.** Addiert man zu einer Loesung der inhomogenen
Gleichung eine Loesung der homogenen, bekommt man wieder eine Loesung der
inhomogenen. -/
theorem speziell_plus_homogen
    (hp : HasDerivAt xp (A t (xp t) + f t) t)
    (hh : HasDerivAt xh (A t (xh t)) t) :
    HasDerivAt (fun s => xp s + xh s) (A t (xp t + xh t) + f t) t := by
  have h := hp.add hh
  simp only [map_add]
  convert h using 1
  abel

/-- **Und umgekehrt.** Die Differenz zweier Loesungen der inhomogenen
Gleichung loest die homogene. Zusammen mit dem vorigen Satz heisst das: Die
Loesungsmenge der inhomogenen Gleichung ist `x_p` plus der Loesungsraum der
homogenen -- ein affiner Unterraum. -/
theorem differenz_ist_homogen
    (h₁ : HasDerivAt x (A t (x t) + f t) t)
    (h₂ : HasDerivAt y (A t (y t) + f t) t) :
    HasDerivAt (fun s => x s - y s) (A t (x t - y t)) t := by
  have h := h₁.sub h₂
  simp only [map_sub]
  convert h using 1
  abel

end Struktur
```

Die drei Sätze oben sind Satz 6.10 und Satz 6.12, Zeile für Zeile. Die Variation der Konstanten im skalaren Fall ist ebenfalls nachgerechnet:

Lean code/Kapitel6.lean

```
/-- **Variation der Konstanten**, skalarer Fall. Die Formel

      x(t) = exp(a·t) · ( x₀ + ∫₀ᵗ exp(-a·u) · b(u) du )

loest x' = a·x + b(t) mit x(0) = x₀. Im mehrdimensionalen Fall steht statt
exp(a·t) die Fundamentalmatrix; die Rechnung ist dieselbe. -/
theorem variation_der_konstanten (a : ℝ) (b : ℝ → ℝ) (hb : Continuous b) (x₀ t :
    ℝ) :
    HasDerivAt
      (fun s : ℝ => Real.exp (a * s) * (x₀ + ∫ u in (0:ℝ)..s, Real.exp (-(a * u))
    * b u))
      (a * (Real.exp (a * t) * (x₀ + ∫ u in (0:ℝ)..t, Real.exp (-(a * u)) * b u))
    + b t) t := by
  have hg : Continuous (fun u : ℝ => Real.exp (-(a * u)) * b u) := by fun_prop
  have hF : HasDerivAt (fun s : ℝ => ∫ u in (0:ℝ)..s, Real.exp (-(a * u)) * b u)
      (Real.exp (-(a * t)) * b t) t :=
    intervalIntegral.integral_hasDerivAt_right (hg.intervalIntegrable _ _)
      (hg.stronglyMeasurableAtFilter _ _) hg.continuousAt
  have hsum : HasDerivAt (fun s : ℝ => x₀ + ∫ u in (0:ℝ)..s, Real.exp (-(a * u))
    * b u)
      (Real.exp (-(a * t)) * b t) t := hF.const_add x₀
  have hE : HasDerivAt (fun s : ℝ => Real.exp (a * s)) (a * Real.exp (a * t)) t
    := by
    have h1 : HasDerivAt (fun s : ℝ => a * s) a t := by
      simpa using (hasDerivAt_id t).const_mul a
    simpa [mul_comm] using (Real.hasDerivAt_exp (a * t)).comp t h1
  have h := hE.mul hsum
  have hx : Real.exp (a * t) * Real.exp (-(a * t)) = 1 := by
    rw [← Real.exp_add]; simp
  convert h using 1
  linear_combination (-(b t)) * hx
```

Konstante Koeffizienten

Ist A konstant, so ist $\Phi(t) = e^{tA}$ eine Fundamentalmatrix, denn

$$e^{tA} = \sum_{k=0}^{\infty} \frac{(tA)^k}{k!}$$

konvergiert für jedes A – es ist $\|A^k\| \le \|A\|^k$, und die Exponentialreihe konvergiert überall.

Lean code/Kapitel6.lean

```
/- Skalar: x' = a·x mit x(0) = x₀ hat die Loesung x₀·exp(a·t). -/
theorem linear_skalar (a x₀ t : ℝ) :
    HasDerivAt (fun s : ℝ => x₀ * Real.exp (a * s))
      (a * (x₀ * Real.exp (a * t))) t := by
  have h1 : HasDerivAt (fun s : ℝ => a * s) a t := by
    simpa using (hasDerivAt_id t).const_mul a
  have h2 := (Real.hasDerivAt_exp (a * t)).comp t h1
  have h3 := h2.const_mul x₀
  convert h3 using 1
  ring

/- In beliebiger Dimension steht statt der Zahl ‘a‘ eine stetige lineare
```

```
   Abbildung ‘A‘, und statt exp(a·t) die Exponentialreihe exp(t·A). -/
open NormedSpace in
example {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [CompleteSpace E]
    (A : E →L[ℝ] E) (t : ℝ) :
    HasDerivAt (fun u : ℝ => exp ℝ (u • A)) (exp ℝ (t • A) * A) t :=
  hasDerivAt_exp_smul_const A t
```

Warum ausgerechnet die Exponentialfunktion. In einer Dimension ist $x' = ax$ die Gleichung, deren Lösung ihre eigene Ableitung bis auf den Faktor a ist – das ist die Definition von e^{at}. In n Dimensionen ändert sich daran nichts außer der Schreibweise. Was sich ändert, ist die Rechnung: $e^A e^B = e^{A+B}$ gilt nur, wenn A und B vertauschen.

Ausrechnen tut man e^{tA} nicht über die Reihe, sondern über die Eigenwerte. Je nach Fall:

Fall	Beitrag zum Fundamentalsystem
λ reell, einfach	$e^{\lambda t}v$ mit $Av = \lambda v$
$\lambda = \alpha \pm i\beta$ komplex	$e^{\alpha t}\cos(\beta t)\,u - e^{\alpha t}\sin(\beta t)\,w$ und $e^{\alpha t}\sin(\beta t)\,u + e^{\alpha t}\cos(\beta t)\,w$, wobei $v = u + iw$
λ k-fach, zu wenig Eigenvektoren	zusätzlich $t\,e^{\lambda t}, \dots, t^{k-1}e^{\lambda t}$ mal Hauptvektoren

Der dritte Fall ist die Jordan-Normalform: $A = S + N$ mit S halbeinfach, N nilpotent und $SN = NS$. Dann ist $e^{tA} = e^{tS}e^{tN}$, und e^{tN} bricht nach endlich vielen Gliedern ab – daher die Polynome in t.

Satz 6.14 (Stabilität)**.** *Haben alle Eigenwerte von A negativen Realteil, so geht jede Lösung von $x' = Ax$ gegen null, und zwar exponentiell. Hat ein Eigenwert positiven Realteil, so gibt es Lösungen, die über alle Grenzen wachsen.*

Der Grenzfall – alle Realteile ≤ 0 mit Gleichheit – ist der, in dem die Linearisierung nichts entscheidet. Genau ihn hatte Abschnitt 6.11 bei Lotka–Volterra.

Lean code/Kapitel6.lean

```
/- Fuer konstante Koeffizienten rechnet man exp(t·A) ueber die Eigenwerte.
   Was Mathlib dafuer hat: -/
#check @Matrix.charpoly              -- charakteristisches Polynom
#check @Module.End.HasEigenvalue     -- Eigenwert
#check @NormedSpace.exp              -- die Exponentialreihe
#check @NormedSpace.exp_add_of_commute   -- exp(X+Y) = exp X · exp Y, falls XY = YX

/- Was Mathlib *nicht* hat: eine Jordan-Normalform. Es gibt die
   Jordan-Chevalley-Zerlegung in halbeinfach plus nilpotent, aber keine
   fertige Loesungstheorie linearer Systeme. -/
#check @Module.End.exists_isNilpotent_isSemisimple
```

Mathlib. Mathlib hat die Exponentialreihe (`NormedSpace.exp`), das charakteristische Polynom und die Jordan–Chevalley-Zerlegung in halbeinfach plus nilpotent. Was fehlt, ist die Jordan-Normalform selbst und die gesamte Lösungstheorie dieses Abschnitts: Fundamentalmatrix, Wronski-Determinante, Variation der Konstanten für Systeme. Die Sätze hier sind also Papier – bis auf das, was in den beiden Schnipseln oben steht.

Gleichungen höherer Ordnung

Eine skalare Gleichung

$$y^{(n)} + a_{n-1}y^{(n-1)} + \cdots + a_1 y' + a_0 y \;=\; g(t)$$

wird mit $x = (y, y', \ldots, y^{(n-1)})$ zu einem System erster Ordnung, und die Matrix dazu ist die *Begleitmatrix*. Ihr charakteristisches Polynom ist

$$\chi(\lambda) \;=\; \lambda^n + a_{n-1}\lambda^{n-1} + \cdots + a_0,$$

also genau das, was man bekommt, wenn man $y = e^{\lambda t}$ einsetzt. Die Tabelle oben übersetzt sich damit direkt: Eine k-fache Nullstelle λ liefert $e^{\lambda t}, te^{\lambda t}, \ldots, t^{k-1}e^{\lambda t}$.

Für die spezielle Lösung braucht man die Variation der Konstanten oft nicht. Hat g selbst die Form „Polynom mal Exponentialfunktion“, findet man sie mit einem Ansatz derselben Form:

rechte Seite $g(t)$	**Ansatz für** x_p
$p(t)$, Grad m	$q(t)$, Grad m
$p(t)\,e^{\mu t}$	$q(t)\,e^{\mu t}$, Grad m
$p(t)\,e^{\alpha t}\cos(\beta t)$ oder sin	$e^{\alpha t}\big(q_1(t)\cos\beta t + q_2(t)\sin\beta t\big)$
Resonanz: Ist μ (bzw. $\alpha + i\beta$) eine k-fache Nullstelle von χ, wird der Ansatz mit t^k multipliziert.	

Was Resonanz bedeutet. Ohne Resonanz regt man das System mit einer Frequenz an, die es selbst nicht hat – es schwingt mit, beschränkt. Trifft die Anregung dagegen genau eine Eigenfrequenz, dann ist $e^{\mu t}$ schon homogene Lösung, und der Ansatz $q(t)e^{\mu t}$ liefert nur noch 0. Erst der Faktor t^k hilft, und genau der sorgt dafür, dass die Amplitude linear wächst.

Das ist dieselbe Rechnung, die eine Brücke zum Einsturz bringt, wenn man im Gleichschritt darüber geht.

6.8 Das kleine Einmaleins

Picard–Lindelöf sagt, dass es eine Lösung gibt, und Grönwall, dass es nur eine ist. Wie sie *aussieht*, sagt keiner von beiden. Geschlossen lösbar ist nur eine kleine Minderheit aller Gleichungen – aber diese Minderheit kommt dauernd vor, und fast alles darin lässt sich auf *zwei* Fälle zurückführen.

Die beiden Grundfälle

Trennbare Variablen. Steht rechts ein Produkt aus einer Funktion von x und einer von y,

$$y' = f(x)\,g(y),$$

so sortiert man und integriert beide Seiten:

$$\int \frac{\mathrm{d}y}{g(y)} \;=\; \int f(x)\,\mathrm{d}x.$$

Bemerkung 6.15 (Wo das schiefgeht). Geteilt wird durch $g(y)$, also gilt die Rechnung nur dort, wo $g(y) \neq 0$. Die Nullstellen von g sind selbst Lösungen – konstante – und man verliert sie beim Trennen.

Genau dort sitzt auch das Gegenbeispiel aus Abschnitt 6.3: Bei $y' = 2\sqrt{|y|}$ ist $g(y) = 2\sqrt{|y|}$, und bei $y = 0$ ist $\int \mathrm{d}y/g(y)$ *konvergent*. Deshalb kann eine Lösung die konstante in endlicher Zeit verlassen, und die Eindeutigkeit ist weg. Divergiert das Integral, kann sie es nicht – das ist die Probe, die man machen sollte.

Linear erster Ordnung.

$$y' + a(x)\,y = b(x).$$

Mit dem *integrierenden Faktor* $\mu(x) = e^{A(x)}$, $A = \int a$, wird die linke Seite zu einer Ableitung: $(\mu y)' = \mu\,(y' + ay) = \mu b$. Integrieren gibt

$$y(x) \;=\; e^{-A(x)}\Big(y_0 + \int_{x_0}^{x} e^{A(s)}\,b(s)\,\mathrm{d}s\Big).$$

Das ist die Variation der Konstanten (Satz 6.13) in einer Dimension – in Lean nachgerechnet als `variation_der_konstanten`.

Die Rückführungen

Alles Weitere ist eine Substitution, die auf einen der beiden Fälle führt.

Typ	Form	Substitution	wird
Ähnlichkeit	$y' = f(y/x)$	$u = y/x$	trennbar
verschoben	$y' = f(ax + by + c)$	$u = ax + by + c$	trennbar
Bernoulli	$y' + ay = b\,y^n$	$u = y^{1-n}$	linear
Riccati	$y' = a + by + cy^2$, eine Lösung y_1 bekannt	$y = y_1 + 1/u$	linear
exakt	$M + Ny' = 0$ mit $M_y = N_x$	Potential F	$F(x, y) = c$
Euler-DGL	$x^2y'' + axy' + by = 0$	$x = e^t$	konst. Koeff.
y fehlt	$y'' = f(x, y')$	$u = y'$	erste Ordnung
x fehlt	$y'' = f(y, y')$	$p(y) = y'$	erste Ordnung
zweite Lösung	linear, y_1 bekannt	$y_2 = y_1 \int u$	erste Ordnung in u

Die beiden vorletzten Zeilen beruhen auf derselben Beobachtung: Fehlt eine der Variablen in der Gleichung, so ist eine Ordnung geschenkt. Beim Fall „x fehlt“ schreibt man die Geschwindigkeit als Funktion des Ortes und rechnet

$$y'' \;=\; \frac{\mathrm{d}}{\mathrm{d}x}\,p(y) \;=\; p'(y)\,y' \;=\; p'(y)\,p(y),$$

also nichts als die Kettenregel.

Lean code/Kapitel6.lean

```
section Rezepte
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]

/-- **Trennbare Variablen, ein Fall zum Nachrechnen.** ‘y’ = 1 + y^2‘ hat die
Loesung ‘tan(t)‘; das ist das Ergebnis von ∫dy/(1+y^2) = ∫dt. -/
theorem trennbar_tan {t : ℝ} (ht : Real.cos t ≠ 0) :
    HasDerivAt Real.tan (1 + Real.tan t ^ 2) t := by
  have h := Real.hasDerivAt_tan ht
  convert h using 1
  rw [Real.tan_eq_sin_div_cos]
  field_simp
  rw [← Real.sin_sq_add_cos_sq t]
  ring

/-- **Superposition auf der rechten Seite.** Zerlegt man die Stoerung in
‘f₁ + f₂‘ und loest beide Teile einzeln, so ist die Summe eine Loesung des
Ganzen. Damit darf man eine komplizierte rechte Seite zerlegen. -/
theorem superposition_rechte_seite {A : ℝ → E →L[ℝ] E} {f₁ f₂ x₁ x₂ : ℝ → E} {t
    : ℝ}
```

```
    (h₁ : HasDerivAt x₁ (A t (x₁ t) + f₁ t) t)
    (h₂ : HasDerivAt x₂ (A t (x₂ t) + f₂ t) t) :
    HasDerivAt (fun s => x₁ s + x₂ s) (A t (x₁ t + x₂ t) + (f₁ t + f₂ t)) t := by
  have h := h₁.add h₂
  simp only [map_add]
  convert h using 1
  abel

/-- **Die Bernoulli-Substitution.** Loest `y` die logistische Gleichung
`y' = r·y·(1 - y/K)` -- eine Bernoulli-Gleichung mit n = 2 --, so loest
`u = 1/y` die *lineare* Gleichung `u' = -r·u + r/K`. -/
theorem bernoulli_substitution (r K : ℝ) {y : ℝ → ℝ} {t : ℝ} (hy : y t ≠ 0)
    (h : HasDerivAt y (r * y t * (1 - y t / K)) t) :
    HasDerivAt (fun s => (y s)⁻¹) (-(r * (y t)⁻¹) + r / K) t := by
  have h' := h.inv hy
  convert h' using 1
  field_simp
  ring

/-- **Ordnungsreduktion, wenn `x` nicht vorkommt.** Schreibt man die
Geschwindigkeit als Funktion des Ortes, `y' = p(y)`, so ist `y'' = p'(y)·p(y)`
-- aus zweiter Ordnung wird erste. Der Beweis ist die Kettenregel. -/
theorem ordnungsreduktion {y p : ℝ → ℝ} {p' : ℝ} {t : ℝ}
    (hy : HasDerivAt y (p (y t)) t) (hp : HasDerivAt p p' (y t)) :
    HasDerivAt (fun s => p (y s)) (p' * p (y t)) t :=
  hp.comp t hy

/-- **Das Pendel hat eine Erhaltungsgroesse.** Fuer `y'' = -(g/l)·sin y` ist
`v²/2 + (g/l)(1 - cos y)` laengs jeder Loesung konstant. Genau diese Groesse
faellt bei der Ordnungsreduktion von selbst an. -/
theorem pendel_energie (g l : ℝ) {y v : ℝ → ℝ} {t : ℝ}
    (hy : HasDerivAt y (v t) t)
    (hv : HasDerivAt v (-(g / l) * Real.sin (y t)) t) :
    HasDerivAt (fun s => v s ^ 2 / 2 + (g / l) * (1 - Real.cos (y s))) 0 t := by
  have h1 : HasDerivAt (fun s => v s ^ 2 / 2)
      (v t * (-(g / l) * Real.sin (y t))) t := by
    have := (hv.pow 2).div_const 2
    convert this using 1
    ring
  have h2 : HasDerivAt (fun s => (g / l) * (1 - Real.cos (y s)))
      ((g / l) * (Real.sin (y t) * v t)) t := by
    have := ((Real.hasDerivAt_cos (y t)).comp t hy)
    have h3 := (this.const_sub 1).const_mul (g / l)
    convert h3 using 1
    ring
  have := h1.add h2
  convert this using 1
  ring
end Rezepte
```

Wie man Probleme zusammensetzt

Vier Handgriffe, und fast immer ist es einer davon.

1. **Die rechte Seite zerlegen – nur bei linearen Gleichungen.** Ist $b = b_1 + b_2$, so löst man beide Teile einzeln und addiert (Satz 6.12). Eine Störung aus mehreren Frequenzen

behandelt man also Frequenz für Frequenz. Bei nichtlinearen Gleichungen ist das falsch, und zwar fundamental.

2. **Substituieren.** Die ganze Tabelle oben ist ein einziger Gedanke: Finde ein $u = \varphi(x, y)$, das die Gleichung auf einen der beiden Grundfälle schiebt. Die Frage lautet nie „wie löse ich das“, sondern „worauf sieht das aus“.

3. **Die Ordnung senken.** Fehlt eine Variable, siehe oben. Und jede *Erhaltungsgröße* senkt die Ordnung um eins: Eine Gleichung $H(y, y') = c$ ist erster Ordnung, wo vorher zweite stand. Bei Lotka–Volterra war das H aus Abschnitt 6.11.

4. **Die Ordnung erhöhen.** Das klingt verkehrt herum, ist aber der häufigste Handgriff überhaupt: Eine Gleichung n-ter Ordnung wird zu einem System erster Ordnung in $\mathbb{R}^n$ (Abschnitt 6.2). Für die Theorie und für jeden numerischen Löser ist das die Form, in der man arbeitet.

Zwei Gleichungen, durchgerechnet

Das logistische Wachstum ist eine Bernoulli-Gleichung.

$$y' = r\,y\Big(1 - \frac{y}{K}\Big) = r\,y - \frac{r}{K}\,y^2,$$

also Bernoulli mit $n = 2$. Die Tabelle sagt: $u = y^{-1}$. Nachrechnen:

$$u' = -\frac{y'}{y^2} = -\frac{ry - \frac{r}{K}y^2}{y^2} = -\frac{r}{y} + \frac{r}{K} = -r\,u + \frac{r}{K}.$$

Das ist linear mit konstanten Koeffizienten, und die Formel von oben gibt

$$u(t) = \frac{1}{K} + \Big(\frac{1}{y_0} - \frac{1}{K}\Big)e^{-rt}, \qquad \text{also} \qquad y(t) = \frac{K\,y_0\,e^{rt}}{K + y_0\,(e^{rt} - 1)}.$$

Das ist genau die Lösung, mit der Notebook `03` seine numerischen Werte vergleicht – und die Substitution ist der Satz `bernoulli_substitution` oben.

Das Pendel, so weit es geht.

$$\varphi'' = -\frac{g}{\ell}\,\sin\varphi.$$

Die Zeit kommt nicht vor, also $p(\varphi) = \varphi'$ und $\varphi'' = p\,p'$. Damit wird aus zweiter Ordnung

$$p\,\frac{\mathrm{d}p}{\mathrm{d}\varphi} = -\frac{g}{\ell}\sin\varphi,$$

und das ist trennbar. Integrieren:

$$\frac{p^2}{2} \;=\; \frac{g}{\ell}\cos\varphi + C.$$

Was da steht, ist der Energiesatz – und er ist nicht hineingesteckt worden, sondern beim Ordnungsenken von selbst angefallen. In Lean steht er als `pendel_energie`.

Startet das Pendel bei φ_0 in Ruhe, so ist $C = -\frac{g}{\ell}\cos\varphi_0$ und

$$\varphi' \;=\; \pm\sqrt{\tfrac{2g}{\ell}\,(\cos\varphi - \cos\varphi_0)}, \qquad \text{also} \qquad t \;=\; \int \frac{\mathrm{d}\varphi}{\sqrt{\frac{2g}{\ell}(\cos\varphi - \cos\varphi_0)}}.$$

Und hier hört es auf. Dieses Integral ist nicht elementar; es ist ein elliptisches Integral erster Art, und die Schwingungsdauer lautet $T = 4\sqrt{\ell/g}\;K\big(\sin\tfrac{\varphi_0}{2}\big)$. Erst für kleine Ausschläge, wo $\sin\varphi \approx \varphi$ ist, bleibt das vertraute $T = 2\pi\sqrt{\ell/g}$ übrig – und erst dann hängt die Dauer nicht mehr vom Ausschlag ab.

Wie weit man kommt. Die Reduktion hat funktioniert: aus zweiter Ordnung wurde erste, und die Erhaltungsgröße fiel ab. Nur das letzte Integral geht nicht mehr in elementaren Funktionen auf.

Das ist der Normalfall. Die Rückführungen bringen einen fast immer ein Stück weit, und fast nie ganz ans Ziel.

Wenn nichts davon hilft

Das ist der Regelfall, nicht die Ausnahme. Schon $y' = x^2 + y^2$ hat keine Lösung in elementaren Funktionen; man braucht Bessel-Funktionen der Ordnung $\frac{1}{4}$. Das ist ein Satz von Liouville aus dem Jahr 1841, kein Mangel an Findigkeit. Dann bleiben drei Wege:

- **Qualitativ.** Gleichgewichte, Stabilität, Phasenporträt, Erhaltungsgrößen. Man bekommt kein $y(t)$, aber man weiß, wie es ausgeht – wie bei Lotka–Volterra.
- **Näherungsweise in Formeln.** Reihenansatz, Störungsrechnung, Linearisieren um ein Gleichgewicht (Abschnitt 6.7).
- **Numerisch.** Der Rest dieses Kapitels.

Es lohnt sich, das richtig herum zu sehen: Nicht die Existenzsätze sind der Notbehelf für die Fälle, die man nicht lösen kann. Die geschlossene Lösung ist der Glücksfall, und die Sätze sind das, worauf man sich sonst stützt.

6.9 Das Pendel, ausgeführt

Das Pendel ist dem Kapitel bisher dreimal begegnet: als Zeile in der Tabelle am Anfang, als Rechenbeispiel in Abschnitt 6.8 und später noch einmal numerisch. Hier einmal von vorn und am Stück, weil fast alles, was dieses Kapitel bereitstellt, daran vorkommt.

Das Modell

Eine Punktmasse m an einer masselosen Stange der Länge ℓ, der Winkel φ gegen die Ruhelage gemessen. Die Schwerkraft mg zerlegt sich in eine Komponente längs der Stange – die hält die Stange aus – und eine quer dazu, $-mg\sin\varphi$. Die Masse bewegt sich auf einem Kreis, ihre Bahnbeschleunigung ist $\ell\varphi''$. Newton:

$$m\,\ell\,\varphi'' = -m\,g\,\sin\varphi \qquad \Longrightarrow \qquad \varphi'' = -\frac{g}{\ell}\,\sin\varphi.$$

Die Masse fällt heraus. Wie schnell ein Pendel schwingt, hängt nicht davon ab, was daran hängt.

Mit $x = (\varphi, v)$ wird daraus, wie in Abschnitt 6.2, ein System erster Ordnung:

$$\varphi' = v, \qquad v' = -\frac{g}{\ell}\sin\varphi.$$

Bemerkung 6.16 (Hier ist die globale Existenz leicht). Die rechte Seite ist in x global Lipschitzstetig: sin ist 1-Lipschitz, also kommt man mit $K = \max\{1,\, g/\ell\}$ aus. Nach Satz 6.9 – genauer: nach demselben Grönwall-Argument – existiert die Lösung für alle Zeiten.

Das ist nicht selbstverständlich. Bei $x' = x^2$ aus Abschnitt 6.3 war die rechte Seite ebenso glatt und die Lösung trotzdem nach endlicher Zeit weg. Der Unterschied ist, dass sin beschränkt ist.

Kleine Ausschläge: der harmonische Oszillator

Für kleine φ ist $\sin\varphi = \varphi - \varphi^3/6 + \cdots \approx \varphi$, und übrig bleibt

$$\varphi'' = -\omega_0^2\,\varphi, \qquad \omega_0 = \sqrt{g/\ell}.$$

Das ist linear mit konstanten Koeffizienten, also greift Abschnitt 6.7. Als System ist

$$A = \begin{pmatrix} 0 & 1 \\ -\omega_0^2 & 0 \end{pmatrix}, \qquad \chi(\lambda) = \lambda^2 + \omega_0^2, \qquad \lambda_{1,2} = \pm i\,\omega_0.$$

Ein rein imaginäres Paar, also die zweite Zeile der Eigenwerttabelle mit $\alpha = 0$: Das Fundamentalsystem ist $\cos\omega_0 t$, $\sin\omega_0 t$, und

$$\varphi(t) = \varphi_0\cos(\omega_0 t) + \frac{v_0}{\omega_0}\,\sin(\omega_0 t).$$

Ausgeschrieben ist die Fundamentalmatrix

$$e^{tA} = \begin{pmatrix} \cos\omega_0 t & \frac{1}{\omega_0}\sin\omega_0 t \\ -\omega_0\sin\omega_0 t & \cos\omega_0 t \end{pmatrix}.$$

Lean code/Kapitel6.lean

```
/-- Geschwindigkeit des harmonischen Oszillators:
‘φ(t) = A·cos(ωt) + B·sin(ωt)‘ hat die Ableitung
‘-Aω·sin(ωt) + Bω·cos(ωt)‘. -/
theorem harmonisch_v (t : ℝ) :
    HasDerivAt (fun s => A * Real.cos (ω * s) + B * Real.sin (ω * s))
      (-(A * ω) * Real.sin (ω * t) + B * ω * Real.cos (ω * t)) t := by
  have hw : HasDerivAt (fun s : ℝ => ω * s) ω t := by
    simpa using (hasDerivAt_id t).const_mul ω
  have hc : HasDerivAt (fun s : ℝ => A * Real.cos (ω * s))
      (A * (-Real.sin (ω * t) * ω)) t :=
    ((Real.hasDerivAt_cos (ω * t)).comp t hw).const_mul A
  have hs : HasDerivAt (fun s : ℝ => B * Real.sin (ω * s))
      (B * (Real.cos (ω * t) * ω)) t :=
    ((Real.hasDerivAt_sin (ω * t)).comp t hw).const_mul B
  have h := hc.add hs
  convert h using 1
  ring

/-- Und die Beschleunigung ist ‘-ω²·φ‘: die Funktion loest ‘φ'' = -ω²φ‘. -/
theorem harmonisch_a (t : ℝ) :
    HasDerivAt (fun s => -(A * ω) * Real.sin (ω * s) + B * ω * Real.cos (ω * s))
      (-ω ^ 2 * (A * Real.cos (ω * t) + B * Real.sin (ω * t))) t := by
  have hw : HasDerivAt (fun s : ℝ => ω * s) ω t := by
    simpa using (hasDerivAt_id t).const_mul ω
  have hs : HasDerivAt (fun s : ℝ => -(A * ω) * Real.sin (ω * s))
      (-(A * ω) * (Real.cos (ω * t) * ω)) t :=
    ((Real.hasDerivAt_sin (ω * t)).comp t hw).const_mul (-(A * ω))
  have hc : HasDerivAt (fun s : ℝ => B * ω * Real.cos (ω * s))
      (B * ω * (-Real.sin (ω * t) * ω)) t :=
    ((Real.hasDerivAt_cos (ω * t)).comp t hw).const_mul (B * ω)
  have h := hs.add hc
  convert h using 1
  ring
```

Warum Kreisfunktionen. Rein imaginäre Eigenwerte heißen: Die Matrix dreht. In den Koordinaten $(\varphi,\, v/\omega_0)$ ist A genau ω_0 mal eine Drehung um 90°, und e^{tA} ist dann eine Drehung um den Winkel $\omega_0 t$. Deshalb sind die Bahnen in diesen Koordinaten Kreise und die Lösungen Kreisfunktionen – nicht weil jemand sie geraten hätte, sondern weil die Matrix eine Drehung ist.

Die Schwingungsdauer ist

$$T = \frac{2\pi}{\omega_0} = 2\pi\sqrt{\ell/g},$$

unabhängig vom Ausschlag *und* von der Masse. Diese *Isochronie* ist der Grund, warum man aus einem Pendel eine Uhr bauen kann – und sie ist, wie sich zeigen wird, nur eine Eigenschaft der Linearisierung.

Mit Dämpfung

Reibung proportional zur Geschwindigkeit gibt

$$\varphi'' + 2\delta\,\varphi' + \omega_0^2\,\varphi = 0, \qquad \chi(\lambda) = \lambda^2 + 2\delta\lambda + \omega_0^2, \qquad \lambda_{1,2} = -\delta \pm \sqrt{\delta^2 - \omega_0^2}.$$

Je nachdem, was unter der Wurzel steht, bekommt man genau die drei Zeilen der Eigenwerttabelle aus Abschnitt 6.7:

Fall	Nullstellen	Lösung
Schwingfall ($\delta < \omega_0$)	$-\delta \pm i\omega_d$, $\omega_d = \sqrt{\omega_0^2 - \delta^2}$	$e^{-\delta t}\big(A\cos\omega_d t + B\sin\omega_d t\big)$
Grenzfall ($\delta = \omega_0$)	$-\delta$, doppelt	$(A + B\,t)\,e^{-\delta t}$
Kriechfall ($\delta > \omega_0$)	$\lambda_1, \lambda_2 < 0$ reell	$A\,e^{\lambda_1 t} + B\,e^{\lambda_2 t}$

In allen drei Fällen sind alle Realteile negativ, nach Satz 6.14 geht also jede Lösung gegen null. Am schnellsten der Grenzfall – darauf stellt man Türschließer und Stoßdämpfer ein.

Er ist zugleich die einzige Stelle in diesem Kapitel, an der die Zeile „mehrfache Nullstelle“ wirklich gebraucht wird. Dass $t\,e^{-\delta t}$ tatsächlich löst, ist nachgerechnet:

Lean code/Kapitel6.lean

```
/-- **Aperiodischer Grenzfall.** Bei doppelter Nullstelle `-δ` des
charakteristischen Polynoms `λ² + 2δλ + δ²` kommt `t·e^{-δt}` als zweite
Loesung dazu -- die Zeile "mehrfache Nullstelle" aus Abschnitt 6.7. -/
theorem grenzfall_v (t : ℝ) :
    HasDerivAt (fun s => s * Real.exp (-(δ * s)))
      ((1 - δ * t) * Real.exp (-(δ * t))) t := by
  have hd : HasDerivAt (fun s : ℝ => -(δ * s)) (-δ) t := by
    simpa using ((hasDerivAt_id t).const_mul δ).neg
  have he : HasDerivAt (fun s : ℝ => Real.exp (-(δ * s)))
      (Real.exp (-(δ * t)) * -δ) t := hd.exp
  have hi : HasDerivAt (fun s : ℝ => s) 1 t := hasDerivAt_id t
  have h : HasDerivAt (fun s : ℝ => s * Real.exp (-(δ * s)))
      (1 * Real.exp (-(δ * t)) + t * (Real.exp (-(δ * t)) * -δ)) t := hi.mul he
  convert h using 1
  ring

theorem grenzfall_a (t : ℝ) :
    HasDerivAt (fun s => (1 - δ * s) * Real.exp (-(δ * s)))
      (δ * (δ * t - 2) * Real.exp (-(δ * t))) t := by
  have hd : HasDerivAt (fun s : ℝ => -(δ * s)) (-δ) t := by
    simpa using ((hasDerivAt_id t).const_mul δ).neg
  have he : HasDerivAt (fun s : ℝ => Real.exp (-(δ * s)))
```

```
      (Real.exp (-(δ * t)) * -δ) t := hd.exp
  have hl : HasDerivAt (fun s : ℝ => 1 - δ * s) (-δ) t := by
    simpa using ((hasDerivAt_id t).const_mul δ).const_sub 1
  have h : HasDerivAt (fun s : ℝ => (1 - δ * s) * Real.exp (-(δ * s)))
      (-δ * Real.exp (-(δ * t)) + (1 - δ * t) * (Real.exp (-(δ * t)) * -δ)) t :=
    hl.mul he
  convert h using 1
  ring
```

Erzwungen: Resonanz

Jetzt mit einer äußeren Anregung,

$$\varphi'' + 2\delta\,\varphi' + \omega_0^2\,\varphi = A\cos(\omega t).$$

Nach Satz 6.12 ist die allgemeine Lösung die spezielle plus die homogene. Die homogene klingt ab, weil alle Realteile negativ sind – also bleibt auf Dauer nur die spezielle übrig, der *eingeschwungene Zustand*.

Der Ansatz vom Typ der rechten Seite aus Abschnitt 6.8 liefert $\varphi_p = C\cos\omega t + D\sin\omega t$ mit der Amplitude

$$\hat{\varphi}(\omega) \;=\; \frac{A}{\sqrt{(\omega_0^2-\omega^2)^2 + 4\delta^2\omega^2}}.$$

Sie ist am größten in der Nähe von $\omega = \omega_0$, und für $\delta \to 0$ wächst dieses Maximum über alle Grenzen.

Ohne Dämpfung und bei $\omega = \omega_0$ ist die Anregung selbst eine homogene Lösung – der Resonanzfall der Ansatztabelle. Der Ansatz muss mit t multipliziert werden, und heraus kommt

$$\varphi_p(t) \;=\; \frac{A}{2\omega_0}\,t\,\sin(\omega_0 t).$$

Lean code/Kapitel6.lean

```
/-- **Resonanz.** Ohne Daempfung und bei Anregung mit genau der
Eigenfrequenz loest `y(t) = (A/2ω)·t·sin(ωt)` die Gleichung
`y'' + ω²y = A·cos(ωt)` -- die Amplitude waechst linear in `t`. -/
theorem resonanz_v (t : ℝ) :
    HasDerivAt (fun s => A / (2 * ω) * (s * Real.sin (ω * s)))
      (A / (2 * ω) * (Real.sin (ω * t) + ω * t * Real.cos (ω * t))) t := by
  have hw : HasDerivAt (fun s : ℝ => ω * s) ω t := by
    simpa using (hasDerivAt_id t).const_mul ω
  have hs : HasDerivAt (fun s : ℝ => Real.sin (ω * s)) (Real.cos (ω * t) * ω) t :=
    (Real.hasDerivAt_sin (ω * t)).comp t hw
  have hi : HasDerivAt (fun s : ℝ => s) 1 t := hasDerivAt_id t
  have h : HasDerivAt (fun s : ℝ => A / (2 * ω) * (s * Real.sin (ω * s)))
      (A / (2 * ω) * (1 * Real.sin (ω * t) + t * (Real.cos (ω * t) * ω))) t :=
    (hi.mul hs).const_mul (A / (2 * ω))
  convert h using 1
  ring

theorem resonanz_a (t : ℝ) (hω : ω ≠ 0) :
    HasDerivAt (fun s => A / (2 * ω) * (Real.sin (ω * s) + ω * s * Real.cos (ω *
    s)))
      (A * Real.cos (ω * t) - ω ^ 2 * (A / (2 * ω) * (t * Real.sin (ω * t)))) t
    := by
  have hw : HasDerivAt (fun s : ℝ => ω * s) ω t := by
    simpa using (hasDerivAt_id t).const_mul ω
```

```
have hs : HasDerivAt (fun s : ℝ => Real.sin (ω * s)) (Real.cos (ω * t) * ω) t :=
  (Real.hasDerivAt_sin (ω * t)).comp t hw
have hc : HasDerivAt (fun s : ℝ => Real.cos (ω * s)) (-Real.sin (ω * t) * ω) t
  :=
  (Real.hasDerivAt_cos (ω * t)).comp t hw
have hp : HasDerivAt (fun s : ℝ => ω * s * Real.cos (ω * s))
    (ω * Real.cos (ω * t) + ω * t * (-Real.sin (ω * t) * ω)) t := hw.mul hc
have h : HasDerivAt
    (fun s : ℝ => A / (2 * ω) * (Real.sin (ω * s) + ω * s * Real.cos (ω * s)))
    (A / (2 * ω) * (Real.cos (ω * t) * ω
      + (ω * Real.cos (ω * t) + ω * t * (-Real.sin (ω * t) * ω)))) t :=
  (hs.add hp).const_mul (A / (2 * ω))
convert h using 1
field_simp
ring
```

Die Brücke im Gleichschritt. Das ist die Rechnung hinter der Bemerkung in Abschnitt 6.8. Die Amplitude wächst linear in t – nicht exponentiell, aber unbeschränkt. Jeder Anstoß kommt genau dann, wenn das System ohnehin in diese Richtung läuft, und jeder fügt dieselbe Portion Energie hinzu.

In Wirklichkeit gibt es immer etwas Dämpfung, also bleibt die Amplitude endlich – aber $\hat{\varphi}(\omega_0) = A/(2\delta\omega_0)$ kann sehr groß werden, wenn δ klein ist. Deshalb ist Resonanz ein Baustatikproblem und kein mathematisches Kuriosum.

Zurück zum nichtlinearen Pendel

Was bleibt, wenn man $\sin\varphi$ stehen lässt?

- **Die Erhaltungsgröße bleibt.** Abschnitt 6.8 hat sie beim Ordnungsenken bekommen; in Lean steht sie als `pendel_energie`.
- **Die Bahnen bleiben geschlossen**, aber es sind keine Kreise mehr – sie werden mit wachsendem Ausschlag immer eckiger.
- **Die Isochronie geht verloren.** T hängt vom Ausschlag ab, und zwar über ein elliptisches Integral.
- **Es kommt ein Gleichgewicht dazu.** Das lineare System hat nur eines; das Pendel hat ein zweites bei $\varphi = \pi$, und das ist ein Sattel. Linearisierung findet es gar nicht erst, weil sie um $\varphi = 0$ entwickelt wurde.

Die letzten beiden Punkte lassen sich nicht mehr in Formeln auflösen. Dafür ist Abschnitt 6.11 da.

6.10 Das Euler-Verfahren

Zum Schluss der Bogen zurück zu Kapitel 5. Die Integralform aus Satz 6.2 legt ein Verfahren nahe: Man ersetzt das Integral über ein kleines Zeitstück durch die einfachste Quadraturformel, nämlich den linken Randwert mal die Schrittweite. Das gibt

$$x_{k+1} \;=\; x_k + h\, f(t_k, x_k), \qquad t_k = t_0 + kh.$$

Lean code/Kapitel6.lean

```
/- x_{k+1} = x_k + h · f(t_k, x_k). Gerechnet wird konkret mit
   Gleitkommazahlen, ‘Float‘. -/
def eulerF (f : Float → Float → Float) (t₀ x₀ h : Float) (n : Nat) : Float :=
  (List.range n).foldl (fun x i => x + h * f (t₀ + i.toFloat * h) x) x₀

/- x' = x, x(0) = 1. Exakt ist x(1) = e = 2.718281828... -/
#eval eulerF (fun _ x => x) 0 1 0.1    10
#eval eulerF (fun _ x => x) 0 1 0.01   100
#eval eulerF (fun _ x => x) 0 1 0.001  1000
#eval Float.exp 1

/- Der Fehler faellt wie h, also eine Stelle je Verzehnfachung. -/
#eval eulerF (fun _ x => x) 0 1 0.1    10   - Float.exp 1
#eval eulerF (fun _ x => x) 0 1 0.01   100  - Float.exp 1
#eval eulerF (fun _ x => x) 0 1 0.001  1000 - Float.exp 1
```

Die Zahlen für $x' = x$, $x(0) = 1$ auf $[0, 1]$:

Schrittweite	Wert	Fehler
$h = 10^{-1}$	2.593742	$-1.25 \cdot 10^{-1}$
$h = 10^{-2}$	2.704814	$-1.35 \cdot 10^{-2}$
$h = 10^{-3}$	2.716924	$-1.36 \cdot 10^{-3}$
exakt: e	2.718282	

Der Fehler fällt wie h, also eine Stelle je Verzehnfachung des Aufwands. Das ist schlecht – die Trapezregel schaffte h^2, Simpson h^4. Der Grund ist, dass der lokale Fehler eines Schritts zwar $O(h^2)$ ist, es aber $1/h$ Schritte gibt und sich die Fehler unterwegs verstärken können. Wie stark, sagt genau Grönwall: höchstens um den Faktor $e^{K(b-a)}$. Daraus wird die Fehlerschranke

$$\|x_n - x(t_n)\| \;\le\; \frac{h}{2} \max \|x''\| \cdot \frac{e^{K(b-a)} - 1}{K}.$$

Bessere Verfahren – Runge–Kutta, Mehrschrittverfahren – setzen an der Quadraturformel an, nicht an dieser Abschätzung.

Bemerkung 6.17 (Dieselbe Formel an anderer Stelle). Der Gradientenabstieg aus Abschnitt 4.5,

$$\theta_{k+1} = \theta_k - h\, \nabla L(\theta_k),$$

ist buchstäblich ein Euler-Schritt – für die Differentialgleichung $\theta' = -\nabla L(\theta)$. Die Lernrate steht an der Stelle der Schrittweite, und das erklärt, warum sie sich so verhält wie eine: zu klein ist langsam, zu groß wird unruhig.

Weiter trägt die Analogie nicht. Beim Optimieren zählt nur, wo man landet, beim Lösen einer Differentialgleichung der ganze Weg. Wer eine Lernrate einstellt, löst keine Differentialgleichung – er benutzt nur dieselbe Formel.

6.11 Rechnen mit scipy

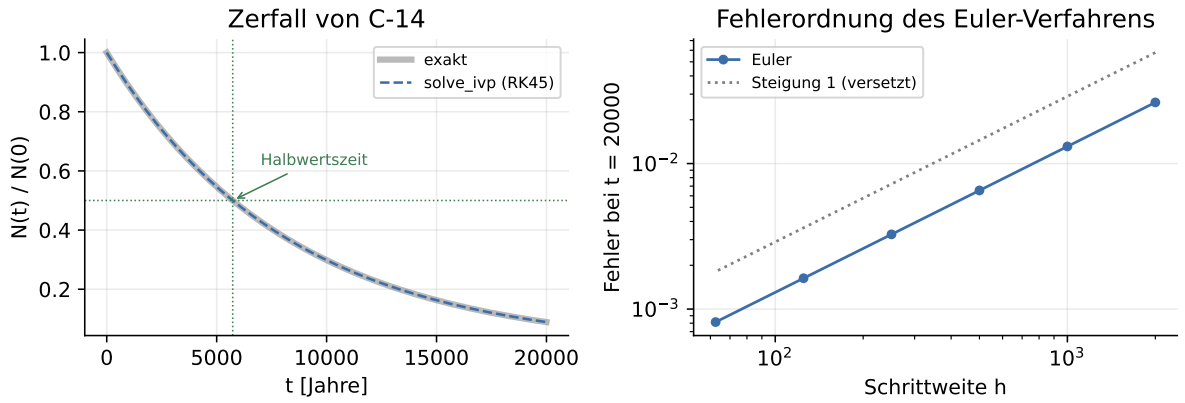
Der Lean-Code dieses Kapitels beweist Aussagen. Ausrechnen tut er nichts – $\mathbb{R}$ ist in Lean nicht berechenbar, und die Sätze oben sind Existenz- und Eindeutigkeitsaussagen, keine Verfahren.

Zum Rechnen gehören sechs Jupyter-Notebooks zu diesem Kapitel. Sie benutzen `scipy.integrate.solve_ivp`, einen Löser, der im Kern dasselbe tut wie das Euler-Verfahren aus Abschnitt 6.10: Er läuft in Schritten durch die Zeit. Nur sind die Formeln besser und die Schrittweite wählt er selbst.

Notebook	Thema
`01_zerfall.ipynb`	$x' = -\lambda x$, Fehlerordnung, Stabilität des Euler-Schritts
`02_picard_lindeloef.ipynb`	Picard-Iteration, Mehrdeutigkeit, Blowup, Grönwall
`03_logistisches_wachstum.ipynb`	Richtungsfeld, Gleichgewichte, Phasenlinie
`04_raeuber_beute.ipynb`	Lotka–Volterra: ein System in $\mathbb{R}^2$, Erhaltungsgröße, Volterra-Prinzip
`05_pendel.ipynb`	Separatrix, Sattelpunkt, Dauer und Ausschlag, symplektischer Euler
`06_feigenbaum.ipynb`	Verdopplungskaskade, $\delta = 4{,}6692\ldots$, Universalität

Die Abbildungen auf den nächsten Seiten sind unverändert das, was die Notebooks erzeugen – dieselbe Regel wie beim Lean-Code.

Zerfall



Notebook `notebooks/01_zerfall.ipynb`

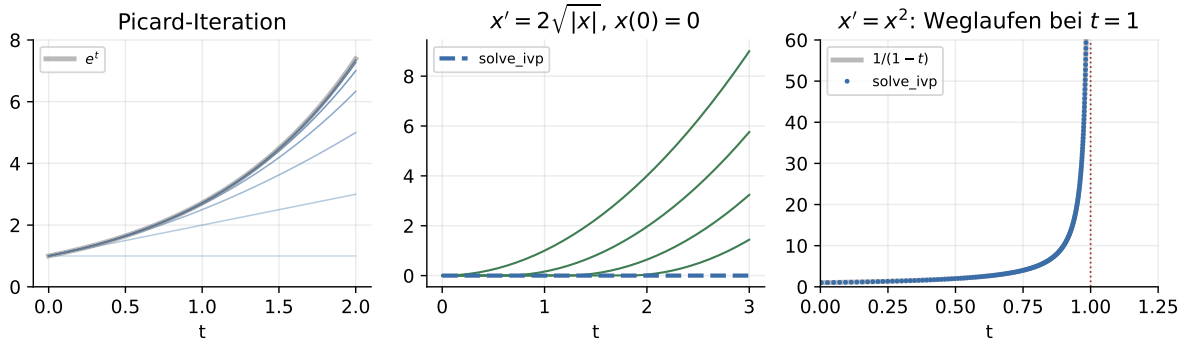
Links die Lösung von $N' = -\lambda N$ für Kohlenstoff-14, numerisch und exakt übereinander. Rechts der Fehler des Euler-Verfahrens über der Schrittweite, doppelt logarithmisch: eine Gerade der Steigung 1, also genau $O(h)$.

Das Notebook rechnet außerdem nach, wo der Euler-Schritt kippt. Für $x' = -\lambda x$ ist ein Schritt

$$x_{k+1} = (1 - h\lambda)\, x_k,$$

die Folge bleibt also nur für $h \le 2/\lambda$ beschränkt. Darüber wächst sie mit wechselndem Vorzeichen, während die wahre Lösung monoton gegen null geht. Das ist kein Genauigkeitsproblem mehr, sondern ein falsches Ergebnis – und der Grund, warum es für steife Gleichungen implizite Verfahren gibt.

Picard–Lindelöf, nachgerechnet



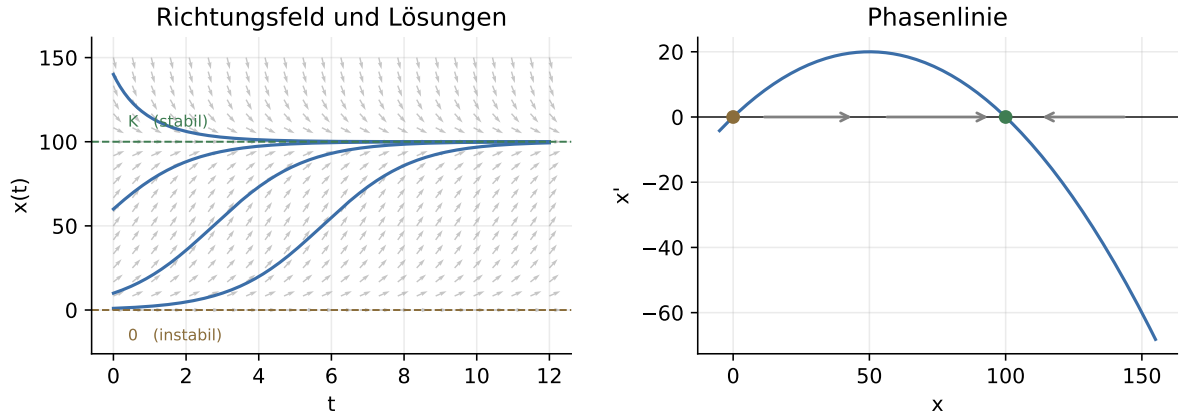
Notebook `notebooks/02_picard_lindeloef.ipynb`

Links die Picard-Iteration für $x' = x$: Die Iterierten sind die Partialsummen von e^t. In der Mitte vier der unendlich vielen Lösungen von $x' = 2\sqrt{|x|}$ und, gestrichelt, die eine, die der Löser zurückgibt. Rechts $x' = x^2$: Die Rechnung endet dort, wo die Lösung endet.

Das mittlere Bild ist das wichtigste des Kapitels. Die Voraussetzung von Satz 6.5 ist verletzt, es gibt unendlich viele Lösungen, und `solve_ivp` liefert kommentarlos eine davon – die Nullfunktion. Es gibt keine Warnung, keinen Fehlercode, nichts.

Das ist genau die Lücke aus Abschnitt 1.8, nur auf der numerischen Seite: Geprüft wird, was man hinschreibt. Ob die Voraussetzungen eines Satzes erfüllt sind, prüft weder ein Löser noch ein Beweisassistent.

Logistisches Wachstum



Notebook `notebooks/03_logistisches_wachstum.ipynb`

$x' = rx(1 - x/K)$. Links das Richtungsfeld mit vier Lösungen; jede läuft gegen die Kapazität K, von unten oder von oben. Rechts dieselbe Aussage ohne Zeitachse: Zwischen 0 und K ist $x' > 0$, darüber $x' < 0$.

Die beiden konstanten Lösungen sind die Nullstellen der rechten Seite. Welche davon stabil ist, entscheidet das Vorzeichen von $\partial f/\partial x$ dort: $f_x(0) = r > 0$ ist instabil, $f_x(K) = -r < 0$ ist stabil. Das ist dieselbe Rechnung wie bei Grönwall – in der Nähe eines Gleichgewichts verhält sich die Lösung wie $e^{f_x(x^*)t}$, und der Exponent entscheidet.

Bemerkung 6.18 (Woher die logistische Funktion kommt). Mit $r = K = 1$ und $x(0) = \frac{1}{2}$ lautet die Gleichung $\sigma' = \sigma(1 - \sigma)$, und ihre Lösung ist

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Das ist die *logistische Funktion* – dieselbe, die in der Statistik der logistischen Regression ihren Namen gibt und in neuronalen Netzen als Aktivierungsfunktion auftaucht. Verhulst schrieb die Gleichung 1838 für Populationen auf und nannte ihre Lösung *courbe logistique*; der Name ist von dort gewandert.

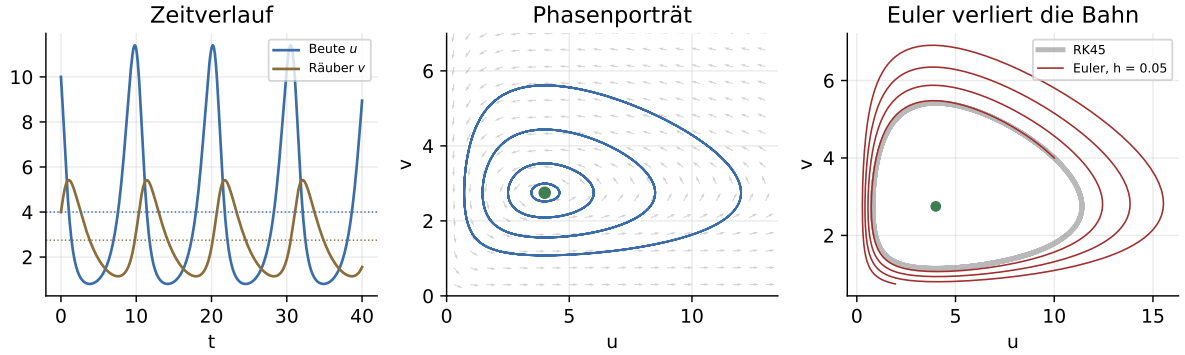
Die Verbindung läuft also über die Funktion, nicht über ein Verfahren. Wer eine logistische Regression rechnet, löst keine Differentialgleichung – er benutzt eine Funktion, die zufällig als deren Lösung definiert werden kann.

Räuber und Beute

Das Modell von Lotka und Volterra ist das erste System in diesen Notebooks: zwei Gleichungen für die Beute u und die Räuber v,

$$u' = \alpha u - \beta uv, \qquad v' = \delta uv - \gamma v.$$

Gefressen wird proportional zur Zahl der Begegnungen, und die ist proportional zu $u\,v$ – daher die gemischten Terme. Der Zustand lebt in $\mathbb{R}^2$, und erst damit gibt es ein Phasenporträt zu zeichnen.



Notebook `notebooks/04_raeuber_beute.ipynb`

Links der Zeitverlauf: Die Räuber hinken der Beute hinterher, die gepunkteten Linien sind die Mittelwerte. In der Mitte das Phasenporträt – geschlossene Bahnen um das Gleichgewicht. Rechts dieselbe Bahn, einmal mit `RK45` und einmal mit dem Euler-Verfahren: Letzteres spiralt nach außen.

Dass die Bahnen geschlossen sind, sieht man nicht der Linearisierung an. Die Jacobi-Matrix im Gleichgewicht hat die Eigenwerte $\pm i\sqrt{\alpha\gamma}$, also Realteil null – genau der Grenzfall, in dem die Linearisierung nichts entscheidet. Den Ausschlag gibt eine Erhaltungsgröße:

$$H(u, v) = \delta u - \gamma \ln u + \beta v - \alpha \ln v$$

ist entlang jeder Lösung konstant, und die Bahnen sind ihre Niveaumengen. Daraus folgt auch, dass die Lösung für alle Zeiten existiert: Sie bleibt auf einer kompakten Niveaumenge, und lokale Existenz plus Schranke ergibt globale Existenz.

Das rechte Bild ist der Grund, warum man bei langen Rechnungen nicht das Euler-Verfahren nimmt. Sein Fehler zeigt hier in jedem Schritt in dieselbe Richtung, also wächst H monoton. Das Verfahren rechnet eine Population aus, die immer heftiger schwankt – qualitativ falsch, nicht nur ungenau.

Bemerkung 6.19 (Das Volterra-Prinzip). Der Mittelwert über eine Periode ist

$$\langle u \rangle = \frac{\gamma}{\delta}, \qquad \langle v \rangle = \frac{\alpha}{\beta},$$

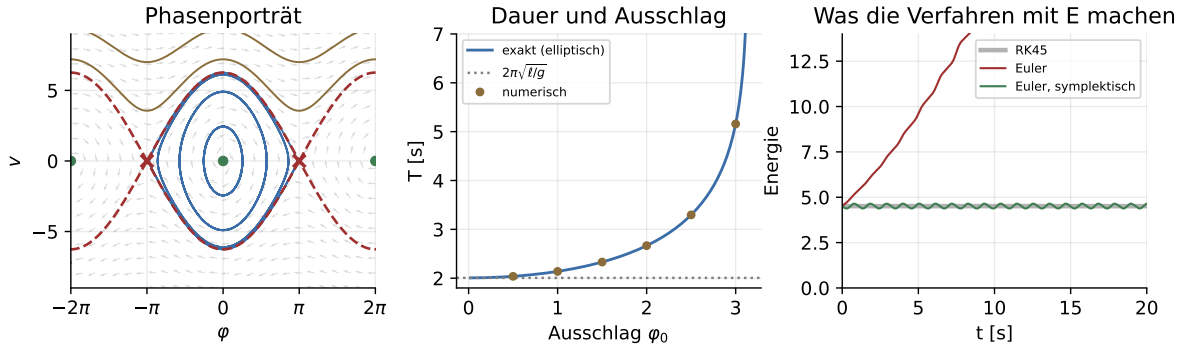
unabhängig davon, auf welcher Bahn man läuft. Der Beweis ist zwei Zeilen: Integriert man $u'/u = \alpha - \beta v$ über eine Periode, steht links $\ln u(T) - \ln u(0) = 0$.

Fischen trifft beide Arten: Aus α wird $\alpha - \varepsilon$, aus γ wird $\gamma + \varepsilon$. Also steigt $\langle u \rangle$ und fällt $\langle v \rangle$ – mehr Fischen bringt im Mittel mehr Beutefische und weniger Raubfische.

Genau das hatte Umberto d'Ancona in den Fangzahlen der Adria gesehen: Während des Ersten Weltkriegs, als kaum gefischt wurde, stieg der Anteil der Raubfische. Volterra erklärte es 1926 mit diesen Gleichungen. Dasselbe Argument heißt heute *Pestizid-Paradox*: Ein Mittel, das Schädling und Nützling gleichermaßen tötet, erhöht im Mittel den Schädlingsbestand.

Das Pendel, numerisch

Abschnitt 6.9 ist beim nichtlinearen Pendel an zwei Stellen stehengeblieben: bei der Schwingungsdauer und beim zweiten Gleichgewicht. Ab dort rechnet das Notebook weiter.



Notebook `notebooks/05_pendel.ipynb`

Links das Phasenporträt: geschlossene Bahnen um die Zentren bei $\varphi = 0, \pm 2\pi$, durchlaufende oben und unten, gestrichelt die Separatrix durch die Sattelpunkte bei $\pm\pi$. In der Mitte wächst die Schwingungsdauer mit dem Ausschlag; punktiert die Schulformel. Rechts, was die drei Verfahren mit der Energie machen.

Drei Dinge liest man ab.

Zwei Sorten Bahnen. Für $E < 2g/\ell$ schwingt das Pendel hin und her, für $E > 2g/\ell$ hat es genug Schwung und dreht durch. Dazwischen liegt die *Separatrix* mit genau $E = 2g/\ell$: Auf ihr erreicht das Pendel den höchsten Punkt – aber erst nach unendlich langer Zeit, denn dort steht ein Gleichgewicht.

Ein Zentrum und ein Sattel. Die Jacobi-Matrix hat unten die Eigenwerte $\pm i\sqrt{g/\ell}$ und oben $\pm\sqrt{g/\ell}$. Unten also derselbe Grenzfall wie bei Lotka–Volterra – rein imaginär, und erst die Erhaltungsgröße entscheidet. Oben dagegen ein echter Sattel, der Fall mit positivem Realteil aus Satz 6.14: Eine kleine Abweichung wächst mit der Zeitkonstante $\sqrt{\ell/g} \approx 0{,}32$ Sekunden.

Wie weit die Schulformel trägt. $T = 2\pi\sqrt{\ell/g}$ kommt aus $\sin\varphi \approx \varphi$ und kennt den Ausschlag gar nicht. Richtig ist $T(\varphi_0) = 4\sqrt{\ell/g}\,K\!\left(\sin^2\frac{\varphi_0}{2}\right)$ – genau das Integral, bei dem Abschnitt 6.8 aufgehört hat. Der Fehler der Näherung:

Ausschlag	0,1 rad	1,0 rad	3,0 rad
exakt [s]	2,0073	2,1391	5,1581
Schulformel [s]	2,0061	2,0061	2,0061
Fehler	−0,06 %	−6,2 %	−61 %

Das ist eine brauchbare Faustregel dafür, wie weit eine Linearisierung überhaupt trägt: hier etwa bis zu einem Radiant, danach nicht mehr.

Bemerkung 6.20 (Symplektisch statt genauer). Das rechte Bild wiederholt die Klage der letzten Abschnitte – das explizite Euler-Verfahren pumpt Energie hinein, von 4,51 auf 62,7 in zwanzig Sekunden. Es rechnet ein Pendel aus, das sich von selbst aufschaukelt.

Diesmal gibt es aber auch eine Abhilfe, und sie kostet nichts. Man nimmt für den Ort schon die *neue* Geschwindigkeit:

$$v_{k+1} = v_k - h\,\tfrac{g}{\ell}\sin\varphi_k, \qquad \varphi_{k+1} = \varphi_k + h\,v_{k+1}.$$

Das heißt *symplektisches* Euler-Verfahren. Es ist immer noch nur von erster Ordnung, aber die Energie bleibt bei $4{,}51 \pm 0{,}14$ – sie schwankt, statt wegzulaufen.

Der Grund ist nicht Genauigkeit, sondern Struktur: Das Verfahren erhält das Phasenraumvolumen, und man kann zeigen, dass es die Energie eines *benachbarten* Systems exakt erhält. Wer lange rechnet, nimmt deshalb lieber ein Verfahren niedriger Ordnung, das die richtige Struktur hat, als eines hoher Ordnung, das sie zerstört.

Feigenbaum

Zum Schluss der Extremfall. Die logistische Gleichung aus Notebook `03` ist so zahm, wie eine Differentialgleichung nur sein kann: eindimensional und autonom, also hat x' zwischen zwei Gleichgewichten ein festes Vorzeichen, und *jede* Lösung ist monoton. Schwingen kann sie nicht, chaotisch werden erst recht nicht.

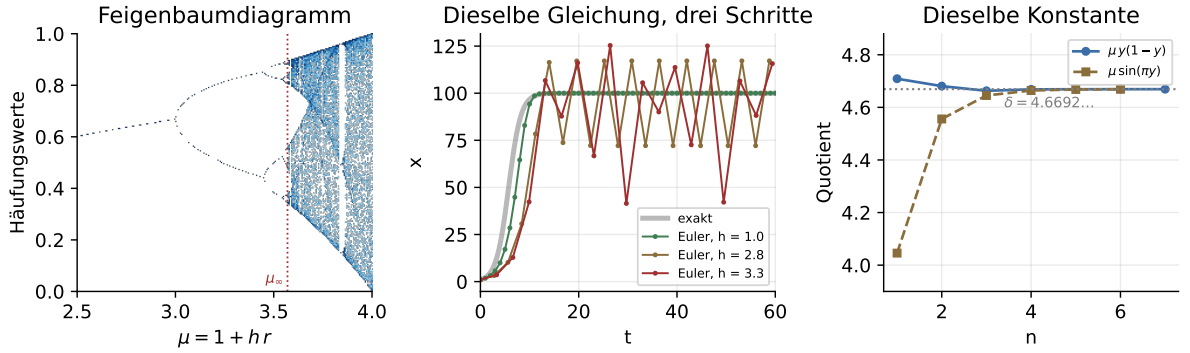
Ein Euler-Schritt darauf lautet

$$x_{n+1} = x_n + h\,r\,x_n\Big(1 - \frac{x_n}{K}\Big) = (1 + hr)\,x_n - \frac{hr}{K}\,x_n^2.$$

Mit

$$\mu = 1 + hr, \qquad y_n = \frac{hr}{(1 + hr)\,K}\,x_n$$

steht dort exakt die *logistische Abbildung* $y_{n+1} = \mu y_n(1 - y_n)$ – keine Näherung, eine Substitution. Und die ist das Standardbeispiel für den Weg ins Chaos.



Notebook `notebooks/06_feigenbaum.ipynb`

Links die Verdopplungskaskade über $\mu = 1 + hr$, also direkt über der Schrittweite. In der Mitte dieselbe Differentialgleichung dreimal gelöst: $h = 1$ gibt die S-Kurve, $h = 2{,}8$ springt zwischen zwei Werten, $h = 3{,}3$ ist unregelmäßig. Rechts streben die Verdopplungsabstände zweier ganz verschiedener Abbildungen gegen dieselbe Zahl.

Die Schwellen lassen sich in Schrittweiten übersetzen. Der Fixpunkt ist stabil bis $\mu = 3$, also bis $h = 2/r$ – genau die Stabilitätsgrenze, die Notebook `01` für $x' = -\lambda x$ gefunden hat. Darüber kommt Periode 2, bei $\mu = 1 + \sqrt{6}$ Periode 4, und ab $\mu_\infty = 3{,}569946\ldots$, also $h \approx 2{,}57/r$, hört die Kaskade auf und das Chaos fängt an.

Bemerkung 6.21 (Was Feigenbaum gefunden hat). Die Verdopplungen häufen sich geometrisch, und das Verhältnis der Abstände strebt gegen

$$\delta \;=\; \lim_{n\to\infty} \frac{\mu_n - \mu_{n-1}}{\mu_{n+1} - \mu_n} \;=\; 4{,}669201609\ldots$$

Das Bemerkenswerte ist nicht die Zahl, sondern dass sie nicht zur Parabel gehört. Dieselbe Konstante kommt bei jeder Abbildung heraus, die einen einzigen, quadratisch gekrümmten Höcker hat. Das Notebook rechnet es für $y \mapsto \mu \sin(\pi y)$ nach – kein Polynom, dieselbe Zahl.

Feigenbaum veröffentlichte das 1978, nachdem er es auf einem Taschenrechner bemerkt hatte. Ein Beweis kam 1982 von Lanford, und zwar computergestützt – einer der ersten Fälle überhaupt. Damit gehört er in dieselbe Reihe wie der Vierfarbensatz und die Keplersche Vermutung aus Abschnitt 1.8.

Für dieses Kapitel ist die Moral eine numerische. Notebook `01` hat gezeigt, dass eine zu große Schrittweite ein Verfahren instabil macht, Notebook `04`, dass sie aus einer geschlossenen Bahn eine Spirale macht, und Notebook `05`, dass es dagegen ein billiges Mittel gibt. Hier rechnet das Verfahren nicht ungenau, sondern es rechnet ein *anderes dynamisches System* aus – und zwar eines, das es in der Gleichung gar nicht gibt.

Das Unangenehme daran: Man sieht es den Zahlen nicht an. Sie bleiben im plausiblen Bereich, sie schwanken nur. Wer das Ergebnis nicht gegen eine kleinere Schrittweite prüft, merkt nichts.

Zusammenfassung

- Ein Anfangswertproblem $x' = f(t,x)$, $x(t_0) = x_0$ ist dasselbe wie eine Integralgleichung, und die ist ein Fixpunktproblem.
- Systeme und höhere Ordnungen sind keine eigenen Fälle: beides wird zu einer Gleichung erster Ordnung in höherer Dimension. Die Beweise brauchen von $\mathbb{R}^n$ nur Norm und Vollständigkeit und gelten deshalb in jedem Banachraum.
- Stetigkeit der rechten Seite reicht nicht: $x' = 2\sqrt{|x|}$ hat unendlich viele Lösungen, $x' = x^2$ läuft in endlicher Zeit weg.
- Picard–Lindelöf (Satz 6.5) liefert lokale Existenz und Eindeutigkeit, über den Banachschen Fixpunktsatz im Raum der stetigen Funktionen.
- Grönwall (Lemma 6.6) gibt die Eindeutigkeit im Großen und die stetige Abhängigkeit von den Anfangsdaten – mit einer Schranke, die exponentiell wächst.
- Lineare Systeme sind auf dem ganzen Intervall lösbar, und ihr homogener Lösungsraum ist ein Vektorraum der Dimension n (Satz 6.10).
- Die allgemeine Lösung der inhomogenen Gleichung ist *eine spezielle plus die allgemeine homogene* (Satz 6.12) – dieselbe Struktur wie bei $Ax = b$. Die spezielle liefert die Variation der Konstanten (Satz 6.13).
- Bei konstanten Koeffizienten ist $\Phi(t) = e^{tA}$, und man rechnet über die Eigenwerte: $e^{\lambda t}$, bei mehrfachen Nullstellen mit Faktoren t^k. Alle Realteile negativ heißt stabil.
- Geschlossen lösbar ist wenig, und fast alles davon geht auf zwei Fälle zurück: trennbar und linear erster Ordnung (Abschnitt 6.8). Der Rest ist Substitution, Ordnung senken oder Ordnung erhöhen.

- Das Euler-Verfahren ist die einfachste Quadraturformel, Schritt für Schritt angewandt; seine Fehlerordnung $O(h)$ folgt aus Grönwall.
- Gerechnet wird in den Notebooks (Abschnitt 6.11). Sie zeigen die Fehlerordnung, die Stabilitätsgrenze $h \leq 2/\lambda$, die Picard-Iteration, und dass ein Löser bei verletzten Voraussetzungen stillschweigend eine von unendlich vielen Lösungen ausgibt.
- Am Pendel (Abschnitt 6.9) hängt fast alles zusammen: Linearisierung gibt den harmonischen Oszillator mit e^{tA} als Drehung, Dämpfung die drei Fälle der Eigenwerttabelle, Anregung die Resonanz mit dem Faktor t. Nichtlinear bleiben Erhaltungsgröße und geschlossene Bahnen, Isochronie geht verloren.
- Lotka–Volterra und das Pendel zeigen, wofür man die höhere Dimension braucht: ein Phasenporträt. Eine Erhaltungsgröße entscheidet dort, wo die Linearisierung nichts sagt, und liefert nebenbei die globale Existenz.
- Die Schrittweite ist kein reiner Genauigkeitsparameter. Das Euler-Verfahren auf der logistischen Gleichung *ist* die logistische Abbildung mit $\mu = 1 + hr$; zu große Schritte erzeugen Chaos in einer Gleichung, deren Lösungen allesamt monoton sind.

Kapitel 7

Sprachmodelle

Kapitel 1 handelte davon, dass Maschinen inzwischen Beweise finden. Dieses Kapitel handelt von den Maschinen. Es sagt, was ein Sprachmodell rechnet – und am Ende, was man daraus über Intelligenz lernt und was nicht.

7.1 Was ein Sprachmodell ist

Text wird zuerst in *Token* zerlegt: Stücke aus einem festen, endlichen Vokabular V, meist zwischen einem Zeichen und einem Wort lang. Ein Sprachmodell ist dann nichts weiter als eine Wahrscheinlichkeitsverteilung über Tokenfolgen.

Jede solche Verteilung lässt sich exakt faktorisieren:

$$p(x_1, \dots, x_n) \;=\; \prod_{t=1}^{n} p(x_t \mid x_1, \dots, x_{t-1}).$$

Das ist keine Annahme, sondern die Kettenregel der Wahrscheinlichkeitsrechnung. Sie sagt: Wer alle bedingten Verteilungen kennt, kennt die ganze Verteilung. Ein Modell muss also nur eines können – zu einem Anfang den nächsten Token vorhersagen.

Ein Modell mit Parametern θ berechnet dafür

$$x_{1:t-1} \;\longmapsto\; p_\theta(\,\cdot \mid x_{1:t-1}) \;\in\; \Delta^{|V|-1},$$

einen Punkt im Simplex, also einen Vektor aus $|V|$ nichtnegativen Zahlen mit Summe 1.

Das Modell schreibt keinen Text. Es ist wichtig, das auseinanderzuhalten: Die Ausgabe eines Sprachmodells ist eine *Verteilung*, kein Wort. Welches Wort daraus wird, entscheidet ein Verfahren, das daraus zieht – das kann der wahrscheinlichste Token sein, ein zufällig gezogener, oder etwas dazwischen.

Der Text, den man am Ende liest, ist also ein Produkt aus Modell und Ziehverfahren. Das Modell selbst behauptet nichts; es gibt an, für wie wahrscheinlich es jede Fortsetzung hält.

7.2 Softmax

Ein Netz rechnet in reellen Zahlen, nicht in Wahrscheinlichkeiten. Es gibt also zunächst einen Vektor von *Logits* $z \in \mathbb{R}^{|V|}$ aus, und der muss in den Simplex. Das besorgt die *Softmax*:

$$\operatorname{softmax}(z)_i \;=\; \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Lean `code/Kapitel7.lean`

```
/-- Die Softmax-Abbildung schickt Logits auf eine Wahrscheinlichkeitsverteilung. -/
noncomputable def softmax (z : ι → ℝ) : ι → ℝ :=
  fun i => Real.exp (z i) / ∑ j, Real.exp (z j)

theorem nenner_pos (z : ι → ℝ) : 0 < ∑ j, Real.exp (z j) :=
  Finset.sum_pos (fun j _ => Real.exp_pos (z j)) Finset.univ_nonempty

/-- Jeder Eintrag ist echt positiv -- kein Token hat Wahrscheinlichkeit null. -/
theorem softmax_pos (z : ι → ℝ) (i : ι) : 0 < softmax z i :=
  div_pos (Real.exp_pos _) (nenner_pos z)

/-- Und die Eintraege summieren sich zu eins. Zusammen: ‘softmax z‘ liegt im
Simplex der Wahrscheinlichkeitsverteilungen ueber dem Vokabular. -/
theorem softmax_sum (z : ι → ℝ) : ∑ i, softmax z i = 1 := by
  unfold softmax
  rw [← Finset.sum_div, div_self (nenner_pos z).ne']

/-- Groesseres Logit, groessere Wahrscheinlichkeit. -/
theorem softmax_mono (z : ι → ℝ) {i j : ι} (h : z i ≤ z j) :
    softmax z i ≤ softmax z j := by
  have hS := nenner_pos z
  unfold softmax
  gcongr

omit [Nonempty ι] in
/-- Eine Konstante auf alle Logits aendert nichts. Deshalb darf man vor dem
Rechnen das Maximum abziehen -- so vermeidet man Ueberlauf. -/
theorem softmax_shift (z : ι → ℝ) (c : ℝ) :
    softmax (fun i => z i + c) = softmax z := by
  funext i
  unfold softmax
  simp only [Real.exp_add, ← Finset.sum_mul]
  rw [mul_div_mul_right _ _ (Real.exp_ne_zero c)]
```

Die drei Sätze oben sind die Rechtfertigung dafür: Die Einträge sind positiv, sie summieren sich zu eins, und größere Logits bekommen größere Wahrscheinlichkeit. Der vierte – Verschiebungsinvarianz – ist der Grund, warum man vor dem Rechnen das Maximum abzieht: e^{z_i} läuft sonst über.

Bemerkung 7.1 (Nichts hat Wahrscheinlichkeit null). Die Exponentialfunktion ist nirgends null, also bekommt *jeder* Token positive Wahrscheinlichkeit – auch der unsinnigste. Dass ein Modell „irgendetwas sagen kann“, ist keine Schwäche der Umsetzung, sondern steht in der Formel.

Mit einer *Temperatur* T rechnet man $\mathrm{softmax}(z/T)$. Für $T \to 0$ bleibt nur das Maximum übrig, für großes T wird die Verteilung flach. T ist also ein Regler zwischen „immer das Wahrscheinlichste“ und „fast gleichverteilt“.

7.3 Die Verlustfunktion

Trainiert wird auf die *Kreuzentropie*: den mittleren negativen Logarithmus der Wahrscheinlichkeit, die das Modell dem tatsächlich beobachteten nächsten Token gegeben hat,

$$L(\theta) \;=\; -\frac{1}{N}\sum_{t=1}^{N} \log p_\theta(x_t \mid x_{<t}).$$

Mehr steht dort nicht. Das ganze Training besteht darin, diese eine Zahl kleiner zu machen.

Satz 7.2 (Zerlegung und untere Schranke)**.** *Für Wahrscheinlichkeitsverteilungen p (Daten) und q (Modell) gilt*

$$\underbrace{-\sum_i p_i \log q_i}_{\text{Kreuzentropie } H(p,q)} = \underbrace{-\sum_i p_i \log p_i}_{\text{Entropie } H(p)} + \underbrace{\sum_i p_i \log \tfrac{p_i}{q_i}}_{\text{Divergenz } D(p\|q) \geq 0}.$$

Insbesondere ist $H(p,q) \geq H(p)$, mit Gleichheit genau für $q = p$.

Lean code/Kapitel7.lean

```
omit [Nonempty ι] in
/-- **Ungleichung von Gibbs.** Die Kullback-Leibler-Divergenz ist nie negativ.
Der Beweis ist eine Zeile Analysis: `log x ≤ x - 1`. -/
theorem kl_nonneg (p q : ι → ℝ) (hp : ∀ i, 0 < p i) (hq : ∀ i, 0 < q i)
    (hps : ∑ i, p i = 1) (hqs : ∑ i, q i = 1) :
    0 ≤ ∑ i, p i * Real.log (p i / q i) := by
  have key : ∀ i ∈ Finset.univ, p i * Real.log (q i / p i) ≤ q i - p i := by
    intro i _
    have hpne : p i ≠ 0 := (hp i).ne'
    have h1 : Real.log (q i / p i) ≤ q i / p i - 1 :=
      Real.log_le_sub_one_of_pos (div_pos (hq i) (hp i))
    calc p i * Real.log (q i / p i) ≤ p i * (q i / p i - 1) :=
          mul_le_mul_of_nonneg_left h1 (hp i).le
      _ = q i - p i := by field_simp
  have hsum := Finset.sum_le_sum key
  rw [Finset.sum_sub_distrib, hps, hqs, sub_self] at hsum
  have hflip : ∀ i, p i * Real.log (p i / q i) = -(p i * Real.log (q i / p i)) :=
    by
    intro i
    rw [← mul_neg, ← Real.log_inv, inv_div]
  simp only [hflip, Finset.sum_neg_distrib]
  linarith

omit [Nonempty ι] in
/-- **Kreuzentropie = Entropie + Divergenz.** Der erste Summand haengt nur von
den Daten ab, der zweite nur vom Modell. -/
theorem kreuzentropie_zerlegung (p q : ι → ℝ) (hp : ∀ i, 0 < p i) (hq : ∀ i, 0 <
    q i) :
    -∑ i, p i * Real.log (q i)
      = (-∑ i, p i * Real.log (p i)) + ∑ i, p i * Real.log (p i / q i) := by
  have h : ∀ i, p i * Real.log (p i / q i)
      = p i * Real.log (p i) - p i * Real.log (q i) := by
    intro i; rw [Real.log_div (hp i).ne' (hq i).ne']; ring
  simp only [h, Finset.sum_sub_distrib]
  ring

omit [Nonempty ι] in
/-- **Der Boden des Verlusts.** Kein Modell kommt unter die Entropie der Daten.
Trainieren kann nur die Divergenz verkleinern, nie die Entropie. -/
theorem verlust_untere_schranke (p q : ι → ℝ) (hp : ∀ i, 0 < p i) (hq : ∀ i, 0 <
    q i)
    (hps : ∑ i, p i = 1) (hqs : ∑ i, q i = 1) :
    (-∑ i, p i * Real.log (p i)) ≤ -∑ i, p i * Real.log (q i) := by
  rw [kreuzentropie_zerlegung p q hp hq]
  linarith [kl_nonneg p q hp hq hps hqs]
```

Der Boden. Der erste Summand hängt nur von den Daten ab, der zweite nur vom Modell. Trainieren

kann also *nur* die Divergenz verkleinern, nie die Entropie. Es gibt einen Boden, und der liegt dort, wo das Modell die Daten perfekt trifft.

Dieser Boden ist nicht null. Sprache ist nicht determiniert: Nach „Ich gehe heute ins“ geht es auf viele Weisen weiter, und ein perfektes Modell würde das auch so angeben. Ein Verlust von null hieße, dass der nächste Token immer feststeht – das wäre kein gutes Modell, sondern ein auswendig gelerntes.

Zwei Umrechnungen sind üblich. Die *Perplexität* e^{L} ist die effektive Zahl der Möglichkeiten: Perplexität 8 heißt, das Modell ist so unsicher, als hätte es bei jedem Schritt zwischen acht gleich wahrscheinlichen Tokens zu wählen. Und mit $\log_2$ gerechnet ist L die mittlere Zahl *Bits* pro Token.

Bemerkung 7.3 (Ein Sprachmodell ist ein Kompressor). Das ist keine Analogie, sondern dieselbe Größe. Nach Shannon lässt sich ein Text, dessen Tokens das Modell mit p bewertet, in $-\log_2 p(\text{Text})$ Bits codieren – und das ist $N \cdot L/\ln 2$. Besseres Modell heißt kürzerer Code, Bit für Bit.

Shannon hat 1951 Menschen raten lassen, wie englischer Text weitergeht, und daraus etwa ein Bit pro Zeichen geschätzt. Diese Zahl ist seitdem der Maßstab.

Warum Kreuzentropie und nicht etwa die Trefferquote? Weil sie differenzierbar ist, weil sie ehrliche Unsicherheit belohnt – wer sich sicher gibt und falsch liegt, zahlt viel – und weil ihr Gradient nach den Logits so einfach ausfällt, dass man ihn hinschreiben kann:

$$\frac{\partial L}{\partial z} \;=\; \operatorname{softmax}(z) - e_k \qquad \text{(Vorhersage minus Wahrheit).}$$

Lean code/Kapitel7.lean

```
/-- Der Gradient des Verlusts nach den Logits ist bemerkenswert einfach. Im
Zweiklassenfall ist der Verlust ‘log(1 + exp(-z))‘ und seine Ableitung
‘σ(z) - 1‘ -- also "Vorhersage minus Wahrheit". Im allgemeinen Fall steht
dort ‘softmax(z) - e_k‘. -/
theorem verlust_ableitung (z : ℝ) :
    HasDerivAt (fun u : ℝ => Real.log (1 + Real.exp (-u)))
      (1 / (1 + Real.exp (-z)) - 1) z := by
  have hpos : (0:ℝ) < 1 + Real.exp (-z) := by positivity
  have hinner : HasDerivAt (fun u : ℝ => 1 + Real.exp (-u)) (-Real.exp (-z)) z :=
    by
    simpa using ((Real.hasDerivAt_exp (-z)).comp z (hasDerivAt_neg z)).const_add 1
  have h := hinner.log hpos.ne'
  convert h using 1
  field_simp
  ring

/- Die Rueckwaertsrechnung ist die Kettenregel, in der guenstigen Reihenfolge
   ausgewertet: Fuer f = f_L ∘ ... ∘ f_1 ist Df = Df_L * ... * Df_1. Bei
   skalarem Ergebnis ist Df_L eine Zeile; von links nach rechts multipliziert
   bleibt alles eine Zeile -- billiger als umgekehrt. -/
#check @HasFDerivAt.comp
#check @ContinuousLinearMap.comp
```

7.4 Der Transformer

Bleibt die Frage, wie $p_\theta(\cdot \mid x_{<t})$ gerechnet wird. Seit 2017 fast immer mit derselben Bauweise.

Die n Tokens werden zu Vektoren: $X \in \mathbb{R}^{n\times d}$, eine Zeile je Position. Daraus bildet man drei Matrizen,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

genannt *Anfragen*, *Schlüssel* und *Werte*. Der Kern ist dann

$$\mathrm{Attn}(X) \;=\; \mathrm{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right) V.$$

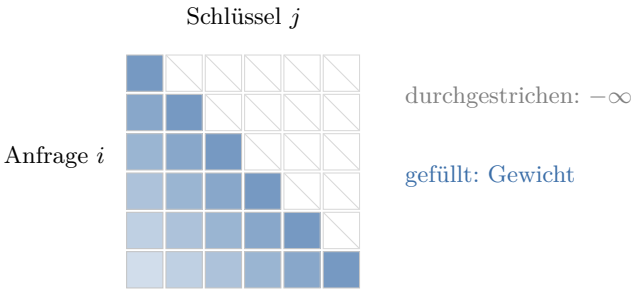
Eine weiche Nachschlagetabelle. Jede Position stellt mit ihrer Anfrage q_i eine Frage. Jede andere Position hält mit ihrem Schlüssel k_j dagegen, wie gut sie zu dieser Frage passt – gemessen am Skalarprodukt $q_i \cdot k_j$. Die Softmax macht aus diesen Passungen Gewichte, und damit wird aus den Werten v_j gemittelt.

In einer gewöhnlichen Nachschlagetabelle passt genau ein Schlüssel. Hier passen alle ein bisschen, und das Ergebnis ist eine Mischung. Daher *weich*.

Drei Teile der Formel verdienen eine Begründung.

Warum $\sqrt{d_k}$. Sind die Einträge von q und k unabhängig mit Varianz 1, so hat $q \cdot k = \sum_{l=1}^{d_k} q_l k_l$ die Varianz d_k. Ohne den Faktor würden die Logits mit der Dimension immer größer, die Softmax sättigt, und ihr Gradient verschwindet. Das Teilen durch $\sqrt{d_k}$ hält die Varianz bei 1.

Warum die Maske M. M ist 0 unterhalb der Diagonale und $-\infty$ darüber. Nach der Softmax bedeutet das: Position i sieht nur die Positionen $\leq i$. Erst das macht aus dem Netz ein Sprachmodell im Sinn von Abschnitt 7.1 – ohne Maske dürfte es beim Vorhersagen die Antwort ablesen.



Warum es Positionsinformation braucht. Vertauscht man die Zeilen von X, so vertauschen sich die Zeilen der Ausgabe mit – und sonst ändert sich nichts. Aufmerksamkeit ist *permutationsäquivariant*: Sie sieht eine Menge von Vektoren, keine Reihenfolge. Ohne Zusatz könnte ein Transformer „Hund beißt Mann“ nicht von „Mann beißt Hund“ unterscheiden. Deshalb addiert man zu den Einbettungen eine Positionscodierung.

Satz 7.4 (Aufmerksamkeit mischt nur)**.** *Die Ausgabe einer Aufmerksamkeitsschicht an jeder Position liegt in der konvexen Hülle der Wertvektoren $v_1, \ldots, v_n$.*

Lean `code/Kapitel7.lean`

```
/-- **Die Ausgabe einer Aufmerksamkeitsschicht liegt in der konvexen Huelle
der Wertvektoren.** Die Gewichte sind nichtnegativ und summieren sich zu
eins, also ist die Ausgabe eine Konvexkombination der Zeilen von `V`.

Daraus folgt: Aufmerksamkeit allein kann nichts erzeugen, was nicht schon in
den Werten steckt -- sie kann nur zwischen ihnen mischen. Alles Neue muss aus
dem MLP und dem Residualstrom kommen. -/
theorem attention_mem_convexHull {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ
    E]
```

```
    (z : ι → ℝ) (v : ι → E) :
    ∑ i, softmax z i • v i ∈ convexHull ℝ (Set.range v) :=
  (convex_convexHull ℝ _).sum_mem (fun i _ => (softmax_pos z i).le) (softmax_sum
    z)
    (fun i _ => subset_convexHull ℝ _ ⟨i, rfl⟩)
```

Was daraus folgt. Die Gewichte sind positiv und summieren sich zu eins – also ist die Ausgabe eine Konvexkombination der Werte. Aufmerksamkeit kann nichts erzeugen, was nicht schon in den Werten steckt; sie kann nur auswählen und mischen.

Alles Neue muss also woanders herkommen: aus den Matrizen W_Q, W_K, W_V, aus dem MLP hinter jeder Schicht, und aus der Tatsache, dass sich das alles viele Male wiederholt. Die Aufmerksamkeit bewegt Information zwischen den Positionen; verrechnet wird sie anderswo.

Der Rest der Bauweise ist schnell erzählt. Man rechnet mehrere Aufmerksamkeitsköpfe parallel und hängt sie aneinander (*Multi-Head*). Jede Schicht schreibt ihr Ergebnis *additiv* in einen Residualstrom,

$$x \;\mapsto\; x + \mathrm{Attn}(\mathrm{LN}(x)), \qquad x \;\mapsto\; x + \mathrm{MLP}(\mathrm{LN}(x)),$$

wobei LN normiert. Dass addiert und nicht ersetzt wird, ist der Grund, warum sich sehr viele Schichten stapeln lassen: Der Gradient findet immer einen direkten Weg zurück.

Bemerkung 7.5 (Was Kontext kostet). Die Matrix $QK^\top$ hat n^2 Einträge. Der Aufwand wächst also quadratisch in der Kontextlänge, der Speicher ebenso. Das ist der Grund, warum lange Kontexte teuer sind, und das Ziel ungefähr aller Varianten seit 2019.

7.5 Training

Minimiert wird $L(\theta)$ mit Gradientenabstieg – demselben, der in Abschnitt 4.5 steht. Zwei Dinge kommen hinzu.

Stochastisch. Den Gradienten über alle Daten zu berechnen ist unbezahlbar. Man nimmt eine zufällige Teilmenge (*Minibatch*) der Größe B. Deren Gradient ist ein erwartungstreuer Schätzer des vollen Gradienten, mit Varianz proportional zu $1/B$. Der Abstieg läuft also in die richtige Richtung, nur verwackelt.

Rückwärts. Der Gradient wird mit der Kettenregel berechnet. Für eine Hintereinanderausführung $f = f_L \circ \cdots \circ f_1$ ist

$$Df \;=\; Df_L \cdot Df_{L-1} \cdots Df_1,$$

ein Produkt von Jacobi-Matrizen. Matrixmultiplikation ist assoziativ, die Kosten sind es nicht: Weil das Ergebnis ein *Skalar* ist, ist der linke Faktor eine Zeile. Multipliziert man von links nach rechts, bleibt alles eine Zeile, und die ganze Rechnung kostet so viel wie ein Vorwärtsdurchlauf. Andersherum wären es so viele Durchläufe, wie es Parameter gibt.

Das ist die *Rückwärtsrechnung*, und mehr ist sie nicht: die Kettenregel, in der günstigen Reihenfolge ausgewertet.

Bemerkung 7.6 (Was in der Praxis dazukommt). Statt des reinen Abstiegs nimmt man Adam, dazu eine Aufwärmphase und einen abfallenden Schrittweitenplan, Gradientenbegrenzung, Gewichtsdämpfung. Keine dieser Entscheidungen folgt aus einem Satz; sie sind ausprobiert und haben sich gehalten.

Und noch einmal der Hinweis aus Abschnitt 6.10: Der Gradientenabstieg hat die Form eines Euler-Schritts, aber er löst keine Differentialgleichung. Was zählt, ist, wo er landet, nicht welchen Weg er nimmt.

7.6 Was skaliert

Die auffälligste empirische Tatsache dieses Gebiets ist, wie regelmäßig der Verlust fällt, wenn man größer rechnet. Über viele Größenordnungen passt

$$L(N, D) \;\approx\; E \;+\; \frac{A}{N^{\alpha}} \;+\; \frac{B}{D^{\beta}},$$

mit N der Zahl der Parameter, D der Zahl der Tokens und Exponenten um 0,3. Daraus lässt sich ausrechnen, wie man ein Rechenbudget zwischen Modellgröße und Datenmenge aufteilt – die Antwort von 2022 lautet: ungefähr gleichmäßig, und bis dahin waren alle Modelle zu groß für ihre Datenmenge.

Der konstante Term E ist dabei genau der Boden aus Satz 7.2: die Entropie der Sprache selbst. Dorthin kann man sich nähern, darunter nicht.

Bemerkung 7.7 (Was das ist und was nicht). Diese Gesetze sind Anpassungen an Messpunkte, keine Sätze. Sie gelten in dem Bereich, in dem gemessen wurde, und es gibt keine Theorie, die sie erklärt. Dass eine so einfache Formel über sechs Größenordnungen passt, ist eine Tatsache ohne Begründung – das ist ungewöhnlich und sollte misstrauisch machen.

Ähnlich bei den *emergenten Fähigkeiten*: Es sah lange so aus, als tauchten bestimmte Fähigkeiten bei einer Größe sprunghaft auf. Ein Teil davon war ein Artefakt der Messung – wer „ganz richtig oder gar nicht“ misst, sieht Sprünge, wo der zugrunde liegende Verlust glatt fällt.

7.7 Was Sprachmodelle über Intelligenz verständlich machen

Der tote Fisch. 2006 setzten Beal, Hover, Triantafyllou, Liao und Lauder eine *tote* Forelle in einen Strömungskanal, hinter einen Zylinder. Hinter dem Zylinder reißen abwechselnd Wirbel ab, eine Kármánsche Wirbelstraße. Der tote Fisch blieb nicht nur stehen – er bewegte sich *stromaufwärts*.

Kein Muskel, kein Nerv, keine Absicht. Der biegsame Körper gerät mit den ankommenden Wirbeln in Resonanz, entzieht ihnen Energie, erzeugt Schub und überwindet seinen eigenen Widerstand.[1]

Warum das hierher gehört: „Schwimmt gegen die Strömung“ galt als sicheres Zeichen für ein lebendes, sich anstrengendes Tier. Es ist keines. Der Schluss war nie gültig – wir hatten nur nie einen Gegenfall gesehen.

Genau das haben Sprachmodelle mit einem anderen Schluss gemacht. „Schreibt flüssig, antwortet zum Thema, erklärt einen Witz“ galt als sicheres Zeichen für Verstehen. Auch dieser Schluss war immer nur Induktion: Alles, was wir je so reden sahen, verstand auch. Ein Gegenfall genügt, und den gibt es jetzt.

Wichtig ist, was damit *nicht* gezeigt ist. Der tote Fisch zeigt nicht, dass lebende Fische nicht schwimmen. Er zeigt, dass das *Verhalten* die Frage nicht entscheidet. Ebenso zeigen Sprachmodelle nicht, dass niemand versteht – sie zeigen, dass flüssige Rede kein Beweis dafür ist.

Wo die Analogie trägt

Am weitesten trägt sie an einer Stelle, die über die bloße Warnung hinausgeht. Der Fisch schwimmt nicht aus eigener Kraft; er *erntet* Struktur, die schon da ist – die Wirbelstraße, die der Zylinder erzeugt hat. Die Arbeit wurde anderswo getan, und der Körper muss nur die richtige Form haben, um sie abzugreifen.

[1] D. N. Beal u. a., *Passive propulsion in vortex wakes*, J. Fluid Mech. 549 (2006), 385–402. Jimmy Liao bekam dafür 2024 den Ig-Nobelpreis für Physik.

Ein Sprachmodell erntet ebenfalls. Was es modelliert, ist Sprache, und Sprache ist der Niederschlag von sehr vielen Menschen, die sehr lange nachgedacht haben. Ein System, das diesen Niederschlag gut modelliert, gibt die Form dieses Denkens wieder, ohne es zu tun. Die Frage ist dann nicht „denkt es?“, sondern „wessen Denken sehe ich hier?“.

Wo sie nicht trägt

Man sollte es damit nicht übertreiben, denn der Unterschied ist groß.

Der Fisch enthält fast nichts. Seine ganze Mitwirkung besteht in Biegsamkeit und einer Eigenfrequenz, die zufällig passt. Die gesamte Struktur, auf die es ankommt, steckt im Wasser. Ein Sprachmodell dagegen enthält Hunderte Milliarden angepasster Zahlen; die Struktur steckt *drinnen*, nicht nur draußen.

Und der Fisch funktioniert nur in genau dieser Wirbelstraße. Nimmt man ihn heraus, ist er ein toter Fisch. Sprachmodelle brechen nicht so ein, wenn man sie aus ihrer Verteilung herausnimmt; sie werden schlechter, aber sie antworten auf Eingaben, die es nie gab. Wie weit das trägt, ist genau die offene Frage – aber es ist nicht nichts.

Was die Analogie wirklich klärt

Vielleicht das Nützlichste: Sie legt den Finger auf die Stelle, an der die Frage falsch gestellt ist.

Wo *ist* das Schwimmen? Nicht im Fisch – der ist tot. Nicht im Wasser – es fließt nur geradeaus. Es ist in der Kombination, und die Grenze zwischen „System“ und „Umgebung“ haben wir gezogen, nicht die Natur.

Dieselbe Frage bei Sprachmodellen: Wo ist das Können? Im Modell? Im Textbestand, aus dem es stammt? In der Eingabeaufforderung, die ein Mensch geschrieben hat? Wer eines davon herausnimmt, bekommt nichts. Die Antwort „im Modell“ ist vielleicht so sinnvoll wie die Antwort „im Fisch“.

Bemerkung 7.8 (Warum der Vergleich mit dem Flugzeug nicht tröstet). Man hört oft: Flugzeuge fliegen, ohne mit den Flügeln zu schlagen, und irgendwann hat niemand mehr gefragt, ob sie „wirklich“ fliegen. So werde es auch hier kommen.

Der Vergleich hinkt an einer entscheidenden Stelle. Die Frage nach dem Flugzeug hat sich aufgelöst, weil wir eine *Theorie* bekamen – die Aerodynamik erklärt Vogel und Flugzeug mit denselben Gleichungen, und die Unterschiede stehen dann in den Randbedingungen. Für das Denken haben wir keine solche Theorie. Ohne sie bleibt die Frage, und zwar zu Recht.

Vier Dinge, die klarer geworden sind

Trotz allem ist etwas gewonnen, und zwar mehr, als die Debatte meist zugibt.

1. **Sehr viel sprachliche Kompetenz ist aus der Form allein erreichbar.** Dass Grammatik ohne angeborene Struktur lernbar ist und dass aus reiner Verteilungsstatistik etwas entsteht, das meist auch inhaltlich zusammenpasst, hätte vor 2018 kaum jemand erwartet. Widerlegt ist damit kein philosophisches Argument, aber die empirische Grundlage mehrerer Argumente ist weg.

2. **Vorhersage ist ein anspruchsvolles Ziel.** Das Training verlangt nichts als „rate den nächsten Token“. Dass das ausreicht, ist selbst ein Argument: Wer gut vorhersagen will, muss das modellieren, was die Daten erzeugt hat. Das rückt Vorhersage und Kompression näher an das heran, was man von Intelligenz erwartet, als man dachte.

3. **Fähigkeit ist kein Skalar.** Sprachmodelle sind in manchem weit über menschlichem Niveau und in anderem unter dem eines Kindes – verlässlich zählen, konsistent bei einer

Behauptung bleiben, planen. Wäre Intelligenz eine einzige Größe, die sich füllt, dürfte es dieses Muster nicht geben.

4. **Es gibt jetzt einen zweiten Fall.** Das ist das Wichtigste. Jede Theorie des Denkens war bis vor kurzem eine Theorie über ein einziges Beispiel, uns selbst. Eine Wissenschaft mit $n = 1$ kann nicht unterscheiden, was notwendig ist und was Zufall unserer Bauart. Jetzt ist $n = 2$, und die beiden Fälle sind an interessanten Stellen verschieden. Erst da fängt eine Theorie an, überhaupt greifen zu können.

Zusammengenommen: Was Sprachmodelle über Intelligenz verständlich machen, ist vor allem, wie wenig wir verstanden hatten. Wir hielten eine Reihe von Schlüssen für sicher, die nur deshalb nie falsch geworden waren, weil es keinen Gegenfall gab. Jetzt gibt es einen.

7.8 Was sie nicht zeigen

Die Kehrseite derselben Einsicht. Wenn Verhalten die Frage nicht entscheidet, dann entscheiden auch Testverfahren sie nicht – und Testverfahren sind Verhalten. Ein Modell, das eine Aufgabensammlung löst, hat gezeigt, dass es diese Sammlung löst. Mehr nicht, und bei öffentlich verfügbaren Sammlungen nicht einmal das, weil sie in den Trainingsdaten stehen können.

Das ist kein Grund zur Resignation, sondern der Grund für den Rest dieses Skripts. Es gibt eine Art von Beweis, die dieses Argument nicht trifft:

Die eine Sorte Beleg, die hält. Ein Lean-Beweis, der durch den Kern geht, ist richtig – gleichgültig, ob ihn ein Mensch, ein Modell oder ein Zufallsgenerator erzeugt hat. Der Kern fragt nicht nach Verstehen, er rechnet nach.

Damit umgeht man die ganze Frage. Man muss nicht entscheiden, ob dort jemand denkt; man muss nur prüfen, ob der Term typt. Genau das war die Pointe von Abschnitt 1.8, und genau deshalb steht sie am Anfang dieses Skripts und nicht am Ende.

Was dabei zu prüfen bleibt, ist dasselbe wie dort: ob die *Aussage* die gemeinte ist. Der tote Fisch kann einem auch das nicht abnehmen.

Zusammenfassung

- Ein Sprachmodell ist eine Wahrscheinlichkeitsverteilung über Tokenfolgen, zerlegt nach der Kettenregel. Es gibt eine Verteilung aus, keinen Text.
- Die Softmax bringt Logits in den Simplex: positiv, Summe eins, monoton, verschiebungsinvariant.
- Trainiert wird auf Kreuzentropie. Sie zerfällt in Entropie plus Divergenz (Satz 7.2); die Entropie ist ein Boden, den kein Modell unterschreitet. e^L ist die Perplexität, $L/\ln 2$ die Bits pro Token – ein Sprachmodell ist ein Kompressor.
- Aufmerksamkeit ist eine weiche Nachschlagetabelle. Ihre Ausgabe liegt in der konvexen Hülle der Werte (Satz 7.4): Sie mischt, sie erfindet nicht.
- Die Rückwärtsrechnung ist die Kettenregel in der günstigen Reihenfolge – ein Argument über Assoziativität, kein neuer Begriff.
- Die Skalierungsgesetze sind Messungen, keine Sätze; ihr konstanter Term ist die Entropie der Sprache.

- Der tote Fisch zeigt: Verhalten entscheidet die Frage nach dem Inneren nicht. Das trifft Sprachmodelle und ihre Tests gleichermaßen – und ist der Grund, warum ein maschinengeprüfter Beweis eine andere Sorte Beleg ist.